

Draft Standard for Local and Metropolitan Area Networks— Station and Media Access Control Connectivity Discovery

Prepared by the

Maintenance Task Group of IEEE 802.1

Sponsor

LAN/MAN Standards Committee

of the

IEEE Computer Society

This and the following cover pages are not part of the draft. They provide revision and other information for IEEE 802.1 Working Group members and will be updated as convenient. New participants: Please read these cover pages, they contain information that should help you contribute effectively to this standards development project. The [Introduction to the current draft](#) should be useful to all readers.

The text proper of this draft begins with the [Title page](#).

Important Notice

This document is an unapproved draft of a proposed IEEE Standard. IEEE hereby grants the named IEEE SA Working Group or Standards Committee Chair permission to distribute this document to participants in the receiving IEEE SA Working Group or Standards Committee, for purposes of review for IEEE standardization activities. No further use, reproduction, or distribution of this document is permitted without the express written permission of IEEE Standards Association (IEEE SA). Prior to any review or use of this draft standard, in part or in whole, by another standards development organization, permission must first be obtained from IEEE SA (stds-copyright@ieee.org). This page is included as the cover of this draft, and shall not be modified or deleted.

IEEE Standards Association
445 Hoes Lane
Piscataway, NJ 08854, USA

1 Introduction to the current draft¹

2 This document is a revision in progress of IEEE Std 802.1AB-2016, incorporating published amendments,
3 draft amendments (if, and as, noted on the Title page), the agreed or proposed resolution of [Maintenance](#)
4 [items](#) (see below), and additional [Technical corrections](#) and [Editorial corrections](#) to the description of existing
5 functionality.

6 These cover pages also include an introduction to [Participation in 802.1 standards development](#), a summary
7 of the [PAR \(Project Authorization Request\)](#), and a general discussion of [Draft development](#).¹⁶

8 Draft P802.1AB-2016-Rev-d0.1 rolls up the amendments IEEE Std 802.1ABcu-2021 and IEEE Std
9 802.1ABdh-2021 into IEEE Std 802.1AB-2016. Text added by the amendment IEEE Std 802.1ABcu-2021
10 outside of clause 12 (which is completely added by 802.1ABcu) is highlighted by the used of [blue text](#). Text
11 added by the amendment IEEE Std 802.1ABdh-2021 is highlighted by the used of [magenta text](#). The titles of
12 figures changed by these amendments is highlighted, however the actual figure is not highlighted. The
13 highlighting of inserted text will be removed in a future revision of this draft.

14 Maintenance items

15 This draft includes the following proposed or agreed resolutions of maintenance items that were not
16 addressed by rolled-up amendments.

17 — Maintenance item 0339 was implemented in clause 12

18 Technical corrections

19 This draft does not include any technical corrections beyond those addressed by the rolled-up amendments.

20 Editorial corrections

21 Pre-publication staff editing of amendments addressed a number of inconsistencies in terminology,
22 capitalisation, etc. That editing was necessarily restricted to the text of each published amendment, though
23 the choice between alternatives was largely determined by predominant use and general rules in the
24 standard as a whole. This draft carries that resolution into the remaining text of the clauses touched by each
25 amendment or the rest of the standard. In some cases the cost benefit of comprehensive change favours
26 devoting time to more significant issues:

27 This draft also includes the following corrections that do not appear in the amendments:

28 —

29 MIB modules

30 The MIB modules specified by this standard are attached to the draft pdf as plain text (UTF-8) .mib files.

31 YANG modules

32 As noted above, the following YANG modules in this draft have been updated as a result of maintenance
33 activity not addressed by the included amendments:

34 —

¹The whole or parts of the introduction, possibly updated, to past drafts may be retained at the Editor's discretion, with the most recent introduction first. The introduction to each draft may solicit input on specific subjects.

1 A Revision statement has been added to each of these modules indicating that they are “TO BE Published ..”
2 as part of the Revision, together with an indication of the change. The “TO BE ” is to be removed and the
3 revision date updated at pre-publication time [the revision statements in published standard reflects only the
4 historical derivation of each published (i.e. standardized) module version from its published predecessors.
5 The description of the change from the prior published version should be retained. Note that these updated
6 modules are not ‘the standard’ at this time.

¹ **This page intentionally left blank**

¹ **This page intentionally left blank**

¹ **This page intentionally left blank**

1 Participation in 802.1 standards development

2 All participants in IEEE 802.1 activities should be aware of the Working Group Policies and Procedures, and
3 their obligations under the IEEE Patent Policy, the IEEE Standards Association (SA) Copyright Policy, and the
4 IEEE SA Participation Policy. For information on these policies see 1.ieee802.org/rules/ and the slides
5 presented at the beginning of each of our Working Group and Task Group meeting.

6 The IEEE SA [PAR \(Project Authorization Request\)](#) is summarized in these cover pages and a link provided to
7 the full text. As part of the IEEE 802® process, the PAR of each project is reviewed regularly to ensure its
8 continued validity. A vote of “Approve” on this draft is also an affirmation that the PAR and CSD is still valid.

9 Comments on this draft are encouraged. NOTE: All issues related to IEEE standards presentation style,
10 formatting, spelling, etc. are routinely handled between the 802.1 Editor and the IEEE Staff Editors prior to
11 publication, after balloting and the process of achieving agreement on the technical content of the standard is
12 complete. Readers are urged to devote their valuable time and energy only to comments that materially affect
13 either the technical content of the document or the clarity of that technical content. Comments should not
14 simply state what is wrong, but also what might be done to fix the problem.

15 Full participation in the work of IEEE 802.1 requires attendance at IEEE 802 meetings. Information on 802.1
16 activities, working papers, and email distribution lists etc. can be found on the 802.1 Website:

17 <http://ieee802.org/1/>

18 Use of the email distribution list is not presently restricted to 802.1 members, and the working group has a
19 policy of considering comments from all who are interested and willing to contribute to the development of the
20 draft. Individuals not attending meetings have helped to identify sources of misunderstanding and ambiguity
21 in past projects. The email lists exist primarily to allow the members of the working group to develop
22 standards, and are not a general forum. All contributors to the work of 802.1 should familiarize themselves
23 with the IEEE patent policy and anyone using the email distribution list will be assumed to have done so.
24 Information can be found at <http://standards.ieee.org/db/patents/>

25 Comments on this draft may be sent to the 802.1 email exploder, to the Editors, or to the Chairs of the 802.1
26 Working Group and Time-Sensitive Networking (TSN) Task Group.

27 Paul Bottorff
28 Editor, IEEE Std 802.1AB
29 Email: paul.bottorff@hpe.com

30 Mark Hantel
31 Chair, 802.1 Maintenance Task Group
32
33 Email: mrhantel@ra.rockwell.com

Glenn Parsons
Chair, 802.1 Working Group
+1 514-379-9037
Email: glenn.parsons@ericsson.com

34 NOTE: Comments whose distribution is restricted in any way cannot be considered, and may not be
35 acknowledged.

36 **All participants in IEEE standards development have responsibilities under the IEEE patent policy and**
37 **should familiarize themselves with that policy, see**
38 <http://standards.ieee.org/about/sasb/patcom/materials.html>

39 As part of our IEEE 802 process, the text of the PAR and CSD (Criteria for Standards Development, formerly
40 referred to as the 5 Criteria or 5C's) is reviewed on a regular basis in order to ensure their continued validity.
41 A vote of “Approve” on this draft is also an affirmation by the balloter that the PAR is still valid.

1 PAR (Project Authorization Request)

2 The PAR for the Revision of IEEE Std 802.1AB-2016 was approved by IEEE NesCom, Sep 26th, 2024:

3 <https://development.standards.ieee.org/myproject-web/app#viewpar/15809/11716>

4 This is a maintenance PAR so did not require an IEEE 802 CSD (Criteria for Standards Development).

5 The Need for the Project is as follows:

6 This revision project is needed to incorporate approved amendments and corrigenda, to incorporate technical
7 and editorial corrections to existing functionality, and to ensure that consistency is maintained in the
8 consolidated text.

9 During the early stages of draft development, 802.1 editors have a responsibility to attempt to craft technically
10 coherent drafts from the resolutions of ballot comments and from the other discussions that take place in the
11 working group meetings. Preparation of drafts often exposes inconsistencies in editor's instructions or
12 exposes the need to make choices between approaches that were not fully apparent in the meeting. Choices
13 and requests by the editors' for contributions on specific issues will be found in the editors' [Introduction to the](#)
14 [current draft](#) and at appropriate points in the draft.

15 Any text with a Cyan background (as in this sentence) is temporary, with conditional tag 'Editor comment',
16 inserted by the Editors to solicit comment, suggest a future change, or act simply as an aide memoire. Text
17 can also be highlighted to draw it to the readers' attention, using conditional tag 'Editor highlight'. In both
18 these cases conditional tagging helps location, and eventual removal, of text or highlighting and can control
19 whether or not it is displayed.

20 The ballot comments received on each draft, and the editors' proposed and final disposition of comments on
21 working group drafts, are part of the audit trail of the development of the standard and are available, along
22 with all the revisions of the draft on the 802.1 website (for address see above).

23 During the early stages of draft development the proposed text can be moved around a great deal, and even
24 minor rearrangement can lead to a lot of 'change', not all of which is noteworthy from the point of the reviewer,
25 so the use of automatic change bars is not very effective. In early drafts change bars may be omitted or
26 applied manually, with a view to drawing the readers attention to the most significant areas of change.
27 Readers interested in viewing every change are encouraged to use Adobe Acrobat to compare the document
28 with their selected prior draft. Note that the FrameMaker change bar feature is useless when it comes to
29 indicating changes to Figures.

30 Records of participants in the development of the standard are added after SA Ballot, as part of
31 pre-publication editing by IEEE Staff.

1
2
3
4

P802.1AB™-2016-Rev/D0.1
March 5, 2025

(Revision of IEEE Std 802.1AB-2016 as amended by IEEE Std 802.1ABcu-2021 and IEEE Std 802.1ABdh)

5 **Draft Standard for**
6 **Local and metropolitan area networks—**
7 **Station and Media Access Control**
8 **Connectivity Discovery**

9 Prepared by the
10 **Maintenance Task Group of IEEE 802.1**

11 Sponsor
12 **LAN/MAN Standards Committee**
13 **of the**
14 **IEEE Computer Society**

15 Copyright © 2025 by the IEEE.
16 Three Park Avenue
17 New York, New York 10016-5997, USA
18 All rights reserved.

Copyright © 2025 by the Institute of Electrical and Electronics Engineers, Inc.
345 East 47th Street
New York, NY 10017, USA
All rights reserved.

All rights reserved. This document is an unapproved draft of a proposed IEEE Standard. As such, this document is subject to change. USE AT YOUR OWN RISK! Because this is an unapproved draft, this document must not be utilized for any conformance/compliance purposes. Permission is hereby granted for IEEE Standards Committee participants to reproduce this document for purposes of IEEE standardization activities only. Prior to submitting this document to another standards development organization for standardization activities, permission must first be obtained from the Manager, Standards Licensing and Contracts, IEEE Standards Activities Department. Other entities seeking permission to reproduce this document, in whole or in part, must obtain permission from the Manager, Standards Licensing and Contracts, IEEE Standards Activities Department.

IEEE Standards Department
Copyright and Permissions
445 Hoes Lane, P.O. Box 1331
Piscataway, NJ 08855-1331, US

1 **Abstract:** A protocol and a set of managed objects that can be used for discovering the physical
2 topology from adjacent stations in IEEE 802[®] LANs are defined in this document.

3

4 **Keywords:** IEEE 802.1AB[™], link layer discovery protocol, management information base,
5 topology discovery, topology information

6

7

8

9

10

11

12

13

1 **Important Notices and Disclaimers Concerning IEEE Standards Documents**

2 IEEE documents are made available for use subject to important notices and legal disclaimers. These notices
3 and disclaimers, or a reference to this page, appear in all standards and may be found under the heading
4 “Important Notice” or “Important Notices and Disclaimers Concerning IEEE Standards Documents.”

5 **Notice and Disclaimer of Liability Concerning the Use of IEEE Standards** 6 **Documents**

7 IEEE Standards documents (standards, recommended practices, and guides), both full-use and trial-use, are
8 developed within IEEE Societies and the Standards Coordinating Committees of the IEEE Standards
9 Association (“IEEE-SA”) Standards Board. IEEE (“the Institute”) develops its standards through a
10 consensus development process, approved by the American National Standards Institute (“ANSI”), which
11 brings together volunteers representing varied viewpoints and interests to achieve the final product.
12 Volunteers are not necessarily members of the Institute and participate without compensation from IEEE.
13 While IEEE administers the process and establishes rules to promote fairness in the consensus development
14 process, IEEE does not independently evaluate, test, or verify the accuracy of any of the information or the
15 soundness of any judgments contained in its standards.

16 IEEE does not warrant or represent the accuracy or content of the material contained in its standards, and
17 expressly disclaims all warranties (express, implied and statutory) not included in this or any other
18 document relating to the standard, including, but not limited to, the warranties of: merchantability; fitness
19 for a particular purpose; non-infringement; and quality, accuracy, effectiveness, currency, or completeness of
20 material. In addition, IEEE disclaims any and all conditions relating to: results; and workmanlike effort.
21 IEEE standards documents are supplied “AS IS” and “WITH ALL FAULTS.”

22 Use of an IEEE standard is wholly voluntary. The existence of an IEEE standard does not imply that there
23 are no other ways to produce, test, measure, purchase, market, or provide other goods and services related to
24 the scope of the IEEE standard. Furthermore, the viewpoint expressed at the time a standard is approved and
25 issued is subject to change brought about through developments in the state of the art and comments
26 received from users of the standard.

27 In publishing and making its standards available, IEEE is not suggesting or rendering professional or other
28 services for, or on behalf of, any person or entity nor is IEEE undertaking to perform any duty owed by any
29 other person or entity to another. Any person utilizing any IEEE Standards document, should rely upon his
30 or her own independent judgment in the exercise of reasonable care in any given circumstances or, as
31 appropriate, seek the advice of a competent professional in determining the appropriateness of a given IEEE
32 standard.

33 IN NO EVENT SHALL IEEE BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL,
34 EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO:
35 PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR
36 BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY,
37 WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR
38 OTHERWISE) ARISING IN ANY WAY OUT OF THE PUBLICATION, USE OF, OR RELIANCE UPON
39 ANY STANDARD, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE AND
40 REGARDLESS OF WHETHER SUCH DAMAGE WAS FORESEEABLE.

1 **Translations**

2 The IEEE consensus development process involves the review of documents in English only. In the event
3 that an IEEE standard is translated, only the English version published by IEEE should be considered the
4 approved IEEE standard.

5 **Official statements**

6 A statement, written or oral, that is not processed in accordance with the IEEE-SA Standards Board
7 Operations Manual shall not be considered or inferred to be the official position of IEEE or any of its
8 committees and shall not be considered to be, or be relied on as, a formal position of IEEE. At lectures,
9 symposia, seminars, or educational courses, an individual presenting information on IEEE standards shall
10 make it clear that his or her views should be considered the personal views of that individual rather than the
11 formal position of IEEE.

12 **Comments on standards**

13 Comments for revision of IEEE Standards documents are welcome from any interested party, regardless of
14 membership affiliation with IEEE. However, IEEE does not provide consulting information or advice
15 pertaining to IEEE Standards documents. Suggestions for changes in documents should be in the form of a
16 proposed change of text, together with appropriate supporting comments. Since IEEE standards represent a
17 consensus of concerned interests, it is important that any responses to comments and questions also receive
18 the concurrence of a balance of interests. For this reason, IEEE and the members of its societies and
19 Standards Coordinating Committees are not able to provide an instant response to comments or questions
20 except in those cases where the matter has previously been addressed. For the same reason, IEEE does not
21 respond to interpretation requests. Any person who would like to participate in revisions to an IEEE
22 standard is welcome to join the relevant IEEE working group.

23 Comments on standards should be submitted to the following address:

24 Secretary, IEEE-SA Standards Board

25 445 Hoes Lane

26 Piscataway, NJ 08854 USA

27 **Laws and regulations**

28 Users of IEEE Standards documents should consult all applicable laws and regulations. Compliance with the
29 provisions of any IEEE Standards document does not imply compliance to any applicable regulatory
30 requirements. Implementers of the standard are responsible for observing or referring to the applicable
31 regulatory requirements. IEEE does not, by the publication of its standards, intend to urge action that is not
32 in compliance with applicable laws, and these documents may not be construed as doing so.

33 **Copyrights**

34 IEEE draft and approved standards are copyrighted by IEEE under U.S. and international copyright laws.
35 They are made available by IEEE and are adopted for a wide variety of both public and private uses. These
36 include both use, by reference, in laws and regulations, and use in private self-regulation, standardization,
37 and the promotion of engineering practices and methods. By making these documents available for use and
38 adoption by public authorities and private users, IEEE does not waive any rights in copyright to the
39 documents.

1 Photocopies

2 Subject to payment of the appropriate fee, IEEE will grant users a limited, non-exclusive license to
3 photocopy portions of any individual standard for company or organizational internal use or individual, non-
4 commercial use only. To arrange for payment of licensing fees, please contact Copyright Clearance Center,
5 Customer Service, 222 Rosewood Drive, Danvers, MA 01923 USA; +1 978 750 8400. Permission to
6 photocopy portions of any individual standard for educational classroom use can also be obtained through
7 the Copyright Clearance Center.

8 Updating of IEEE Standards documents

9 Users of IEEE Standards documents should be aware that these documents may be superseded at any time
10 by the issuance of new editions or may be amended from time to time through the issuance of amendments,
11 corrigenda, or errata. An official IEEE document at any point in time consists of the current edition of the
12 document together with any amendments, corrigenda, or errata then in effect.

13 Every IEEE standard is subjected to review at least every ten years. When a document is more than ten years
14 old and has not undergone a revision process, it is reasonable to conclude that its contents, although still of
15 some value, do not wholly reflect the present state of the art. Users are cautioned to check to determine that
16 they have the latest edition of any IEEE standard.

17 In order to determine whether a given document is the current edition and whether it has been amended
18 through the issuance of amendments, corrigenda, or errata, visit the IEEE-SA Website at [http://](http://ieeexplore.ieee.org/xpl/standards.jsp)
19 ieeexplore.ieee.org/xpl/standards.jsp or contact IEEE at the address listed previously. For more information
20 about the IEEE-SA or IEEE's standards development process, visit the IEEE-SA Website at [http://](http://standards.ieee.org)
21 standards.ieee.org.

22 Errata

23 Errata, if any, for all IEEE standards can be accessed on the IEEE-SA Website at the following URL: [http://](http://standards.ieee.org/findstds/errata/index.html)
24 standards.ieee.org/findstds/errata/index.html. Users are encouraged to check this URL for errata
25 periodically.

26 Patents

27 Attention is called to the possibility that implementation of this standard may require use of subject matter
28 covered by patent rights. By publication of this standard, no position is taken by the IEEE with respect to the
29 existence or validity of any patent rights in connection therewith. If a patent holder or patent applicant has
30 filed a statement of assurance via an Accepted Letter of Assurance, then the statement is listed on the IEEE-
31 SA Website at <http://standards.ieee.org/about/sasb/patcom/patents.html>. Letters of Assurance may indicate
32 whether the Submitter is willing or unwilling to grant licenses under patent rights without compensation or
33 under reasonable rates, with reasonable terms and conditions that are demonstrably free of any unfair
34 discrimination to applicants desiring to obtain such licenses.

35 Essential Patent Claims may exist for which a Letter of Assurance has not been received. The IEEE is not
36 responsible for identifying Essential Patent Claims for which a license may be required, for conducting
37 inquiries into the legal validity or scope of Patents Claims, or determining whether any licensing terms or
38 conditions provided in connection with submission of a Letter of Assurance, if any, or in any licensing
39 agreements are reasonable or non-discriminatory. Users of this standard are expressly advised that
40 determination of the validity of any patent rights, and the risk of infringement of such rights, is entirely their
41 own responsibility. Further information may be obtained from the IEEE Standards Association.

42

1 **Participants**

2 At the time this standard was submitted to the IEEE-SA for approval, the IEEE 802.1 Working Group had
3 the following membership:

4 **Glenn Parsons**, *Chair*
5 **Jessy Royer**, *Vice Chair*
6 **Mark Hantel**, *Maintenance Task Group Chair*
7 **Paul Bottorff**, *Editor*

8 The following members of the individual balloting committee voted on this standard. Balloters may have
9 voted for approval, disapproval, or abstention.

1 When the IEEE-SA Standards Board approved this standard on xx-xx-xx, it had the following membership:

2 , *Chair*
3 , *Vice Chair*
4 , *Past Chair*
5 , *Secretary*

6
7 *Member Emeritus
8

11 Historical participants

12 Included is a historical list of participants in the IEEE 802.1 Working Group who have dedicated their
13 valuable time, energy, and knowledge to the creation of this material.

14 The following individuals participated in the 2005 publication of this standard.

15 **Tony Jeffree, *Chair***
16 **Paul Congdon, *Vice Chair***
17 **Bill Lane, *Technical Editor***
18 **Mick Seaman, *Interworking Task Group Chair***
19

Les Bell
Paul Bottorff
Jim Burns
Marco Carugi
Dirceu Cavendish
Arjan de Heer
Anush Elangovan
Hesham Elbakoury
David Elie-Dit-Cosaque
Norm Finn
David Frattura
Gerard Goubert
Steve Haddock
Ran Ish-Shalom
Atsushi Iwata

Neil Jarvis
Manu Kaycee
Hal Keen
Roger Lapuh
Loren Larsen
Joe Lawrence
Yannick Le Goff
Marcus Leech
Mahalingam Mani
Dinesh Mohan
Bob Moskowitz
Don O'Connor
Don Pannell
Glenn Parsons

Ken Patton
Frank Reichstein
John Roesel
Allyn Romanow
Dan Romascanu
Jessy V. Rouyer
Ali Sajassi
Dolors Sala
Muneyoshi Suzuki
Jonathan Thatcher
Michel Thorsen
Dennis Volpano
Karl Weber
Ludwig Winkel
Michael D. Wright

1 The following individuals participated in the 2009 publication of this standard.

2

Tony Jeffree, *Chair and Editor*

3

Paul Congdon, *Vice Chair*

4

Stephen Haddock, *Chair, Interworking Task Group*

Osama Aboul-Magd
Zehavit Alon
Caitlin Bestler
Jan Bialkowski
Rob Boatright
Jean-Michel Bonnamy
Paul Bottorff
Rudolf Brandner
Craig W. Carlson
Weiyang Cheng
Rao Cherukuri
Paul Congdon
Diego Crupnicoff
Claudio Desanti
Zhemin Ding
Linda Dunbar
Hesham M. Elbakoury
David Elie-Dit-Cosaque
János Farkas
Donald Fedyk
Norman Finn
Robert Frazier
John Fuller
Geoffrey Garner
Anoop Ghanwani
Franz Goetz
Yannick Le Goff
Eric Gray
Karanvir Grewal
Craig Gunther
Mitch Gusat
Asif Hazarika
Charles Hudson

Romain Insler
Michael Johas Teener
Abhay Karandikar
Prakash Kashyap
Hal Keen
Keti Kilcrease
Yongbum Kim
Philippe Klein
Mike Ko
Vinod Kumar
Bruce Kwan
Kari Laihonen
Michael Lerer
Gael Mace
Ben Mack-Cran
David Martin
Riccardo Martinotti
Alan McGuire
James McIntosh
Menucher Menuchery
John Messenger
Matthew Mora
Eric Multanen
Kevin Nolish
Hiroshi Ohta
David Olsen
Donald Pannell
Glenn Parsons
Joseph Pelissier
David Peterson
Hayim Porat
Max Pritikin

Ananda Rajagopal
Karen T. Randall
Guenter Roeck
Josef Roese
Derek J. Rohde
Dan Romascanu
Moran Roth
Jessy V. Rouyer
Jonathan Sadler
Ali Sajassi
Joseph Salowey
Panagiotis Saltsidis
Satish Sathe
John Sauer
Michael Seaman
Koichiro Seto
Himanshu Shah
Nurit Sprecher
Kevin B. Stanton
Robert A. Sultan
Muneyoshi Suzuki
George Swallow
Attila Takacs
Patricia Thaler
Oliver Thorp
Manoj Wadekar
Yuehua Wei
Brian Weis
Bert Wijnen
Michael D. Wright
Chien-Hsien Wu
Ken Young
Glen Zorn

1 The following individuals participated in the development of Corrigendum 1 to the 2009 publication of this
2 standard.

3

Tony Jeffree, *Chair and Editor*

4

Glenn Parsons, *Vice-Chair and Maintenance Task Group Chair*

Ting Ao	Robert Grow	Karen Randall
Kenneth Boehlke	Yingjie Gu	Josef Roesse
Christian Boiger	Craig Gunther	Dan Romascanu
Brad Booth	Stephen Haddock	Jessy Rouyer
Paul Bottorff	Hitoshi Hayakawa	Ali Sajassi
Jeffrey Catlin	Mirko Jakovljevic	Panagiotis Saltsidis
Xin Chang	Markus Jochim	Rick Schell
Weiyang Cheng	Michael Johas Teener	Michael Seaman
Diego Crupnicoff	Girault Jones	Koichiro Seto
Rodney Cummings	Daya Kamath	Daniel Sexton
Donald Eastlake, III	Hal Keen	Rakesh Sharma
János Farkas	Yongbum Kim	Johannes Specht
Donald Fedyk	Philippe Klein	Kevin Stanton
Norman Finn	Oliver Kleineberg	Wilfried Steiner
Andre Fredette	Jeff Lynch	Patricia Thaler
Geoffrey Garner	Ben Mack-Crane	Jeremy Touve
Anoop Ghanwani	John Messenger	Albert Tretter
Franz Goetz	Eric Multanen	Maarten Vissers
Mark Gravel	Henry Muyshondt	Yuehua Wei
Eric Gray	David Olsen	Min Xiao
	Donald Pannell	

5

6

7 The following individuals participated in the development of Corrigendum 2 to the 2009 publication of this
8 standard.

9

Glenn Parsons, *Working Group Chair*

10

John Messenger, *Vice-Chair and Maintenance Task Group Chair*

11

Tony Jeffree, *Editor*

Ting Ao	Hitoshi Hayakawa	Dan Romascanu
Christian Boiger	Jeremy Hitt	Jessy V. Rouyer
Paul Bottorff	Rahil Hussain	Panagiotis Saltsidis
David Chen	Michael Johas Teener	Behcet Sarikaya
Feng Chen	Peter Jones	Michael Seaman
Weiyang Cheng	Hal Keen	Daniel Sexton
Diego Crupnicoff	Marcel Kiessling	Johannes Specht
Rodney Cummings	Yongbum Kim	Kevin B. Stanton
Patrick Diamond	Philippe Klein	Wilfried Steiner
Aboubacar Kader Diarra	Jouni Korhonen	Vahid Tabatabaee
János Farkas	Jeff Lynch	Patricia Thaler
Norman Finn	Ben Mack-Crane	Jeremy Touve
Geoffrey Garner	Christophe Mangin	Karl Weber
Anoop Ghanwani	James McIntosh	Yuehua Wei
Mark Gravel	Eric Multanen	Brian Weis
Eric W. Gray	Donald Pannell	Jordon Woods
Craig Gunther	Karen Randall	Juan-Carlos Zuniga
Stephen Haddock	Maximilian Riegel	

1 The following individuals participated in the 2016 publication of this standard.

2

Glenn Parsons, *Chair*

3

John Messenger, *Vice Chair and Maintenance Task Group Chair*

4

Tony Jeffree, *Editor*

Christian Boiger
Paul Bottorff
David Chen
Feng Chen
Weiyang Cheng
Rodney Cummings
János Farkas
Norman Finn
Geoffrey Garner
Eric Gray
Craig Gunther
Stephen Haddock
Mark Hantel
Marc Holness
Michael Johas Teener

Hal Keen
Stephan Kehrer
Marcel Kiessling
Philippe Klein
Jouni Korhonen
Yizhou Li
Christophe Mangin
Tom McBeath
James McIntosh
Hiroki Nakano
Bob Noseworthy
Donald R. Pannell
Walter Pieniac
Karen Randall
Maximilian Riegel
Dan Romascanu

Jessy Rouyer
Panagiotis Saltsidis
Michael Seaman
Daniel Sexton
Johannes Specht
Wilfried Steiner
Patricia Thaler
David Thornburg
Jeremy Touve
Paul Unbehagen
Karl Weber
Brian Weis
Jordon Woods
Helge Zinner
Juan Carlos Zuniga

5 The following members of the individual balloting committee voted on this standard. Balloters may have
6 voted for approval, disapproval, or abstention.

Thomas Alexander
Butch Anton
Lee Armstrong
Stefan Aust
Christian Boiger
Nancy Bravin
William Byrd
Juan Carreon
Rodney Cummings
Douglas Dorr
János Farkas
Yukihiro Fujimoto
David Gregson
Randall Groves
Craig Gunther
Stephen Haddock
Marek Hajduczenia
Jerome Henry
Marco Hernandez
Guido Hiertz
Werner Hoelzl
Noriyuki Ikeuchi
Sergiu Iordanescu
Atsushi Ito

Raj Jain
Tony Jeffree
Michael Johas Teener
Adri Jovin
Shinkyo Kaku
Piotr Karocki
Stuart Kerry
Yongbum Kim
Paul Lambert
Robert Landman
David Lewis
Arthur H. Light
William Lumpkins
Michael Lynch
Elvis Maculuba
Jonathon McLendon
Richard Mellitz
John Messenger
Charles Moorwood
Michael Newman
Nick S. A. Nikjoo
Satoshi Obara
Alon Regev
Maximilian Riegel

Robert Robinson
Benjamin Rolfe
Dan Romascanu
Jessy Rouyer
John Santhoff
Bartien Sayogo
Michael Seaman
Thomas Starai
Eugene Stoudenmire
Rene Struik
Walter Struppler
Michael Swearingen
Patricia Thaler
Mark-Rene Uchida
Lorenzo Vangelista
Dmitri Varsanofiev
George Vlantis
Khurram Waheed
Karl Weber
Hung-Yu Wei
Natalie Wienckowski
Andreas Wolf
Oren Yuen
Zhen Zhou

7

¹ Introduction

²

This introduction is not part of IEEE Std 802.1AB™-2016, IEEE Standard for Local and metropolitan area networks— Station and Media Access Control Connectivity Discovery.
--

³ This revision of IEEE Std 802.1AB incorporates the following amendments into the base text of the 2016
⁴ revision:

⁵ — IEEE Std 802.1ABcu-2021

⁶ — IEEE Std 802.1ABdh-2021

Contents

1	2	1.	Overview	25
3		1.1	Scope.....	26
4		1.2	Purpose.....	26
5	2.	Normative references	27	
6	3.	Definitions and numerical representation	29	
7		3.1	Definitions	29
8		3.2	Numerical representation	31
9	4.	Acronyms and abbreviations	32	
10	5.	Conformance.....	34	
11		5.1	Terminology.....	34
12		5.2	Protocol Implementation Conformance Statement (PICS).....	34
13		5.3	Required capabilities.....	34
14		5.4	Optional capabilities	35
15	6.	Principles of operation	36	
16		6.1	Transmission and reception	37
17		6.2	LLDP operational modes	38
18		6.3	LLDP information categories	38
19		6.4	TLV selection	38
20		6.5	Transmission principles	39
21		6.6	Reception principles	39
22		6.7	Systems with multiple LLDP Agents	40
23		6.8	LLDP and Link Aggregation	43
24		6.9	Extended LLDP (XLLDP).....	44
25	7.	LLDPDU transmission, reception, and addressing.....	45	
26		7.1	Destination address	45
27		7.2	Source address	48
28		7.3	EtherType use and encoding.....	48
29		7.4	LLDPDU reception.....	48
30	8.	LLDPDU and TLV formats	49	
31		8.1	LLDPDU bit and octet ordering conventions	49
32		8.2	LLDPDU format.....	49
33		8.3	TLV categories	50
34		8.4	Basic TLV format	51
35		8.5	Basic management TLV set formats and definitions.....	53
36		8.5a	Extended LLDP set TLV formats and definitions.....	62
37		8.6	Organizationally Specific TLVs	65
38	9.	LLDP agent operation.....	67	
39		9.1	Overview.....	67
40		9.2	State machines	72

1	10.	LLDP management	96
2	10.1	Data storage and retrieval	96
3	10.2	The LLDP management entity’s responsibilities.....	96
4	10.3	Managed objects	98
5	10.4	Data types	98
6	10.5	LLDP variables	99
7	11.	LLDP MIB definitions.....	102
8	11.1	Internet Standard Management Framework	102
9	11.2	Structure of the LLDP MIB	102
10	11.3	Relationship to other MIBs	107
11	11.4	Security considerations for LLDP base MIB module.....	108
12	11.5	LLDP MIB modules ,	110
13	12.	LLDP YANG definitions.....	161
14	12.1	Internet Standard Management Framework	161
15	12.2	Information model for LLDP management	161
16	12.3	Structure of the LLDP YANG model.....	164
17	12.4	Relationship to other YANG modules	166
18	12.5	Security considerations	166
19	12.6	Definition of the YANG modules,	167
20	Annex A		189
21	(normative) PICS proforma		189
22	Annex B		196
23	(normative) PTOPO MIB update.....		196
24	Annex C		197
25	(informative) Example LLDP transmission frame formats		197
26	Annex D		198
27	(informative) Bibliography		198

1 List of figures

2	Figure 6-1, LLDP agent and its relationship to its LLC entity	36
3	Figure 6-2, Relationship between LLDP agents, LLC Entities, MSAPs, and the LLDP management entity	40
4	Figure 6-3, LLDP in a MAC Bridge	41
5	Figure 6-4, LLDP in an end system with port-based network access control	42
6	Figure 6-5, LLDP in a MAC Bridge that uses port-based network access control on both ports	42
7	Figure 6-6, Scope of group MAC addresses	43
8	Figure 6-7, Multiplexing and demultiplexing using shims	43
9	Figure 7-1, MSDU format	45
10	Figure 8-1, LLDPDU formats	50
11	Figure 8-2, Basic TLV format	51
12	Figure 8-3, End Of LLDPDU TLV format	53
13	Figure 8-4, Chassis ID TLV format	53
14	Figure 8-5, Port ID TLV format	55
15	Figure 8-6, Time To Live TLV format	56
16	Figure 8-7, Port Description TLV format	56
17	Figure 8-8, System Name TLV format	57
18	Figure 8-9, System Description TLV format	58
19	Figure 8-10, System Capabilities TLV format	58
20	Figure 8-11, Management Address TLV format	60
21	Figure 8-11a, Manifest TLV format	62
22	Figure 8-11b, Extension Request TLV format	63
23	Figure 8-11c, Extension Identifier TLV format	64
24	Figure 8-12, Format for Organizationally Specific TLVs	65
25	Figure 9-1, Transmit state machine	91
26	Figure 9-2, Receive state machine	93
27	Figure 9-3, Extended receive state machine	94
28	Figure 9-4, Transmit timer state machine	95
29	Figure 11-1, LLDP MIB block diagram	102
30	Figure 12-1, YANG root hierarchy with LLDP YANG modules	162
31	Figure 12-2, LLDP configuration and monitoring objects	163
32	Figure 12-3, LLDP port configuration and monitoring objects	164
33	Figure C.1, IEEE 802.3 LLDP frame format	197
34	Figure C.2, IEEE 802.11 LLDP frame format	197

1 List of tables

2	Table 7-1, Group MAC addresses used by LLDP	46
3	Table 7-2, Support for the Scope MAC Address in different systems	47
4	Table 7-3, LLDP EtherType	48
5	Table 8-1, TLV type values	52
6	Table 8-2, chassis ID subtype enumeration	54
7	Table 8-3, port ID subtype enumeration	55
8	Table 8-4, System capabilities	59
9	Table 9-1, Subclause/operating mode applicability	67
10	Table 9-2, State machine symbols	73
11	Table 11-1, MIB object groups and operating mode applicability	103
12	Table 11-2, LLDP MIB structure and object cross reference	103
13	Table 12-2, Structure of the ieee802-dot1ab-lldp module	165
14	Table 12-1, Structure of the YANG modules	165

IEEE Standard for Local and metropolitan area networks— Station and Media Access Control Connectivity Discovery

IMPORTANT NOTICE: IEEE Standards documents are not intended to ensure safety, security, health, or environmental protection, or ensure against interference with or from other devices or networks. Implementers of IEEE Standards documents are responsible for determining and complying with all appropriate safety, security, environmental, health, and interference protection practices and all applicable laws and regulations.

This IEEE document is made available for use subject to important notices and legal disclaimers. These notices and disclaimers appear in all publications containing this document and may be found under the heading “Important Notice” or “Important Notices and Disclaimers Concerning IEEE Documents.” They can also be obtained on request from IEEE or viewed at <http://standards.ieee.org/TPR/disclaimers.html>.

1. Overview

The Link Layer Discovery Protocol (LLDP) specified in this standard allows stations attached to an IEEE 802[®] LAN to advertise, to other stations attached to the same IEEE 802 LAN, the major capabilities provided by the system incorporating that station, the management address or addresses of the entity or entities that provide management of those capabilities, and the identification of the station’s point of attachment to the IEEE 802 LAN required by those management entity or entities.

The information distributed via this protocol is stored by its recipients in a standard Management Information Base (MIB), making it possible for the information to be accessed by a Network Management System (NMS) using a management protocol such as the Simple Network Management Protocol (SNMP).

<<Editor’s Note: Perhaps the above paragraph should say something about YANG in addition to MIBs. Any thoughts?>>

1.1 Scope

The scope of this standard is to define a protocol and management elements, suitable for advertising information to stations attached to the same IEEE 802 LAN, for the purpose of populating physical topology and device discovery management information databases. The protocol facilitates the identification of stations connected by IEEE 802 LANs/MANs, their points of interconnection, and access points for management protocols.

This standard defines a protocol that

- a) Advertises connectivity and management information about the local station to adjacent stations on the same IEEE 802 LAN.
- b) Receives network management information from adjacent stations on the same IEEE 802 LAN.
- c) Operates with all IEEE 802 access protocols and network media.
- d) Establishes a network management information schema and object definitions that are suitable for storing connection information about adjacent stations.
- e) Provides compatibility with the IETF PTOPO MIB (IETF RFC 2922 [B9]).²

<<Editor's Note: Perhaps the above list should have a line item indicating the XLLDP protocol extensions. Any thoughts?>>

1.2 Purpose

This standard specifies the necessary protocol and management elements to

- a) Facilitate multi-vendor inter-operability and the use of standard management tools to discover and make available physical topology information for network management.
- b) Make it possible for network management to discover certain configuration inconsistencies or malfunctions that can result in impaired communication at higher layers.
- c) Provide information to assist network management in making resource changes and/or re-configurations that correct configuration inconsistencies or malfunctions identified in b) above.
- d) Provide IETF MIB (IETF RFC 2922 [B9]) to describe a network's physical topology and associated systems within that topology.
- e) Provide YANG configuration and operational models supporting Station and Media Access Control Connectivity Discovery.

<<Editor's Note: Perhaps the above list should have a line item indicating the extension capabilities of XLLDP. Any thoughts?>>

² The numbers in brackets correspond to those in the bibliography in Annex D.

2. Normative references

The following referenced documents are indispensable for the application of this document (i.e., they must be understood and used, so each referenced document is cited in the text and its relationship to this document is explained). For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments or corrigenda) applies.

IEEE Std 802[®], IEEE Standard for Local and Metropolitan Area Networks: Overview and Architecture.^{3, 4}

<<Editor's Note: 802.1ABcu adds a reference to 802d-2017, however this is now rolled into IEEE Std 802. The editor therefore did not include the reference to 802d and instead has changed the text referencing 802d to reference 802.>>

IEEE Std 802.1AC[™], IEEE Standard for Local and metropolitan area networks—Media Access Control (MAC) Service Definition.

IEEE Std 802.1AE[™], IEEE Standard for Local and Metropolitan Area Networks—Media Access Control (MAC) Security.

IEEE Std 802.1AX[™], IEEE Standard for Local and Metropolitan Area Networks—Link Aggregation.

IEEE Std 802.1Q[™], IEEE Standards for Local and Metropolitan Area Networks: Bridges and Bridged Networks.

IEEE Std 802.1X[™], IEEE Standard for Local and Metropolitan Area Networks—Port-Based Network Access Control.

IEEE Std 802.3[™], IEEE Standard for Information technology—Telecommunications and information exchange between systems—Local and metropolitan area networks—Specific requirements—Part 3: Carrier Sense Multiple Access with Collision Detection (CSMA/CD) Access Method and Physical Layer Specifications.

IETF RFC 2863, The Interfaces Group MIB, June 2000.⁵

IETF RFC 3046, DHCP Relay Agent Information Option, January 2001.

IETF RFC 3232, Assigned Numbers: RFC 1700 is Replaced by an On-line Database, January 2002.⁶

IETF RFC 3410, Introduction and Applicability Statements for Internet Standard Management Framework, December 2002.

IETF RFC 3417, Transport Mappings for the Simple Network Management Protocol (SNMP), December 2002.

IETF RFC 3418, Management Information Base (MIB) for the Simple Network Management Protocol (SNMP), December 2002.

IETF RFC 3629, UTF-8, a transformation format of ISO 10646, November 2003.

³ IEEE and 802 are registered trademarks in the U.S. Patent & Trademark Office, owned by The Institute of Electrical and Electronics Engineers, Incorporated.

⁴ IEEE publications are available from The Institute of Electrical and Electronics Engineers (<http://standards.ieee.org/>).

⁵ IETF documents (i.e., RFCs) are available for download at <http://www.rfc-archive.org/>.

⁶ The IETF RFC 3232 ianaAddressFamilyNumbers on-line database module is accessible at (<http://www.iana.org/>).

- 1 IETF RFC 4502, Remote Network Monitoring Management Information Base Version 2, May 2006.
- 2 IETF RFC 4639, Cable Device Management Information Base for Data-Over-Cable Service Interface
- 3 Specification (DOCSIS) Compliant Cable Modems and Cable Modem Termination Systems, December
- 4 2006.
- 5 IETF RFC 4789, Simple Network Management Protocol (SNMP) over IEEE 802 Networks, November
- 6 2006.
- 7 IETF RFC 6241, Network Configuration Protocol (NETCONF), June 2011.⁷
- 8 IETF RFC 6933, Entity MIB (Version 4), May 2013. IETF RFC 6991, Common YANG Data Types, July
- 9 2013.
- 10 IETF RFC 6991, Common YANG Data Types, July 2013.
- 11 IETF RFC 7950, The YANG 1.1 Data Modeling Language, August 2016.
- 12 IETF RFC 8343, A YANG Data Model for Interface Management, March 2018.
- 13 IETF RFC 8349, A YANG Data Model for Routing Management (NMDA Version), March 2018.
- 14 ISO/IEC 8824-1 [ITU-T Rec. X.680 (2002)], Abstract Syntax Notation One (ASN.1): Specification of Basic
- 15 Notation.⁸

⁷ IETF RFCs are available from the Internet Engineering Task Force (<https://www.rfc-archive.org/>).

⁸ ASN.1 standards are available at <http://asn1.elibel.tm.fr/en/standards/index.htm#asn1>.

3. Definitions and numerical representation

3.1 Definitions

For the purposes of this document, the following terms and definitions apply. The *IEEE Standards Dictionary Online* should be consulted for terms not defined in this clause.⁶

alpha-numeric information: Information that is encoded using the 8-bit Universal Character Set (UCS)/Unicode Transformation Format (UTF-8) octet sequence [IETF RFC 3629].

chassis: A physical component incorporating one or more IEEE 802[®] LAN stations and their associated application functionality.

chassis identifier: An administratively assigned name that identifies the particular chassis within the context of an administrative domain that comprises one or more networks.

Extended Link Layer Discovery Protocol (XLLDP): An LLDP extension used to retrieve the frames beyond the Normal LLDPDU of a multi-frame LLDP database.

Extension LLDPDU (XPDU): An LLDPDU containing an Extension Identifier TLV and not containing a Time to Live TLV or Extension Request TLV.

Extension Request LLDPDU (XREQ): An LLDPDU containing an Extension Request TLV and not containing a Time to Live TLV or Extension Identifier TLV.

IEEE 802[®] LAN: Local area network (LAN) technologies that provide a media access control (MAC) Service equivalent to the MAC Service defined in IEEE Std 802.1AC.

IEEE 802[®] LAN station: An IEEE 802-compatible entity that incorporates all the necessary mechanisms to participate in media access control of an IEEE 802 LAN, and that is at least capable of providing the MAC service plus the mandatory capabilities of the LLC.

NOTE 1—For example, routers and host computers are stations in a bridged network. Bridges, in addition to their relay function, also include station capabilities.⁷

Link Layer Discovery Protocol (LLDP): A media-independent protocol capable of running on all IEEE 802[®] LAN stations and to allow an LLDP agent to learn the connectivity and management information from adjacent stations.

LLDP agent: The protocol entity that implements LLDP for a particular MSAP associated with a Port.

MAC service access point (MSAP): The access point for MAC services provided to the LLC sublayer.

MSAP identifier: The identifier of a MAC service access point.

NOTE 2—In this standard, the concatenation of the chassis identifier and the port identifier is used by LLDP as an MSAP identifier, to identify the port associated with an IEEE 802[®] LAN station.

management entity: The protocol entity that implements a particular network management protocol and that provides access support to a MIB associated with the protocol and implemented on a host chassis.

⁶ *IEEE Standards Dictionary Online* is available at: <http://dictionary.ieee.org>.

⁷ Notes in text, tables, and figures are given for information only and do not contain requirements needed to implement the standard.

1 **Management Information Base (MIB):** The instantiation of all MIB modules in a managed entity (e.g.,
2 system or device).

3 **Management Information Base module (MIB module):** The specification or schema for a data base that
4 can be populated with the information required to support a network management information system.

5 **Manifest LLDPDU:** A Normal LLDPDU containing a Manifest TLV.

6 **network:** An interconnected group of systems, each comprising one or more IEEE 802® LAN stations.

7 **Network Management System (NMS):** A management system that is capable of utilizing the information
8 in a MIB.

9 **Normal LLDPDU:** An LLDPDU containing a Time To Live TLV and not containing an Extension Request
10 TLV or an Extension Identifier TLV.

11 **object identifier (OID):** An identifier used to name an object. Structurally, an OID consists of a node in a
12 hierarchically-assigned namespace, formally defined in ISO/IEC 8824-1, Abstract Syntax Notation 1
13 (ASN.1). OIDs are used in this standard to identify MIB modules and the objects they contain.

14 **physical network topology:** The identification of systems, of IEEE 802® LAN stations that compose each
15 system, and of the IEEE 802 LAN stations that attach to the same IEEE 802 LAN.

16 **port:** The entity in a chassis/system to support an MSAP. A port incorporates one and only one MSAP and
17 identifies the collection of manageable entities that provide the MAC Service at the MSAP.

18 **port identifier:** An administratively assigned name that identifies the particular port within the context of a
19 system, where the identification is convenient, local to the system, and persistent for the system's use and
20 management (whereas the MAC address that globally identifies the MSAP can not be).

21 **Scope MAC Address:** The MAC address used as the destination address in Normal LLDPDUs, which
22 determines the limits of propagation in the network.

23 **shutdown LLDPDU:** A Normal LLDPDU with a TTL value of zero.

24 **system:** A managed collection of hardware and software components incorporating one or more chassis,
25 stations and ports.

26 **station only:** A non-forwarding IEEE 802® LAN station such as a user workstation, network file server, or
27 print server.

28 **type, length, value (TLV):** A short, variable length encoding of an information element consisting of
29 sequential type, length, and value fields where the type field identifies the type of information, the length
30 field indicates the length of the information field in octets, and the value field contains the information,
31 itself.

32 **YANG:** IETF defined data modeling language, published as IETF RFC 7950.

33 **YANG model:** One or more YANG modules used to configure and monitor the managed element or system.

34 **YANG module:** The description of the data model used to configure and monitor the managed element or
35 system. A YANG module defines a hierarchy of nodes that can be used for NETCONF-based (see IETF
36 RFC 7803 [B13]) and RESTCONF-based (see IETF RFC 8040 [B14]) operations.

1 3.2 Numerical representation

2 Decimal, hexadecimal, and binary numbers are used within this document. For clarity, decimal numbers are
3 generally used to represent counts, hexadecimal numbers are used to represent addresses, and binary
4 numbers are used to describe bit patterns within binary fields.

5 Decimal numbers are represented in their usual 0, 1, 2, ... format. Hexadecimal numbers are represented by
6 a string of one or more hexadecimal (0–9, A–F) digits followed by the subscript 16, except in C-code
7 contexts, where they are written as `0x123EF2`, etc. Binary numbers are represented by a string of one or
8 more binary (0,1) digits, followed by the subscript 2. Thus the decimal number “26” may also be represented
9 as “1A₁₆” or “11010₂”.

10 MAC addresses, EtherType values, and OUI/EUI values are represented as strings of 8-bit hexadecimal
11 numbers separated by hyphens and without a subscript, as, for example, “01-80-C2-00-00-15” or
12 “AA-55-11”, using the hexadecimal representation defined in IEEE Std 802.

1 4. Acronyms and abbreviations

2 AP	Access Point
3 BSSID	basic service set identification
4 DA	Destination Address
5 DS	Distribution System
6 EPD	Ethertype Protocol Discrimination
7 EUI	Extended Unique Identifier
8 FCS	Frame Check Sequence
9 IANA	Internet Assigned Numbers Authority
10 ID	Identifier
11 IETF	Internet Engineering Task Force
12 KaY	Media access control Security Key Agreement Entity
13 LLC	logical link control (sublayer)
14 LLDP	Link Layer Discovery Protocol
15 LLDPDU	Link Layer Discovery Protocol data unit
16 LSAP	link service access point
17 MAC	media access control (sublayer)
18 MAU	medium attachment unit
19 MDI	media dependent interface
20 MIB	Management Information Base (module)
21 MSAP	Media access control service access point
22 MSDU	Media access control service data unit
23 NMS	Network Management System
24 OID	object identifier
25 OUI	organizationally unique identifier
26 PD	Powered Device
27 PHY	physical (sublayer)

1	PMD	physical media dependent (sublayer)
2	PSE	Power Sourcing Equipment
3	PVID	port virtual local area network identifier
4	PPVID	port and protocol virtual local area network identifier
5	PTOPO	the name of the Internet Engineering Task Force physical topology Management
6		Information Base
7	RFC	Request for comments
8	SA	Source Address
9	SecY	Media access control Security Entity
10	SNAP	Subnetwork Access Protocol
11	SNMP	Simple Network Management Protocol
12	STP	Spanning Tree Protocol
13	TPMR	Two-port Media access control relay
14	TLV	type, length, value
15	TTL	time to live (value)
16	UCS	Universal Character Set
17	UTF-8	8-bit Universal Character Set/Unicode Transformation Format
18	VID	virtual local area network identifier
19	XLLDP	Extended Link Layer Discovery Protocol
20	XPDU	Extension LLDPDU
21	XREQ	Extension Request LLDPDU

22

1 5. Conformance

2 This clause specifies the mandatory and optional capabilities provided by conformant implementations of
3 this standard.

4 5.1 Terminology

5 For consistency with IEEE and existing IEEE 802.1™ standards terminology, requirements placed upon
6 conformant implementations of this standard are expressed using the following terminology:

- 7 a) *Shall* is used for mandatory requirements.
- 8 b) *May* is used to describe implementation or administrative choices (“may” means “is permitted to,”
9 and hence, “may” and “may not” mean precisely the same thing).
- 10 c) *Should* is used for recommended choices (the behaviors described by “should” and “should not” are
11 both permissible but not equally desirable choices).

12 The PICS proforma (see Annex A) reflects the occurrences of the words *shall*, *may*, and *should* within the
13 standard.

14 The standard avoids needless repetition and apparent duplication of its formal requirements by using *is*, *is*
15 *not*, *are*, and *are not* for definitions and the logical consequences of conformant behavior. Behavior that is
16 permitted but is neither always required nor directly controlled by an implementor or administrator, or
17 whose conformance requirement is detailed elsewhere, is described by *can*. Behavior that never occurs in a
18 conformant implementation or system of conformant implementations is described by *can not*. The word
19 *allow* is used as a replacement for the phrase “support the ability for,” and the word *capability* means “is
20 able to, or can be configured to.”

21 5.2 Protocol Implementation Conformance Statement (PICS)

22 The supplier of an implementation that is claimed to conform to this standard shall complete a copy of the
23 PICS proforma provided in Annex A and shall provide the information necessary to identify both the
24 supplier and the implementation.

25 5.3 Required capabilities

26 A system for which conformance to this standard is claimed shall, for all ports for which support is claimed,
27 include the following capabilities:

- 28 a) If port access is controlled by IEEE Std 802.1X,⁶ LLDP exchanges shall be supported through the
29 controlled port, as specified in Clause 6.
- 30 b) The destination and source addressing shall conform to 7.1 and 7.2.
- 31 c) EtherType encapsulation shall conform to 7.3.
- 32 d) LLDPDU recognition and reception shall conform to the operation of the receive state machine as
33 defined in 9.2.10.
- 34 e) LLDPDU encapsulation shall conform to the specifications in 8.2.
- 35 f) The basic TLV format capability shall be implemented as defined in 8.4.
- 36 g) The basic management set of TLVs shall be implemented as defined in 8.5.
- 37 h) The Organizationally Specific TLV format capability shall be implemented as defined in 8.6.

⁶ Information on references can be found in 2.

- 1 i) The protocol shall conform to the specifications for all Clause 9 subclauses indicated in Table 9-1
- 2 for the particular operating mode (transmit only, receive only, or transmit and receive) being
- 3 implemented.
- 4 j) If receipt of LLDPDUs is supported, for every set of TLVs (the basic management set, **extended**
- 5 **LLDP set**, and any organizationally specific sets) supported, support shall be implemented for
- 6 receipt of every TLV defined in the set.
- 7 k) If transmission of LLDPDUs is supported, for every set of TLVs (the basic management set,
- 8 **extended LLDP set**, and any organizationally specific sets) supported, support shall be implemented
- 9 for transmission of every TLV defined in the set.
- 10 l) If transmission of LLDPDUs is supported, for every set of TLVs (the basic management set,
- 11 **extended LLDP set**, and any organizationally specific sets) supported, a capability shall be
- 12 implemented for users to determine which optional TLVs are transmitted in any particular
- 13 LLDPDU.
- 14 m) If SNMP is supported, then
 - 15 1) The system shall conform to the LLDP management specifications in Clause 11 and shall
 - 16 implement the sections of version 2 of the basic LLDP MIB indicated in Table 11-1 for the
 - 17 operating mode being implemented.
 - 18 2) The system shall support at least one of the transport mappings defined by IETF RFC 3417 or
 - 19 IETF RFC 4789.
- 20 n) If **neither SNMP nor YANG** are supported, the system shall provide storage and retrieval capability
- 21 equivalent to the functionality specified in 10.1 for the operating mode being implemented.
- 22 o) If **YANG** is supported, then the system shall conform to the LLDP management specifications in 12
- 23 and shall implement the sections of the LLDP YANG module for the operating mode being
- 24 implemented.
- 25 p) If the Extended Link Layer Discovery Protocol is supported, then
 - 26 1) The extended LLDP set of TLVs shall be implemented as defined in 8.5a.
 - 27 2) The Extension Request and Extension LLDPDU formats (8.2) and addressing (7.1.2) shall be
 - 28 implemented.
 - 29 3) The extended receive state machine (Figure 9-3) shall be implemented.

30 5.4 Optional capabilities

31 A system for which conformance to this standard is claimed may, for each port for which support is claimed,
 32 support the following capabilities:

- 33 a) If port access is controlled by IEEE Std 802.1X, LLDP exchanges may be supported through the
- 34 uncontrolled port, as specified in Clause 6.

6. Principles of operation

LLDP is a link layer protocol that allows an IEEE 802 LAN station to advertise the capabilities and current status of the system associated with an MSAP. The MSAP provides the MAC service to an LLC Entity, and that LLC Entity provides an LSAP to an LLDP agent that transmits and receives information to and from the LLDP agents of other stations attached to the same LAN. The information distributed and received in each LLDPDU is stored in one or more Management Information Bases (MIBs) or YANG modules. Figure 6-1 illustrates the LLDP agent and its relationship to its LLC Entity and MSAP, and to additional MIBs or YANG modules designed by the IETF, IEEE 802, and others.

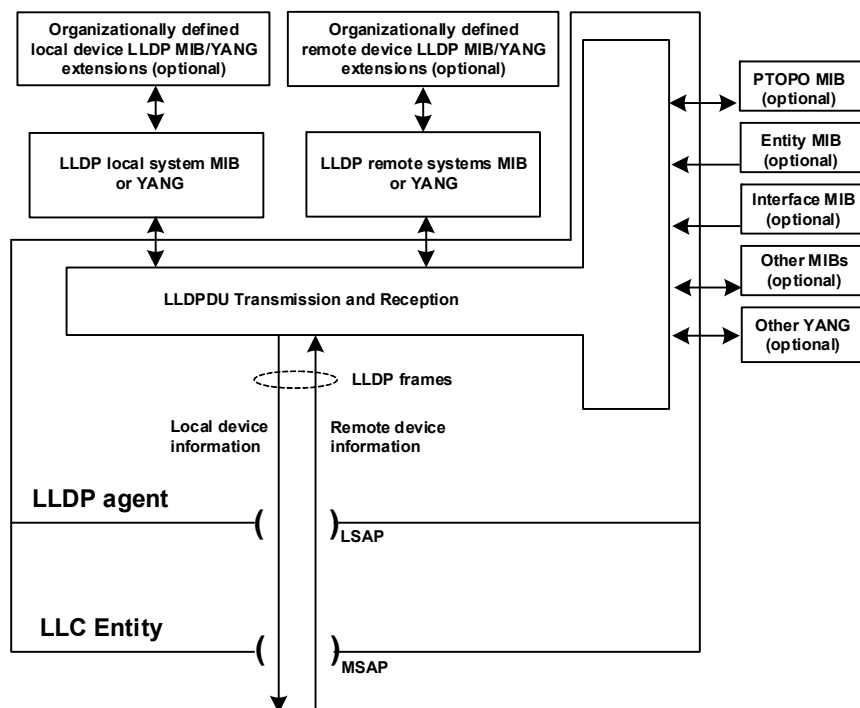


Figure 6-1—LLDP agent and its relationship to its LLC entity

<<Editor's Note: The above diagram includes the local/remote YANG, however does not list appropriate YANG modules on the right of the figure. Any thoughts?>>

This clause describes general principles of LLDP and Extended LLDP (XLLDP) operation. The following clauses specify transmission, reception, and addressing of LLPDUs (Clause 7); LLDPDU formats (Clause 8); operation of each LLDP agent in detail including state machines (Clause 9); management of LLDP (Clause 10); and the MIB module (Clause 11) or YANG module (Clause 12) used for LLDP management.

NOTE 1—For the purposes of this standard, the terms “LLC” and “LLC entity” include the service provided by the operation of entities that support protocol discrimination using an EtherType; i.e., protocol discrimination based on the Type interpretation of the Length/Type field as specified in IEEE Std 802.3, or the use of the SNAP encapsulation of EtherTypes as described in IEEE Std 802. Hence, “LSAP” in the above description can refer to the use of IEEE Std 802.2 LLC addresses, or an EtherType, as a means of higher layer protocol identification.

NOTE 2—The LLC Entity also provides distinct LSAPs to other higher layer protocol entities (such as those responsible for the Spanning Tree Protocol and IP). In a bridge, the LLC Entity associated with each Bridge Port is modeled as being directly connected to the attached LAN, as discussed in 8.13.9 of IEEE Std 802.1Q-2014, so the operation of LLDP is unaffected by the spanning tree Port State.

6.1 Transmission and reception

The information fields in each LLDP frame are contained in a Link Layer Discovery Protocol Data Unit (LLDPDU) as a sequence of variable length information elements, that each include type, length, and value fields (known as TLVs), where

- Type identifies what kind of information is being sent.
- Length indicates the length of the information string in octets.
- Value is the actual information that needs to be sent (for example, a binary bit map or an alpha-numeric string that can contain one or more fields).

Each LLDPDU begins with a mandatory sequence of three TLVs, as specified in 8.2, and can contain zero or more additional optional TLVs as selected by network management. The first three TLVs are:

- 1) A Chassis ID TLV.
- 2) A Port ID TLV.
- 3) A Time To Live TLV or, if XLLDP is supported, an Extension Request TLV or an Extension Identifier TLV.

The first two TLVs contain the chassis ID and the port ID values. These are concatenated to form a logical MSAP identifier. The combination of the MSAP identifier and the Scope MAC Address identify the LLDP agent/port that is the source of the information in the TLVs. Both the chassis ID and port ID values can be defined in a number of convenient forms. Once selected, however, the chassis ID/port ID value combination remains the same as long as the particular port remains operable (8.5.2, 8.5.3, 9.2.4.1).

The third TLV differentiates between three types of LLDPDUs:

- a) A Normal LLDPDU, where the third TLV is a Time To Live TLV that tells the receiving LLDP agent how long all information pertaining to this LLDPDU's MSAP identifier is valid so that all of the associated information can later be automatically discarded by the receiving LLDP agent if the sender fails to update it in a timely manner. It is useful to further differentiate two special cases of a Normal LLDPDU:
 - 1) A shutdown LLDPDU, where the third TLV is a Time To Live TLV with a TTL value of zero indicating that any information pertaining to this LLDPDU's MSAP identifier is to be discarded immediately. A TTL value of zero can be used, for example, to signal that the sending port has initiated a port shutdown procedure.
 - 2) A Manifest LLDPDU, where the third TLV is a Time To Live TLV with a non-zero TTL value and one of the subsequent TLVs is a Manifest TLV.
- b) An Extension Request LLDPDU (XREQ), where the third TLV is an Extension Request TLV that the recipient of a Normal LLDPDU containing a Manifest TLV uses to request one or more of the Extension LLDPDUs described in the Manifest TLV. The Extension Request LLDPDU does not contain a Time to Live TLV.
- c) An Extension LLDPDU (XPDU), where the third TLV is an Extension Identifier TLV. The Extension LLDPDU does not contain a Time to Live TLV.

NOTE —The receive state machine (in 9.2.10 and in prior versions of this standard) specifies that an implementation not supporting XLLDP will discard any received LLDPDUs not containing a TTL TLV as the third TLV (i.e., will discard any received Extension Request or Extension LLDPDUs). The state machine further specifies that only implementations supporting XLLDP will process a Manifest TLV and subsequently transmit Extension Request LLDPDUs, and that only implementations supporting XLLDP will process Extension Request TLVs and subsequently transmit Extension LLDPDUs.

An optional End Of LLDPDU TLV marks the end of the LLDPDU.

1 The format for the LLDPDU is defined in 8.2. The TLV categories and the basic TLV format are defined in
2 8.3 and 8.4. The specific format and field contents for the Chassis ID TLV are defined in 8.5.2; for the
3 Port ID TLV, in 8.5.3; for the Time To Live TLV in 8.5.4; and for the End Of LLDPDU TLV, in 8.5.1.

4 6.2 LLDP operational modes

5 The information advertised by LLDP is transferred in a single direction. An LLDP agent can periodically
6 advertise information about the capabilities and current status of the system associated with its MSAP
7 identifier. The LLDP agent can also **collect** information about the capabilities and current status of the
8 system associated with a remote MSAP identifier. LLDP does not itself contain a mechanism for the LLDP
9 agent to solicit specific information from other LLDP agents beyond what is being advertised by those
10 LLDP agents, nor does it provide a specific means of confirming the receipt of information. Extended LLDP
11 allows an LLDP agent collecting information to request detailed information (via an Extension Request
12 LLDPDU) that is only advertised in a summary form (in a Manifest TLV), however this does not change the
13 inherent LLDP property that the choice of information provided is at the discretion of the advertising LLDP
14 agent.

15 NOTE—The LLDP protocol is designed to advertise information useful for discovering pertinent information about a
16 remote port and to populate topology MIBs. It is not intended to act as a configuration protocol for remote systems, nor
17 as a mechanism to signal control information between ports. During the operation of LLDP, it may be possible to
18 discover configuration inconsistencies between systems on the same IEEE 802 LAN. LLDP does not provide a
19 mechanism to resolve those inconsistencies. Rather, it provides a means to report discovered information to higher layer
20 management entities.

21 LLDP allows the advertising function and collecting function of the LLDP agent to be separately enabled,
22 making it possible to configure an implementation so the local LLDP agent can either advertise only or
23 collect only, or so the local LLDP agent can **advertise** and **collect** LLDP information.

24 6.3 LLDP information categories

25 The following basic management TLV set (this set is required in all LLDP implementations) is defined by
26 this standard and can be used to describe the system and/or to assist in the detection of configuration
27 inconsistencies associated with the MSAP identifier:

- 28 a) Port Description TLV.
- 29 b) System Name TLV.
- 30 c) System Description TLV.
- 31 d) System Capabilities TLV (indicates both the system's capabilities and its current primary network
32 function, such as end station, bridge, router).
- 33 e) Management Address TLV.

34 Table 8-1 includes a list of the optional TLVs in the basic management set and provides subclause references
35 for their specific definitions.

36 Organizationally Specific TLVs can be defined by either the professional organizations or the individual
37 vendors that are involved with the particular functionality being implemented within a system. The basic
38 format and procedures for defining Organizationally Specific TLVs are provided in 8.6.

39 6.4 TLV selection

40 Information for constructing the various TLVs to be sent is stored in the LLDP local system MIB. The
41 selection of which particular TLVs to send is under control of network management. Information received
42 from remote LLDP agents is stored in the LLDP remote systems MIB.

6.5 Transmission principles

A transmission cycle can be initiated either by the expiration of a transmit countdown timing counter or by a change in the value of one or more of the information elements (managed objects) associated with the local system. When a transmit cycle is initiated, the LLDP management entity extracts the managed objects from the LLDP local system MIB and formats this information into TLVs. The TLVs are inserted into an LLDPDU that is passed to the LLDP transmit module. When Extended LLDP is supported, some of the TLVs are inserted into one or more Extension LLDPDUs that are passed to the LLDP transmit module when requested by the receipt of a Extension Request LLDPDU. The LLDP transmit module prepends addressing parameters to the LLDPDU as defined in 8.2, 8.3, and 8.4. The LLDPDU and TLV formats are defined in Clause 8. The LLDP transmit state machine is described in 9.2.1 and 9.2.9.

NOTE 1—Because a transmission cycle can be initiated whenever a change occurs within the LLDP local system MIB, it is possible that a series of successive changes over a short period of time could trigger a number of LLDP frames to be sent, each reporting only a single change. LLDP utilizes a transmission delay timer that can be set by network management to limit the number of LLDP transmission cycles in a given time period.

NOTE 2—Under normal circumstances, the information in the receiving LLDP agent's remote systems MIB is refreshed periodically to avoid being discarded due to ageing. To prevent the receiving LLDP agent's remote systems MIB information being aged out because a refresh frame has been lost in transmission, the sending LLDP agent typically sets the TTL value so that several refresh cycles can occur before the received MIB information ages out.

LLDP can disclose information about a device and network that could be considered sensitive in certain environments. Implementers are encouraged to provide controls that take into account the link protection characteristics when including information in an LLDP message. Different information can be included if an LLDP message is being sent on the uncontrolled port, the controlled port, or a MACsec protected connectivity association.

6.6 Reception principles

The LLDP receive module uses the services of the LLC entity to recognize that the incoming MA_UNITDATA.indication contains the correct combination of destination address and MSDU header values to identify it as resulting from a received LLDPDU. The LLDPDU recognition procedures are defined in 7.4 and 9.2.8.7.

6.6.1 LLDPDU and TLV error handling

The LLDPDU is checked to verify that it contains the correct sequence of three mandatory TLVs at the beginning of the frame, as specified in 8.2, and then each optional TLV is validated in succession. LLDPDUs that contain detectable errors in the first three mandatory TLVs are discarded. Optional TLVs that contain detectable errors are discarded [see 9.2.8.7.2 c)]. TLVs that are not recognized, but that also contain no basic format errors, are assumed to be valid and are stored for possible later retrieval by network management (see 9.2.8.7.1 and 9.2.8.4). If the End Of LLDPDU TLV is present, any octets that follow it are discarded.

TLVs in which the information string length field contains a value that is greater than the sum of the lengths of the fields within the information string are not discarded.

NOTE—This approach allows later versions of a TLV to define additional fields at the end of the information string; implementations based on an earlier version of the TLV can therefore continue to process the fields that were defined in that version and ignore any new fields added at the end of the information string in later versions. Any information contained in such new fields is not made available to network management protocols via the standard MIB.

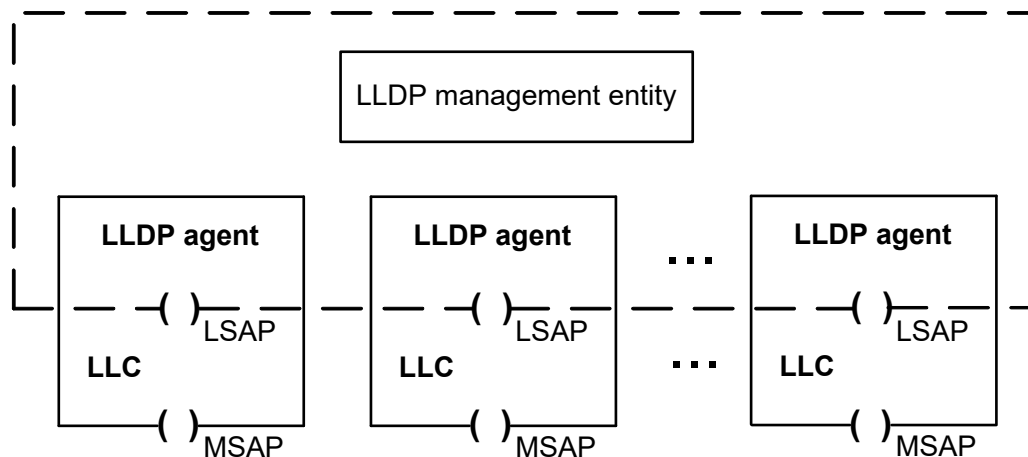
6.6.2 LLDP remote systems MIB update

The LLDP remote systems MIB is updated after all TLVs have been validated. LLDP remote system MIB update procedures are defined in 9.2.8.8, 9.2.7.7.5, and 9.2.8.4. The LLDP receive state machine is described in 9.2.1 and 9.2.10.

<<Editor's Note: IEEE Std 802.1ABdh-2021 deleted 9.2.7.7.4 and 9.2.7.7.5, however did not update the references in 6.6.2. The reference probably should be 9.2.7.4 and 9.2.7.5?>>

6.7 Systems with multiple LLDP Agents

Each LLDP agent advertises a single set of information in the various TLVs it encodes in each transmission cycle, and is associated with the MSAP that supports the LLC entity that the LLDP agent uses to transmit and receive. Each LLDP agent uses its LSAP directly, without the use of any additional multiplexing or addressing above the LSAP to support the use of that LSAP by multiple LLDP agents; and each LLC entity provides service to one and only one protocol entity at each of its LSAPs that it supports, using the service provided by a single MSAP. It follows that each LLDP agent makes use of a unique MSAP, and that the LLDP agent can be uniquely identified by the receiving agent using the MSAP's identifier and the MAC address that determines the LLDP agent's address scope, as specified in 6.1. A single LLDP management entity can support the operation of multiple LLDP agents within the same system. Figure 6-2 illustrates the relationship between the LLDP agents, LLC Entities, MSAPs, and the LLDP management entity.



LLDP - Link Layer Discovery Protocol
LSAP - Link service access point
MSAP - MAC service access point

Figure 6-2—Relationship between LLDP agents, LLC Entities, MSAPs, and the LLDP management entity

A given system can require the use of multiple LLDP agents because it naturally comprises multiple MSAPs, or because it needs to be able to transmit different information (different sets of TLVs) to different sets of peers. These sets of peers can be determined by the way that the MAC service is supported within the system (see IEEE Std 802.1AC for a description of the connectionless connectivity that characterizes an IEEE 802 LAN). A MAC Bridge (see IEEE Std 802.1Q), for example, attaches to multiple LANs, each providing the MAC service at a distinct MSAP. A different example is a station that uses port-based network access control (IEEE Std 802.1X) to provide both a Controlled Port and an Uncontrolled Port, i.e., two distinct MSAPs with potentially different connectivity, for transmission and reception to and from a single

1 LAN. Similarly, different destination addresses can be associated with different sets of potential recipients.
2 Table 7-1 identifies group MAC addresses that can be used for LLDPDU transmission, and 7.1 describes the
3 different transmission scopes associated with each address. LLDP can also be used in conjunction with
4 individual MAC addresses, and with other group MAC addresses. Distinct LLDP agents, and hence separate
5 and distinct protocol state machines, local and remote MIB tables, and MSAPs are maintained for each
6 LLDPDU destination address, even if the use of two or more MSAPs can result in transmission and
7 reception on the same LAN.

8 NOTE—For a given MAC address that is used as a destination address in received LLDPDUs, there is the possibility for
9 a station to receive LLDPDUs from multiple senders. All LLDPDUs that carry that destination MAC address are
10 received by a single LLDP agent, via a single MSAP; however, as the LLDPDU carries information that identifies the
11 source of the LLDPDU, information carried in LLDPDUs from different senders is able to be recorded independently in
12 the remote systems MIB.

13 Figure 6-3 illustrates the use of LLDP in a MAC Bridge, with an LLDP agent for each of the two ports
14 shown. Figure 6-4 shows the use of LLDP in an end system with port-based network access control
15 (IEEE Std 802.1X) supported by MACsec (IEEE Std 802.1AE); an LLDP agent is supported by the
16 Controlled Port and an additional LLDP agent may be supported by the Uncontrolled Port. Figure 6-5 shows
17 LLDP in a MAC Bridge that uses port-based network access control on both ports.

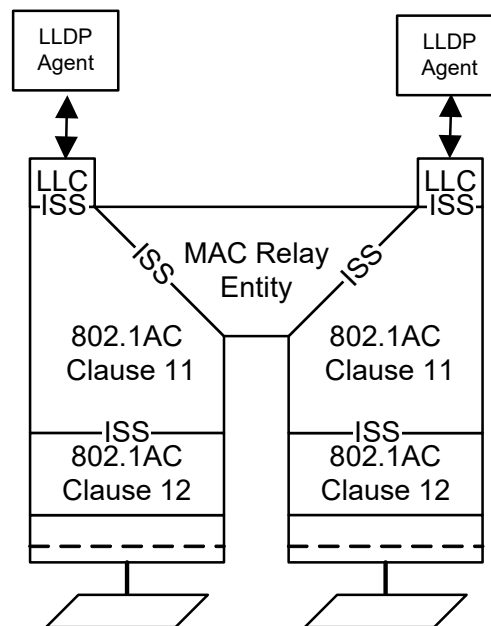


Figure 6-3—LLDP in a MAC Bridge

18 LLDP frames sent on an unprotected link may be modified by an attacker. If an implementation is going to
19 take action based upon a received LLDP attribute, it is advisable to take into account whether the message
20 was received on a protected link, and allow for the system to be configured such that only information
21 received in protected messages is acted upon.

22 The group addresses identified in Table 7-1 are taken from the set specified as reserved addresses by
23 bridging standards. Frames (including those conveying LLDPDUs) that use these destination addresses are
24 filtered or not by the various types of bridge components, as specified by Table 8-1 through Table 8-3 of
25 IEEE Std 802.1Q-2014. The choice of a particular address thus allows a given agent to limit the propagation

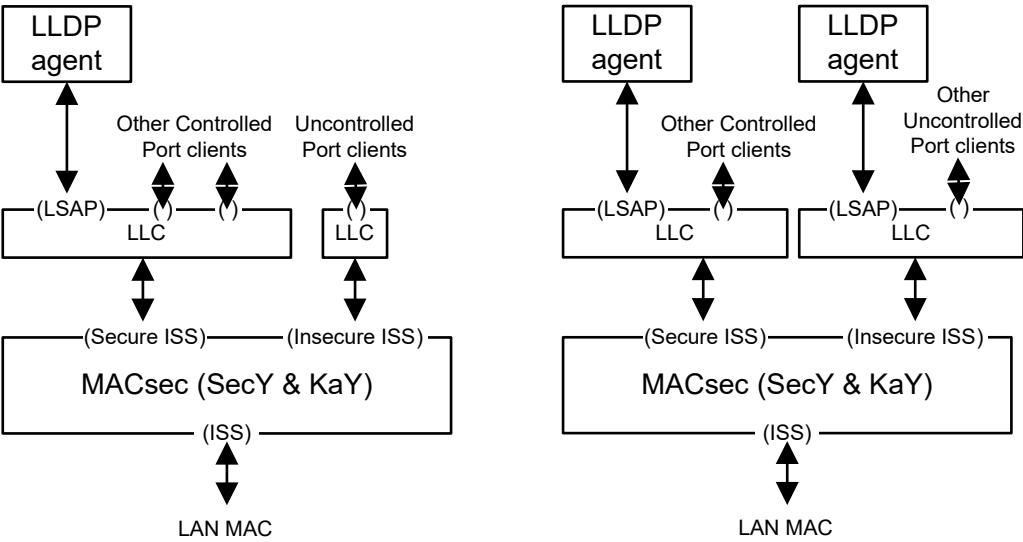


Figure 6-4—LLDP in an end system with port-based network access control

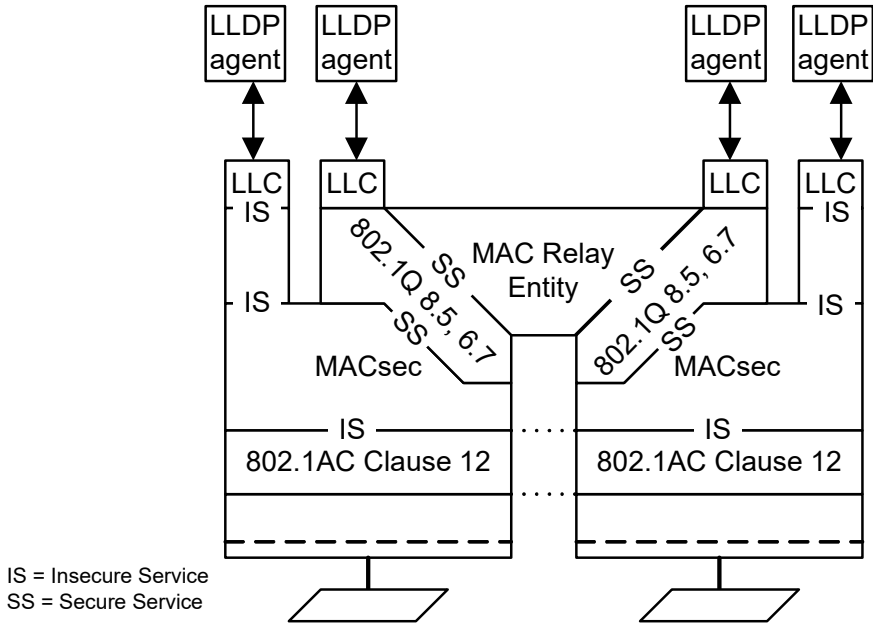


Figure 6-5—LLDP in a MAC Bridge that uses port-based network access control on both ports

1 of LLDPDUs to an individual LAN, or to allow them a wider scope. Figure 6-6 illustrates the various scopes
2 achievable with the three destination group MAC addresses identified in Table 7-1, their use is specified
3 further in 7.1.

4 More than one LLDP agent, each using a different address scope, can be instantiated for a given system port
5 by adding a simple shim that provides the necessary distinct MSAPs by multiplexing and demultiplexing
6 between those MSAPs and a common MSAP for the port, as illustrated in Figure 6-7. Each MSAP shown in

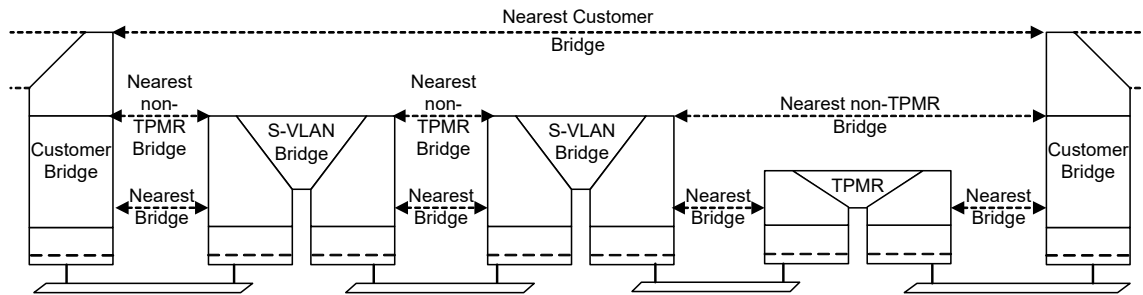


Figure 6-6—Scope of group MAC addresses

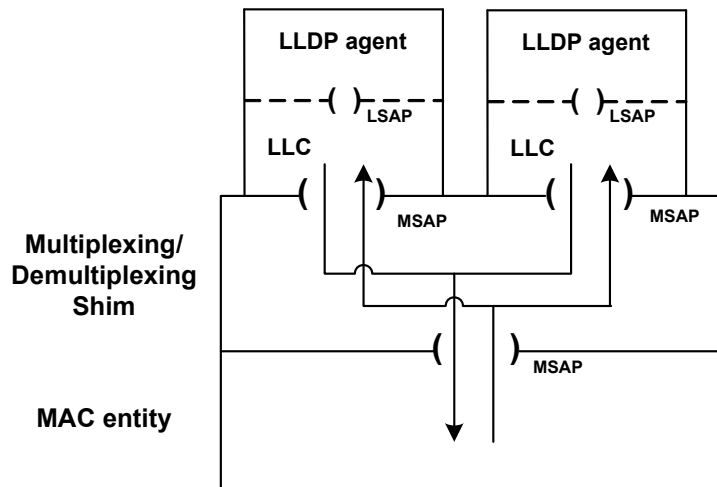


Figure 6-7—Multiplexing and demultiplexing using shims

1 Figure 6-7 is allowed to shared the same MSAP Identifier and individual MAC address. The LLDP agents
2 are differentiated by address scope used, i.e., by the MAC address used as a Destination Address in frames
3 containing a Normal LLDPDU.

4 6.8 LLDP and Link Aggregation

5 An LLDP agent can be configured on an IEEE 802.1AX™ Aggregated Port and/or on any number of the
6 physical ports belonging to the Aggregation. Some TLVs (e.g., the Power Via MDA TLV defined in 79.3.2
7 of IEEE Std 802.3-2012) only make sense when transmitted from an LLDP agent configured on a physical
8 port, and can cause incorrect operation if configured on an Aggregation. Other TLVs (e.g., the DCBX TLVs
9 defined in 38.4 of IEEE Std 802.1Q-2014) make sense only when transmitted from an Aggregated Port,
10 because they carry information applicable to the simulated single interface represented by the Aggregation.
11 Not all standards that define LLDP TLVs are careful to state whether each TLV is applicable to an
12 Aggregation, to a physical port, or to both. Only LLDP TLVs that are appropriate for transmission on a
13 given Port should be transmitted on that Port.

14 An LLDP agent configured on an Aggregated Port, or on one of the physical ports of an Aggregation,
15 transmits the Link Aggregation TLV, and processes received LLDPDUs, as specified in F.1 of
16 IEEE Std 802.1AX-2014. An LLDP agent configured on a physical port of an Aggregation uses the LLDP

1 Parser/Multiplexer as specified in 6.1.3 of IEEE Std 802.1AX-2014 for the transmission and reception of
2 LLDPDUs containing TLVs that are specific to that physical port. Since a port not running Link
3 Aggregation is equivalent to an Aggregation with a single physical port, any LLDP agent can transmit the
4 Link Aggregation TLV and process received LLDPDUs as specified in 802.1AX.

5 NOTE—The consideration of aggregated vs. Physical ports is similar to the distinction among destination MAC
6 addresses discussed in 7.1. In both cases, LLDP TLVs that carry information specific to a physical link should be used
7 only in the LLDP agent with the shortest reach.

8 6.9 Extended LLDP (XLLDP)

9 Extended LLDP allows an LLDP agent to advertise more TLVs than would fit within a single LLDPDU by
10 transmitting the additional TLVs in one or more Extension LLDPDUs. The selection of TLVs to be included
11 in LLDPDUs is controlled by management, but the mapping of TLVs to the initial Normal LLDPDU or to an
12 Extension LLDPDU is implementation dependent (10.2.2). When a transmission cycle is initiated, a unique
13 description of each Extension LLDPDU is encoded in a Manifest TLV, which is then included in the Normal
14 LLDPDU transmitted at the beginning of the cycle. A Normal LLDPDU containing a Manifest TLV is
15 referred to as a Manifest LLDPDU. Changes to any of the TLVs in an Extension LLDPDU changes the
16 description included in the Manifest TLV, and thus triggers a new transmission cycle.

17 An implementation not supporting XLLDP that receives a Manifest TLV treats it as an unrecognized TLV
18 and only stores the TLVs that were included in the Normal LLDPDU. An implementation supporting
19 XLLDP that receives a Manifest TLV then requests the transmission of any new or modified Extension
20 LLDPDUs by sending an Extension Request LLDPDU to the LLDP agent sourcing the Manifest LLDPDU.
21 One or more Extension LLDPDUs can be requested in a single Extension Request LLDPDU. A new
22 Extension Request LLDPDU can be transmitted when all Extension LLDPDUs previously requested have
23 been received. This allows the requesting LLDP agent to pace the rate of information transfer. A timer on the
24 request allows an Extension Request LLDPDU to be retried if it appears that either the request or a response
25 has been lost.

26 Extension Request LLDPDUs and Extension LLDPDUs are transmitted with an individual MAC address as
27 the destination address. The destination address in an Extension Request LLDPDU is taken from the
28 contents of the Return MAC Address field of the received Manifest TLV. The requested Extension
29 LLDPDUs are transmitted with a destination address taken from the contents the Return MAC Address field
30 of the Extension Request TLV.

1 **7. LLDPDU transmission, reception, and addressing**

2 This standard is intended to be compatible with all IEEE 802 MACs.

3 LLDP uses the service provided by the LLDP/LSAP and LLC to transmit and receive LLDPDUs. Each
4 LLDPDU is transmitted as a single MAC service request by an LLC entity that uses a single instance of the
5 MAC Service provided at an MSAP. Each incoming LLDP frame is received at the MSAP by the LLC entity
6 as a MAC service indication.

7 NOTE—For the purposes of this standard, the terms “LLC” and “LLC entity” include the service provided by the
8 operation of entities that support protocol discrimination using an EtherType, i.e., protocol discrimination based on the
9 Type interpretation of the Length/Type field as specified in IEEE Std 802.3, or the use of the SNAP encapsulation of
10 EtherTypes as described in IEEE Std 802. Hence, “LSAP” in the above description can refer to the use of
11 ISO/IEC 8802-2 [B14] LLC addresses, or an EtherType, as a means of higher layer protocol identification.

12 The parameters of each service request and service indication comprise

- 13 a) Destination address
- 14 b) Source address
- 15 c) EtherType
- 16 d) LLDPDU

17 The LLDP EtherType, used to identify the LLDP protocol, is prepended to the LLDPDU as shown in
18 Figure 7-1 to form the MSDU of the corresponding MAC service request.

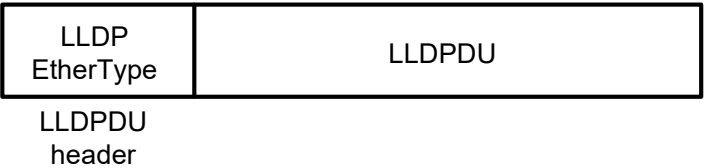


Figure 7-1—MSDU format

19 The values of the parameters used by LLDP, and their encoding by the LLC entity that supports the LLDP
20 LSAP, are specified in the following subclauses.

21 **7.1 Destination address**

22 **7.1.1 Destination address for Normal LLDPDUs**

23 The MAC address used as the destination address in Normal LLDPDUs is referred to as the Scope MAC
24 Address because it determines the limits of propagation of the LLDPDU within a network. A set of standard
25 group MAC addresses that can be used by LLDP as the Scope MAC Address is specified in Table 7-1. These
26 addresses are in the range of IEEE Std 802.1Q C-VLAN and MAC Bridge component Reserved addresses
27 (see Table 8-1 in IEEE Std 802.1Q-2014), the S-VLAN component Reserved addresses (see Table 8-2 in
28 IEEE Std 802.1Q-2014), and the TPMR component Reserved addresses (see Table 8-3 in IEEE Std
29 802.1Q-2014). The choice of address used determines the scope of propagation of LLDPDUs within a
30 bridged LAN, as follows:

- 31 a) The *nearest bridge* group MAC address is an address that no conformant Two-Port MAC Relay
32 (TPMR) component, S-VLAN component, C-VLAN component, or MAC Bridge component can
33 forward. LLDPDUs transmitted using this destination address can therefore travel no further than
34 those stations that can be reached via a single individual LAN from the originating station.

1 LLDPDUs received on this address therefore represent information about stations that are attached
2 to the same individual LAN segment as the recipient station.

3 NOTE 1—This address was selected in order to make it possible to transmit an LLDP frame containing information
4 specific to a single individual LAN, and for that information not to be propagated further than the extent of that
5 individual LAN, to avoid the meaning of that information being misinterpreted. For example, a TLV containing the
6 auto-negotiation status of an IEEE 802.3™ LAN would be open to misinterpretation if it were to be propagated via a
7 Bridge onto a second IEEE 802.3 LAN, and would be meaningless if propagated onto a LAN based on a different MAC
8 technology. This is the “LLDP Multicast address” that was used in IEEE Std 802.1AB-2005. This address was
9 erroneously omitted from the table of S-VLAN component Reserved addresses added to IEEE Std 802.1Q by
10 IEEE Std 802.1ad-2005, but has been added to that table by IEEE Std 802.1aj™-2009 [B1] and is included in
11 subsequent revisions of IEEE Std 802.1Q.

12 b) The **nearest non-TPMR bridge** group MAC address is an address that no conformant C-VLAN
13 component, S-VLAN component, or MAC Bridge can forward. However, this address is relayed by
14 TPMR components. Therefore, LLDPDUs received on this address represent information about
15 stations that are not separated from the recipient station by any intervening C-VLAN component,
16 S-VLAN component, or MAC Bridge. There may, however, be one or more TPMR components in
17 the path between the originating and receiving stations.

18 NOTE 2—This address was selected in order to make it possible to communicate information between adjacent bridges
19 and for that communication to be transparent to the presence or absence of TPMRs in the transmission path. This address
20 is primarily intended for use within provider bridged networks.

21 c) The **nearest Customer Bridge** group MAC address is an address that no conformant C-VLAN
22 component or MAC Bridge forwards. However, this address is relayed by TPMR components and
23 S-VLAN components. Therefore, LLDPDUs received on this address represent information about
24 stations that are not separated from the recipient station by any intervening C-VLAN components
25 (e.g., a C-VLAN Bridge or Provider Edge Bridge). There may, however, be TPMR components
26 and/or S-VLAN components in the path between the originating and receiving stations.

27 NOTE 3—This address was selected in order to make it possible to communicate information between adjacent
28 Customer Bridges and for that communication to be transparent to the presence or absence of TPMRs or S-VLAN
29 components in the communication path. The scope of this address is the same as that of a customer-to-customer
30 MACSec connection.

Table 7-1—Group MAC addresses used by LLDP

Name	Value	Purpose
<i>Nearest bridge</i>	01-80-C2-00-00-0E	Propagation constrained to a single physical link; stopped by all types of bridge
<i>Nearest non-TPMR bridge</i>	01-80-C2-00-00-03	Propagation constrained by all bridges other than TPMRs; intended for use within provider bridged networks
<i>Nearest Customer Bridge</i>	01-80-C2-00-00-00	Propagation constrained by customer bridges; this gives the same coverage as a customer-customer MACSec connection

31 NOTE 4—The LSAP or EtherType field is necessary to distinguish between different protocols conveyed in frames with
32 the same group or individual destination address. Prior to revision of this standard to allow multiple address scopes, the
33 destination address 01-80-C2-00-00-00 was only explicitly specified for Spanning Tree Protocol BPDUs. It is therefore
34 possible that some implementations recognize only a frame’s destination address before applying priority handling
35 resources intended to guard against BPDU loss. Since LLDP rate limits LLDPDU transmission, the additional
36 consumption of these resources by LLDPDUs is unlikely to pose a problem for robust implementations.

1 It is assumed in the foregoing definitions of the scope of these group MAC addresses that an end station
2 attached to an individual LAN is a potential recipient of any of these addresses, and therefore such end
3 stations fall within the scope of these addresses as well as the identified bridge components.

4 In addition to determining the scope of transmission, the support of more than one destination MAC address
5 allows different sets of TLVs to be supported over the different transmission scopes.

6 In addition to the prescribed support for standard group MAC addresses shown in Table 7-1,
7 implementations of LLDP may support the following as the Scope MAC Address:

- 8 d) Any group MAC address.
- 9 e) Any individual MAC address.

10 Support for the use of each of these destination addresses, for both transmission and reception of Normal
11 LLDPDUs, is either mandatory, recommended, permitted, or not permitted, according to the type of system
12 in which LLDP is implemented, as shown in Table 7-2.

Table 7-2—Support for the Scope MAC Address in different systems

Address	C-VLAN Bridge	S-VLAN Bridge	TPMR Bridge	End station
<i>Nearest bridge</i>	Mandatory	Mandatory	Mandatory	Mandatory
<i>Nearest non-TPMR bridge</i>	Mandatory	Mandatory	Not permitted	Recommended
<i>Nearest Customer Bridge</i>	Mandatory	Not permitted	Not permitted	Recommended
<i>Any other group MAC address</i>	Permitted	Permitted	Permitted	Permitted
<i>Any individual MAC address</i>	Permitted	Permitted	Permitted	Permitted

13 NOTE 5 —Where the implementation supports the use of individual MAC addresses, there is a need for some means
14 whereby the sender can ascertain what address to use, and what scope is associated with that address; how this is
15 achieved is outside the scope of this standard.

16 NOTE 6—The option of using an individual MAC addresses supports the use of LLDP in IEEE Std 802.11™ [B2] and
17 other wireless LANs, where it is desirable to target specific TLV content at specific logical Ports. In particular, the use of
18 an individual MAC address allows an access point to direct LLDP frames at an individual station with which it is
19 associated.

20 NOTE 7—If an individual MAC address is specified as the Scope MAC Address, it is used as the destination for
21 transmitted LLDPDUs and also used to validate received LLDPDUs. The primary purpose of using an individual
22 address as the Scope MAC Address is that an LLDP agent configured as “transmit only” sends Normal LLDPDUs to a
23 specific receiving LLDP agent. It is also allowable for an LLDP agent configured as “receive only” to use an individual
24 address (typically the individual MAC address of the LLDP agent’s MSAP) so that only information from an LLDP
25 agent configured as “transmit only” using this same address is collected. Using an individual MAC address as the
26 destination address for an LLDP agent configured to “transmit and receive” is allowed, although it would not typically
27 be useful.

28 NOTE 8—The destination MAC address used by a given LLDP agent defines only the scope of transmission and the
29 intended recipient(s) of the LLDPDUs; it plays no part in protocol identification. In particular, the group MAC addresses
30 identified in Table 7-1 are not used exclusively by LLDP; other protocols that require to use a similar transmission scope
31 are free to use the same addresses.

32 7.1.2 Destination address for Extension Request and Extension LLDPDUs

33 The destination address in an Extension Request LLDPDU or an Extension LLDPDU is an individual
34 address. Extension Request LLDPDUs are sent in response to the receipt of a Manifest LLDPDU, and use
35 the contents of the Return MAC Address field of the Manifest TLV as the destination address of an

1 Extension Request LLDPDU. Likewise, Extension LLDPDUs are sent in response to the receipt of an
2 Extension Request LLDPDU, and use the contents of the Return MAC Address field of the Extension
3 Request TLV as the destination address of the Extension LLDPDU.

4 **7.2 Source address**

5 The source address shall be the individual MAC address of the MSAP supporting the LLDP agent.

6 **7.3 EtherType use and encoding**

7 The EtherType used to identify the LLDP protocol shall be the LLDP EtherType specified in Table 7-3.

Table 7-3—LLDP EtherType

Name	Value
LLDP EtherType	88-CC

8 Where the LLC entity uses an MSAP that is supported by a specific media access control method (for
9 example, IEEE Std 802.3) or a media access control independent entity (for example, IEEE Std 802.1AE)
10 that directly supports encoding of EtherTypes, the LLC entity shall encode the LLDP EtherType as the two
11 octet LLDPDU header in the MSDU of the corresponding MAC service request.

12 Where the LLC entity uses an MSAP that is supported by a specific media access control method that does
13 not directly support Ethertype encoding (for example, IEEE Std 802.11 [B2]), the encoding shall be as
14 specified in IEEE Std 802.1AC.

15 NOTE—Annex C provides example LLDP transmission frame formats for both direct-encoded and SNAP-encoded
16 LLDP EtherType encoding methods.

17 **7.4 LLDPDU reception**

18 The LLDPDU shall be delivered to the LLDP receive module if, and only if

- 19 a) The destination MAC address is equal to the individual MAC address of the MSAP supporting the
20 LLDP agent, or the Scope MAC Address associated with the corresponding LLDP agent; and

21 NOTE—The destination MAC address can be one of the addresses in Table 7-1, or any other valid MAC address—see
22 7.1.

- 23 b) The EtherType is equal to the value shown in Table 7-3.

1 8. LLDPDU and TLV formats

2 8.1 LLDPDU bit and octet ordering conventions

3 All LLDPDUs shall contain an integral number of octets. The octets in an LLDPDU are numbered starting
4 from 1 and increasing in the order they are put into the LLDPDU. The bits in an octet are numbered from 1
5 to 8, where bit 1 is the low-order bit.

6 When consecutive bits within an octet are used to represent a binary number, the highest bit number has the
7 most significant value. When consecutive octets are used to represent a binary number, the lower octet
8 number has the most significant value. All TLVs respect these bit and octet ordering conventions.

9 When the encoding of a field or a number of fields is represented using a diagram

- 10 a) Octets are shown with the lowest numbered octet nearest the left of the page, the octet numbering
11 increasing from left to right.
12 b) Within an octet, bits are shown with bit 8 to the left and bit 1 to the right.

13 8.2 LLDPDU format

14 The LLDPDU shall contain the following ordered sequence of three mandatory TLVs followed by zero or
15 more optional TLVs as shown in Figure 8-1. An End of LLDPDU TLV may be present as the last TLV in the
16 LLDPDU.

- 17 a) Three mandatory TLVs shall be included at the beginning of each LLDPDU and shall be in the order
18 shown.
19 1) Chassis ID TLV
20 2) Port ID TLV
21 3) Time To Live TLV or, if XLLDP is supported, Extension Request TLV or Extension Identifier
22 TLV
23 b) Optional TLVs as selected by network management (may be inserted in any order).

24 NOTE 1—"Optional" in the sense that they are not required for LLDP operation; however, their presence could be
25 required by other system elements that use LLDP.

- 26 c) If the End Of LLDPDU TLV is present, it shall be the last TLV in the LLDPDU.

1

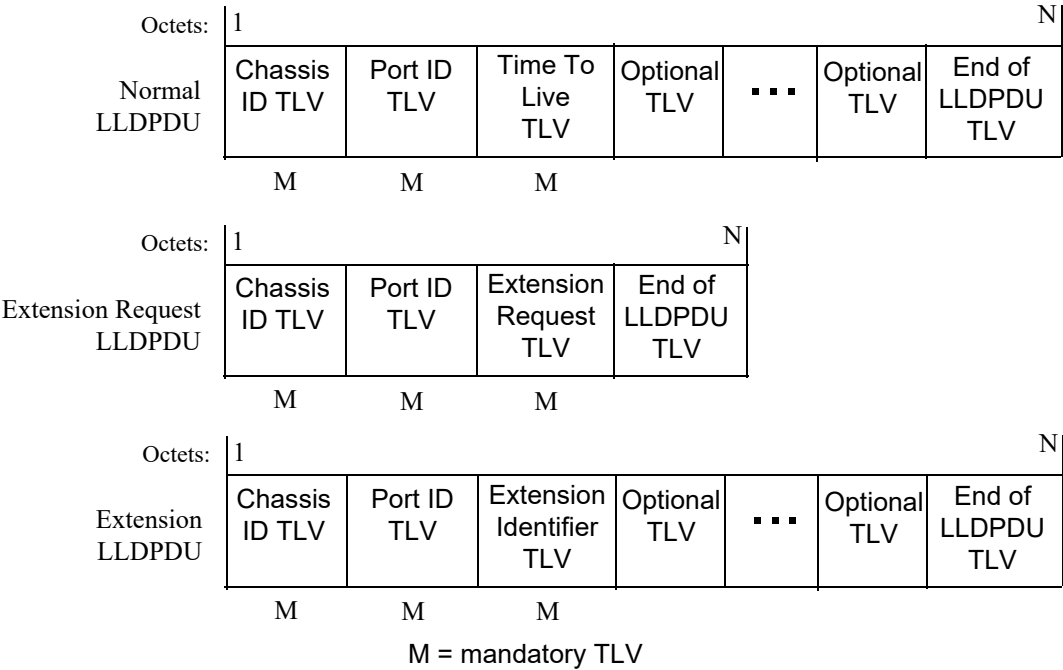


Figure 8-1—LLDPDU formats

2 The maximum length of the LLDPDU is less than or equal to the maximum information field length allowed
3 by the particular transmission rate and protocol. In IEEE 802.3 MACs, for example, the maximum LLDPDU
4 length is the maximum data field length for the basic, untagged MAC frame (1500 octets).

5 NOTE 2—Implementations can restrict the maximum length of the LLDPDU to a value smaller than what would be
6 allowed by the underlying MAC technology. For example, an end station or bridge implementing Time-Sensitive
7 Networking (TSN) features could restrict the maximum length to limit delay variation when transmitting frames in time
8 sensitive streams.

9 NOTE 3—There is no defined minimum length of an LLDPDU, other than that implied by the requirement that
10 conformant implementations support the mandatory TLVs specified in Table 8-1.

11 8.3 TLV categories

12 The general format of LLDP TLVs is defined in 8.4. The TLVs are grouped into three categories as follows:

- 13 a) A set of TLVs that are considered to be basic to the management of network stations. Support of the
14 basic management set is required capability of all LLDP implementations. Each TLV in this
15 category is identified by a unique TLV type value that indicates the particular kind of information
16 contained in the TLV. The specific definitions and usage rules for the basic management set TLVs
17 are defined in 8.5.
- 18 b) A set of TLVs that support Extended LLDP (XLLDP) operation. Support of the extended LLDP set
19 is a required capability of all LLDP implementations supporting XLLDP. Each TLV in this category
20 is identified by a unique TLV type value that indicates the particular kind of information contained
21 in the TLV. The specific definitions and usage rules for the extended LLDP set TLVs are defined in
22 8.5a.
- 23 c) Organizationally specific extension sets of TLVs that are defined by standards groups such as
24 IEEE 802.1 and IEEE 802.3 and others to enhance management of network stations that are

- operating with particular media and/or protocols. The format of Organizationally Specific TLVs, along with field definitions and usage rules common to all Organizationally Specific TLVs, are defined in 8.6.
- 1) TLVs in this category are identified by a common TLV type value that indicates the TLV as belonging to the set of Organizationally Specific TLVs.
 - 2) Each organization is identified by its organizationally unique identifier (OUI), consisting of either an Organizationally Unique Identifier (OUI) or a Company ID (CID)⁶.
 - 3) Organizationally Specific TLV subtype values indicate the kind of information contained in the TLV.

8.4 Basic TLV format

Figure 8-2 shows the basic TLV format.

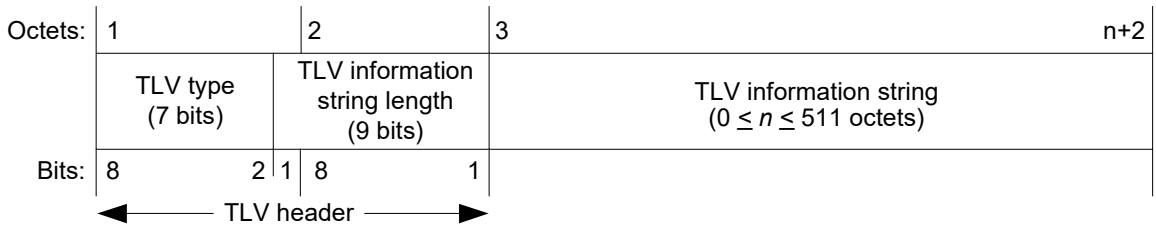


Figure 8-2—Basic TLV format

The TLV type field occupies the seven most significant bits of the first octet of the TLV format. The least significant bit in the first octet of the TLV format is the most significant bit of the TLV information string length field.

8.4.1 TLV type

The TLV type field is seven bits long and identifies the specific TLV. Two classes of TLVs are defined:

- a) Mandatory TLVs that shall be included in all LLDPDUs.
- b) Optional TLVs that may be included in LLDPDUs.

Table 8-1 lists the currently defined TLVs, their identifying TLV type values, and whether they are mandatory or optional for inclusion in any particular LLDPDU.

8.4.2 TLV information string length

The TLV information string length field shall contain the length of the information string, in octets.

8.4.3 TLV information string

The information string

- a) May be fixed or variable length.
- b) May include one or more information fields with associated subtype identifiers and field length designators as in, for example, the Management Address TLV (see 8.5.9).

⁶ Organizationally Unique Identifier and Company ID assignments are administered by the IEEE Registration Authority, and Organizationally Unique Identifier assignments are included in MAC Address-Large (MA-L) assignments; see <http://standards.ieee.org/regauth>.

Table 8-1—TLV type values

TLV type ^a	TLV name	Usage in LLDPDU	Reference
0	End Of LLDPDU	Optional	8.5.1
1	Chassis ID	Mandatory	8.5.2
2	Port ID	Mandatory	8.5.3
3	Time To Live	Mandatory	8.5.4
4	Port Description	Optional	8.5.5
5	System Name	Optional	8.5.6
6	System Description	Optional	8.5.7
7	System Capabilities	Optional	8.5.8
8	Management Address	Optional	8.5.9
9	Manifest	Optional	8.5a.1
10	Extension Request	Optional	8.5a.2
11	Extension Identifier	Optional	8.5a.3
12–126	Reserved for future standardization	—	—
127	Organizationally Specific TLVs	Optional	—

^aTLVs with type values 0–8 are members of the basic management set. TLVs with type values 9–11 are members of the extended LLDP set.

- 1 c) May contain either binary or alpha-numeric information that is instance specific for the particular
- 2 TLV type and/or subtype.
- 3 1) Bit 1 in binary bit maps shall be the least significant bit in the field.
- 4 2) The first octet of an alpha-numeric field shall be the most significant octet.
- 5 3) Alpha-numeric information shall be encoded in UTF-8 [IETF RFC 3629].

1 **8.5 Basic management TLV set formats and definitions**

2 **8.5.1 End Of LLDPDU TLV**

3 The End Of LLDPDU TLV is a 2-octet, all-zero TLV that is used to mark the end of the TLV sequence in
4 LLDPDUs. The format for this TLV is shown in Figure 8-3.

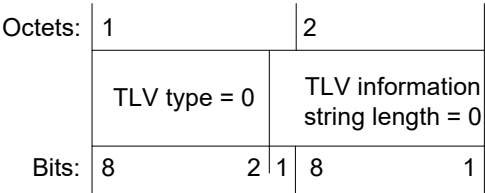


Figure 8-3—End Of LLDPDU TLV format

5 NOTE—Some IEEE 802 MACs require the data field in a frame to contain a minimum number of octets. For example,
6 the IEEE 802.3 MAC adds pad octets to complete a minimum length data field if the user's data is less than the
7 minimum required length. Since pad octets are unspecified, an End Of LLDPDU TLV can be used to prevent non-zero
8 pad octets from being interpreted by the receiving LLDP agent as another TLV.

9 **8.5.2 Chassis ID TLV**

10 The Chassis ID TLV is a mandatory TLV that identifies the chassis containing the IEEE 802 LAN station
11 associated with the **MSAP Identifier associated with the LLDP agent containing the LLDP local system**
12 **MIB that is the source of the information in TLVs**. There are several ways in which a chassis may be
13 identified and a chassis ID subtype is used to indicate the type of component being referenced by the chassis
14 ID field. Each LLDPDU shall contain one, and only one, Chassis ID TLV and the chassis ID field value shall
15 remain constant for all LLDPDUs while the MAC status parameter for the Port remains operational.

16 The Chassis ID TLV shall be the first TLV in the LLDPDU. Its format is shown in Figure 8-4.

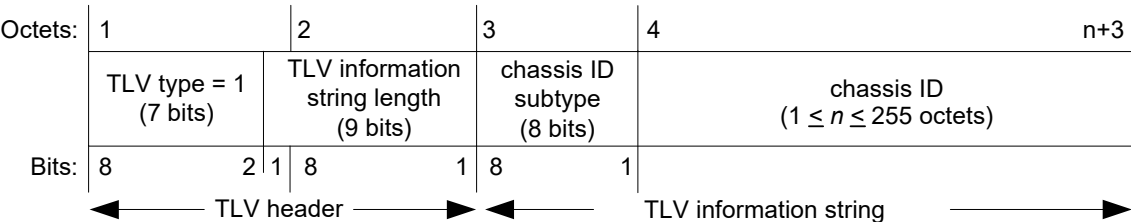


Figure 8-4—Chassis ID TLV format

17 **8.5.2.1 TLV information string length**

18 The TLV information string length field shall indicate the exact length, in octets, of the (chassis ID subtype
19 + chassis ID) fields.

20 **8.5.2.2 chassis ID subtype**

21 The chassis ID subtype field shall contain an integer value indicating the basis for the chassis ID entity that
22 is listed in the chassis ID field. The defined chassis ID subtypes and their preferred use order are listed in
23 Table 8-2.

Table 8-2—chassis ID subtype enumeration

ID subtype	ID basis	Reference
0	Reserved	—
1	Chassis component	EntPhysicalAlias when entPhysClass has a value of ‘chassis(3)’ (IETF RFC 6933)
2	Interface alias	IfAlias (IETF RFC 2863)
3	Port component	EntPhysicalAlias when entPhysicalClass has a value ‘port(10)’ or ‘backplane(4)’ (IETF RFC 6933)
4	MAC address	MAC address (IEEE Std 802)
5	Network address	networkAddress ^a
6	Interface name	ifName (IETF RFC 2863)
7	Locally assigned	local ^b
8–255	Reserved	—

^anetworkAddress is an octet string that identifies a particular network address family and an associated network address that are encoded in network octet order. An IP address, for example, would be encoded with the first octet containing the IANA Address Family Numbers enumeration value for the specific address type and octets 2 through *n* containing the address value (for example, the encoding for C0-00-02-0A would indicate the IPv4 address 192.0.2.10).

^blocal is an alpha-numeric string and is locally assigned.

1 8.5.2.3 chassis ID

2 The chassis ID field shall contain an octet string indicating the specific identifier for the particular chassis in
3 this system. Because chassis ID and port ID values are concatenated to form the local MSAP identifier, the
4 value chosen from Table 8-2 for the chassis ID shall be non-null.

5 8.5.2.4 Chassis ID TLV usage rules

6 An LLDPDU shall contain exactly one Chassis ID TLV.

7 8.5.3 Port ID TLV

8 The Port ID TLV is a mandatory TLV that identifies the port component of the MSAP identifier associated
9 with the MSAP Identifier associated with the LLDP agent containing the LLDP local system MIB that is the
10 source of the information in TLVs. As with the chassis, there are several ways in which a port may be
11 identified. A port ID subtype is used to indicate how the port is being referenced in the port ID field. Each
12 LLDPDU shall contain one, and only one, Port ID TLV. The port ID value shall remain constant for all
13 LLDPDUs while the transmitting port remains operational.

14 The chosen port shall be identified in an unambiguous manner (for example, backplane number,
15 management entity, MAC address).

16 The Port ID TLV shall be the second TLV in the LLDPDU. Its format is shown in Figure 8-5.

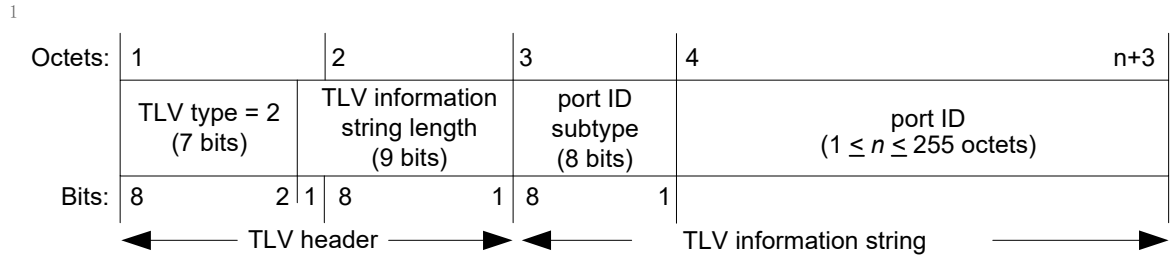


Figure 8-5—Port ID TLV format

2 8.5.3.1 TLV information string length

3 The TLV information string length field shall indicate the length, in octets, of the (port ID subtype + port ID)
4 fields.

5 8.5.3.2 port ID subtype

6 The port ID subtype field shall contain an integer value indicating the basis for the identifier that is listed in
7 the port ID field. The defined port ID subtypes and their preferred use order are listed in Table 8-3.

Table 8-3—port ID subtype enumeration

ID subtype	ID basis	References
0	Reserved	—
1	Interface alias	ifAlias (IETF RFC 2863)
2	Port component	entPhysicalAlias when entPhysicalClass has a value ‘port(10)’ or ‘backplane(4)’ (IETF RFC 6933)
3	MAC address	MAC address (IEEE Std 802)
4	Network address	networkAddress ^a
5	Interface name	ifName (IETF RFC 2863)
6	Agent circuit ID	agent circuit ID (IETF RFC 3046)
7	Locally assigned	local ^b
8–255	Reserved	—

^anetworkAddress is an octet string that identifies a particular network address family and an associated network address that are encoded in network octet order. An IP address, for example, would be encoded with the first octet containing the IANA Address Family Numbers enumeration value for the specific address type and octets 2 through n containing the address value (for example, the encoding for C0-00-02-0A would indicate the IP version 4 address 192.0.2.10).

^blocal is an alpha-numeric string and is locally assigned.

8 8.5.3.3 port ID

9 The port ID field is an alpha-numeric string that contains the specific identifier for the port from which this
10 LLDPDU was transmitted. Because chassis ID and port ID values are concatenated to form the local MSAP
11 identifier, the value chosen from Table 8-3 for the port ID shall be non-null.

12 NOTE—The port ID can contain alphanumeric data or binary data, depending upon the value of the port ID subtype.

8.5.3.4 Port ID TLV usage rules

An LLDPDU shall contain exactly one Port ID TLV.

8.5.4 Time To Live TLV

The Time To Live TLV indicates the number of seconds that the recipient LLDP agent is to regard the information associated with this MSAP identifier to be valid.

- When the TTL field is non-zero the receiving LLDP agent is notified to completely replace all information associated with this MSAP identifier with the information in the received LLDPDU.
- When the TTL field is set to zero, the receiving LLDP agent is notified to delete all system information associated with the LLDP agent/port. This TLV may be used, for example, to signal that the sending port has initiated a port shutdown procedure.

The Time To Live TLV is mandatory for all Normal LLDPDUs, and shall be the third TLV in the LLDPDU. Its format is shown in Figure 8-6.

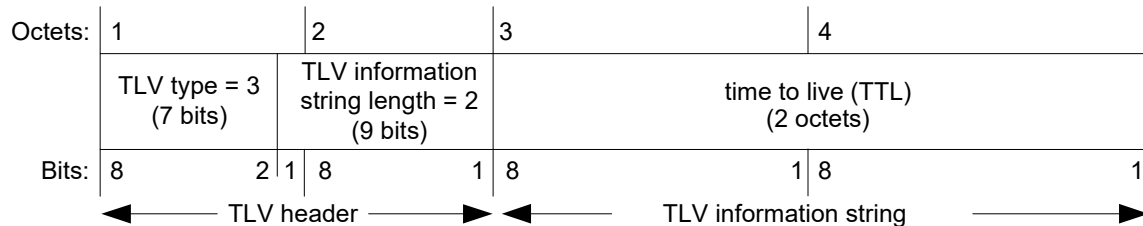


Figure 8-6—Time To Live TLV format

8.5.4.1 time to live (TTL)

The TTL field shall contain an integer value in the range $0 \leq t \leq 65535$ seconds and is set to the computed value of txTTL at the time the LLDPDU is constructed (see 9.2.5.22).

8.5.4.2 Time To Live TLV usage rules

A Normal LLDPDU shall contain exactly one Time To Live TLV as the third TLV, and shall not contain more than one Time To Live TLV. A Time To Live TLV shall not be contained in an Extension LLDPDU or an Extension Request LLDPDU.

8.5.5 Port Description TLV

The Port Description TLV allows network management to advertise the IEEE 802 LAN station's port description. The format for this TLV is shown in Figure 8-7.

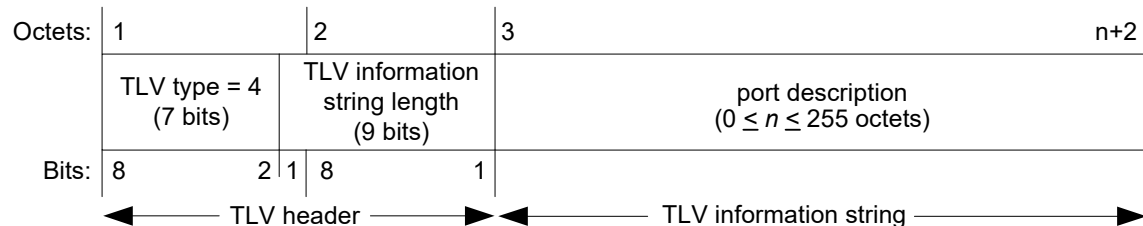


Figure 8-7—Port Description TLV format

1 **8.5.5.1 TLV information string length**

2 The TLV information string length field shall contain the exact length, in octets, of the port description field.

3 **8.5.5.2 port description**

4 The port description field shall contain an alpha-numeric string that indicates the port’s description. If
5 IETF RFC 2863 is implemented, the ifDescr object should be used for this field.

6 **8.5.5.3 Port Description TLV usage rules**

7 An LLDPDU should not contain more than one Port Description TLV.

8 **8.5.6 System Name TLV**

9 The System Name TLV allows network management to advertise the system’s assigned name. The format
10 for this TLV is shown in Figure 8-8.

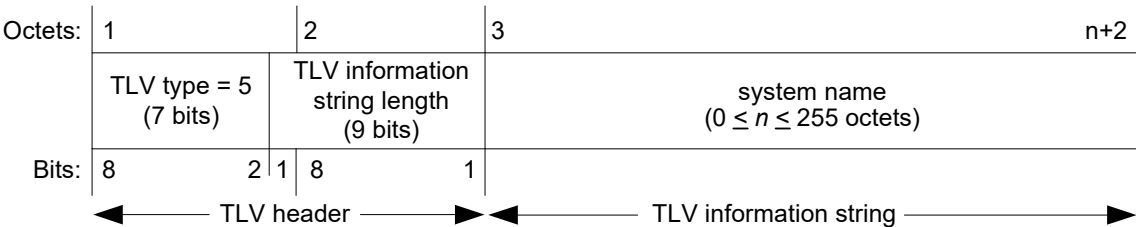


Figure 8-8—System Name TLV format

11 **8.5.6.1 TLV information string length**

12 The TLV information string length field shall contain the exact length, in octets, of the system name.

13 **8.5.6.2 system name**

14 The system name field shall contain an alpha-numeric string that indicates the system’s administratively
15 assigned name. The system name should be the system’s fully qualified domain name. If implementations
16 support IETF RFC 3418, the sysName object should be used for this field.

17 **8.5.6.3 System Name TLV usage rules**

18 An LLDPDU shall not contain more than one System Name TLV.

8.5.7 System Description TLV

The System Description TLV allows network management to advertise the system's description. The format for this TLV is shown in Figure 8-9.

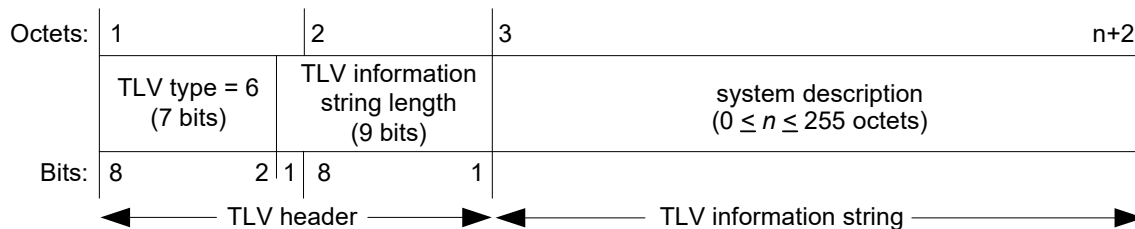


Figure 8-9—System Description TLV format

8.5.7.1 TLV information string length

The TLV information string length field shall indicate the exact length, in octets, of the system description.

8.5.7.2 system description

The system description field shall contain an alpha-numeric string that is the textual description of the network entity. The system description should include the full name and version identification of the system's hardware type, software operating system, and networking software. If implementations support IETF RFC 3418, the sysDescr object should be used for this field.

8.5.7.3 System Description TLV usage rules

An LLDPDU shall not contain more than one System Description TLV.

8.5.8 System Capabilities TLV

The System Capabilities TLV is an optional TLV that identifies the primary function(s) of the system and whether or not these primary functions are enabled. Figure 8-10 shows the format of this TLV.

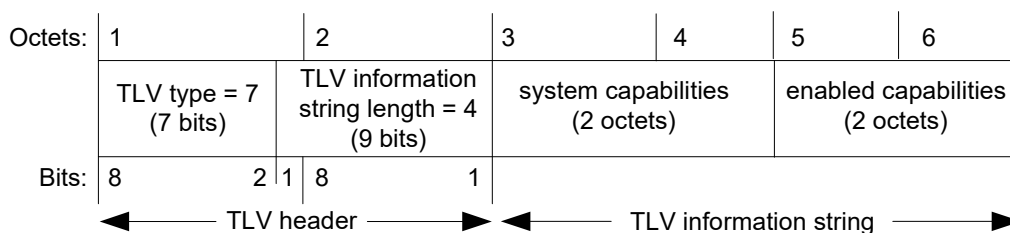


Figure 8-10—System Capabilities TLV format

8.5.8.1 system capabilities

The system capabilities field shall contain a bit-map of the capabilities that define the primary function(s) of the system. The bit positions for each function and the associated MIB or standard that are likely, but not guaranteed, to be supported are listed in Table 8-4. A binary one in the associated bit indicates the existence of that capability. Individual systems may indicate more than one implemented functional capability (for example, both a bridge and router capability).

Table 8-4—System capabilities

Bit	Capability	Reference
1	Other	—
2	Repeater	IETF RFC 2108 [B7]
3	MAC Bridge component	IEEE Std 802.1Q
4	802.11 Access Point (AP)	IEEE Std 802.11 MIB
5	Router	IETF RFC 1812 [B6]
6	Telephone	IETF RFC 4293 [B10]
7	DOCSIS cable device	IETF RFC 4639 and IETF RFC 4546 [B12]
8	Station Only ^a	IETF RFC 4293 [B10]
9	C-VLAN component	IEEE Std 802.1Q
10	S-VLAN component	IEEE Std 802.1Q
11	Two-port MAC Relay component	IEEE Std 802.1Q
12–16	reserved	—

^aThe Station Only capability is intended for devices that implement only an end station capability, and for which none of the other capabilities in the table apply. Bit 8 should therefore not be set in conjunction with any other bits.

1 8.5.8.2 enabled capabilities

2 The enabled capabilities field shall contain a bit map of the primary functions listed in Table 8-4. A binary
3 one in a bit position indicates that the function associated with that bit is currently enabled.

4 8.5.8.3 System Capabilities TLV usage rules

5 An LLDPDU shall not contain more than one System Capabilities TLV.

6 If the system capabilities field does not indicate the existence of a capability that the enabled capabilities
7 field indicates is enabled, the TLV is interpreted as containing an error and shall be discarded.

8 8.5.9 Management Address TLV

9 The Management Address TLV identifies an address associated with the local LLDP agent that may be used
10 to reach higher layer entities to assist discovery by network management. The TLV also provides room for
11 the inclusion of both the system interface number and an object identifier (OID) that are associated with this
12 management address, if either or both are known. Figure 8-11 shows the Management Address TLV format.

1

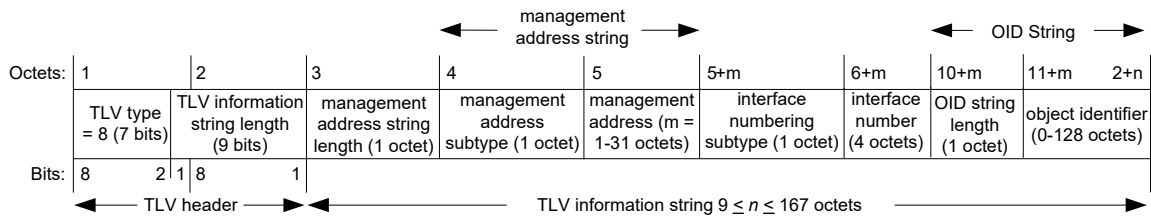


Figure 8-11—Management Address TLV format

8.5.9.1 TLV information string length

The TLV information string length field shall contain the length, in octets, of all the fields in the TLV information string.

8.5.9.2 management address string length

The management address string length field shall contain the length, in octets, of the (management address subtype + management address) fields.

8.5.9.3 management address subtype

The management address subtype field shall contain an integer value indicating the type of address that is listed in the management address field. Enumeration for this field is contained in the ianaAddressFamilyNumbers module of the IETF RFC 3232 on-line database that is accessible through a web page (<http://www.iana.org>). The management address subtype is contained in the first octet of the management address string.

NOTE—As a consequence of the limitation on the size of the management address field (a maximum of 31 octets), some options for the management address subtype (such as long DNS names) may be impractical.

8.5.9.4 management address

The management address field shall contain an octet string indicating the particular management address associated with this TLV.

- The returned address should be the most appropriate for management use, typically a layer 3 address such as the IPv4 address, 192.0.2.10 (see also Table 8-2, footnote a).
- If no management address is available, the return address should be the MAC address for the station or port. The ianaAddressFamilyNumber for a MAC address is all802 (value 6).

8.5.9.5 interface numbering subtype

The interface numbering subtype field shall contain an integer value indicating the numbering method used for defining the interface number. The following three values are currently defined:

- Unknown
- ifIndex
- system port number

1 8.5.9.6 interface number

2 The interface number field shall contain the assigned number within the system that identifies the specific
3 interface associated with this management address. If the value of the interface subtype is unknown, this
4 field shall be set to zero.

5 8.5.9.7 object identifier (OID) string length

6 The object identifier string length field shall contain the length, in octets, of the OID. A value of zero in this
7 field indicates that the OID field is not provided.

8 8.5.9.8 object identifier

9 The object identifier field contains an OID that identifies the type of hardware component or protocol entity
10 associated with the indicated management address. The OID shall be the value portion of the ASN.1
11 encoding of the object identifier, encoded according to ASN.1 basic encoding rules [ISO/IEC 8824-1]. If no
12 OID is available, this field shall not be provided.

13 NOTE—The interface number and OID are included in this TLV to assist NMS discovery by indicating Enterprise
14 Specific or other starting points for the search, such as the Interface or Entity MIB (IETF RFC 6933).

15 8.5.9.9 Management Address TLV usage rules

16 Management Address TLVs are subject to the following:

- 17 a) At least one Management Address TLV should be included in every LLDPDU.
 - 18 b) Since there are typically a number of different addresses associated with a MSAP identifier, an
19 individual LLDPDU may contain more than one Management Address TLV.
 - 20 c) When Management Address TLV(s) are included in an LLDPDU, the included address(es) should
21 be the address(es) offering the best management capability.
 - 22 d) If more than one Management Address TLV is included in an LLDPDU, each management address
23 shall be different from the management address in any other management address TLV in the
24 LLDPDU.
 - 25 e) If an OID is included in the TLV, it shall be reachable by the management address.
 - 26 f) In a properly formed Management Address TLV, the TLV information string length is equal to:
27 (management address string length) + (OID string length) + 7.
- 28 If the TLV information string length in a received Management Address TLV is incorrect, then it is
29 ignored and processing of that LLDPDU is terminated.

8.5a Extended LLDP set TLV formats and definitions

8.5a.1 Manifest TLV

The Manifest TLV allows an LLDP agent supporting XLLDP to advertise more TLVs than fits in a single LLDPDU. The Manifest TLV contains a list of Extension PDUs that convey the additional TLVs. The format for this TLV is shown in Figure 8-11a.

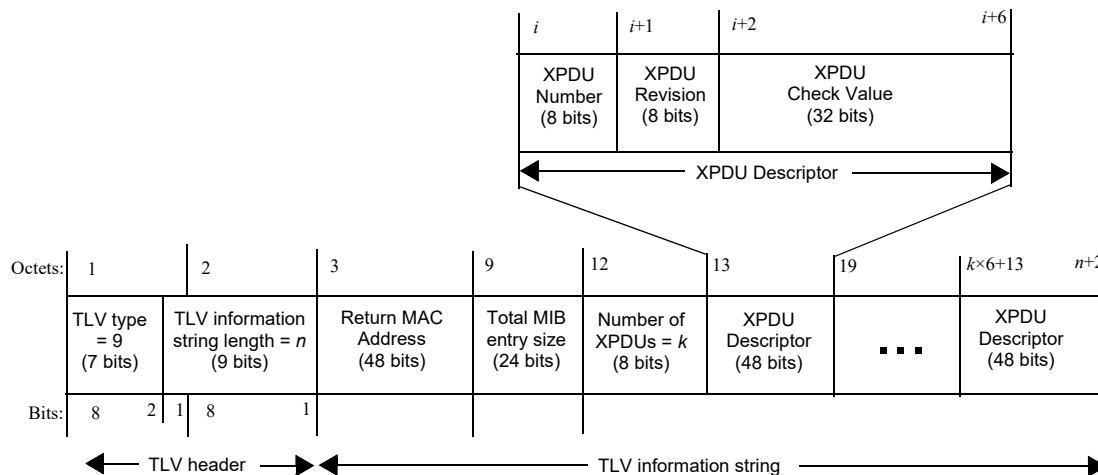


Figure 8-11a—Manifest TLV format

8.5a.1.1 TLV information string length

The TLV information string length field shall contain the number of octets in the TLV information string.

8.5a.1.2 Return MAC Address

The individual MAC Address of the MSAP supporting the LLDP agent, to be used as the destination address for Extension Request LLDPDUs generated in response to this Manifest TLV.

8.5a.1.3 Total MIB entry size

The total MIB entry size is the number of octets in all TLVs associated with this MSAP Identifier (regardless of the number of XPDUs used to convey those TLVs). It is the sum of the information string length fields in all TLVs, plus two times the number of TLVs, and includes the TLVs in the initial Normal LLDPDU. This allows the LLDP remote systems MIB manager to determine whether it expects to have space to store the information prior to requesting any XPDUs.

8.5a.1.4 Number of XPDUs

The number of XPDUs field contains the count of the XPDU Descriptors in this TLV. Since the maximum size of the TLV information string is 511 octets, the minimum number of XPDU Descriptors is 1 and the maximum number is 83. When the number of XPDU Descriptors is k , the TLV information string length is at least $k \times 6 + 10$, but it need not be exactly $k \times 6 + 10$. This allows an implementation to use a fixed length TLV information string with a variable number of XPDU Descriptors.

1 **8.5a.1.5 XPDU Descriptor**

2 An XPDU Descriptor comprises an 8-bit XPDU Number, an 8-bit XPDU Revision, and a 32-bit XPDU
3 Check Value. The XPDU Number uniquely identifies each XPDU in a manifest. The XPDU Revision is
4 assigned a random value at initialization, and is incremented (modulo 256) each time there is a change to
5 one or more TLVs in the XPDU. The XPDU Check Value is the low order 32 bits of an MD5 hash calculated
6 across all octets of the XPDU.

7 **8.5a.1.6 Manifest TLV usage rules**

8 An LLDPDU shall not contain more than one Manifest TLV. Each XPDU Descriptor associated with an
9 MSAP identifier and Scope MAC Address shall have a unique XPDU Number.

10 **8.5a.2 Extension Request TLV**

11 The Extension Request TLV allows an LLDP agent supporting XLLDP to request that the neighbor transmit
12 specific Extension PDUs. The format for this TLV is shown in Figure 8-11b.

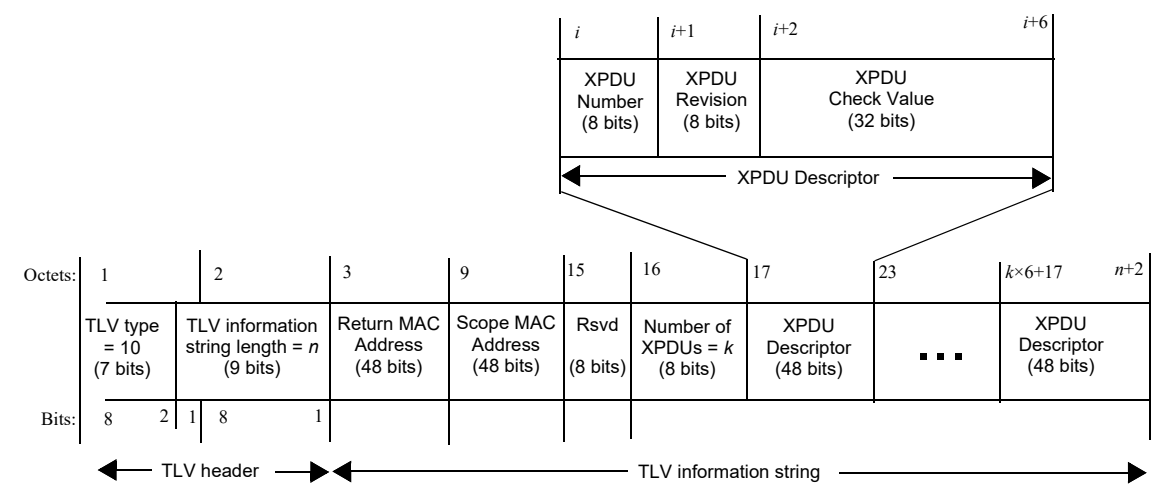


Figure 8-11b—Extension Request TLV format

13 **8.5a.2.1 TLV information string length**

14 The TLV information string length field shall contain the number of octets in the TLV information string.

15 **8.5a.2.2 Return MAC Address**

16 The individual MAC address of the MSAP supporting the LLDP agent, to be used as the destination address
17 for the requested Extension LLDPDUs.

18 **8.5a.2.3 Scope MAC Address**

19 The MAC Address used by this LLDP agent as the destination address for Normal LLDPDUs.

20 **8.5a.2.4 Number of XPDUs**

21 The number of XPDUs field contains the count of the XPDU Descriptors in this TLV. Since the maximum
22 size of the TLV information string is 511 octets, the minimum number of XPDU Descriptors is 1 and the
23 maximum number is 82. When the number of XPDU Descriptors is k , the TLV information string length is

at least $k \times 6 + 14$, but it need not be exactly $k \times 6 + 14$. This allows an implementation to use a fixed length TLV information string with a variable number of XPDU Descriptors.

8.5a.2.5 XPDU Descriptor

An XPDU Descriptor comprises an 8-bit XPDU Number, an 8-bit XPDU Revision, and a 32-bit XPDU Check, as specified in 8.5a.1.5.

8.5a.2.6 Extension Request TLV usage rules

An Extension Request LLDPDU shall contain an Extension Request TLV as the third TLV, and shall not contain more than one Extension Request TLV. An Extension Request TLV shall not be contained in a Normal LLDPDU or an Extension LLDPDU. Each XPDU Descriptor associated with a MSAP identifier and Scope MAC Address shall have a unique XPDU Number.

8.5a.3 Extension Identifier TLV

The Extension Identifier TLV identifies an XPDU containing one or more TLVs that would not fit in the Normal LLDPDU transmitted by this LLDP agent. The format for this TLV is shown in Figure 8-11c.

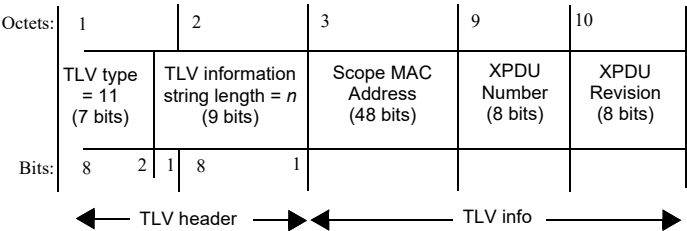


Figure 8-11c—Extension Identifier TLV format

8.5a.3.1 TLV information string length

The TLV information string length field shall contain the number of octets in the TLV information string.

8.5a.3.2 Scope MAC Address

The MAC Address used by this LLDP agent as the destination address for Normal LLDPDUs.

8.5a.3.3 XPDU Number

The XPDU Number portion of the XPDU Descriptor specified in 8.5a.1.5.

8.5a.3.4 XPDU Revision

The XPDU Revision portion of the XPDU Descriptor specified in 8.5a.1.5.

8.5a.3.5 Extension Identifier TLV usage rules

An Extension LLDPDU shall contain an Extension Identifier TLV as the third TLV, and shall not contain more than one Extension Identifier TLV. An Extension Identifier TLV shall not be contained in a Normal LLDPDU or an Extension Request LLDPDU.

1 **8.6 Organizationally Specific TLVs**

2 This TLV category is provided to allow different organizations, such as IEEE 802.1, IEEE 802.3, IETF, as
3 well as individual software and equipment vendors, to define TLVs that advertise information to remote
4 entities attached to the same media, subject to the following restrictions:

- 5 a) Information transmitted in an Organizationally Specific TLV is intended to be a one way
6 advertisement.
- 7 b) Information transmitted in an Organizationally Specific TLV shall be independent from information
8 in a TLV received from a remote port.
- 9 c) Information transmitted in one Organizationally Specific TLV shall not be concatenated with
10 information transmitted in another TLV on the same media in order to provide a means for sending
11 messages that are larger than would fit within a single TLV.
- 12 d) Information received in an Organizationally Specific TLV shall not be explicitly forwarded to other
13 ports in the system.
- 14 e) Organizationally Specific TLVs shall conform to 8.4 and 8.6.1.

15 Each set of Organizationally Specific TLVs shall include associated LLDP MIB extensions and the
16 associated TLV selection management variables and MIB/TLV cross reference tables. Systems that
17 implement LLDP and that also support standard protocols for which Organizationally Specific TLV
18 extension sets have been defined shall support all TLVs and the LLDP MIB extensions defined for that
19 particular TLV set.

20 Annex D of IEEE Std 802.1Q-2014 and Clause 79 of IEEE Std 802.3-2012 contain Organizationally
21 Specific TLVs for IEEE Std 802.1 and IEEE Std 802.3, respectively, along with their associated LLDP MIB
22 extensions and LLDP MIB/TLV cross reference tables. Organizations wishing to define TLVs for their use
23 should use these annexes as examples.

24 **8.6.1 Organizationally Specific TLV format**

25 The format for Organizationally Specific TLVs is shown in Figure 8-12.

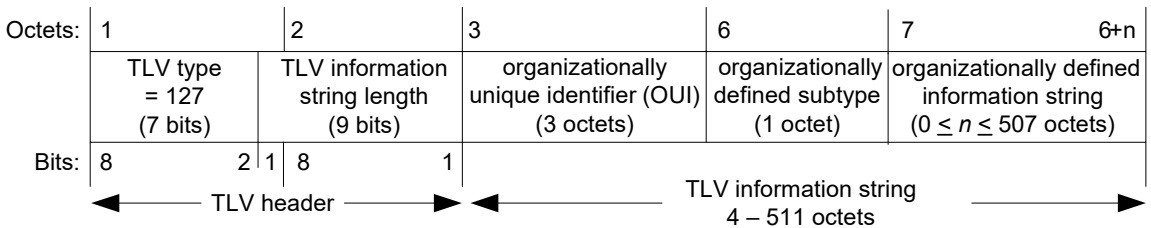


Figure 8-12—Format for Organizationally Specific TLVs

26 The following subclauses indicate how the individual fields shall be defined.

27 **8.6.1.1 TLV type**

28 The same Organizationally Specific TLV type value, 127, shall be used for all organizationally defined
29 TLVs.

30 **8.6.1.2 TLV information string length**

31 The TLV information string length field shall contain the length of the information string, in octets.

1 8.6.1.3 organizationally unique identifier (OUI)

2 The organizationally unique identifier field shall contain the organization's OUI (see Clause 8 of
3 IEEE Std 802-2014).

4 8.6.1.4 organizationally defined subtype

5 The organizationally defined subtype field shall contain a unique subtype value assigned by the defining
6 organization.

7 NOTE—Defining organizations are responsible for maintaining listings of organizationally defined subtypes in order to
8 assure uniqueness.

9 8.6.1.5 organizationally defined information string

10 The actual format of the organizationally defined information string field is organizationally specific and can

- 11 a) Contain either binary or alpha-numeric information that is instance specific for the particular TLV
12 type and/or subtype. Alpha-numeric information should be encoded in UTF-8 (IETF RFC 3629).
- 13 b) Include one or more information fields with their associated field-type identifiers and field length
14 designators similar to those in the Management Address TLV (see 8.5.9).

15 8.6.2 Organizationally Specific TLV usage rules

16 Organizations defining their own Organizationally Specific TLVs should include a subclause that defines
17 any specific usage rules and/or specific conditions that affects how the receiving LLDP agent shall treat the
18 TLV.

19 The Organizationally Specific TLV usage rules should include the following:

- 20 a) The number of Organizationally Specific TLVs that may be contained in an LLDPDU and any
21 additional information field subtypes that would identify differences between two TLVs with the
22 same OUI and organizationally defined subtype.
- 23 b) Any error conditions that are specific to the particular Organizationally Specific TLV and the action
24 that is taken for each defined error condition.

9. LLDP agent operation

This clause describes the operation of the protocol from the point of view of a single LLDP agent (see Clause 6); i.e., it describes the operation of the protocol for the transmission and reception of LLDPDUs on a single Port. Where the use of more than one **Scope MAC Address (7.1.1)** is supported on a given Port, a separate instance of the LLDP agent is responsible for the operation of the protocol for each **Scope MAC Address** that is supported.

9.1 Overview

Each LLDP agent is responsible for causing the following tasks to be performed:

- a) Maintaining current information in the LLDP local system MIB.
- b) Extracting and formatting LLDP local system MIB information for transmission to the remote port LLDP agent(s) at regular intervals or whenever there is a change in the system condition or status.
- c) Recognizing and processing received LLDP frames.
- d) Maintaining current values in the LLDP remote system MIB.
- e) Using the procedure somethingChangedLocal() and the procedure somethingChangedRemote() to notify the PTOPO MIB manager and MIB managers of other optional MIBs whenever a value or status change has occurred in one or more objects in the LLDP local system LLDP remote systems MIB.

NOTE 1—A consequence of the support of multiple MAC addresses on a Port is that there are multiple copies of the LLDP local system MIB and remote system MIB, one copy for each address supported.

LLDP allows implementations that support three different operating modes: transmit only, receive only, or both transmit and receive. An LLDP agent shall conform to the specifications of each of the state machines indicated in Table 9-1 for the operating mode that it supports.

Table 9-1—Subclause/operating mode applicability

Subclause number	Subclause title	Transmit only	Receive only	Transmit and receive
9.2.10	Receive state machine	Mandatory if XLLDP is supported	Mandatory	Mandatory
9.2.9	Transmit state machine	Mandatory	—	Mandatory
9.2.11	Transmit timer state machine	Mandatory	—	Mandatory

Note 2—When XLLDP is supported and the operating mode is transmit only, the receive state machine is only enabled to recognize and respond to Extension Request LLDPDUs. Information advertised by remote LLDP agents is not stored in the LLDP remote systems MIB.

9.1.1 Frame transmission

An LLDPDU transmission cycle consists of the transmission of a Normal LLDPDU, potentially including a Manifest TLV if the implementation supports XLLDP, and the subsequent transmission of one or more Extension LLDPDUs upon receipt of Extension Request LLDPDUs. For an implementation not supporting XLLDP, the entire transmission cycle consists of the transmission of a single Normal LLDPDU.

1 The LLDP local system MIB contains the information needed for constructing individual TLVs and includes
2 a capability that allows network managers to select the optional TLVs to be included in the Normal
3 LLDPDU and, if XLLDP is supported, in one or more Extension LLDPDUs (see 10.2.2).

4 9.1.1.1 Normal LLDPDUs

5 The transmission of Normal LLDPDUs is performed by the transmit state machine, and the timing of
6 transmission cycles is controlled by the transmit timer state machine. Transmissions occur as a result of a
7 number of different local events, as follows:

- 8 a) A regular background transmission cycle occurs as a result of a transmission timer expiring. This
9 prevents the receiving system from timing out the information received, as a result of the “time to
10 live” associated with the last received LLDPDU expiring.
- 11 b) If a new neighbor is recognized (i.e., an LLDPDU is received by the receive state machine from a
12 hitherto unknown remote LLDP agent), then four LLDPDU transmission cycles are performed at
13 shorter time intervals to quickly update the new neighbor with current information about its own
14 neighbors. This rapid transmission behavior is suppressed if the remote systems MIB is not able to
15 accommodate the new neighbor information without removing current information relating to an
16 older neighbor, in order to prevent excessive PDU transmissions resulting from “churn” effects.
- 17 c) If a condition (status or value) change occurs in one or more objects in the LLDP local system MIB,
18 then a transmission cycle is initiated immediately, in order to communicate the local changes as
19 quickly as possible to any neighboring systems. The transmit timer state machine operates a simple
20 credit-based transmission strategy that allows a number of transmission cycles in quick succession
21 (the maximum number being determined by the maximum credit value), thus allowing rapid
22 updating of information in situations where the local state is changing quickly. However, once the
23 LLDP agent’s transmission credit is exhausted, the rate of transmission is cut to a maximum of one
24 transmission cycles per second.
- 25 d) The overall timing of regular transmission cycles is governed by a system “tick” that operates on a
26 nominal timebase of 1 s. However, in shared media LANs, in order to avoid bunching of
27 transmissions, the intervals between ticks include a random jitter component so that, while the
28 average time interval between ticks is 1 s, the interval between adjacent pairs of ticks can vary (see
29 9.2.2).

30 In order to reduce synchronization effects, it is recommended that transmission cycle intervals for different
31 LLDP agents on the same multi-port implementation are staggered.

32 When the transmit state machine (9.2.9) initiates a transmission cycle, the LLDP MIB manager extracts the
33 selected information from the LLDP local system MIB and constructs a Normal LLDPDU containing the
34 following:

- 35 a) The mandatory TLVs as specified in 8.2:
 - 36 1) Chassis ID TLV (see 8.5.2).
 - 37 2) Port ID TLV (see 8.5.3).
 - 38 3) Time To Live TLV, with the TTL value set equal to txTTL (see 8.5.4).
- 39 b) A Manifest TLV (see 8.5a.1), if XLLDP is supported and any of the selected information is to be
40 included in an Extension LLDPDU. The Manifest TLV includes a descriptor for each Extension
41 LLDPDU, even if the information selected to be included in that Extension LLDPDU has not
42 changed.
- 43 c) Additional optional TLVs, from the basic management set or from one or more organizationally
44 specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system
45 MIB by network management (see Table 8-1).
- 46 d) Optionally, an End Of LLDPDU TLV.

Optional TLVs from the basic management set, such as the Management Address TLV, with different TLV type and subtype combinations as well as optional TLVs with different **organizationally unique identifier** and organizationally defined subtype combinations from one or more organizationally specific sets can be included within the same LLDPDU.

9.1.1.2 Shutdown LLDPDUs

A special procedure exists for the case in which a LLDP agent knows an associated port is about to become non-operational (for example, the adminStatus for the port is transitioning to ‘disabled’). In the event a port, currently configured with LLDP frame transmission enabled, either becomes disabled for LLDP activity, or the interface is administratively disabled, the transmit state machine attempts to send a final LLDP shutdown LLDPDU with:

- a) The Chassis ID TLV.
- b) The Port ID TLV.
- c) The Time To Live TLV with the TTL field set to zero.
- d) Optionally, an End Of LLDPDU TLV.

The shutdown LLDPDU does not include any other optional TLVs and, if possible, should be transmitted before the interface is disabled.

NOTE—There is an inherent race condition between an interface knowing it is going down and its ability to send “one more frame.” If possible, the actual transition of the port to disabled is delayed until after this frame is transmitted. In the event where adminStatus is transitioning to the disabled state and the LLDP agent is shutting down, this shutdown procedure should be executed for all local ports.

9.1.1.3 Extension Request LLDPDUs

Extension Request LLDPDUs are transmitted in response to the reception of a Manifest LLDPDU. The transmission is controlled by the extended receive state machine. When XLLDP is supported, and a Manifest LLDPDU or an Extension LLDPDU is received, and the Manifest TLV includes XPDU Descriptors that do not match the XPDU Descriptors in the LLDP remote systems MIB entry for the MSAP Identifier received in the LLDPDU, the LLDP MIB manager constructs an Extension Request LLDPDU containing the following:

- a) The mandatory TLVs as specified in 8.2:
 - 1) Chassis ID TLV (8.5.2).
 - 2) Port ID TLV (8.5.3).
 - 3) Extension Request TLV with a list of one or more XPDU descriptors (8.5a.2).
- b) Optionally, an End Of LLDPDU TLV.

9.1.1.4 Extension LLDPDUs

Extension LLDPDUs are transmitted in response to the reception of an Extension Request LLDPDU. The transmission is controlled by the receive state machine. When XLLDP is supported, and an Extension Request LLDPDU is received, then for each XPDU Number and XPDU Revision pair in the Extension Request TLV that matches an XPDU Number and XPDU Revision in the LLDP local system MIB, the LLDP MIB manager extracts the selected information from the LLDP local system MIB and constructs an Extension LLDPDU containing the following:

- a) The mandatory TLVs as specified in 8.2:
 - 1) Chassis ID TLV (8.5.2).
 - 2) Port ID TLV (8.5.3).
 - 3) Extension Identifier TLV with the requested XPDU Number and XPDU Revision pair (8.5a.3).

- 1 b) Additional optional TLVs, from the basic management set or from one or more organizationally
- 2 specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system
- 3 MIB by network management (Table 8-1).
- 4 c) Optionally, an End Of LLDPDU TLV.

5 9.1.2 Frame reception

6 LLDP frame reception consists of four phases: frame recognition, frame validation, TLV collection, and
7 LLDP remote systems MIB updating.

8 9.1.2.1 Frame recognition

9 Frame recognition is performed at the LLDP/LSAP (see Figure 6-2), where the destination address and
10 EtherType values determine whether the frame is an LLDP frame destined for this LLDP agent or not.

11 9.1.2.2 Frame validation

12 Frames that are recognized as LLDP frames are then validated to determine whether they are properly
13 constructed and contain the correct set of mandatory TLVs. Frames that do not pass the validation criteria
14 are discarded.

15 9.1.2.3 TLV collection

16 All received TLVs are validated by checking for format errors, and invalid TLVs are ignored. If the TLVs
17 received in the Normal LLDPDU did not include a Manifest TLV (or the implementation does not support
18 XLLDP and does not recognize the Manifest TLV), then all validated TLVs are passed to the LLDP remote
19 systems MIB immediately. If the implementation supports XLLDP and the Normal LLDPDU includes a
20 Manifest TLV, then this TLV is used to determine which Extension LLDPDUs need to be received before a
21 complete set of TLVs can be passed to the LLDP remote systems MIB.

22 An implementation supporting XLLDP compares the XPDU descriptors in the Manifest TLV to those
23 received in a previous Manifest TLV to determine which XPDU (identified by the XPDU Number) contain
24 new information. If no Manifest TLV has been received previously, then all of the XPDU descriptors in the
25 Manifest TLV contain new information. If there are no XPDU descriptors that have new information, then the validated
26 TLVs received in the Normal LLDPDU are passed to the LLDP remote systems MIB. Otherwise these TLVs
27 are retained until the XPDU descriptors with new information have been received.

28 If there are XPDU descriptors that have new information, then an Extension Request LLDPDU, requesting one or more
29 of the XPDU descriptors with new information, is transmitted to the LLDP agent that sourced the Normal LLDPDU. A
30 timer is started when an Extension Request LLDPDU is transmitted, and the request is retried once if the
31 timer expires before all requested XPDU descriptors have been received. When all requested XPDU descriptors have been
32 received, another Extension Request LLDPDU is transmitted requesting one or more of the remaining
33 XPDU descriptors with new information. This process repeats until all XPDU descriptors with new information have been
34 received. If a new Manifest LLDPDU is received before all XPDU descriptors have been received, any received
35 XPDU descriptors that have the same Revision and Checksum in the new Manifest TLV are retained, and any received
36 XPDU descriptors that have different Revision and Checksum are discarded. The extended receive state machine then
37 continues to issue Extension Requests for any necessary XPDU descriptors.

38 Received frames containing Extension LLDPDUs are recognized and validated following the same steps as
39 frames containing Normal LLDPDUs. Once validated, the XPDU check value is calculated. This check
40 value, together with the XPDU Number and XPDU Revision from the Extension Identifier TLV, is
41 compared to the XPDU descriptors in the Manifest TLV. If there is a matching descriptor then all valid TLVs
42 received in this Extension LLDPDU are retained. When all XPDU descriptors with new information have been
43 received, all of the retained TLVs are passed to the LLDP remote systems MIB.

1 9.1.2.4 LLDP remote systems MIB updating

2 The collected TLVs are used to create, modify, or delete an entry in the LLDP remote systems MIB. The
3 relevant entry in the LLDP remote systems MIB is determined by the originating MSAP identifier formed
4 from the contents of the Chassis ID and Port ID TLVs. The LLDP remote systems MIB is updated as
5 follows:

- 6 a) If the TTL value in the Time To Live TLV is greater than zero and an entry does not exist in the
7 LLDP remote systems MIB for the originating MSAP identifier and there is sufficient space (or
8 sufficient space can be created by the steps in 9.2) for the new information, then a new entry in the
9 MIB is created.
- 10 b) If the TTL value in the Time To Live TLV is greater than zero and an entry already exists in the
11 remote systems MIB for the originating MSAP identifier and there is sufficient space (or sufficient
12 space can be created by the steps in 9.2) for the new information, then that entry is modified as
13 follows:
 - 14 1) The TLVs received in the Normal LLDPDU are used to replace any information elements in the
15 existing entry in the MIB that were previously received in a Normal LLDPDU. Note that if the
16 Normal LLDPDU did not contain a Manifest TLV (or the implementation does not support
17 XLLDP and does not recognize the Manifest TLV), then the new information contained in the
18 LLDPDU is used to replace the whole of the existing entry in the MIB; i.e., if there are
19 information elements in the existing entry for which there are corresponding elements in the
20 received LLDPDU, then those elements are updated using the information received; any other
21 information elements in the existing entry in the MIB are deleted.
 - 22 2) The TLVs received in an Extension LLDPDU are used to replace any information elements in
23 the existing entry in the MIB that were previously received in an Extension LLDPDU with the
24 same XPDU Number.
 - 25 3) Any information elements in the existing entry in the MIB that were previously received in an
26 Extension LLDPDU with an XPDU Number that is not included in the received Manifest TLV
27 are deleted.
- 28 c) If the TTL value in the Time To Live TLV is greater than zero and an entry exists in the LLDP
29 remote systems MIB for the originating MSAP identifier and there is insufficient space for the new
30 information, then the entry is deleted from the MIB.
- 31 d) If the TTL value in the Time To Live TLV is equal to zero and an entry exists in the LLDP remote
32 systems MIB for the originating MSAP identifier, then the entry is deleted from the MIB.

33 When an entry in the LLDP remote systems MIB is created or modified (even if the modification is to
34 replace all information elements with the same information), the rxInfoTTL timer associated with that entry
35 is set to the TTL value from the Time To Live TLV. This determines how long the information associated
36 with that MSAP identifier and destination MAC address is to be stored in the LLDP remote systems MIB
37 before being aged out and deleted. The ageing out of old data purges the MIB of data that was originated by
38 systems that are no longer neighbors as a result of topology changes, system failure, or system inactivation.

39 9.1.3 Too many neighbors

40 The amount of space needed in the LLDP remote systems MIB to accommodate the creation of several new
41 MIB structures is beyond the scope of this standard. It may not always be possible to accommodate another
42 new neighbor in implementations with limited memory. There are several possibilities for handling this case,
43 including but not limited to the following examples:

- 44 a) Ignore and not process the new neighbor's information.
- 45 b) Delete the information from the oldest neighbor(s) until there is sufficient memory available to store
46 the new neighbor's information.

- 1 c) Randomly delete neighbors until there is sufficient memory available to store the new neighbor's
2 information.

3 The method of handling the case where a new entry in the remote systems MIB can not be created is beyond
4 the scope of this standard, however it is necessary for the implementation to keep track of this case by
5 properly updating the variable tooManyNeighbors and the tooManyNeighborsTimer (9.2.5.16, 9.2.2). The
6 variable tooManyNeighbors identifies when there is insufficient space in the LLDP remote systems MIB to
7 store information from all active neighbors. The tooManyNeighborsTimer indicates the minimum time that
8 this condition exists.

9 Further detail on the handling of the too many neighbors condition can be found in 9.2.8.5.1.

10 **9.1.4 LLDP remote systems rxInfoTTL timer expiration**

11 If the rxInfoTTL timer (9.2.2.1) associated with an MSAP identifier expires, all information associated with
12 that MSAP identifier is deleted and the procedure somethingChangedRemote() notifies the managers of the
13 PTOPO MIB and other optional MIB modules (see Figure 11-1) that something has changed in the LLDP
14 remote systems MIB associated with that MSAP identifier.

15 **9.1.5 LLDP local port/connection failure**

16 If the local port or the connection to the remote system fails unexpectedly before a shutdown frame is
17 received, the LLDP management entity does not delete objects in the LLDP remote systems MIB pertaining
18 to information received from any MSAP identifier through that local port until the port is re-initialized or the
19 associated rxInfoTTL timer (9.2.2.1) expires.

20 **9.2 State machines**

21 The operation of the protocol is represented by the following three state machines and associated timers to
22 indicate when local system managed object values are to be transmitted to refresh the values in remote
23 LLDP agent's LLDP remote systems MIB and when values in the local LLDP agent's remote systems MIB
24 have become invalid:

- 25 a) Transmit state machine (9.2.9).
- 26 b) Receive state machine (9.2.10).
- 27 c) Transmit timer state machine (9.2.11).

28 **9.2.1 Notational conventions used in state diagrams**

29 State diagrams are used to represent the operation of a function as a group of connected, mutually exclusive
30 states. Only one state of a function can be active at any given time.

31 Each state is represented in the state diagram as a rectangular box, divided into two parts by a horizontal
32 line. The upper part contains the state identifier, written in uppercase letters. The lower part contains any
33 procedures that are executed on entry to the state.

34 All permissible transitions between states are represented by arrows, the arrowhead denoting the direction of
35 the possible transition. Labels attached to arrows denote the condition(s) that shall be met in order for the
36 transition to take place. A transition that is global in nature (i.e., a transition that occurs from any of the
37 possible states if the condition attached to the arrow is met) is denoted by an open arrow; i.e., no specific
38 state is identified as the origin of the transition.

1 On entry to a state, the procedures defined for the state (if any) are executed exactly once, in the order that
2 they appear on the page. Each action is deemed to be atomic; i.e., execution of a procedure completes before
3 the next sequential procedure starts to execute. No procedures execute outside of a state block. On
4 completion of all of the procedures within a state, all exit conditions for the state (including all conditions
5 associated with global transitions) are evaluated continuously until such a time as one of the conditions is
6 met. All exit conditions are regarded as Boolean expressions that evaluate to TRUE or FALSE; if a condition
7 evaluates to TRUE, then the condition is met. When the condition associated with a global transition is met,
8 it supersedes all other exit conditions, including UCT. The label UCT denotes an unconditional transition
9 (i.e., UCT always evaluates to TRUE). The label ELSE denotes a transition that occurs if none of the other
10 conditions for transitions from the state are met (i.e., ELSE evaluates to TRUE if all other possible exit
11 conditions from the state evaluate to FALSE).

12 A variable that is set to a particular value in a state block retains this value until a subsequent state block
13 executes a procedure that modifies the value, or until the value is modified by an external event or timer tick.

14 Where it is necessary to segment a state machine description across more than one diagram, a transition
15 between two states that appear on different diagrams is represented by an exit arrow drawn with dashed
16 lines, plus a reference to the diagram that contains the destination state. Similarly, dashed arrows and a
17 dashed state box are used on the destination diagram to show the transition to the destination state. In a state
18 machine that has been segmented in this way, any global transitions that can cause entry to states defined in
19 one of the diagrams are deemed to be potential exit conditions for all of the states of the state machine,
20 regardless of which diagram the state boxes appear in.

21 Should a conflict exist between the interpretation of a state diagram and either the corresponding global
22 transition tables or the textual description associated with the state machine, the state diagram takes
23 precedence.

24 The interpretation of the special symbols and operators used in the state diagrams is defined in Table 9-2;
25 these symbols and operators are derived from the notation of the “C” programming language.

Table 9-2—State machine symbols

Symbol	Interpretation
()	Used to force the precedence of operators in Boolean expressions and to delimit the argument(s) of actions within state boxes.
;	Used as a terminating delimiter for actions within state boxes. Where a state box contains multiple actions, the order of execution follows the normal English language conventions for reading text.
=	Assignment action. The value of the expression to the right of the operator is assigned to the variable to the left of the operator. Where this operator is used to define multiple assignments, e.g., a = b = X the action causes the value of the expression following the right-most assignment operator to be assigned to all of the variables that appear to the left of the right-most assignment operator.
!	Logical NOT operator.
&&	Logical AND operator.
	Logical OR operator.
if...then...	Conditional action. If the Boolean expression following the if evaluates to TRUE, then the action following the then is executed.

Table 9-2—State machine symbols (*continued*)

Symbol	Interpretation
{statement 1, ... statement N}	Compound statement. Braces are used to group statements that are executed together as if they were a single statement.
!=	Inequality. Evaluates to TRUE if the expression to the left of the operator is not equal in value to the expression to the right.
==	Equality. Evaluates to TRUE if the expression to the left of the operator is equal in value to the expression to the right.
<	Less than. Evaluates to TRUE if the value of the expression to the left of the operator is less than the value of the expression to the right.
<=	Less than or equal to. Evaluates to TRUE if the value of the expression to the left of the operator is either less than or equal to the value of the expression to the right.
>	Greater than. Evaluates to TRUE if the value of the expression to the left of the operator is greater than the value of the expression to the right.
>=	Greater than or equal to. Evaluates to TRUE if the value of the expression to the left of the operator is either greater than or equal to the value of the expression to the right.
+	Arithmetic addition operator.
–	Arithmetic subtraction operator.

9.2.2 Timers

A set of timers is used by the LLDP state machines; these operate as countdown timers (i.e., they expire when their value reaches zero). These timers

- Have a resolution of one second.
- Have an integer value n , where $0 < n < 65535$ seconds.
- Are started by loading an initial integer value.
- Are decremented once per timer tick as long as $n > 0$.
- Represent the remaining time in the period.

For Ports connected to point-to-point LANs, the interval between timer ticks is 1 s. For Ports connected to shared media LANs, the interval between any pair of timer ticks is 0.8 s, plus a “jitter” component that is a random value between 0.0 s and 0.4 s. Hence, the average interval between timer ticks is 1 s; however, the interval between any pair of ticks varies randomly. When a timer tick occurs, all instances of the variable txTick (9.2.5.22) for the Port are set TRUE.

NOTE—The random component of the tick time used in shared media LANs is intended to minimize the possibility of systems connected to the same LAN becoming synchronized in their transmission timings.

9.2.2.1 rxInfoTTL

An instance of this timer exists for each entry in the LLDP remote systems MIB that has been created by this instance of the receive state machine for a given MSAP identifier. The timer value indicates the number of seconds remaining until the information in all the objects in the LLDP remote systems MIB associated with the MSAP identifier is no longer valid and needs to be deleted.

1 9.2.2.2 tooManyNeighborsTimer

2 This timer indicates the number of seconds remaining during which it is known that an LLDP neighbor
3 exists that may not be stored in the remote systems MIB.

4 9.2.2.3 txTTR

5 An instance of this timer exists for each Agent. The txTTR timer is used to determine the next LLDPDU
6 transmission is due; it is initialized by the transmit timer state machine, with the value msgTxInterval
7 (9.2.5.6) if txFast (9.2.5.19) is equal to zero, or with the value msgFastTx (9.2.5.4) if txFast is greater than
8 zero.

9 9.2.2.4 txShutdownWhile

10 An instance of this timer exists for each Agent. The txShutdownWhile timer indicates the number of
11 seconds remaining until LLDP re-initialization can occur.

12 9.2.2.5 rxTimer

13 An instance of this timer exists for each extended receive state machine to impose a time limit on each
14 extension request, and on the overall TLV collection process.

15 9.2.3 Per-System variables

16 An instance of each of these variables exists per system; they are available for use by all of the state
17 machines of all of the LLDP agents in the system.

18 9.2.3.1 BEGIN

19 This Boolean variable is controlled by the system initialization process. A value of TRUE causes all state
20 machines to continuously execute their initial state. A value of FALSE allows all state machines to perform
21 transitions out of their initial state, in accordance with the relevant state machine definitions.

22 9.2.4 Per-Port variables

23 An instance of each of these variables exists per Port; they are available to all of the state machines
24 associated with a Port.

25 9.2.4.1 portEnabled

26 This variable is externally controlled. Its value reflects the operational state of the MAC service supporting
27 the port. Its value is TRUE if the MAC service supporting the Port is in an operable condition; otherwise, it
28 is FALSE.

29 9.2.5 Per-Agent variables

30 An instance of each of these variables exists per LLDP agent; they are available to all of the state machines
31 associated with an LLDP agent.

1 9.2.5.1 adminStatus

2 This variable indicates whether or not the LLDP agent is enabled. The defined values for this variable are as
3 follows:

- 4 a) enabledRxTx: The LLDP agent is enabled for reception and transmission of LLDPDUs. This is the
5 recommended default value.
- 6 b) enabledTxOnly: The LLDP agent is enabled for transmission of LLDPDUs only.
- 7 c) enabledRxOnly: The LLDP agent is enabled for reception of LLDPDUs only.
- 8 d) disabled: The LLDP agent is disabled for both reception and transmission.

9 9.2.5.2 localChange

10 This variable is set TRUE by the somethingChangedLocal() procedure (see 9.2.8.8) when there is a change
11 in the value of the LLDP local system MIB associated with this LLDP agent. The variable is set FALSE by
12 the operation of the transmit timer state machine (see 9.2.11).

13 9.2.5.3 Scope MAC Address

14 The MAC address associated with this instance of the LLDP agent that defines the scope of propagation of
15 Normal LLDPDUs in the network. It is used as the destination MAC address when the LLDP agent
16 transmits Normal LLDPDUs, and the destination MAC address that, when present in a received LLDPDU,
17 causes the LLDPDU to be passed to this LLDP agent instance for processing.

18 9.2.5.4 msgFastTx

19 This variable defines the time interval in timer ticks between transmissions during fast transmission periods
20 (i.e., txFast is non-zero). The recommended default value of msgFastTx is 1; this value can be changed by
21 management to any value in the range 1 through 3600.

22 9.2.5.5 msgTxHold

23 This variable is used, as a multiplier of msgTxInterval, to determine the value of txTTL that is carried in
24 LLDP frames transmitted by the LLDP agent. The recommended default value of msgTxHold is 4; this
25 value can be changed by management to any value in the range 2 through 10.

26 9.2.5.6 msgTxInterval

27 This variable defines the time interval in timer ticks between transmissions during normal transmission
28 periods (i.e., txFast is zero). The recommended default value for msgTxInterval is 30 s; this value can be
29 changed by management to any value in the range 1 through 3600.

30 9.2.5.7 newNeighbor

31 This variable is set TRUE by the rxProcessFrame() procedure (see 9.2.8.7) when information is received
32 from a new neighbor. It is set FALSE by the Transmit timer state machine (9.2.11).

33 NOTE—The newNeighbor variable is used to force a more rapid regular transmission rate when a new neighbor
34 appears, to ensure that the new neighbor also receives new information from the receiving system.

9.2.5.8 rcvFrame

This variable indicates that an LLDP frame has been recognized by the LLDP LSAP function and is ready to be processed by the LLDP receive state machine. If both of the following conditions are TRUE, the variable rcvFrame is set to TRUE and the frame is made available to the receive state machine for validation and processing:

- a) The destination address value is the Scope MAC Address associated with this specific LLDP agent or the individual MAC address of the MSAP supporting the LLDP agent.
- b) The EtherType value carried by the frame is the LLDP EtherType defined in Table 7-3.

NOTE—The model that is assumed for reception is that incoming LLDPDUs are presented to all LLDP agents associated with the reception Port, and if the Scope MAC Address is not one that is associated with a given LLDP agent, that LLDPDU is ignored by that LLDP agent. In practice, at most one LLDP agent processes any received LLDPDU, as there is only one LLDP agent associated with any given Scope MAC Address on a given reception Port.

9.2.5.9 reinitDelay

This parameter indicates the amount of delay from when adminStatus becomes 'disabled' until re-initialization is attempted. The recommended default value for reinitDelay is 2 s.

9.2.5.10 remoteChanges

A Boolean indication of whether the status/value of one or more selected objects in the LLDP remote systems MIB has changed or that all objects associated with a particular MSAP identifier have been deleted. This variable takes the value (*rxInfoAge OR (rxChanges && (rxTTL !=0))*)

9.2.5.11 rxChanges

This variable indicates that the incoming LLDPDU has been received with different TLV values from those currently in the LLDP remote systems MIB associated with the LLDPDU's MSAP identifier.

9.2.5.12 rxInfoAge

This variable is set TRUE when a timer expiry event occurs for the rxInfoTTL timing counter associated with a particular MSAP identifier in the LLDP remote systems MIB (i.e., the counter transitions from non-zero to zero). It is set FALSE by the operation of the Receive state machine.

9.2.5.13 rxTTL

This variable indicates the time to live value associated with all TLVs received in the current frame.

9.2.5.14 rxType

This variable indicates the type of the most recently received LLDP frame. It is set by the rxProcessFrame() procedure to one of the following values:

- a) xreq: The LLDP agent supports XLLDP and identifies the frame as containing a request for extended PDUs.
- b) xpdu: The LLDP agent supports XLLDP and identifies the frame as containing an extended PDU.
- c) shutdown: The frame contains a shutdown LLDPDU (which has an rxTTL value equal to zero).
- d) manifest: The LLDP agent supports XLLDP and the frame is a Normal LLDPDU that has a rxTTL value greater than zero and contains a Manifest TLV.
- e) normal: The frame contains a Normal LLDPDU that has an rxTTL value greater than zero, and either the LLDP agent does not support XLLDP or the frame does not contain a Manifest TLV.

- 1 f) invalid: The frame is improperly formatted for a valid LLDPDU, or the frame contains an extended
2 PDU or a request for extended PDUs but the LLDP agent does not support XLLDP.

3 **9.2.5.15 sentManifest**

4 This Boolean variable is set to TRUE when the transmit state machine generates a Manifest LLDPDU, and
5 is set to FALSE when the transmit state machine generates a Normal LLDPDU that does not contain a
6 Manifest TLV. It is used to enable the receive state machine to recognize Extension Request LLDPDUs and
7 respond with Extension LLDPDUs.

8 **9.2.5.16 tooManyNeighbors**

9 This Boolean variable indicates that there is insufficient space in the LLDP remote systems MIB to store
10 information from all connected active remote ports (see 9.2.8.5.1).

11 **9.2.5.17 txCredit**

12 The number of consecutive LLDPDUs that can be transmitted at any time. This variable is incremented by
13 the txAddCredit() procedure, up to the maximum value specified by the txCreditMax variable (see 9.2.5.18),
14 and is decremented whenever an LLDPDU is transmitted by the transmit state machine.

15 **9.2.5.18 txCreditMax**

16 The maximum value of txCredit. The recommended default value is 5; this value can be changed by
17 management to any value in the range 1 through 10.

18 **9.2.5.19 txFast**

19 This variable is used as a down counter to count the number of transmissions to be made during a fast
20 transmission period. Fast transmission periods are initiated when a new neighbor is detected, and cause
21 LLDPDU transmissions to take place at shorter time intervals than during normal (steady state) operation of
22 the protocol. The fast transmission period ensures that more than one LLDPDU is transmitted when a new
23 neighbor is detected. The first transmission is immediate; the subsequent transmissions occur at intervals
24 determined by msgFastTx. The number of transmissions is determined by the value of txFastInit (see
25 9.2.5.20).

26 **9.2.5.20 txFastInit**

27 This variable is used as the initial value for the txFast variable (see 9.2.5.19). This value determines the
28 number of LLDPDUs that are transmitted during a fast transmission period. The recommended default value
29 of txFastInit is 4; this value can be changed by management to any value in the range 1 through 8.

30 **9.2.5.21 txNow**

31 This variable is used to signal to the transmit state machine that a transmission is required. It is set TRUE by
32 the transmit timer state machine, and set FALSE by the transmit state machine when the transmission has
33 been initiated.

34 **9.2.5.22 txTick**

35 This variable is set TRUE by the system timer tick at one second intervals (see 9.2.2). It is set FALSE by the
36 operation of the transmit timer state machine.

9.2.5.23 txTTL

This variable indicates the time remaining before information in the outgoing LLDPDU is no longer valid. The TTL field in the Time To Live TLV is set to txTTL during LLDPDU construction (see 9.1.2). The value of txTTL depends on the following:

- a) During normal operation, txTTL is set to whichever is the smaller of the values represented by Equation (1) and Equation (2):

$$(msgTxInterval \times msgTxHold) + 1 \quad (1)$$

$$65535 \quad (2)$$

where msgTxInterval is defined in 9.2.5.6 and msgTxHold is defined in 9.2.5.5.

- b) If adminStatus is transitioning to ‘disabled’ or portEnabled is transitioning to FALSE, txTTL shall be set to zero.

9.2.6 Per-Neighbor variables

An instance of each of these variables exists per neighbor (identified by an MSAP) for each LLDP agent; they are available to all of the state machines associated with an LLDP agent.

9.2.6.1 createExtRx

This Boolean is used to initialize a per-neighbor extended receive state machine. It is set to TRUE when an extended receive state machine is created in response to the receipt of a Manifest LLDPDU from a neighbor for which an extended receive state machine does not already exist. It is set to FALSE by the extended receive state machine following initialization.

9.2.6.2 rxExtended

This Boolean is used to pass control between the per-agent receive state machine and the per-neighbor extended receive state machine. It is set to TRUE by the receive state machine when an LLDPDU containing a Manifest TLV or an Extension Identifier TLV is received, and is set to FALSE by the extended receive state machine.

9.2.6.3 manifestComplete

This Boolean indicates to the receive state machine whether the extended receive state machine has received all TLVs necessary to update the information for this MSAP in the LLDP remote systems MIB.

9.2.6.4 xreqComplete

This Boolean is used within the extended receive state machine to indicate that all XPDUs requested for this MSAP have been received.

9.2.6.5 rxTick

This Boolean is set TRUE by the system timer tick at one second intervals (see 9.2.2). It is set FALSE by the operation of the extended receive state machine.

1 **9.2.6.6 rxTTLExpired**

2 This Boolean is used to indicate that the extended receive state machine has been unable to collect all the
3 TLVs within a time interval greater than the received rxTTL value. It is set to TRUE when the rxTimer is
4 decremented to zero and either all Extension LLDPDUs for the most recently transmitted Extension Request
5 LLDPDU have been received, or that Extension Request LLDPDU was a retry of a previous request.

6 **9.2.7 Per-Agent statistical counters**

7 An instance of each of these statistical counters exists per LLDP agent; they are used to count significant
8 events in the transmit and receive state machines and are made available to the management functions
9 described in Clause 10 and Clause 11. All counters are maintained as unsigned integer values, four octets in
10 length, and are initialized to zero only on system initialization (i.e., when the BEGIN system variable is set
11 TRUE).

12 **9.2.7.1 statsAgeoutsTotal**

13 This counter provides a count of the times that a neighbor's information has been deleted from the LLDP
14 remote systems MIB because the rxInfoTTL timer associated with the neighbor's MSAP has expired.

15 **9.2.7.2 statsFramesDiscardedTotal**

16 This counter provides a count of all LLDPDUs received and then discarded for any of the following reasons:

- 17 a) One or more of the three mandatory TLVs at the beginning of the LLDPDU is missing, out of order,
18 or contains an out of range information string length.
- 19 b) There is insufficient space in the remote systems MIB to store the LLDPDU.

20 **9.2.7.3 statsFramesInErrorsTotal**

21 This counter provides a count of all LLDPDUs received with one or more detectable errors.

22 **9.2.7.4 statsFramesInTotal**

23 This counter provides a count of all LLDP frames received.

24 **9.2.7.5 statsFramesOutTotal**

25 This counter provides a count of all LLDP frames transmitted by this instance of the transmit state machine.
26 The counter shall be maintained as an unsigned integer value, four octets in length.

27 **9.2.7.6 statsTLVsDiscardedTotal**

28 This counter provides a count of all TLVs received and then discarded for any reason.

29 **9.2.7.7 statsTLVsUnrecognizedTotal**

30 This counter provides a count of all TLVs received on the port that are not recognized by the LLDP local
31 agent.

32 **9.2.7.8 lldpduLengthErrors**

33 A count of the number of LLDPDU length restriction errors detected by the mibConstrInfoLLDPDU()
34 procedure (9.2.8.2).

1 9.2.8 State machine procedures

2 9.2.8.1 dec (variable-name)

3 If the state machine variable *variable-name* has a non-zero value, this procedure decrements the value of the
4 variable by 1.

5 9.2.8.2 mibConstrInfoLLDPDU()

6 The mibConstrInfoLLDPDU() procedure constructs an information LLDPDU, structured according to the
7 specification in 8.2, the associated basic TLV formats defined in 8.4, plus any optional Organizationally
8 Specific TLVs as specified and their associated individual organizationally defined formats (see 8.6). The
9 information LLDPDU contains the following:

- 10 a) The set of mandatory TLVs as specified in 8.2, with the value of txTTL set in accordance with the
11 definition in 9.2.5.23.
- 12 b) Additional optional TLVs, from the basic management set or from one or more organizationally
13 specific sets, as allowed by LLDPDU length restrictions and as selected in the LLDP local system
14 MIB by network management (see Table 8-1).
- 15 c) Optionally, an End Of LLDPDU TLV.

16 If the set of TLVs that is selected in the LLDP local system MIB by network management would result in an
17 LLDPDU that violates LLDPDU length restrictions, then the variable lldpduLengthErrors (9.2.8.2) is
18 incremented by 1, and an LLDPDU is sent containing the mandatory TLVs plus as many of the optional
19 TLVs in the set as will fit in the remaining LLDPDU length.

20 9.2.8.3 mibConstrShutdownLLDPDU()

21 The mibConstrShutdownLLDPDU() procedure constructs a shutdown LLDPDU according to the LLDPDU
22 and the associated TLV formats specified in 8.2 and 8.5. The shutdown LLDPDU contains only the
23 following items:

- 24 a) The Chassis ID and Port ID TLVs.
- 25 b) The Time To Live TLV with the TTL field set to zero.
- 26 c) Optionally, an End Of LLDPDU TLV.

27 9.2.8.4 mibDeleteObjects()

28 The mibDeleteObjects() procedure deletes all information in the LLDP remote systems MIB associated with
29 a given MSAP identifier (and destroys the associated extended receive state machine if one exists) if an
30 LLDPDU is received with an rxTTL value of zero (see 9.2.8.7.1), the extended receive state machine sets
31 rxTTLExpired to TRUE, or the timing counter rxInfoTTL expires (see 9.2.2.1).

32 9.2.8.5 mibUpdateObjects()

33 The mibUpdateObjects() procedure updates the MIB objects corresponding to the TLVs contained in the
34 received Normal LLDPDU, and all TLVs contained in any received Extension LLDPDUs, for the LLDP
35 remote system if MIB space is available, as follows:

- 36 a) Compare the MSAP identifier in the current LLDPDU with the MSAP identifiers in the LLDP
37 remote systems MIB:
 - 38 1) If a match is found, but no difference exists between the information in the TLVs just received
39 and the information in the LLDP remote systems MIB, then set the control variable rxChanges

- to FALSE, and set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.
- 2) If a match is found and sufficient space is not available in the LLDP remote systems MIB for the updated information in the TLVs just received, then follow the tooManyNeighbors procedure (9.2.8.5.1) to determine whether to discard the TLVs from a new neighbor or to delete information from an existing neighbor that is already in the LLDP remote systems MIB.
- 3) If a match is found, and sufficient space is available (or has been made available) in the LLDP remote systems MIB for the updated information in the TLVs just received, then:
 - i) Replace all information associated with the MSAP identifier in the LLDP remote systems MIB that was previously received in a Normal LLDPDU with the information in the just received Normal LLDPDU.
 - ii) For each received Extension LLDPDU associated with the MSAP identifier, if any, replace all information in the LLDP remote systems MIB that was previously received in an Extension LLDPDU of the same XPDU Number with the information in the just received Extension LLDPDU. If a received Extension LLDPDU has an XPDU Number that has not been previously received, add the new information to the LLDP remote system MIB entry.
 - iii) Set the control variable rxChanges to TRUE.
 - iv) Set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.
- 4) If no match is found and sufficient space is not available in the LLDP remote systems MIB, then follow the tooManyNeighbors procedure (9.2.8.5.1) to determine whether to discard the TLVs from a new neighbor or to delete information from an existing neighbor that is already in the LLDP remote systems MIB.
- 5) If no match is found and sufficient space is available (or has been made available), create a new MIB structure to receive information associated with the new MSAP identifier, set these MIB objects to the values indicated in their respective TLVs, set the control variable rxChanges to TRUE, and set the timing counter rxInfoTTL associated with the MSAP identifier to rxTTL.

If an incoming TLV is not recognized by the receiving LLDP agent, the TLV is stored in the LLDP remote systems MIB as follows:

- b) If the TLV type value is in the range of the reserved TLV types in Table 8-1, the TLV can be from a later version of the basic management set and is stored according to the basic TLV format shown in Figure 8-2. These TLVs are indexed by their TLV type.
- c) If the TLV type value is 127, the TLV is an Organizationally Specific TLV and is stored according to the basic format for Organizationally Specific TLVs shown in Figure 8-12. These TLVs are indexed by their organizationally unique identifier and organizationally defined TLV subtype.

9.2.8.5.1 Too many neighbors

When there is insufficient space in the LLDP remote systems MIB to store information from a new neighbor something needs to be discarded. Either the received LLDPDU is discarded or existing information within the current remote systems MIB is discarded in order to make space for the new information received in the LLDPDU:

- a) Set the flag variable tooManyNeighbors to TRUE.
- b) If the information selected to be discarded is the information in the current LLDPDU:
 - 1) Set the tooManyNeighborsTimer as specified in Equation (5).

$$\text{tooManyNeighborsTimer} = \max(\text{tooManyNeighborsTimer}, \text{rxTTL}) \quad (3)$$

where rxTTL is the TTL value in the current LLDPDU

- 2) Discard the current LLDPDU and increment the statsFramesDiscardedTotal counter.

- 1 3) Set the control variable rxChanges to FALSE. Wait for the next LLDPDU.
- 2 c) If the information selected to be discarded is currently in the LLDP remote systems MIB:
- 3 1) Delete all information associated with the selected neighbor's MSAP identifier from the LLDP
- 4 remote systems MIB, and destroy the associated extended receive state machine if one exists.
- 5 2) Set the tooManyNeighborsTimer as specified in Equation (6).
- 6
$$\text{tooManyNeighborsTimer} = \max (\text{tooManyNeighborsTimer}, \text{rxInfoTTL}) \quad (4)$$
- 7 where rxInfoTTL = the selected neighbor's TTL value
- 8 3) If sufficient space exists to store the information received in the current LLDPDU in the LLDP
- 9 remote systems MIB, set control variable rxChanges to TRUE.
- 10 4) If sufficient space still does not exist to store the information received in the current LLDPDU
- 11 in the LLDP remote systems MIB, perform steps 1–3 again.
- 12 The variable tooManyNeighbors is automatically set to FALSE whenever the tooManyNeighborsTimer
- 13 expires.
- 14 NOTE—The tooManyNeighborsTimer is not precise, as it is only cleared (and the tooManyNeighbors variable set
- 15 FALSE) by timer expiration, and not by the removal of the too many neighbors condition.

16 9.2.8.6 rxInitializeLLDP()

17 The rxInitializeLLDP() procedure initializes the LLDP receive module. After the variable portEnabled is
18 equal to TRUE, the LLDP receive module is initialized or re-initialized for frame reception. During this
19 process, the local LLDP agent performs the following tasks:

- 20 a) The variable tooManyNeighbors is set to FALSE.
- 21 b) All information in the remote systems MIB associated with this LLDP agent is deleted.

22 9.2.8.7 rxProcessFrame()

23 The rxProcessFrame() procedure:

- 24 a) Validates the LLDPDU and sets the value of rxType as specified in 9.2.8.7.1.
- 25 b) Validates the TLVs contained in the LLDPDU as defined in 9.2.8.7.1 through 9.2.8.7.3.
- 26

27 9.2.8.7.1 LLDPDU validation

28 The receive module processes each incoming LLDPDU as it is received. The statsFramesInTotal counter for
29 the port is incremented and the LLDPDU is checked to verify the presence of the three mandatory TLVs at
30 the beginning of the LLDPDU as defined in 8.2.

- 31 a) The first TLV is extracted:
- 32 1) If the extracted TLV type value does not match the Chassis ID TLV type:
- 33 i) The LLDPDU is discarded.
- 34 ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both
- 35 incremented.
- 36 iii) The rxType value is set equal to invalid.
- 37 iv) The procedure rxProcessFrame() is terminated.
- 38 2) If the extracted TLV type value matches the Chassis ID TLV type, and the chassis ID TLV
- 39 information string length is not within the range $2 \leq n \leq 256$:
- 40 i) The LLDPDU is discarded.

- ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - iii) The rxType value is set equal to invalid.
 - iv) The procedure rxProcessFrame() is terminated.
 - 3) If the extracted TLV type value matches the Chassis ID TLV type, and the chassis ID TLV information string length is within the range $2 \leq n \leq 256$, the chassis ID value is extracted to become the first part of the MSAP identifier.
 - b) The second TLV is extracted:
 - 1) If the extracted TLV type value does not match the Port ID TLV type:
 - i) The LLDPDU is discarded.
 - ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - iii) The rxType value is set equal to invalid.
 - iv) The procedure rxProcessFrame() is terminated.
 - 2) If the extracted TLV type value matches the Port ID TLV type, and the port ID TLV information string length is not within the range $2 \leq n \leq 256$:
 - i) The LLDPDU is discarded.
 - ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - iii) The rxType value is set equal to invalid.
 - iv) The procedure rxProcessFrame() is terminated.
 - 3) If the extracted TLV type value matches the Port ID TLV type, and the port ID TLV information string length is within the range $2 \leq n \leq 256$, the port ID value is extracted and appended to the chassis ID value to complete construction of the MSAP identifier.
 - c) The third TLV is extracted:
 - 1) If the extracted TLV type value does not match the Time To Live, Extension Request, or Extension TLV type, the LLDPDU is invalid:
 - i) The LLDPDU is discarded.
 - ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - iii) The rxType value is set equal to invalid.
 - iv) The procedure rxProcessFrame() is terminated.
 - 2) If the extracted TLV type value matches the Time To Live or Extension TLV type, and the adminStatus variable is enableTxOnly, the LLDPDU is discarded because the receive state machine is only enabled to receive Extension Request LLDPDUs.
 - i) The LLDPDU is discarded.
 - ii) The rxType value is set equal to invalid.
 - iii) The procedure rxProcessFrame() is terminated.
 - 3) If the extracted TLV type value matches the Time To Live TLV type, and the Time To Live TLV information string length is less than 2:
 - i) The LLDPDU is discarded.
 - ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - iii) The rxType value is set equal to invalid.
 - iv) The procedure rxProcessFrame() is terminated.

- 1 4) If the extracted TLV type value matches the Time To Live TLV type, and the Time To Live
2 TLV information string length is greater than or equal to 2:
 - 3 i) The first two octets of the TLV information string is extracted and rxTTL is set to this
4 value.
 - 5 ii) If rxTTL equals zero, the rxType value is set equal to shutdown and the procedure
6 rxProcessFrame() is terminated.
 - 7 iii) If rxTTL is non-zero, the rxType value is set to normal.
- 8 5) If the extracted TLV type value matches the Extension Request TLV type, XLLDP is
9 supported, and the received MSAP identifier and Scope MAC Address match the MSAP and
10 Scope MAC Address of this LLDP agent:
 - 11 i) The rxType value is set equal to xreq.
 - 12 ii) The procedure rxProcessFrame() is terminated.
- 13 6) If the extracted TLV type value matches the Extension Request TLV type, and either XLLDP is
14 not supported or the received MSAP identifier and Scope MAC Address combination does not
15 match the MSAP and Scope MAC Address combination of this LLDP agent:
 - 16 i) The LLDPDU is discarded.
 - 17 ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both
18 incremented.
 - 19 iii) The rxType value is set equal to invalid.
 - 20 iv) The procedure rxProcessFrame() is terminated.
- 21 7) If the extracted TLV type value matches the Extension TLV type, XLLDP is supported, and an
22 extended receive state machine exists for the neighbor identified by the received MSAP and
23 Scope MAC Address:
 - 24 i) The rxType value is set equal to xpdu.
- 25 8) If the extracted TLV type value matches the Extension TLV type, and either XLLDP is not
26 supported or an extended receive state machine does not exist for the neighbor identified by the
27 received MSAP and Scope MAC Address:
 - 28 i) The LLDPDU is discarded.
 - 29 ii) The statsFramesDiscardedTotal and statsFramesInErrorsTotal counters are both
30 incremented.
 - 31 iii) The rxType value is set equal to invalid.
 - 32 iv) The procedure rxProcessFrame() is terminated.

33 Each of the remaining TLV information elements are decoded in succession as required by their particular
34 TLV format definitions:

- 35 d) If the TLV type value equals 0, the TLV is the End Of LLDPDU TLV, and the procedure
36 rxProcessFrame() is terminated.
- 37 e) If the TLV_type_value is less than 127 and is recognized by the LLDP implementation, it is
38 validated according to the general rules for all TLVs defined in 9.2.8.7.2 as well as any specific rules
39 defined for the particular TLVs defined in 8.5.
- 40 f) If the TLV_type_value is less than 127 and is not recognized by the LLDP implementation, it may
41 be a basic TLV from a later LLDP version. The statsTLVsUnrecognizedTotal counter is
42 incremented, and the TLV is assumed to be validated.
- 43 g) If the implementation supports XLLDP and the TLV type value matches the Manifest TLV type:
 - 44 1) If the MSAP identifier in the current LLDPDU does not match an MSAP identifier in the
45 LLDP remote systems MIB, the total MIB entry size is extracted from the Manifest TLV. If
46 there is insufficient space in the LLDP remote systems MIB to store the neighbor information,

- 1 and the policy is to discard the current LLDPDU rather than delete information for other
2 MSAPs:
- 3 i) The flag variable `tooManyNeighbors` is set to `TRUE`.
- 4 ii) If the `rxTTL` value in the LLDPDU is greater than the value of the
5 `tooManyNeighborsTimer`, the `tooManyNeighborsTimer` is set to the `rxTTL` value.
- 6 iii) The `statsFrameDiscardedTotal` counter is incremented.
- 7 iv) The `rxType` value is changed to `invalid`.
- 8 2) If the MSAP identifier in the current LLDPDU matches an MSAP identifier in the LLDP
9 remote systems MIB, or there is sufficient space in the LLDP remote systems MIB to store the
10 new neighbor information, then:
- 11 i) The `rxType` value is changed to `manifest`.
- 12 ii) If an extended receive state machine does not already exist for the MSAP identifier in the
13 current LLDPDU, an extended receive state machine for that MSAP identifier is created
14 and its `createExtRx` variable is set to `TRUE`.
- 15 h) If the TLV type value is 127, the TLV is an Organizationally Specific TLV:
- 16 1) If the TLV's `organizationally unique identifier` and organizationally defined subtype are
17 recognized, the TLV is validated according to the general rules for all TLVs defined in 9.2.8.7.2
18 as well as the general rules for Organizationally Specific TLVs defined in 9.2.8.7.3 plus any
19 specific rules defined for the particular TLV.
- 20 2) If the TLV's `organizationally unique identifier` and/or organizationally defined subtype are not
21 recognized, the `statsTLVsUnrecognizedTotal` counter is incremented, and the TLV is assumed
22 to be validated.
- 23 i) If the end of the LLDPDU has been reached the procedure `rxProcessFrame()` is terminated.

24 9.2.8.7.2 General validation rules for all TLVs

25 The value in the TLV information string length field is the value used to validate the TLV and to indicate the
26 location of the next TLV in the LLDPDU.

27 All TLVs are subject to the general validation rules listed in this subclause as well as any specific usage rules
28 defined for the particular TLV (for example, see systems capabilities TLV usage rules in 8.5.8.3):

- 29 a) If the LLDPDU contains more than one Chassis ID TLV, Port ID TLV, or Time To Live TLV:
 - 30 1) The LLDPDU is discarded.
 - 31 2) The `statsFramesDiscardedTotal` and `statsFramesInErrorsTotal` counters are both incremented.
 - 32 3) The `rxType` value is set equal to `invalid`.
 - 33 4) The procedure `rxProcessFrame()` is terminated.
- 34 b) If the implementation supports XLLDP and the LLDPDU contains more than one Manifest TLV,
35 Extension Request TLV, or Extension Identifier TLV:
 - 36 1) The LLDPDU is discarded.
 - 37 2) The `statsFramesDiscardedTotal` and `statsFramesInErrorsTotal` counters are both incremented.
 - 38 3) The `rxType` value is set equal to `invalid`.
 - 39 4) The procedure `rxProcessFrame()` is terminated.
- 40 c) If the TLV information string length value is not greater than or equal to the sum of the lengths of all
41 fields contained in the TLV information string:
 - 42 1) The LLDPDU is discarded.
 - 43 2) The `statsFramesDiscardedTotal` and `statsFramesInErrorsTotal` counters are both incremented.
 - 44 3) The `rxType` value is set equal to `invalid`.
 - 45 4) The procedure `rxProcessFrame()` is terminated.
- 46 d) If any TLV contains an error condition specified for that particular TLV:

- 1) The TLV is discarded.
 - 2) The statsTLVsDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - 3) The location of the next TLV is based on the TLV information string length value of the current TLV.
- e) With the exception of the TTL TLV for which specific rules apply (see 9.2.8.7.1), if the length of any field of a TLV is outside the length range specified for that particular TLV (for example, the TLV information string length is greater than the maximum permitted length for the TLV information string as specified for that TLV):
- 1) The TLV is discarded.
 - 2) The statsTLVsDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
 - 3) The location of the next TLV is based on the TLV information string length value of the current TLV.
- f) If any TLV extends past the physical end of the frame:
- 1) The TLV is discarded.
 - 2) The statsTLVsDiscardedTotal and statsFramesInErrorsTotal counters are both incremented.
- g) Any information following an End Of LLDPDU TLV is ignored.
- NOTE—Usage rules for individual TLVs allow some TLVs to appear more than once in an LLDPDU. Duplicate TLVs result in any one of the values being placed in the MIB, can cause the discard stats to increment, and can cause the change marker for the MIB entry to change if any of the TLV copies change the value even if the value finally recorded is unchanged. The only thing guaranteed is that the MIB value is set to one (unspecified) of the TLV values, and if that value is different to what was previously in the MIB then the change marker is set.

9.2.8.7.3 General validation rules for all Organizationally Specific TLVs

If an Organizationally Specific TLV is not recognized by the receiving LLDP agent, the content of the TLV can be ignored but the TLV is stored in the LLDP remote systems MIB for possible retrieval by network management using the basic Organizationally Specific TLV format (see 8.6) and the StatsTLVsUnrecognizedTotal counter is incremented. If more than one unrecognized Organizationally Specific TLV is received with the same OUI and organizationally defined subtype, but with identifiable differences in the organizationally defined information strings, all copies are assigned a temporary identification index and stored.

9.2.8.8 somethingChangedLocal()

This procedure is called to indicate that the status/value of one or more of the selected objects in the LLDP local system MIB has changed, and that there is therefore a need to transmit new information. It is used to notify the managers of the PTOPO and other optional MIBs that something has changed in the LLDP local systems MIB.

This procedure also sets the value of the variable localChange (see 9.2.5.2) to signal the need for the LLDP agent(s) to transmit the new information, and copies the value of the variable txFastInit (see 9.2.5.20) to the variable txFast (see 9.2.5.19) to ensure that the new values will be seen by the neighbors on the port.

It is assumed that the system provides the ability for any processes that need to receive such indications to register with this procedure so that appropriate signals can be received by those processes when the procedure is called.

1 9.2.8.9 somethingChangedRemote()

2 This procedure is called after all the information associated with the particular MSAP identifier has been
3 updated in the LLDP remote systems MIB. The procedure call serves as an indication to the managers of the
4 PTOPO and other optional MIBs that one or more of the following conditions is true:

- 5 a) That the status/value of one or more objects in the LLDP remote systems MIB associated with the
6 particular MSAP identifier has changed.
- 7 b) That an incoming LLDPDU contained an MSAP identifier requiring creation of a new MIB
8 structure in the LLDP remote systems MIB to receive the information in that LLDPDU.
- 9 c) That information in the LLDP remote systems MIB associated with the MSAP identifier has been
10 deleted.

11 It is assumed that the system provides the ability for any processes that need to receive such indications to
12 register with this procedure so that appropriate signals can be received by those processes when the
13 procedure is called.

14 9.2.8.10 txAddCredit()

15 This procedure increments the value of the txCredit variable (9.2.5.17) by one if the current value of the
16 txCredit variable is less than the value of txCreditMax (9.2.5.18).

17 9.2.8.11 txFrame()

18 The txFrame() procedure makes use of the services of LLC to transmit the LLDPDU constructed by either
19 the mibConstrInfoLLDPDU(), mibConstrShutdownLLDPDU(), constructXreqPDU(), or rxProcessXreq()
20 procedures (9.2.8.2, 9.2.8.3, 9.2.8.16, 9.2.8.18). For Normal LLDPDUs, the destination MAC address used
21 is the MAC address associated with the instance of the LLDP agent (see 7.1.1 and 9.2.5.3). For Extension
22 Request and Extension LLDPDUs, the destination MAC address used is the Return MAC Address in the
23 received Manifest TLV or Extension Request TLV respectively (see 8.5a.1.2 and 8.5a.2.2). The source MAC
24 address used is the individual MAC address of the transmitting port. The EtherType used is the LLDP
25 EtherType identified in Table 7-3.

26 If the frame conveys a Manifest LLDPDU, the sentManifest variable is set to TRUE; otherwise the
27 sentManifest variable is set to FALSE.

28 After the frame has been transmitted, the statsFramesOutTotal counter (9.2.7.5) is incremented.

29 9.2.8.12 txInitializeLLDP()

30 The txInitializeLLDP() procedure initializes the LLDP transmit module. It performs the following tasks:

- 31 a) If applicable, either the non-volatile configuration for the LLDP local system MIB are retrieved or
32 the appropriate default values are assigned to all LLDP configuration variables.
- 33 b) The internal (implementation specific) data structures are initialized with appropriate local physical
34 topology information.
- 35 c) The setManifest variable is set to FALSE.
- 36 d) System default values are set for the following timing parameters:
 - 37 1) reinitDelay (9.2.5.9)
 - 38 2) msgTxHold (9.2.5.5)
 - 39 3) msgTxInterval (9.2.5.6)

1 4) msgFastTx (9.2.5.4)

2 NOTE—It is recommended to introduce a degree of stagger into starting the LLDP local agent initialization in
3 multi-port implementations so that the timing of LLDP frame transmission cycles can be distributed among system
4 ports, and distributed among supported MAC addresses on each port. The method of accomplishing the stagger is an
5 implementation issue and is beyond the scope of this standard.

6 **9.2.8.13 rxProcessManifest()**

7 The rxProcessManifest() procedure performs the following tasks:

- 8 a) All TLVs received in the Manifest LLDPDU and passed from the receive state machine are retained
9 in temporary storage associated with this MSAP identifier, replacing any TLVs retained from a
10 previous Manifest LLDPDU.
11 b) If there is no current entry in the LLDP remote systems MIB for this MSAP identifier, or if the
12 current entry does not contain a Manifest TLV, all XPDU Descriptors in the new Manifest TLV are
13 assigned a status of “update”.
14 c) If there is a current manifest in the LLDP remote systems MIB for this MSAP identifier, each XPDU
15 Descriptor in the new manifest is compared to the XPDU Descriptor in the current manifest. Any
16 XPDU Descriptors in the new manifest that match an XPDU Descriptor in the current manifest are
17 assigned a status of “current”. Any XPDU Descriptors in the new manifest that do not have a
18 matching XPDU Descriptor in the current manifest are assigned a status of “update”.
19 d) If there are any XPDUs in temporary storage associated with this MSAP identifier that match an
20 XPDU Descriptor with a status of “update” in the new manifest, those XPDUs are retained and the
21 status of the corresponding XPDU Descriptors is changed to “new”. Any other XPDUs in temporary
22 storage are discarded.

23 NOTE—Usage rules for the Manifest TLV specify that each XPDU Descriptor has a unique XPDU Number. There is no
24 requirement for the LLDP agent receiving a Manifest TLV to check for duplicate XPDU Numbers, and the response of
25 the receiving LLDP agent to duplicated XPDU Numbers is unspecified and implementation dependent.

26 **9.2.8.14 rxProcessXPDU()**

27 The rxProcessXPDU() procedure performs the following tasks:

- 28 a) The statsFramesInTotal counter for the port is incremented.
29 b) The 32-bit XPDU Check value of the XPDU is calculated. If the XPDU Number, Sequence, and
30 Check do not match an XPDU Descriptor in the new manifest, the statsFramesDiscardedTotal and
31 statsFramesInErrorsTotal counters are both incremented and the procedure rxProcessXPDU() is
32 terminated.
33 c) If the XPDU Number, Sequence, and Check match an XPDU Descriptor in the new manifest, the
34 status of that XPDU Descriptor is changed to “new”.
35 d) Validates the TLVs contained in the LLDPDU as defined in 9.2.8.7.
36 e) Retains all validated TLVs in temporary local storage associated with this XPDU Descriptor.
37 f) If there are any XPDU Descriptors in the new manifest with a status of “requested” or “retried”, the
38 variable xreqComplete is set to FALSE. Otherwise the variable xreqComplete is set to TRUE.

39 **9.2.8.15 checkManifest()**

40 The checkManifest() procedure reviews the status of all XPDU Descriptors in the new manifest. If the status
41 of one or more XPDU Descriptors is “update”, “requested”, or “retried”, the manifestComplete variable is
42 set to FALSE. Otherwise the manifestComplete variable is set to TRUE and rxTimer is set to rxTTL.

43 **9.2.8.16 constructXreqPDU()**

44 The constructXreqPDU() procedure performs the following tasks:

- 1 a) If there are any XPDU Descriptors with a status of “requested” in the new manifest, these XPDU
2 Descriptors are selected to be included in the Extension Request TLV, the status of these XPDU
3 Descriptors is changed to “retried”, and rxTimer is set to rxTTL.
- 4 b) If there are no XPDU Descriptors with a status of “requested” or “retried” in the new manifest, one
5 or more XPDU Descriptors with a status of “update” are selected to be included in the request. The
6 status of these XPDU Descriptors is changed to “requested”, and rxTimer is set to the value of
7 rxTTL divided by 32 and rounded up to the nearest integer.
- 8 c) Constructs an Extension Request LLDPDU (8.2) including an Extension Request TLV, structured
9 according to the specification in 8.5a.2, containing the selected XPDU Descriptors.

10 The LLDP agent generating the Extension Request LLDPDU can pace the rate at which Extension
11 LLDPDUs arrive by adjusting the number of XPDU Descriptors in a single Extension Request TLV and the
12 timing of the Extension Request LLDPDU transmission. This pacing is implementation dependent and
13 beyond the scope of this standard. An LLDP agent only transmits an Extension Request LLDPDU when it is
14 prepared (e.g., has sufficient buffer resources) to receive all requested Extension LLDPDUs.

15 **9.2.8.17 checkRxTimer()**

16 The checkRxTimer() procedure reviews the status of all XPDU Descriptors in the new manifest. If the value
17 of rxTimer is zero and there are any XPDU Descriptors with a status of “requested”, then this is a timeout for
18 an Extension Request LLDPDU and the rxTTLExpired variable is set to FALSE. If the value of rxTimer is
19 zero and there are no XPDU Descriptors with a status “requested”, then this is a timeout for the TLV
20 collection process and rxTTLExpired is set to TRUE.

21 **9.2.8.18 rxProcessXREQ()**

22 If the MSAP identifier and Scope MAC Address in the received Extension Request LLDPDU match this
23 LLDP agent, then for each XPDU Descriptor in the Extension Request TLV that matches one of the XPDU
24 Descriptors in the most recent Manifest TLV created by the LLDP manager and transmitted in a Normal
25 LLDPDU, an Extension LLDPDU (8.2) is constructed, including an Extension Identifier TLV structured
26 according to the specification in 8.5a.3, and the other TLVs selected by the LLDP manager, and transmits
27 the frame using the txFrame() procedure.

28 NOTE—When the Extension Request TLV includes more than one XPDU Descriptor, the rate at which Extension
29 LLDPDUs are transmitted is at the discretion of the transmitting agent and is implementation dependent.

9.2.9 Transmit state machine

The transmit state machine for an individual port and destination MAC address (see 7.1) shall implement the function specified by the state diagram contained in Figure 9-1 and the attendant definitions contained in 9.2.3 through 9.2.7.8.

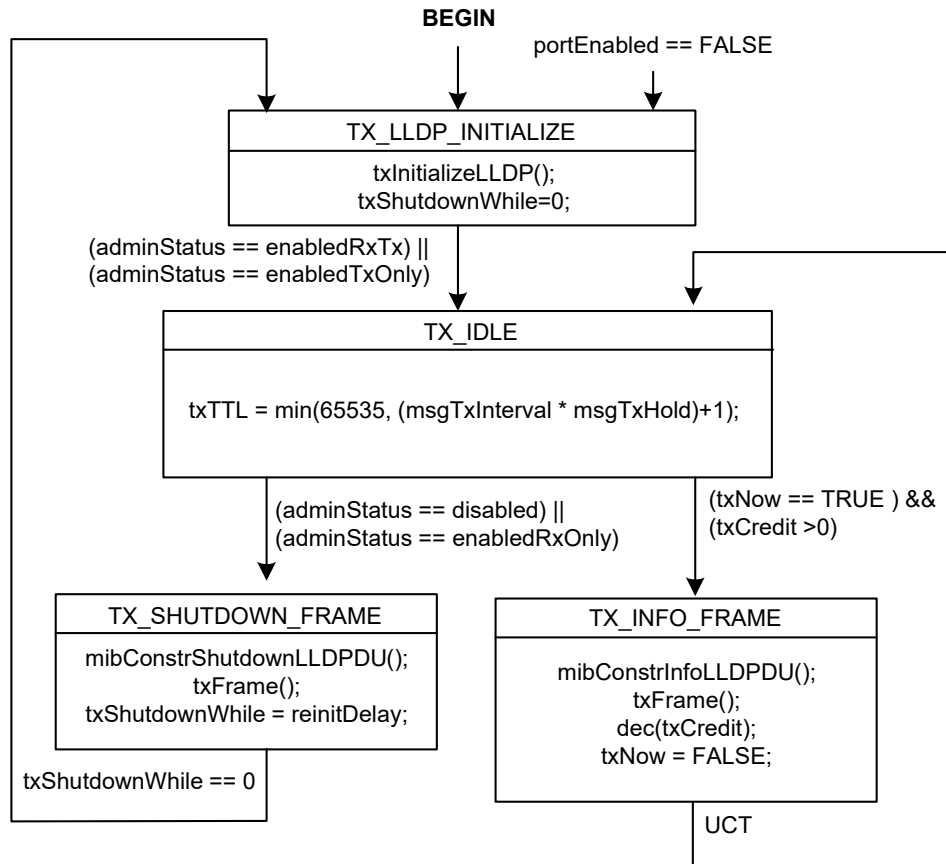


Figure 9-1—Transmit state machine

9.2.10 Receive state machine

The receive state machine for an individual port and destination MAC address (see 7.1) shall implement the function specified by the state diagram contained in Figure 9-2 and the attendant definitions contained in 9.2.2 through 9.2.8. Additionally, when an implementation supports Extended LLDP and a Manifest TLV is received in an LLDPDU, an extended receive state machine for the received chassis ID and port ID shall implement the function specified by the state diagram contained in Figure 9-3 and the attendant definitions contained in 9.2.2 through 9.2.8.

Because there can be multiple extended receive state machines, the receive state machine uses a notational convention to reference per-neighbor variables (9.2.6) associated with an extended receive state machine. The variable is prefaced by “MSAP.” to indicate a variable associated with the specific MSAP Identifier contained in the frame currently being processed by the receive state machine. The variable is prefaced with “any.” to indicate the logical OR of the variable from all extended state machines.

An extended receive state machine associated with a neighbor is dynamically instantiated, if it does not already exist, when a Normal LLDPDU is received that contains a Manifest TLV. It is destroyed, and any

- ¹ TLVs held in temporary storage associated with this state machine are discarded, when a Normal LLDPDU
- ² is received that does not contain a Manifest TLV, or by the assertion of BEGIN.

1

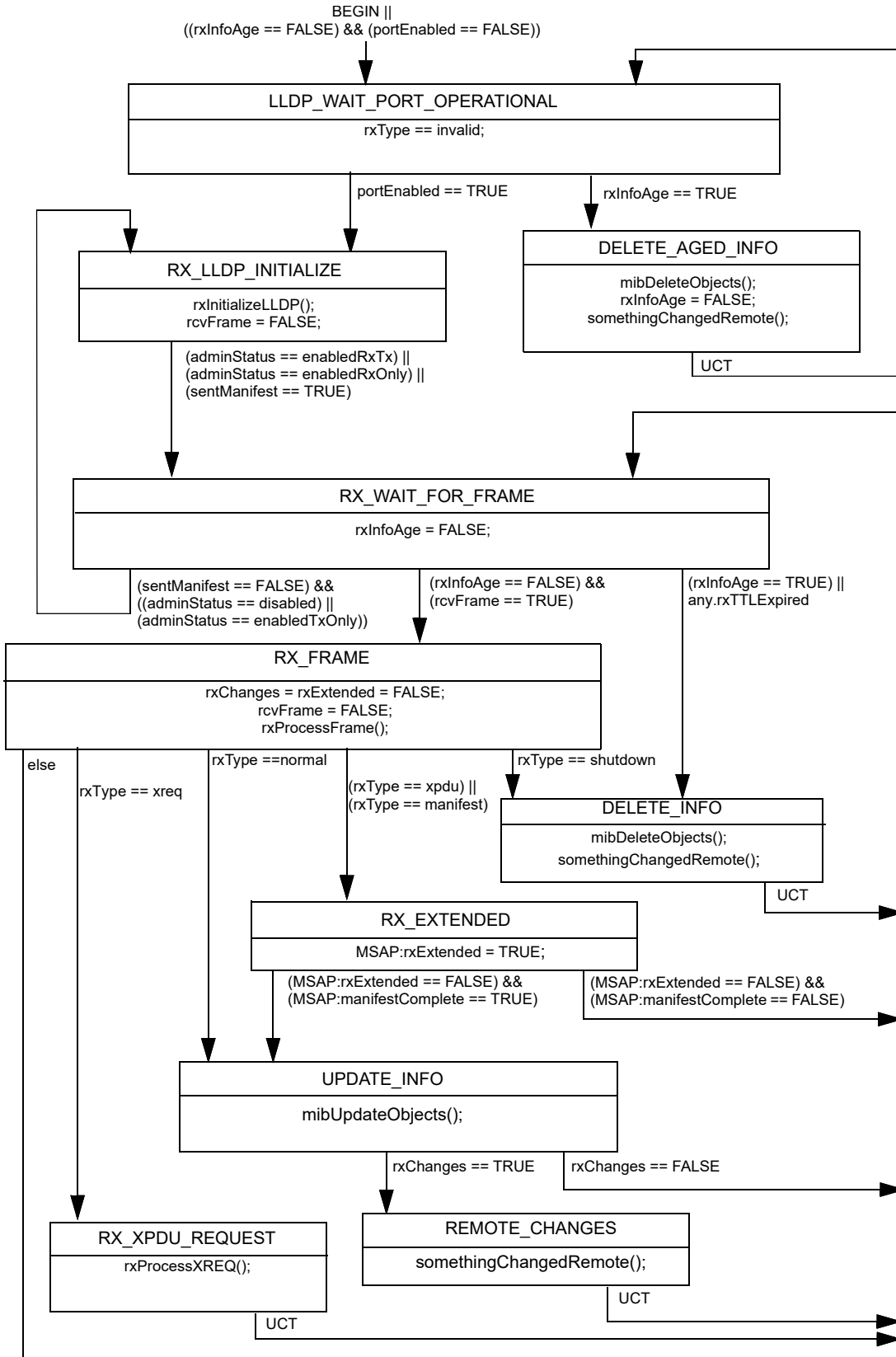


Figure 9-2— Receive state machine

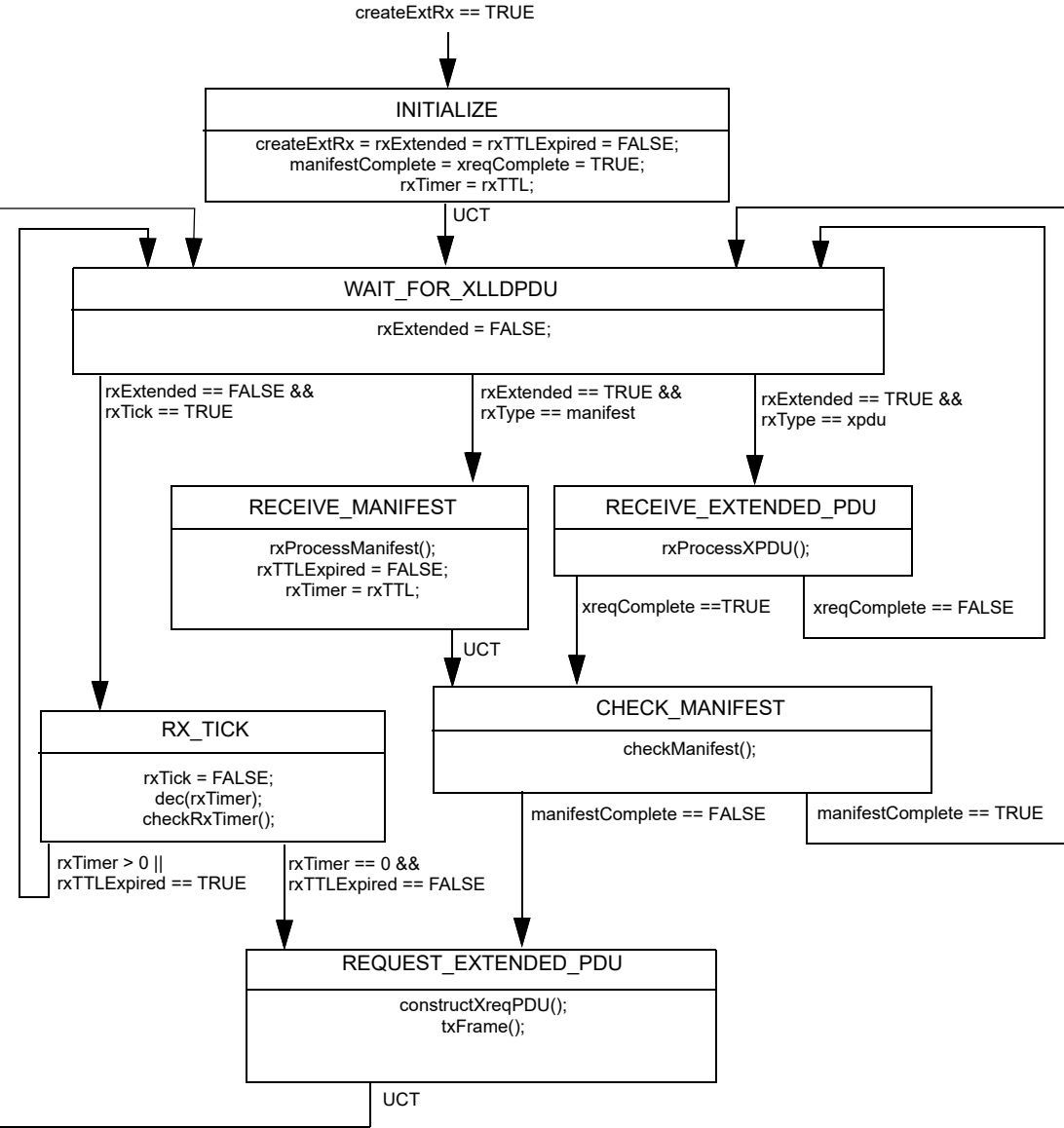


Figure 9-3— Extended receive state machine

2

3

4

1 **9.2.11 Transmit timer state machine**

2 The transmit timer state machine for an individual port and destination MAC address (see 7.1) shall
3 implement the function specified by the state diagram contained in Figure 9-4 and the attendant definitions
4 contained in 9.2.3 through 9.2.7.8.

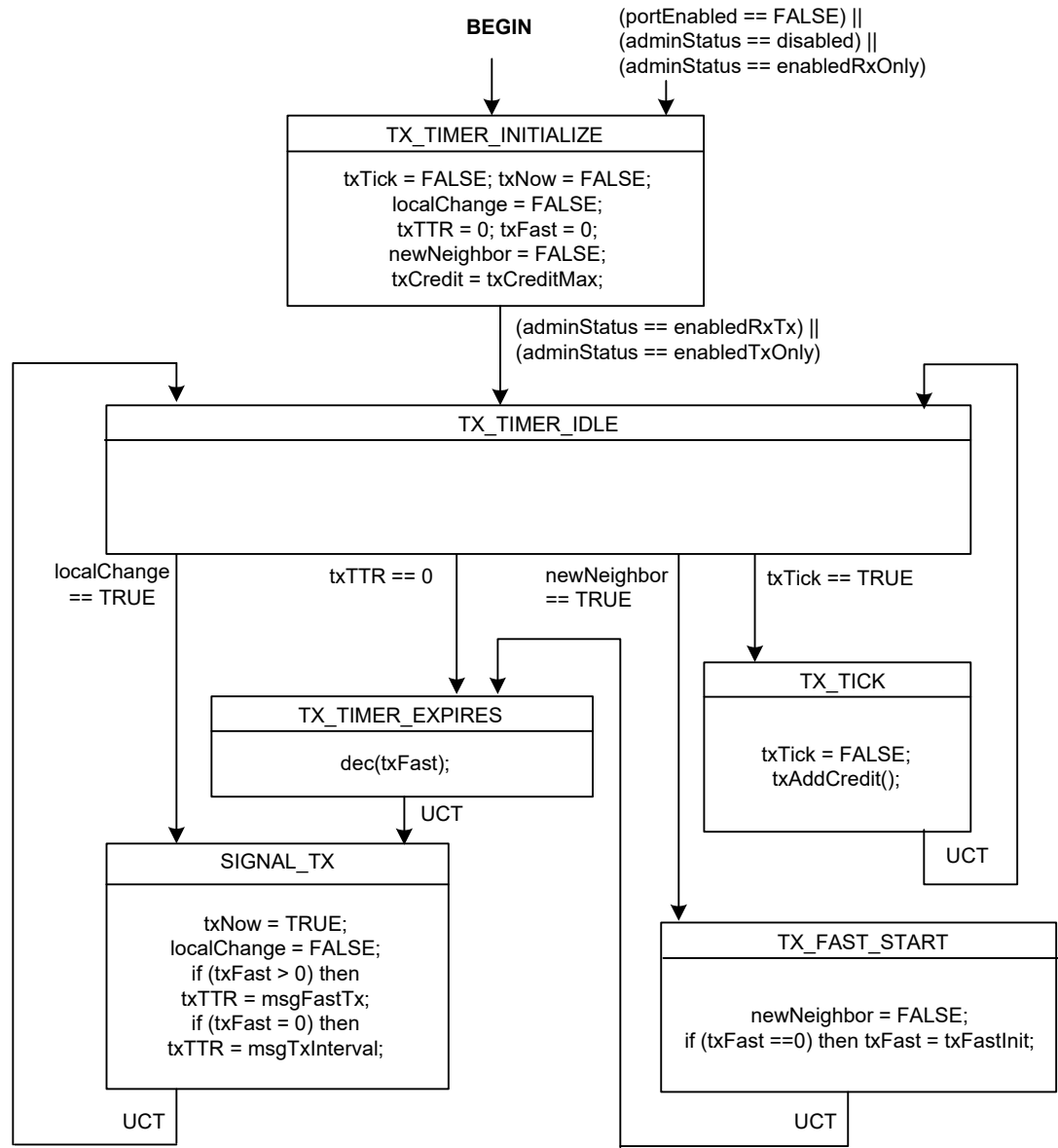


Figure 9-4—Transmit timer state machine

10. LLDP management

This clause defines the set of managed objects and their functionality that allow administrative configuration and monitoring of LLDP operation.

10.1 Data storage and retrieval

LLDP agents need to have a place to store both information about the local system and information they have received about remote systems. Data storage and retrieval capability to support the functionality defined in 7, 8, 9, and 10 shall be provided. No particular implementation is implied.

10.2 The LLDP management entity's responsibilities

The LLDP management entity has the following responsibilities:

- a) Starting the transmit and receive state machines.
- b) Providing a means for the network manager to select TLVs to be included in outgoing LLDPDUs.
- c) Extracting the necessary information from the LLDP local system MIB and constructing the individual TLVs selected for insertion into an outgoing LLDPDU.
- d) Prepending appropriate addressing and LLDP EtherType to the LLDPDU before submitting the MA_UNITDATA.request for frame transmission.
- e) Updating the LLDP remote systems MIB and monitoring for MIB object adds, deletions and value changes.
- f) Maintaining operational statistics.

10.2.1 Protocol initialization management

Protocol initialization consists of the following:

- a) Retrieving non-volatile configuration values for the LLDP local system MIB.
- b) Assigning default or management assigned values to LLDP configuration variables.
- c) Loading physical topology information into their associated LLDP local system MIB objects.

10.2.2 TLV selection management

TLV selection management consists of providing the network manager with the means to select which specific TLVs are enabled for inclusion in an LLDPDU. The following LLDP variables cross reference to LLDP local systems configuration MIB tables indicating which specific TLVs are enabled for the particular port(s) on the system. The specific port(s) through which each TLV is enabled for transmission may be set (or reset) by the network manager.

- a) **mibBasicTLVsTxEnable:** This variable lists the single-instance-use basic management TLVs, each with a bit map indicating the system ports through which the referenced TLV is enabled for transmission.
- b) **mibMgmtAddrInstanceTxEnable:** This variable lists the different management addresses (by subtype) that are defined for the system, each with a bit map indicating the system ports through which the particular management address/subtype TLV is enabled for transmission.

NOTE—Implementers of new TLVs should be aware that provision needs to be made to allow similar network management selection of new TLVs and that doing so requires linkage to the LLDP MIB, regardless of whether or not the TLV is part of the basic management set or part of an Organizationally Specific TLV set.

1 When XLLDP is not supported, the LLDP agent can only transmit the TLVs that fit within a single Normal
2 LLDPU. If more TLVs are selected for inclusion in the LLDPU than fit, some TLVs are not transmitted.
3 Which TLVs are not transmitted is implementation dependent and not determined by this standard.

4 When XLLDP is supported, each TLV selected for transmission is mapped to the Normal LLDPU or to an
5 Extension LLDPU. Each Extension LLDPU is identified by an XPDU Number. The mapping of TLVs to
6 XPDU is implementation dependent. The LLDP management entity maintains a list of which TLVs are
7 mapped to which XPDU Number, and uses this list to create a Manifest TLV for inclusion in a Manifest
8 LLDPU at the start of a transmission cycle. The addition, deletion, or modification of any TLV mapped to
9 Extension LLDPU since the last Manifest LLDPU was transmitted cause the XPDU Revision for that
10 Extension LLDPU to be incremented prior to creating a new Manifest TLV. Maintaining a consistent
11 mapping of TLVs to Extension LLDPU whenever possible reduces the number of Extension LLDPU
12 that need to be requested and transmitted during a transmission cycle.

13 10.2.3 Transmission management

14 Transmission management consists of the following:

- 15 a) Determining when a new transmission cycle is required, as signaled by the
16 somethingChangedLocal() procedure (see 9.2.8.8), or when an Extension LLDPU has been
17 requested by the rxProcessXREQ() procedure (see 9.2.7.18).
- 18 b) Extracting the appropriate LLDP local system MIB information for the three mandatory TLVs and
19 inserting them at the beginning of the LLDPU in proper format order.
- 20 c) Creating a Manifest TLV for inclusion in the Normal LLDPU that starts a transmission cycle if
21 XLLDP is supported and there are TLVs that are mapped to one or more Extension LLDPU.
- 22 d) Extracting the appropriate LLDP local system MIB information for the selected optional TLVs and
23 inserting them into the LLDPU.
- 24 e) Optionally, appending an End Of LLDPU TLV after the last optional TLV in the LLDPU.
- 25 f) Maintaining length control during LLDPU construction so that the TLVs do not exceed the
26 maximum length allowed for the LLDPU.
- 27 g) Submitting the frame for transmission.

28 10.2.4 Reception management

29 Reception management consists of the following:

- 30 a) Receiving and parsing the incoming LLDPU.
- 31 b) Validating, and checking the types of the first three TLVs to ensure that they are the required type
32 and in LLDPU format order.
- 33 c) Validating optional TLVs and extracting information values.
- 34 d) Checking whether or not the current LLDPU represents a new MSAP identifier.
- 35 e) Monitoring whether or not there is sufficient space available in the LLDP remote systems MIB for
36 all active neighbors.
- 37 f) Arbitrating which information is to be discarded in cases where insufficient space is available in the
38 LLDP remote systems MIB for all active neighbors.
- 39 g) Checking whether or not the received information represents a status or value change to the existing
40 MIB object.
- 41 h) Updating remote systems MIB objects as necessary.

42 When XLLDP is supported, the LLDP management entity associates each TLV that is stored in the remote
43 systems MIB with the XPDU Number of the Extension LLDPU in which it was received. (For
44 convenience an XPDU Number of zero can be used for TLVs received in a Normal LLDPU, since no valid

1 Extension LLDPDU has an XPDU Number equal to zero.) This association is necessary so that the
2 appropriate TLVs are updated or deleted when a new Extension LLDPDU with that XPDU Number is
3 received.

4 10.2.5 Performance management

5 Performance management consists of the following:

- 6 a) Monitoring the LLDP local system MIB update activities for status or value changes to selected
7 MIB objects and:
 - 8 1) Calling the somethingChangedLocal() procedure (see 9.2.8.8) whenever a status/value change
9 occurs.
 - 10 2) Initiating a new transmit cycle if txCredit > 0.
- 11 b) Monitoring the txTTR timer for countdown expiration and initiating a new transmit cycle if txCredit
12 > 0.
- 13 c) Monitoring the rxInfoTTL timer for countdown expiration and:
 - 14 1) Deleting the associated objects from the LLDP remote systems MIB whenever the timing
15 counter expires.
 - 16 2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.
- 17 d) Monitoring rxTTL in the incoming LLDPDUs for shutdown indication and:
 - 18 1) Deleting the associated objects from the LLDP remote systems MIB whenever rxTTL = 0.
 - 19 2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.
- 20 e) Monitoring the “tooManyNeighbors” variable to determine whether an existing neighbor’s
21 information needs to be deleted and:
 - 22 1) Deleting the objects associated with a selected MSAP identifier from the LLDP remote systems
23 MIB whenever existing information is selected to be deleted.
 - 24 2) Performing the procedure somethingChangedRemote() associated with the MSAP identifier.
- 25 f) Monitoring the tooManyNeighborsTimer to determine when there is not sufficient space available to
26 accommodate information from all active neighbors and setting the variable tooManyNeighbors
27 associated with the port to FALSE when the tooManyNeighborsTimer expires.
- 28 g) Notifying the managers of the PTOPO MIB and other optional MIB modules (see Figure 11-1)
29 whenever the procedures somethingChangedLocal() or somethingChangedRemote() are performed.
- 30 h) Maintaining operational statistics regarding the LLDP frames sent and received.

31 10.3 Managed objects

32 Managed objects model the semantics of management operations. Operations upon a managed object supply
33 information concerning, or facilitate control over, the process or entity associated with that managed object.

34 Management of LLDP is described in terms of the managed resources that are associated with individual
35 TLVs and that support frame transmission and reception.

36 10.4 Data types

37 This subclause specifies the semantics of operations independent of their encoding in management protocol.
38 The data types of the parameters of operations are defined for that specification.

39 The following data types are used:

- 40 a) Boolean.
- 41 b) Enumerated, for a collection of named values.

- 1 c) Unsigned, for all parameters specified as “number of” some quantity.
- 2 d) MAC address.
- 3 e) Time interval, an Unsigned value representing a positive integral number of seconds for all time out
- 4 parameters.
- 5 f) Counter, for all parameters specified as a “count” of some quantity (all counters increment and wrap
- 6 with a modulus of 2 to the power 32).

7 10.5 LLDP variables

8 LLDP managed objects fall into the following categories:

- 9 a) Status/control variables necessary for operation of the protocol.
- 10 b) Variables that accumulate operational statistics.
- 11 c) Variables that are required by the particular TLVs.

12 10.5.1 LLDP operational status and control

- 13 a) Global parameters and variables:
 - 14 1) **adminStatus**: The authority that controls whether or not a local LLDP agent is enabled for
 - 15 transmit and receive, transmit only, or receive only; or is disabled (9.2.5.1).
- 16 b) Transmit state machine parameters and variables:
 - 17 1) **msgTxHold**: A multiplier on msgTxInterval used to compute the TTL value of txTTL (9.2.5.5,
 - 18 9.2.5.23).
 - 19 2) **msgTxInterval**: The interval between successive transmit cycles in normal transmission mode
 - 20 (9.2.5.6).
 - 21 3) **reinitDelay**: The delay after adminStatus becomes ‘disabled’ before re-initialization is
 - 22 attempted (9.2.5.9).
 - 23 4) **txCreditMax**: The maximum value of transmission credit (9.2.5.18).
 - 24 5) **txFastInit**: The interval between successive LLDPDU transmissions in fast transmission mode
 - 25 (9.2.5.20).
- 26 c) Receive state machine parameters and variables:
 - 27 1) **remoteChanges**: An indication that the status/value of one or more selected objects in the
 - 28 LLDP remote systems MIB has changed or that all objects associated with a particular MSAP
 - 29 identifier have been deleted (9.2.5.10).
 - 30 2) **tooManyNeighbors**: An indication that there is insufficient space in the LLDP remote systems
 - 31 MIB to store information from all active neighbors (9.2.5.16).

32 10.5.2 LLDP operational statistics counters

33 The following counters provide operational statistics on a per port basis:

- 34 a) Transmission counters:
 - 35 1) **statsFramesOutTotal**: A count of all LLDP frames transmitted through the port (9.2.7.5).
 - 36 2) **statsLengthErrorsTotal**: A count of all LLDP length errors detected when constructing
 - 37 LLDPDU frames for transmission through the port (9.2.8.2).
- 38 b) Reception counters:
 - 39 1) **statsAgeoutsTotal**: A count of the times that a neighbor’s information is deleted from the
 - 40 LLDP remote systems MIB because of rxInfoTTL timer expiration (9.2.7.1).
 - 41 2) **statsFramesDiscardedTotal**: A count of all LLDPDUs received and then discarded (9.2.7.2).
 - 42 3) **statsFramesInErrorsTotal**: A count of all LLDPDUs received at the port with one or more
 - 43 detectable errors 9.2.7.3).

- 1 4) **statsFramesInTotal:** A count of all LLDP frames received at the port (9.2.7.4).
- 2 5) **statsTlvsDiscardedTotal:** A count of all TLVs received at the port and discarded for any
- 3 reason. (9.2.7.6)
- 4 6) **statsTLVsUnrecognizedTotal:** This counter provides a count of all TLVs not recognized by
- 5 the receiving LLDP local agent (9.2.7.7).

6 **10.5.3 TLV required variables**

7 Variables in this category are defined by the requirements of the particular TLVs. They are maintained with
8 reference to the local system in the LLDP local system MIB; and with reference to the remote system, in the
9 LLDP remote systems MIB. TLV variables are outputs from the local LLDP agent and inputs to the remote
10 LLDP agent.

11 Variables that pertain to basic set TLVs are listed in the following subclauses.

12 **10.5.3.1 Chassis ID TLV objects**

- 13 a) **chassis ID subtype:** The type of identifier used for the chassis (see 8.5.2.2).
- 14 b) **chassis ID:** The identification assigned to the chassis containing the port (see 8.5.2.3).

15 **10.5.3.2 Port ID TLV objects**

- 16 a) **port ID subtype:** The type of identifier used for the port (see 8.5.3.2).
- 17 b) **port ID:** The identification assigned to the port (see 8.5.3.3).

18 **10.5.3.3 Port description TLV object**

- 19 a) **port description:** The port's description (see 8.5.5.2).

20 **10.5.3.4 System name TLV object**

- 21 a) **system name:** The system's assigned name (see 8.5.6.2).

22 **10.5.3.5 System description TLV object**

- 23 a) **system description:** The system's description (see 8.5.7.2).

24 **10.5.3.6 System capabilities TLV objects**

- 25 a) **system capabilities:** The primary capabilities of the system (see 8.5.8.1).
- 26 b) **enabled capabilities:** The system's enabled capabilities (see 8.5.8.2).

27 **10.5.3.7 Management address TLV objects**

- 28 a) **management address length:** The length of the management address (see 8.5.9.2).
- 29 b) **management address subtype:** The management address type (see 8.5.9.3).
- 30 c) **management address:** The management address (see 8.5.9.4).
- 31 d) **interface numbering subtype:** The interface type (see 8.5.9.5).
- 32 e) **interface number:** The interface number (see 8.5.9.6).
- 33 f) **OID length:** The object identifier length (see 8.5.9.7).
- 34 g) **OID:** The object identifier (see 8.5.9.8).

11. LLDP MIB definitions

This clause defines the LLDP basic MIB for use with SNMP in TCP/IP based internets. In particular, it defines objects for managing the operation of LLDP based on the specifications of 7, 8, 9, and 10.

11.1 Internet Standard Management Framework

LLDP MIBs are designed to operate in a manner consistent with the principles of the Internet Standard Management Framework, which describes the separation of a data modeling language (for example, SMIV2) from content-specific data models (for example, the LLDP remote systems MIB), and from messages and protocol operations used to manipulate the data (for example, SNMPv3).

For a detailed overview of the documents that describe the current Internet Standard Management Framework, please refer to section 7 of IETF RFC 3410.

Managed objects are accessed via a virtual information store, termed the Management Information Base or MIB. MIB objects are generally accessed through the Simple Network Management Protocol (SNMP). Objects in the MIB are defined using the mechanisms defined in the Structure of Management Information (SMI). This clause specifies a MIB module that is compliant to the SMIV2, which is described in IETF RFC 2578 (STD 58) [B3], IETF RFC 2579 (STD 58) [B4], and IETF RFC 2580 (STD 58) [B5].

11.2 Structure of the LLDP MIB

The LLDP MIB consists of two types of MIB modules, the mandatory basic MIB defined in this clause and from zero to n optional organizationally specific MIB extensions such as the IEEE 802.1 MIB in Annex D of IEEE Std 802.1Q-2014 and the IEEE 802.3 MIB in Clause 79 of IEEE Std 802.3-2012.

Each MIB module is divided into two major sections as shown in Figure 11-1 to allow selective MIB support for the particular operating mode (transmit only, receive only, or both transmit and receive) being implemented.

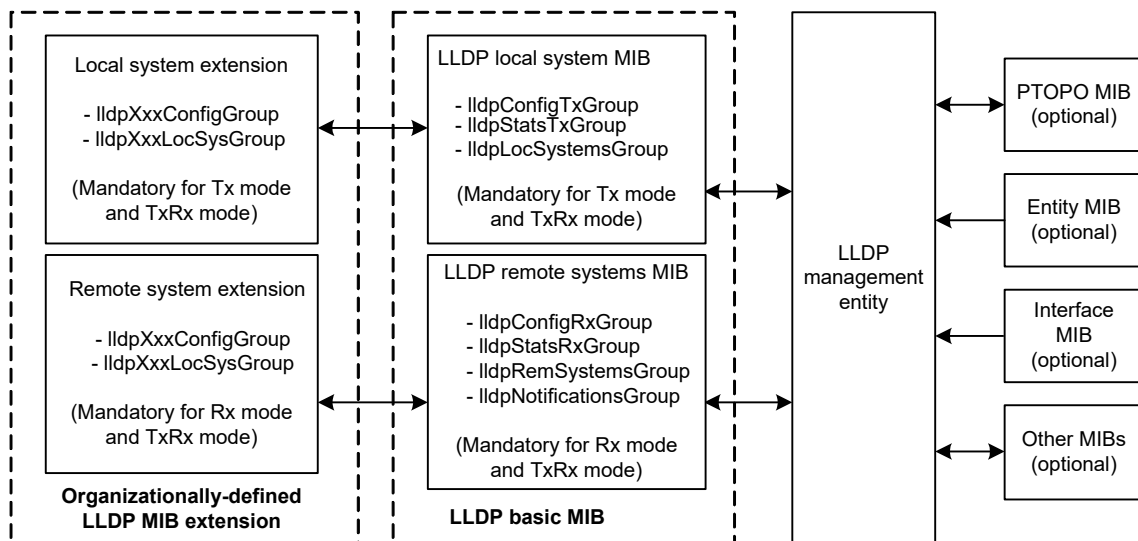


Figure 11-1—LLDP MIB block diagram

¹ Table 11-1 summarizes the particular object groups that are required for each operating mode. The basic MIB
² shall comply with the MIB conformance section for the particular operating mode (Rx, Tx, or TxRx mode)
³ being supported.

Table 11-1—MIB object groups and operating mode applicability

MIB group	Rx mode	Tx mode	TxRx mode
lldpV2ConfigGroup			
lldpV2ConfigRxGroup	M ^a	—	M
lldpV2ConfigTxGroup	—	M	M
lldpV2StatsRxGroup	M	—	M
lldpV2StatsTxGroup	—	M	M
lldpV2LocSysGroup	—	M	M
lldpV2RemSysGroup	M	—	M
lldpV2NotificationsGroup	M	—	M
ifGeneralInformationGroup	M	M	M

^aM=Mandatory

⁴ Table 11-2 shows the structure of the MIB and the relationship of the MIB objects to the LLDP operational
⁵ status/control variables, LLDP statistics variables, and TLV variables.

Table 11-2—LLDP MIB structure and object cross reference

MIB table	MIB object	LLDP reference
<i>LLDP Configuration group</i>		
	lldpV2MessageTxInterval	msgTxInterval, 9.2.5.6
	lldpV2MessageTxHoldMultiplier	msgTxHold, 9.2.5.5
	lldpV2ReinitDelay	reinitDelay, 9.2.5.9
	lldpV2NotificationInterval	msgTxInterval, 9.2.5.6
	lldpV2TxCreditMax	txCreditMax, 9.2.5.18
	lldpV2MessageFastTx	msgFastTx, 9.2.5.4
	lldpV2TxFastInit	txFastInit, 9.2.5.20

Table 11-2—LLDP MIB structure and object cross reference (continued)

MIB table	MIB object	LLDP reference
lldpV2PortConfigTableV2		
	lldpV2PortConfigIfIndexV2	(Table index)
	lldpV2PortConfigDestAddressIndexV2	(Table index)
	lldpV2PortConfigAdminStatusV2	adminStatus, 9.2.5.1
	lldpV2PortMessageTxInterval	msgTxInterval, 9.2.5.6
	lldpV2PortMessageTxHoldMultiplier	msgTxHold, 9.2.5.5
	lldpV2PortReinitDelay	reinitDelay, 9.2.5.9
	lldpV2PortNotificationInterval	msgTxInterval, 9.2.5.6
	lldpV2PortTxCreditMax	txCreditMax, 9.2.5.18
	lldpV2PortMessageFastTx	msgFastTx, 9.2.5.4
	lldpV2PortTxFastInit	txFastInit, 9.2.5.20
	lldpV2PortConfigNotificationEnableV2	—
	lldpV2PortConfigTLVsTxEnableV2	9.1.2.1
lldpV2DestAddressTable		
	lldpV2AddressTableIndex	(Table index)
	lldpV2DestMacAddress	(Table index)
lldpV2ManAddrConfigTxPortsTable		
	lldpV2ManAddrConfigIfIndex	(Table index)
	lldpV2ManAddrConfigDestAddressIndex	(Table index)
	lldpV2ManAddrConfigLocManAddrSubtype	8.5.9.3 (Table index)
	lldpV2ManAddrConfigLocManAddr	8.5.9.4 (Table index)
	lldpV2ManAddrConfigTxEnable	9.1.2.1
	lldpV2ManAddrConfigRowStatus	—
<i>LLDP Statistics group</i>		
	lldpV2StatsRemTablesLastChangeTime	—
	lldpV2StatsRemTablesInserts	—
	lldpV2StatsRemTablesDeletes	—
	lldpV2StatsRemTablesDrops	—
	lldpV2StatsRemTablesAgeouts	—

Table 11-2—LLDP MIB structure and object cross reference (continued)

MIB table	MIB object	LLDP reference
lldpV2StatsTxPortTable		
	lldpV2StatsTxIfIndex	(Table index)
	lldpV2StatsTxDestMACAddress	(Table index)
	lldpV2StatsTxPortFramesTotal	statsFramesOutTotal, 9.2.7.5
	lldpV2StatsTxLLDPDULengthErrors	lldpduLengthErrors, 9.2.7.8
lldpV2StatsRxPortTable		
	lldpV2StatsRxDestIfIndex	(Table index)
	lldpV2StatsRxDestMACAddress	(Table index)
	lldpV2StatsRxPortFramesDiscardedTotal	statsFramesDiscardedTotal, 9.2.7.2
	lldpV2StatsRxPortFramesErrors	statsFramesInErrorsTotal, 9.2.7.3
	lldpV2StatsRxPortFramesTotal	statsFramesInTotal, 9.2.7.4
	lldpV2StatsRxPortTLVsDiscardedTotal	statsTLVsDiscardedTotal, 9.2.7.6
	lldpV2StatsRxPortTLVsUnrecognizedTotal	statsTLVsUnrecognizedTotal, 9.2.7.7
	lldpV2StatsRxPortAgeoutsTotal	statsAgeoutsTotal, 9.2.7.1
<i>Local System Data group</i>		
	lldpV2LocChassisIdSubtype	chassis ID subtype, 8.5.2.2
	lldpV2LocChassisId	chassis ID, 8.5.2.3
	lldpV2LocSysName	system name, 8.5.6.2
	lldpV2LocSysDesc	system description, 8.5.7.2
	lldpV2LocSysCapSupported	system capabilities, 8.5.8.1
	lldpV2LocSysCapEnabled	enabled capabilities, 8.5.8.2
lldpV2LocPortTable		
	lldpV2LocPortIfIndex	(Table index)
	lldpV2LocPortIdSubtype	port ID subtype, 8.5.3.2
	lldpV2LocPortId	port ID, 8.5.3.3
	lldpV2LocPortDesc	port description, 8.5.5.2
lldpV2LocManAddrTable		
	lldpV2LocManAddrSubtype	management address subtype, 8.5.9.3 (Table index)
	lldpV2LocManAddr	management address, 8.5.9.4 (Table index)
	lldpV2LocManAddrLen	management address string length, 8.5.9.2
	lldpV2LocManAddrIfSubtype	interface numbering subtype, 8.5.9.5

Table 11-2—LLDP MIB structure and object cross reference (continued)

MIB table	MIB object	LLDP reference
	lldpV2LocManAddrIfId	interface number, 8.5.9.6
	lldpV2LocManAddrOID	object identifier, 8.5.9.8
<i>Remote Systems Data group</i>		
lldpV2RemTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemChassisIdSubtype	chassis ID subtype, 8.5.2.2
	lldpV2RemChassisId	chassis ID, 8.5.2.3
	lldpV2RemPortIdSubtype	port ID sybtype, 8.5.3.2
	lldpV2RemPortId	port ID, 8.5.3.3
	lldpV2RemPortDesc	port description, 8.5.5.2
	lldpV2RemSysName	system name, 8.5.6.2
	lldpV2RemSysDesc	system description, 8.5.7.2
	lldpV2RemSysCapSupported	system capabilities, 8.5.8.1
	lldpV2RemSysCapEnabled	enabled capabilities, 8.5.8.2
	lldpV2RemRemoteChanges	remoteChanges, 9.2.5.10
	lldpV2RemTooManyNeighbors	tooManyNeighbors, 9.2.5.16
lldpV2RemManAddrTable		(Table index)
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemManAddrSubtype	management address subtype, 8.5.9.3 (Table index)
	lldpV2RemManAddr	management address, 8.5.9.4 (Table index)
	lldpV2RemManAddrIfSubtype	interface numbering subtype, 8.5.9.5
	lldpV2RemManAddrIfId	interface number, 8.5.9.6
	lldpV2RemManAddrOID	object identifier, 8.5.9.8

Table 11-2—LLDP MIB structure and object cross reference (continued)

MIB table	MIB object	LLDP reference
lldpV2RemUnknownTLVTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemUnknownTLVType	LLDPDU validation, 9.2.8.7.1 (Table index)
	lldpV2RemUnknownTLVInfo	LLDPDU validation, 9.2.8.7.1
lldpV2RemOrgDefInfoTable		
	lldpV2RemTimeMark	(Table index)
	lldpV2RemLocalIfIndex	(Table index)
	lldpV2RemLocalDestMACAddress	(Table index)
	lldpV2RemIndex	(Table index)
	lldpV2RemOrgDefInfoOUI	organizationally unique identifier, 8.6.1.3 (Table index)
	lldpV2RemOrgDefInfoSubtype	organizationally defined subtype, 8.6.1.4 (Table index)
	lldpV2RemOrgDefInfoIndex	(Table index)
	lldpV2RemOrgDefInfo	organizationally defined information, 8.6.1.5
LLDP MIB Notifications		
	lldpV2RemTablesChange	

1 11.3 Relationship to other MIBs

2 This clause includes specifications for an LLDP Textual Conventions MIB module, an LLDP MIB module,
3 and for IEEE 802.1 and IEEE 802.3 extension MIB modules that are compliant with the SMIV2 as defined in
4 IETF RFC 2578 (STD 58) [B3], IETF RFC 2579 (STD 58) [B4], and IETF RFC 2580 (STD 58) [B5].

5 LLDP is designed to operate in conjunction with MIB modules defined by the IETF, IEEE 802, and others.
6 The following subclauses discuss the relationship between LLDP and IETF MIB modules. LLDP agents
7 automatically notify the managers of these MIB modules whenever there is a value or status change in an
8 LLDP MIB object.

9 LLDP managed objects for the local system are stored in the LLDP local system MIB. Information received
10 from a remote LLDP agent is stored in the local LLDP agent's LLDP remote system MIB.

11 NOTE—In this standard, managed objects for an LLDP MIB module are defined as they would be for an SNMP MIB.
12 However, it is not required for LLDP implementations to support SNMP to store and retrieve system data. LLDP agents
13 need to have a place to store both information about the local system and information they have received about remote
14 systems. No particular implementation is implied.

11.3.1 Relationship to LLDP Version 1 MIB

The version 1 MIB module that was published in the initial 2005 publication of this standard has been superseded by the version 2 MIB module specified in 11.5.1, and support of the version 2 module is a requirement for conformance to the required or optional capabilities (Clause 5) in this revision of the standard. The version 2 MIB module reflects changes in indexation of the MIB objects that support the use of LLDP with multiple destination MAC addresses, as discussed in Clause 6.

In addition to the changes in indexation, the version 2 MIB module also supports changes to the management of the LLDP protocol that have resulted from changes to the LLDP protocol state machines. As these protocol changes affect only the timing and frequency of transmission of LLDPDUs, and not the content of the LLDPDUs, version 1 and version 2 implementations can successfully co-exist and interoperate.

From the perspective of a version 2 implementation, a version 1 neighbor behaves as a version 2 neighbor that is only using a single destination MAC address (the 01-80-C2-00-00-0E address specified for use in the IEEE 802.1AB-2005 version of the standard). From the perspective of a version 1 implementation, a version 2 neighbor that uses the 01-80-C2-00-00-0E destination MAC address behaves as a version 1 neighbor.

11.3.2 IETF Physical Topology MIB

The IETF Physical Topology (PTOPO) MIB (IETF RFC 2922 [B9]) allows a LLDP agent to expose learned physical topology information, using an IETF MIB. LLDP is intended to support the PTOPO MIB.

NOTE—The LLDP MIB module is a logical superset of the IETF PTOPO MIB; therefore, from a functional point of view, if the LLDP MIB module is implemented, it is likely not to be necessary to implement the PTOPO MIB defined in IETF RFC 2922 [B9]. However, for backward compatibility, this standard also supports the use of the PTOPO MIB.

11.3.3 IETF Entity MIB

The Entity MIB (IETF RFC 6933) allows the physical component inventory and hierarchy to be identified. Chassis IDs passed in the LLDPDU can identify entPhysicalTable entries. SNMP agents that implement the LLDP MIB should implement the entPhysicalAlias object from the Entity MIB version 2 or higher.

11.3.4 IETF Interfaces MIB

The Interfaces MIB (IETF RFC 2863) provides a standard mechanism for managing network ports. Port IDs passed in the LLDPDU can identify ifTable (or entPhysicalTable) entries. SNMP agents that implement the LLDP MIB shall also implement the ifTable and ifXTable for the ports that are represented in the Interfaces MIB.

11.4 Security considerations for LLDP base MIB module

There are a number of management objects defined in this MIB module with a MAX-ACCESS clause of read-write.⁶ Such objects may be considered sensitive or vulnerable in some network environments. The support for SET operations in a non-secure environment without proper protection can have a negative effect on network operations.

- a) Setting the following objects to incorrect values can result in an excessive number of LLDP packets being sent by the LLDP agent:
 - 1) lldpV2MessageTxInterval, lldpV2PortMessageTxInterval
 - 2) lldpV2TxCreditMax, lldpV2PortTxCreditMax

⁶ In IETF MIB definitions, the MAX-ACCESS clause defines the type of access that is allowed for particular data elements in the MIB. An explanation of the MAX-ACCESS mappings is given in section 7.3 of IETF RFC 2578 [B3].

- 1 3) lldpV2MessageFastTx, lldpV2PortMessageFastTx
- 2 4) lldpV2TxFastInit, lldpV2PortTxFastInit
- 3 b) Setting the object, lldpV2MessageTxHoldMultiplier or lldpV2PortMessageTxHoldMultiplier, to
- 4 incorrect values can cause the LLDP agent to transmit LLDPDUs with too-high TTL values, which
- 5 affect the expiration time of objects grouped under lldpV2RemoteSystemsData identifier.
- 6 c) Setting the object, lldpV2ReinitDelay or lldpV2PortReinitDelay, to too low a value can cause the
- 7 transmit state machine to attempt excessive re-initializations.
- 8 d) Setting incorrect bits in the object, lldpV2PortConfigTLVsTxEnableV2, can cause the LLDP agent
- 9 to transmit LLDPDUs with an undesired optional TLV sequence.
- 10 e) Setting incorrect bits in the object, lldpV2ConfigManAddrPortsTxEnable, can cause the LLDP
- 11 agent to advertise management addresses that were not meant to be disclosed and/or to omit
- 12 addresses that were desired.
- 13 f) Setting the following objects to incorrect values can result in improper operation of the MIB
- 14 notification process:
 - 15 1) lldpV2NotificationInterval
 - 16 2) lldpV2PortNotificationInterval
 - 17 3) lldpV2PortConfigNotificationEnableV2
- 18 g) Setting the object, lldpV2PortConfigAdminStatusV2, to the incorrect value can result in enabling a
- 19 non-desired operational mode.

20 The following readable objects in this MIB module may be considered to be sensitive or vulnerable in some
21 network environments:

- 22 h) Objects that are associated with the transmit mode
 - 23 1) lldpV2LocChassisIdSubtype
 - 24 2) lldpV2LocChassisId
 - 25 3) lldpV2LocPortIdSubtype
 - 26 4) lldpV2LocPortId
 - 27 5) lldpV2LocPortDesc
 - 28 6) lldpV2LocSysName
 - 29 7) lldpV2LocSysDesc
 - 30 8) lldpV2LocSysCapSupported
 - 31 9) lldpV2LocSysCapEnabled
 - 32 10) lldpV2LocManAddrLen
 - 33 11) lldpV2LocManAddrIfSubtype
 - 34 12) lldpV2LocManAddrIfId
 - 35 13) lldpV2LocManAddrOID
- 36 i) Objects that are associated with the receive mode
 - 37 1) lldpV2NotificationInterval
 - 38 2) lldpV2PortConfigNotificationEnable
 - 39 3) lldpV2RemChassisIdSubtype
 - 40 4) lldpV2RemChassisId
 - 41 5) lldpV2RemPortIdSubtyp
 - 42 6) lldpV2RemPortId
 - 43 7) lldpV2RemPortDesc
 - 44 8) lldpV2RemSysName
 - 45 9) lldpV2RemSysDesc
 - 46 10) lldpV2RemSysCapSupported
 - 47 11) lldpV2RemSysCapEnabled
 - 48 12) lldpV2RemManAddrIfSubtype
 - 49 13) lldpV2RemManAddrIfId

- 1 14) lldpV2RemManAddrOID
- 2 15) lldpV2RemUnknownTLVInfo
- 3 16) lldpV2RemOrgDefInfo

4 This concern applies both to objects that describe the configuration of the local host, as well as for objects
5 that describe information from the remote hosts, acquired via LLDP and displayed by the objects in this
6 MIB module. It is thus important to control even GET and/or NOTIFY access to these objects and possibly
7 to even encrypt the values of these objects when sending them over the network via SNMP.

8 SNMP versions prior to SNMPv3 did not include adequate security. Even if the network itself is secure (for
9 example, by using IPSec), even then, there is no control as to who on the secure network is allowed to
10 access and GET/SET (read/change/create/delete) the objects in this MIB module.

11 Implementers should consider the security features as provided by the SNMPv3 framework (see
12 IETF RFC 3410, section 8), including full support for the SNMPv3 cryptographic mechanisms (for
13 authentication and privacy).

14 Further, implementers should not deploy SNMP versions prior to SNMPv3. Instead, implementers should
15 deploy SNMPv3 to enable cryptographic security. It is then a customer/operator responsibility to ensure that
16 the SNMP entity giving access to an instance of this MIB module is properly configured to give access to the
17 objects only to those principals (users) that have legitimate rights to indeed GET or SET
18 (change/create/delete) them.

19 **11.5 LLDP MIB modules** ^{7, 8}

20 Two MIB modules are defined—a textual conventions MIB module (11.5.1) that contains the textual
21 conventions used by the LLDP version 2 MIB module and by the version 2 extension MIB module in
22 Annex D of IEEE Std 802.1Q-2014, and the LLDP version 2 MIB module itself (11.5.2). The textual
23 conventions defined in the Textual-Convention MIB module are also available by extension MIB modules
24 defined in other standards, such as the extension MIB modules defined in IEEE Std 802.1Q.

25

⁷ *Copyright release for MIBs:* Users of this standard may freely reproduce the MIB contained in this subclause so that it can be used for its intended purpose.

⁸ An ASCII version of this MIB module can be obtained by Web browser from the IEEE 802.1 Website at <http://www.ieee802.org/1/pages/MIBS.html>.

1 11.5.1 Definitions for the LLDP-V2-TC MIB module

2 In the following MIB module, should any discrepancy between the DESCRIPTION text and the
3 corresponding definition in Clause 9 and Clause 10 occur, the definition in Clause 9 and Clause 10 shall take
4 precedence.

```
5 LLDP-V2-TC-MIB DEFINITIONS ::= BEGIN
6 IMPORTS
7     MODULE-IDENTITY,
8     Unsigned32,
9     org
10    FROM SNMPv2-SMI
11    TEXTUAL-CONVENTION
12    FROM SNMPv2-TC;
13
14 lldpV2TcMIB MODULE-IDENTITY
15     LAST-UPDATED "201603110000Z" -- March 11, 2016
16 ORGANIZATION "IEEE 802.1 Working Group"
17 CONTACT-INFO
18     " WG-URL: http://www.ieee802.org/1/
19     WG-EMail: stds-802-1-L@ieee.org
20
21     Contact: IEEE 802.1 Working Group Chair
22     Postal: C/O IEEE 802.1 Working Group
23             IEEE Standards Association
24             445 Hoes Lane
25             Piscataway
26             NJ 08854
27             USA
28     E-mail: stds-802-1-L@ieee.org"
29 DESCRIPTION
30     "Textual conventions used throughout the IEEE Std 802.1AB
31     version 2 and later MIB modules.
32
33     Unless otherwise indicated, the references in this
34     MIB module are to IEEE 802.1AB-2016.
35
36     The TCs in this MIB are taken from the original LLDP-MIB,
37     LLDP-EXT-DOT1-MIB, and LLDP-EXT-DOT3-MIB published in
38     IEEE Std 802-1D-2004, with the addition of TCs to support
39     the management address table. They have been made available
40     as a separate TC MIB module to facilitate referencing from
41     other MIB modules.
42
43     Copyright (C) IEEE (2016). This version of this MIB module
44     is published as 11.5.1 of IEEE Std 802.1AB-2016;
45     see the standard itself for full legal notices."
46
47 REVISION "201603110000Z" -- March 11, 2016
48
49 DESCRIPTION
50     "Published as part of IEEE Std 802.1AB-2016.
51     This revision updated referencees."
52
53 REVISION "200906080000Z" -- June 08, 2009
54
55 DESCRIPTION
56     "Published as part of IEEE Std 802.1AB-2009 revision."
```

```
1
2 ::= { org ieee(111) standards-association-numbers-series-standards(2)
3     lan-man-stds(802) ieee802dot1(1) 1 12 }
4
5 --
6 -- Definition of the root OID arc for IEEE 802.1 MIBs
7 --
8
9 ieee802dot1mibs OBJECT IDENTIFIER
10 ::= { org ieee(111) standards-association-numbers-series-standards(2)
11     lan-man-stds(802) ieee802dot1(1) 1 }
12
13 --
14 -- *****
15 --
16 -- Textual Conventions
17 --
18 -- *****
19
20 LldpV2ChassisIdSubtype ::= TEXTUAL-CONVENTION
21     STATUS          current
22     DESCRIPTION
23         "This TC describes the source of a chassis identifier.
24
25         The enumeration 'chassisComponent(1)' represents a chassis
26         identifier based on the value of entPhysicalAlias object
27         (defined in IETF RFC 6933) for a chassis component (i.e.,
28         an entPhysicalClass value of 'chassis(3)').
29
30         The enumeration 'interfaceAlias(2)' represents a chassis
31         identifier based on the value of ifAlias object (defined in
32         IETF RFC 2863) for an interface on the containing chassis.
33
34         The enumeration 'portComponent(3)' represents a chassis
35         identifier based on the value of entPhysicalAlias object
36         (defined in IETF RFC 6933) for a port or backplane
37         component (i.e., entPhysicalClass value of 'port(10)' or
38         'backplane(4)'), within the containing chassis.
39
40         The enumeration 'macAddress(4)' represents a chassis
41         identifier based on the value of a unicast source address
42         (encoded in network byte order and IEEE 802.3 canonical bit
43         order), of a port on the containing chassis as defined in
44         IEEE Std 802.
45
46         The enumeration 'networkAddress(5)' represents a chassis
47         identifier based on a network address, associated with
48         a particular chassis. The encoded address is actually
49         composed of two fields. The first field is a single octet,
50         representing the IANA AddressFamilyNumbers value for the
51         specific address type, and the second field is the network
52         address value.
53
54         The enumeration 'interfaceName(6)' represents a chassis
55         identifier based on the value of ifName object (defined in
56         IETF RFC 2863) for an interface on the containing chassis.
57
58         The enumeration 'local(7)' represents a chassis identifier
59         based on a locally defined value."
```



```
1  SYNTAX  INTEGER {
2      chassisComponent(1),
3      interfaceAlias(2),
4      portComponent(3),
5      macAddress(4),
6      networkAddress(5),
7      interfaceName(6),
8      local(7)
9  }
10
11 LldpV2ChassisId ::= TEXTUAL-CONVENTION
12     DISPLAY-HINT "1x:"
13     STATUS      current
14     DESCRIPTION
15         "This TC describes the format of a chassis identifier string.
16         Objects of this type are always used with an associated
17         LldpChassisIdSubtype object, which identifies the format of
18         the particular LldpChassisId object instance.
19
20         If the associated LldpChassisIdSubtype object has a value of
21         'chassisComponent(1)', then the octet string identifies
22         a particular instance of the entPhysicalAlias object
23         (defined in IETF RFC 6933) for a chassis component (i.e.,
24         an entPhysicalClass value of 'chassis(3)').
25
26         If the associated LldpChassisIdSubtype object has a value
27         of 'interfaceAlias(2)', then the octet string identifies
28         a particular instance of the ifAlias object (defined in
29         IETF RFC 2863) for an interface on the containing chassis.
30         If the particular ifAlias object does not contain any values,
31         another chassis identifier type should be used.
32
33         If the associated LldpChassisIdSubtype object has a value
34         of 'portComponent(3)', then the octet string identifies a
35         particular instance of the entPhysicalAlias object (defined
36         in IETF RFC 6933) for a port or backplane component within
37         the containing chassis.
38
39         If the associated LldpChassisIdSubtype object has a value of
40         'macAddress(4)', then this string identifies a particular
41         unicast source address (encoded in network byte order and
42         IEEE 802.3 canonical bit order), of a port on the containing
43         chassis as defined in IEEE Std 802.
44
45         If the associated LldpChassisIdSubtype object has a value of
46         'networkAddress(5)', then this string identifies a particular
47         network address, encoded in network byte order, associated
48         with one or more ports on the containing chassis. The first
49         octet contains the IANA Address Family Numbers enumeration
50         value for the specific address type, and octets 2 through
51         N contain the network address value in network byte order.
52
53         If the associated LldpChassisIdSubtype object has a value
54         of 'interfaceName(6)', then the octet string identifies
55         a particular instance of the ifName object (defined in
56         IETF RFC 2863) for an interface on the containing chassis.
57         If the particular ifName object does not contain any values,
58         another chassis identifier type should be used.
59
```

```

1           If the associated LldpChassisIdSubtype object has a value of
2           'local(7)', then this string identifies a locally assigned
3           Chassis ID."
4   SYNTAX      OCTET STRING (SIZE (1..255))
5
6 LldpV2PortIdSubtype ::= TEXTUAL-CONVENTION
7   STATUS      current
8   DESCRIPTION
9       "This TC describes the source of a particular type of port
10      identifier used in the LLDP MIB.
11
12      The enumeration 'interfaceAlias(1)' represents a port
13      identifier based on the ifAlias MIB object, defined in IETF
14      RFC 2863.
15
16      The enumeration 'portComponent(2)' represents a port
17      identifier based on the value of entPhysicalAlias (defined in
18      IETF RFC 6933) for a port component (i.e., entPhysicalClass
19      value of 'port(10)'), within the containing chassis.
20
21      The enumeration 'macAddress(3)' represents a port identifier
22      based on a unicast source address (encoded in network
23      byte order and IEEE 802.3 canonical bit order), which has
24      been detected by the agent and associated with a particular
25      port (IEEE Std 802).
26
27      The enumeration 'networkAddress(4)' represents a port
28      identifier based on a network address, detected by the agent
29      and associated with a particular port.
30
31      The enumeration 'interfaceName(5)' represents a port
32      identifier based on the ifName MIB object, defined in IETF
33      RFC 2863.
34
35      The enumeration 'agentCircuitId(6)' represents a port
36      identifier based on the agent-local identifier of the circuit
37      (defined in IETF RFC 3046), detected by the agent and associated
38      with a particular port.
39
40      The enumeration 'local(7)' represents a port identifier
41      based on a value locally assigned."
42
43   SYNTAX      INTEGER {
44       interfaceAlias(1),
45       portComponent(2),
46       macAddress(3),
47       networkAddress(4),
48       interfaceName(5),
49       agentCircuitId(6),
50       local(7)
51   }
52
53 LldpV2PortId ::= TEXTUAL-CONVENTION
54   DISPLAY-HINT "1x:"
55   STATUS      current
56   DESCRIPTION
57       "This TC describes the format of a port identifier string.
58       Objects of this type are always used with an associated
59       LldpPortIdSubtype object, which identifies the format of the

```

1 particular LldpPortId object instance.
2
3 If the associated LldpPortIdSubtype object has a value of
4 'interfaceAlias(1)', then the octet string identifies a
5 particular instance of the ifAlias object (defined in IETF
6 RFC 2863). If the particular ifAlias object does not contain
7 any values, another port identifier type should be used.
8
9 If the associated LldpPortIdSubtype object has a value of
10 'portComponent(2)', then the octet string identifies a
11 particular instance of the entPhysicalAlias object (defined
12 in IETF RFC 6933) for a port or backplane component.
13
14 If the associated LldpPortIdSubtype object has a value of
15 'macAddress(3)', then this string identifies a particular
16 unicast source address (encoded in network byte order
17 and IEEE 802.3 canonical bit order) associated with the port
18 (IEEE Std 802).
19
20 If the associated LldpPortIdSubtype object has a value of
21 'networkAddress(4)', then this string identifies a network
22 address associated with the port. The first octet contains
23 the IANA AddressFamilyNumbers enumeration value for the
24 specific address type, and octets 2 through N contain the
25 networkAddress address value in network byte order.
26
27 If the associated LldpPortIdSubtype object has a value of
28 'interfaceName(5)', then the octet string identifies a
29 particular instance of the ifName object (defined in IETF
30 RFC 2863). If the particular ifName object does not contain
31 any values, another port identifier type should be used.
32
33 If the associated LldpPortIdSubtype object has a value of
34 'agentCircuitId(6)', then this string identifies a agent-local
35 identifier of the circuit (defined in IETF RFC 3046).
36
37 If the associated LldpPortIdSubtype object has a value of
38 'local(7)', then this string identifies a locally
39 assigned port ID."
40 SYNTAX OCTET STRING (SIZE (1..255))
41
42 LldpV2ManAddrIfSubtype ::= TEXTUAL-CONVENTION
43 STATUS current
44 DESCRIPTION
45 "This TC defines an enumeration value that identifies
46 the interface numbering method used for defining the
47 interface number associated with a management address.
48 An object with this syntax defines the format of an
49 interface number object.
50
51 The enumeration 'unknown(1)' represents the case where the
52 interface is not known. In this case, the corresponding
53 interface number is of zero length.
54
55 The enumeration 'ifIndex(2)' represents interface identifier
56 based on the ifIndex MIB object.
57
58 The enumeration 'systemPortNumber(3)' represents interface
59 identifier based on the system port numbering convention."

```
1  REFERENCE
2      "8.5.9.5"
3
4  SYNTAX  INTEGER {
5      unknown(1),
6      ifIndex(2),
7      systemPortNumber(3)
8  }
9
10 LldpV2ManAddress ::= TEXTUAL-CONVENTION
11     DISPLAY-HINT "1x:"
12     STATUS      current
13     DESCRIPTION
14         "The value of a management address associated with the LLDP
15         agent that may be used to reach higher layer entities to
16         assist discovery by network management.
17
18         It should be noted that appropriate security credentials,
19         such as SNMP engineId, may be required to access the LLDP
20         agent using a management address. These necessary credentials
21         should be known by the network management and the objects
22         associated with the credentials are not included in the
23         LLDP agent."
24     SYNTAX      OCTET STRING (SIZE (1..31))
25
26 LldpV2SystemCapabilitiesMap ::= TEXTUAL-CONVENTION
27     STATUS      current
28     DESCRIPTION
29         "This TC describes the system capabilities.
30
31         The bit 'other(0)' indicates that the system has capabilities
32         other than those listed below.
33
34         The bit 'repeater(1)' indicates that the system has repeater
35         capability.
36
37         The bit 'bridge(2)' indicates that the system has bridge
38         capability.
39
40         The bit 'wlanAccessPoint(3)' indicates that the system has
41         WLAN access point capability.
42
43         The bit 'router(4)' indicates that the system has router
44         capability.
45
46         The bit 'telephone(5)' indicates that the system has telephone
47         capability.
48
49         The bit 'docsisCableDevice(6)' indicates that the system has
50         DOCSIS Cable Device capability (IETF RFC 4639 & 2670).
51
52         The bit 'stationOnly(7)' indicates that the system has only
53         station capability and nothing else.
54
55         The bit 'cVLANComponent(8)' indicates that the system has
56         C-VLAN component functionality.
57
58         The bit 'sVLANComponent(8)' indicates that the system has
59         S-VLAN component functionality.
```

```

1
2         The bit 'twoPortMACRelay(10)' indicates that the system has
3         Two-port MAC Relay (TPMR) functionality."
4     SYNTAX BITS {
5         other(0),
6         repeater(1),
7         bridge(2),
8         wlanAccessPoint(3),
9         router(4),
10        telephone(5),
11        docsisCableDevice(6),
12        stationOnly(7),
13        cVLANComponent(8),
14        sVLANComponent(9),
15        twoPortMACRelay(10)
16    }
17
18
19
20 LldpV2DestAddressTableIndex ::= TEXTUAL-CONVENTION
21     DISPLAY-HINT "d"
22     STATUS      current
23     DESCRIPTION
24         "An index value, used as the index to the table of destination
25         MAC addresses used both as the destination addresses on
26         transmitted LLDPDUs and on received LLDPDUs. This index value
27         is also used as a secondary index value in tables indexed
28         by fields of type ifIndex, in order to associate
29         a destination address with each row of the table."
30     SYNTAX      Unsigned32(1..4096)
31
32 LldpV2PowerPortClass ::= TEXTUAL-CONVENTION
33     STATUS      current
34     DESCRIPTION
35         "This TC describes the Power over Ethernet (PoE) port class."
36     SYNTAX      INTEGER {
37         pClassPSE(1),
38         pClassPD(2)
39     }
40
41 END
42

```

1 11.5.2 LLDP MIB module - version 2

2 In the following MIB module, should any discrepancy between the DESCRIPTION text and the
3 corresponding definition in Clause 9 and Clause 10 occur, the definition in Clause 9 and Clause 10 shall take
4 precedence.

```
5 LLDP-V2-MIB DEFINITIONS ::= BEGIN
6 IMPORTS
7     MODULE-IDENTITY,
8     OBJECT-TYPE,
9     Unsigned32,
10    Counter32,
11    NOTIFICATION-TYPE
12    FROM SNMPv2-SMI
13    TimeStamp,
14    TruthValue,
15    MacAddress,
16    RowStatus
17    FROM SNMPv2-TC
18    SnmpAdminString
19    FROM SNMP-FRAMEWORK-MIB
20    MODULE-COMPLIANCE,
21    OBJECT-GROUP,
22    NOTIFICATION-GROUP
23    FROM SNMPv2-CONF
24    TimeFilter,
25    ZeroBasedCounter32
26    FROM RMON2-MIB
27    AddressFamilyNumbers
28    FROM IANA-ADDRESS-FAMILY-NUMBERS-MIB
29    ifGeneralInformationGroup,
30    InterfaceIndex
31    FROM IF-MIB
32    LldpV2ChassisIdSubtype,
33    LldpV2ChassisId,
34    LldpV2PortIdSubtype,
35    LldpV2PortId,
36    LldpV2ManAddrIfSubtype,
37    LldpV2ManAddress,
38    LldpV2SystemCapabilitiesMap,
39    LldpV2DestAddressTableIndex,
40    ieee802dot1mibs
41    FROM LLDP-V2-TC-MIB;
42
43 lldpV2MIB MODULE-IDENTITY
44     LAST-UPDATED "201603110000Z" -- March 11, 2016
45     ORGANIZATION "IEEE 802.1 Working Group"
46     CONTACT-INFO
47         "WG-URL: http://www.ieee802.org/1/
48         WG-EMail: stds-802-1-L@ieee.org
49
50         Contact: IEEE 802.1 Working Group Chair
51         Postal: IEEE Standards Board
52                 445 Hoes Lane
53                 Piscataway, NJ 08854
54                 USA
55         E-mail: stds-802-1-L@ieee.org"
56     DESCRIPTION
57         "Management Information Base module for LLDP configuration,
```

1 statistics, local system data and remote systems data
2 components.
3
4 This MIB module supports the architecture described in
5 Clause 6, where multiple LLDP agents can be associated with
6 a single Port, each supporting transmission by means of a
7 different MAC address.
8
9 Unless otherwise indicated, the references in this
10 MIB module are to IEEE Std 802.1AB-2016.
11
12 Copyright (C) IEEE (2016). This version of this MIB module
13 is published as 11.5.2 of IEEE Std 802.1AB-2016;
14 see the standard itself for full legal notices."
15
16 REVISION "201603110000Z" -- March 11, 2016
17
18 DESCRIPTION
19 "Published as part of the 2016 revision of
20 IEEE Std 802.1AB.
21 This revision incorporated changes to the MIB to
22 address issues identified in maintenance item
23 0121 - see <http://www.ieee802.org/1/maint.html>.
24 The changes involved correcting the way that
25 lldpV2MessageTxHoldMultiplier is calculated, in
26 accordance with 9.2.5.23."
27
28
29 REVISION "201502160000Z" -- February 16, 2015
30
31 DESCRIPTION
32 "Published as part of IEEE Std 802.1AB-2009 Cor-2.
33 This corrigendum incorporated changes to the MIB to
34 address issues identified in maintenance item
35 0121 - see <http://www.ieee802.org/1/maint.html>."
36
37 REVISION "200906080000Z" -- June 08, 2009
38
39 DESCRIPTION
40 "Published as part of IEEE Std 802.1AB-2009 revision.
41 This revision incorporated changes to the MIB to
42 support the use of LLDP with multiple destination MAC
43 addresses."
44
45 ::= { ieee802dot1mibs 13 }
46
47 lldpV2Notifications OBJECT IDENTIFIER ::= { lldpV2MIB 0 }
48 lldpV2Objects OBJECT IDENTIFIER ::= { lldpV2MIB 1 }
49 lldpV2Conformance OBJECT IDENTIFIER ::= { lldpV2MIB 2 }
50
51 --
52 -- LLDP MIB Objects
53 --
54
55 lldpV2Configuration OBJECT IDENTIFIER ::= { lldpV2Objects 1 }
56 lldpV2Statistics OBJECT IDENTIFIER ::= { lldpV2Objects 2 }
57 lldpV2LocalSystemData OBJECT IDENTIFIER ::= { lldpV2Objects 3 }
58 lldpV2RemoteSystemsData OBJECT IDENTIFIER ::= { lldpV2Objects 4 }
59 lldpV2Extensions OBJECT IDENTIFIER ::= { lldpV2Objects 5 }

```

1
2 --
3 -- *****
4 --
5 --             L L D P       C O N F I G
6 --
7 -- *****
8 --
9
10 lldpV2MessageTxInterval OBJECT-TYPE
11     SYNTAX      Unsigned32(5..32768)
12     UNITS       "seconds"
13     MAX-ACCESS  read-write
14     STATUS      current
15     DESCRIPTION
16         "The interval at which LLDP frames are transmitted on
17         behalf of this LLDP agent.
18
19         The default value for lldpV2MessageTxInterval object is
20         30 seconds.
21
22         The value of this object is used as the initial value of
23         the lldpV2PortMessageTxInterval object on row creation in
24         the lldpV2PortConfigTableV2.
25
26         The value of this object is restored from non-volatile
27         storage after a re-initialization of the management system."
28     REFERENCE
29         "9.2.5.6"
30     DEFVAL      { 30 }
31     ::= { lldpV2Configuration 1 }
32
33 lldpV2MessageTxHoldMultiplier OBJECT-TYPE
34     SYNTAX      Unsigned32(2..10)
35     MAX-ACCESS  read-write
36     STATUS      current
37     DESCRIPTION
38         "The time to live value expressed as a multiple of the
39         lldpV2MessageTxInterval object. The actual time to live
40         value used in LLDP frames, transmitted on behalf of this
41         LLDP agent, can be expressed by the following formula:
42         TTL = min(65535,
43         (lldpV2MessageTxInterval*lldpV2MessageTxHoldMultiplier)+1)
44         For example, if the value of lldpV2MessageTxInterval is
45         '30', and the value of lldpV2MessageTxHoldMultiplier
46         is '4', then the value '121' is encoded in the
47         TTL field in the LLDP header.
48
49         The default value for lldpV2MessageTxHoldMultiplier
50         object is 4.
51
52         The value of this object is used as the initial value of
53         the lldpV2PortMessageTxHoldMultiplier object on row creation in
54         the lldpV2PortConfigTableV2.
55
56         The value of this object is restored from non-volatile
57         storage after a re-initialization of the management system."
58     REFERENCE
59         "9.2.5.5"

```



```

1      DEFVAL      { 4 }
2      ::= { lldpV2Configuration 2 }
3
4 lldpV2ReinitDelay OBJECT-TYPE
5     SYNTAX      Unsigned32(1..10)
6     UNITS       "seconds"
7     MAX-ACCESS  read-write
8     STATUS      current
9     DESCRIPTION
10            "The lldpV2ReinitDelay indicates the delay (in units of
11            seconds) from when lldpPortConfigAdminStatus object of a
12            particular port becomes 'disabled' until re-initialization
13            is attempted.
14
15            The default value for lldpV2ReinitDelay is 2 s.
16
17            The value of this object is used as the initial value of
18            the lldpV2PortReinitDelay object on row creation in
19            the lldpV2PortConfigTableV2.
20
21            The value of this object is restored from non-volatile
22            storage after a re-initialization of the management system."
23     REFERENCE
24            "9.2.5.9"
25     DEFVAL      { 2 }
26     ::= { lldpV2Configuration 3 }
27
28 lldpV2NotificationInterval OBJECT-TYPE
29     SYNTAX      Unsigned32(5..3600)
30     UNITS       "seconds"
31     MAX-ACCESS  read-write
32     STATUS      current
33     DESCRIPTION
34            "This object controls the interval between transmission of
35            LLDP notifications during normal transmission periods.
36
37            The value of this object is used as the initial value of
38            the lldpV2PortNotificationInterval object on row creation in
39            the lldpV2PortConfigTableV2.
40
41            The value of this object is restored from non-volatile
42            storage after a re-initialization of the management system."
43     DEFVAL { 30 }
44     ::= { lldpV2Configuration 4 }
45
46 lldpV2TxCreditMax OBJECT-TYPE
47     SYNTAX Unsigned32(1..100)
48     UNITS "PDUs"
49     MAX-ACCESS read-write
50     STATUS current
51     DESCRIPTION
52            "The maximum number of consecutive LLDPDUs that can be
53            transmitted at any time.
54
55            The default value for lldpV2TxCreditMax object is 5 PDUs.
56
57            The value of this object is used as the initial value of
58            the lldpV2PortTxCreditMax object on row creation in
59            the lldpV2PortConfigTableV2.

```

```

1
2         The value of this object is restored from non-volatile
3         storage after a re-initialization of the management system."
4     REFERENCE
5         "9.2.5.18"
6     DEFVAL { 5 }
7     ::= { lldpV2Configuration 5 }
8
9 lldpV2MessageFastTx OBJECT-TYPE
10    SYNTAX Unsigned32(1..3600)
11    UNITS "seconds"
12    MAX-ACCESS read-write
13    STATUS current
14    DESCRIPTION
15        "The interval at which LLDP frames are transmitted on
16        behalf of this LLDP agent during fast transmission period
17        (e.g., when a new neighbor is detected).
18
19        The default value for lldpV2MessageFastTx object is
20        1 second.
21
22        The value of this object is used as the initial value of
23        the lldpV2PortMessageFastTx object on row creation in
24        the lldpV2PortConfigTableV2.
25
26        The value of this object is restored from non-volatile
27        storage after a re-initialization of the management system."
28    REFERENCE
29        "9.2.5.4"
30    DEFVAL { 1 }
31    ::= { lldpV2Configuration 6 }
32
33 lldpV2TxFastInit OBJECT-TYPE
34    SYNTAX Unsigned32(1..8)
35    MAX-ACCESS read-write
36    STATUS current
37    DESCRIPTION
38        "The initial value used to initialize the txFast variable
39        which determines the number of transmissions that are
40        made in fast transmission mode.
41
42        The default value for lldpV2TxFastInit object is 4.
43
44        The value of this object is used as the initial value of
45        the lldpV2PortTxFastInit object on row creation in
46        the lldpV2PortConfigTableV2.
47
48        The value of this object is restored from non-volatile
49        storage after a re-initialization of the management system."
50    REFERENCE
51        "9.2.5.20"
52    DEFVAL { 4 }
53    ::= { lldpV2Configuration 7 }
54 --
55 -- lldpV2PortConfigTable: LLDP configuration indexed on a per port,
56 -- per destination address basis. The ifIndex, coupled with an
57 -- index into the lldpDestAddressTable, is used to index per port
58 -- per destination MAC address.
59 -- ***This table and its associated objects are now deprecated

```

```

1 -- and replaced by lldpV2PortConfigTableV2.***
2 --
3
4 lldpV2PortConfigTable    OBJECT-TYPE
5     SYNTAX                SEQUENCE OF LldpV2PortConfigEntry
6     MAX-ACCESS            not-accessible
7     STATUS                deprecated
8     DESCRIPTION
9         "The table that controls LLDP frame transmission on individual
10        ports that uses particular destination MAC addresses."
11     ::= { lldpV2Configuration 8 }
12
13 lldpV2PortConfigEntry    OBJECT-TYPE
14     SYNTAX                LldpV2PortConfigEntry
15     MAX-ACCESS            not-accessible
16     STATUS                deprecated
17     DESCRIPTION
18         "LLDP configuration information for a particular port and
19        destination MAC address.
20
21        This configuration parameter controls the transmission and
22        the reception of LLDP frames on those interface/address
23        combinations whose rows are created in this table.
24
25        Rows in this table can only be created for MAC addresses
26        that can validly be used in association with the type of
27        interface concerned, as defined by Table 7-2.
28
29        The contents of this table is persistent across
30        re-initializations or re-boots."
31     INDEX { lldpV2PortConfigIfIndex,
32            lldpV2PortConfigDestAddressIndex }
33     ::= { lldpV2PortConfigTable 1 }
34
35 lldpV2PortConfigEntry ::= SEQUENCE {
36     lldpV2PortConfigIfIndex      InterfaceIndex,
37     lldpV2PortConfigDestAddressIndex  LldpV2DestAddressTableIndex,
38     lldpV2PortConfigAdminStatus  INTEGER,
39     lldpV2PortConfigNotificationEnable  TruthValue,
40     lldpV2PortConfigTLVsTxEnable  BITS }
41
42 lldpV2PortConfigIfIndex    OBJECT-TYPE
43     SYNTAX                InterfaceIndex
44     MAX-ACCESS            not-accessible
45     STATUS                deprecated
46     DESCRIPTION
47         "The interface index value used to identify the port
48        associated with this entry. Its value is an index into
49        the interfaces MIB.
50
51        The value of this object is used as an index to the
52        lldpV2PortConfigTable."
53     ::= { lldpV2PortConfigEntry 1 }
54
55 lldpV2PortConfigDestAddressIndex    OBJECT-TYPE
56     SYNTAX                LldpV2DestAddressTableIndex
57     MAX-ACCESS            not-accessible
58     STATUS                deprecated
59     DESCRIPTION

```

```
1         "The index value used to identify the destination
2         MAC address associated with this entry. Its value identifies
3         the row in the lldpV2DestAddressTable where the MAC address
4         can be found.
5
6         The value of this object is used as an index to the
7         lldpV2PortConfigTable."
8 ::= { lldpV2PortConfigEntry 2 }
9
10 lldpV2PortConfigAdminStatus OBJECT-TYPE
11     SYNTAX INTEGER {
12         txOnly(1),
13         rxOnly(2),
14         txAndRx(3),
15         disabled(4)
16     }
17     MAX-ACCESS read-write
18     STATUS deprecated
19     DESCRIPTION
20         "The administratively desired status of the local LLDP agent.
21
22         If the associated lldpV2PortConfigAdminStatus object is
23         set to a value of 'txOnly(1)', then LLDP agent transmits
24         LLDPframes on this port and it does not store any
25         information about the remote systems connected.
26
27         If the associated lldpV2PortConfigAdminStatus object is
28         set to a value of 'rxOnly(2)', then the LLDP agent
29         receives, but it does not transmit LLDP frames on this port.
30
31         If the associated lldpV2PortConfigAdminStatus object is set
32         to a value of 'txAndRx(3)', then the LLDP agent transmits
33         and receives LLDP frames on this port.
34
35         If the associated lldpV2PortConfigAdminStatus object is set
36         to a value of 'disabled(4)', then LLDP agent does not
37         transmit or receive LLDP frames on this port. If there is
38         remote systems information that is received on this port
39         and stored in other tables, before the port's
40         lldpV2PortConfigAdminStatus becomes disabled, then that
41         information is deleted."
42     REFERENCE
43         "9.2.5.1"
44     DEFVAL { txAndRx }
45 ::= { lldpV2PortConfigEntry 3 }
46
47 lldpV2PortConfigNotificationEnable OBJECT-TYPE
48     SYNTAX TruthValue
49     MAX-ACCESS read-write
50     STATUS deprecated
51     DESCRIPTION
52         "The lldpV2PortConfigNotificationEnable controls, on a per
53         agent basis, whether or not notifications from the agent
54         are enabled. The value true(1) means that notifications are
55         enabled; the value false(2) means that they are not."
56     DEFVAL { false }
57 ::= { lldpV2PortConfigEntry 4 }
58
59 lldpV2PortConfigTLVsTxEnable OBJECT-TYPE
```

```

1  SYNTAX      BITS {
2              portDesc(0),
3              sysName(1),
4              sysDesc(2),
5              sysCap(3)
6  }
7  MAX-ACCESS  read-write
8  STATUS      deprecated
9  DESCRIPTION
10             "The lldpV2PortConfigTLVsTxEnable, defined as a bitmap,
11             includes the basic set of LLDP TLVs whose transmission is
12             allowed on the local LLDP agent by the network management.
13             Each bit in the bitmap corresponds to a TLV type associated
14             with a specific optional TLV.
15
16             It should be noted that the organizationally-specific TLVs
17             are excluded from the lldpV2PortConfigTLVsTxEnable bitmap.
18
19             LLDP Organization Specific Information Extension MIBs should
20             have similar configuration objects to control transmission
21             of their organizationally defined TLVs.
22
23             The bit 'portDesc(0)' indicates that LLDP agent should
24             transmit 'Port Description TLV'.
25
26             The bit 'sysName(1)' indicates that LLDP agent should transmit
27             'System Name TLV'.
28
29             The bit 'sysDesc(2)' indicates that LLDP agent should transmit
30             'System Description TLV'.
31
32             The bit 'sysCap(3)' indicates that LLDP agent should transmit
33             'System Capabilities TLV'.
34
35             There is no bit reserved for the management address TLV type
36             since transmission of management address TLVs are controlled
37             by another object, lldpV2ConfigManAddrTable.
38
39             The default value for lldpV2PortConfigTLVsTxEnable object is
40             empty set, which means no enumerated values are set.
41
42             The value of this object is restored from non-volatile
43             storage after a re-initialization of the management system."
44  REFERENCE
45      "9.1.2.1"
46  DEFVAL  { { } }
47  ::= { lldpV2PortConfigEntry 5 }
48
49 --
50 -- lldpV2PortConfigTableV2: LLDP configuration indexed on a per port,
51 -- per destination address basis. The ifIndex, coupled with an
52 -- index into the lldpDestAddressTable, is used to index per port
53 -- per destination MAC address.
54 --
55 -- V2 extends the original table definition to include per-port
56 -- per-MAC address parameters msgTxInterval, msgTxHold, reinitDelay,
57 -- notificationInterval, txCreditMax, msgFastTx, and txFastInit.
58 --
59

```

```

1 lldpV2PortConfigTableV2    OBJECT-TYPE
2     SYNTAX      SEQUENCE OF LldpV2PortConfigEntryV2
3     MAX-ACCESS  not-accessible
4     STATUS      current
5     DESCRIPTION
6         "The table that controls LLDP frame transmission on individual
7         ports and using particular destination MAC addresses."
8     ::= { lldpV2Configuration 11 }
9
10 lldpV2PortConfigEntryV2    OBJECT-TYPE
11     SYNTAX      LldpV2PortConfigEntryV2
12     MAX-ACCESS  not-accessible
13     STATUS      current
14     DESCRIPTION
15         "LLDP configuration information for a particular port and
16         destination MAC address.
17
18         This configuration parameter controls the transmission and
19         the reception of LLDP frames on those interface/address
20         combinations whose rows are created in this table.
21
22         Rows in this table can only be created for MAC addresses
23         that can validly be used in association with the type of
24         interface concerned, as defined by Table 7-2.
25
26         The contents of this table is persistent across
27         re-initializations or re-boots."
28     INDEX { lldpV2PortConfigIfIndexV2,
29             lldpV2PortConfigDestAddressIndexV2 }
30     ::= { lldpV2PortConfigTableV2 1 }
31
32 LldpV2PortConfigEntryV2 ::= SEQUENCE {
33     lldpV2PortConfigIfIndexV2      InterfaceIndex,
34     lldpV2PortConfigDestAddressIndexV2 LldpV2DestAddressTableIndex,
35     lldpV2PortConfigAdminStatusV2  INTEGER,
36     lldpV2PortMessageTxInterval    Unsigned32,
37     lldpV2PortMessageTxHoldMultiplier Unsigned32,
38     lldpV2PortReinitDelay           Unsigned32,
39     lldpV2PortNotificationInterval Unsigned32,
40     lldpV2PortTxCreditMax           Unsigned32,
41     lldpV2PortMessageFastTx         Unsigned32,
42     lldpV2PortTxFastInit            Unsigned32,
43     lldpV2PortConfigNotificationEnableV2 TruthValue,
44     lldpV2PortConfigTLVsTxEnableV2 BITS }
45
46 lldpV2PortConfigIfIndexV2    OBJECT-TYPE
47     SYNTAX      InterfaceIndex
48     MAX-ACCESS  not-accessible
49     STATUS      current
50     DESCRIPTION
51         "The interface index value used to identify the port
52         associated with this entry. Its value is an index into
53         the interfaces MIB.
54
55         The value of this object is used as an index to the
56         lldpV2PortConfigTable."
57     ::= { lldpV2PortConfigEntryV2 1 }
58
59 lldpV2PortConfigDestAddressIndexV2    OBJECT-TYPE

```

```

1  SYNTAX      LldpV2DestAddressTableIndex
2  MAX-ACCESS  not-accessible
3  STATUS      current
4  DESCRIPTION
5      "The index value used to identify the destination
6      MAC address associated with this entry. Its value identifies
7      the row in the lldpV2DestAddressTable where the MAC address
8      can be found.
9
10     The value of this object is used as an index to the
11     lldpV2PortConfigTable."
12 ::= { lldpV2PortConfigEntryV2 2 }
13
14 lldpV2PortConfigAdminStatusV2 OBJECT-TYPE
15     SYNTAX INTEGER {
16         txOnly(1),
17         rxOnly(2),
18         txAndRx(3),
19         disabled(4)
20     }
21     MAX-ACCESS read-write
22     STATUS      current
23     DESCRIPTION
24         "The administratively desired status of the local LLDP agent.
25
26         If the associated lldpV2PortConfigAdminStatus object is
27         set to a value of 'txOnly(1)', then LLDP agent transmits
28         LLDPframes on this port and it does not store any
29         information about the remote systems connected.
30
31         If the associated lldpV2PortConfigAdminStatus object is
32         set to a value of 'rxOnly(2)', then the LLDP agent
33         receives, but it does not transmit, LLDP frames on this port.
34
35         If the associated lldpV2PortConfigAdminStatus object is set
36         to a value of 'txAndRx(3)', then the LLDP agent transmits
37         and receives LLDP frames on this port.
38
39         If the associated lldpV2PortConfigAdminStatus object is set
40         to a value of 'disabled(4)', then LLDP agent does not
41         transmit or receive LLDP frames on this port. If there is
42         remote systems information that is received on this port
43         and stored in other tables, before the port's
44         lldpV2PortConfigAdminStatus becomes disabled, then that
45         information is deleted."
46     REFERENCE
47         "9.2.5.1"
48     DEFVAL { txAndRx }
49 ::= { lldpV2PortConfigEntryV2 3 }
50
51 lldpV2PortMessageTxInterval OBJECT-TYPE
52     SYNTAX      Unsigned32(5..32768)
53     UNITS        "seconds"
54     MAX-ACCESS  read-write
55     STATUS      current
56     DESCRIPTION
57         "The interval at which LLDP frames are transmitted on
58         behalf of this LLDP agent.
59

```

```

1          This object takes its initial value from the
2          lldpV2MessageTxInterval object on table row creation.
3
4          The value of this object is restored from non-volatile
5          storage after a re-initialization of the management system."
6  REFERENCE
7      "9.2.5.6"
8  DEFVAL      { 30 }
9  ::= { lldpV2PortConfigEntryV2 4 }
10
11 lldpV2PortMessageTxHoldMultiplier OBJECT-TYPE
12     SYNTAX      Unsigned32(2..10)
13     MAX-ACCESS  read-write
14     STATUS      current
15     DESCRIPTION
16         "The time to live value expressed as a multiple of the
17         lldpV2MessageTxInterval object. The actual time to live
18         value used in LLDP frames, transmitted on behalf of this
19         LLDP agent, can be expressed by the following formula:
20         TTL = min(65535,
21         (lldpV2MessageTxInterval*lldpV2MessageTxHoldMultiplier)+1)
22         For example, if the value of lldpV2MessageTxInterval is
23         '30', and the value of lldpV2MessageTxHoldMultiplier
24         is '4', then the value '121' is encoded in the
25         TTL field in the LLDP header.
26
27         This object takes its initial value from the
28         lldpV2PortMessageTxHoldMultiplier object on table row creation.
29
30         The value of this object is restored from non-volatile
31         storage after a re-initialization of the management system."
32  REFERENCE
33      "9.2.5.5"
34  DEFVAL      { 4 }
35  ::= { lldpV2PortConfigEntryV2 5 }
36
37 lldpV2PortReinitDelay OBJECT-TYPE
38     SYNTAX      Unsigned32(1..10)
39     UNITS       "seconds"
40     MAX-ACCESS  read-write
41     STATUS      current
42     DESCRIPTION
43         "The lldpV2ReinitDelay indicates the delay (in units of
44         seconds) from when lldpPortConfigAdminStatus object of a
45         particular port becomes 'disabled' until re-initialization
46         is attempted.
47
48         This object takes its initial value from the
49         lldpV2PortReinitDelay object on table row creation.
50
51         The value of this object is restored from non-volatile
52         storage after a re-initialization of the management system."
53  REFERENCE
54      "9.2.5.9"
55  DEFVAL      { 2 }
56  ::= { lldpV2PortConfigEntryV2 6 }
57
58 lldpV2PortNotificationInterval OBJECT-TYPE
59     SYNTAX      Unsigned32(5..3600)

```



```

1     UNITS          "seconds"
2     MAX-ACCESS    read-write
3     STATUS        current
4     DESCRIPTION
5         "This object controls the interval between transmission of
6         LLDP notifications during normal transmission periods.
7
8         This object takes its initial value from the
9         lldpV2PortNotificationInterval object on table row creation.
10
11        The value of this object is restored from non-volatile
12        storage after a re-initialization of the management system."
13     DEFVAL { 30 }
14     ::= { lldpV2PortConfigEntryV2 7 }
15
16 lldpV2PortTxCreditMax OBJECT-TYPE
17     SYNTAX Unsigned32(1..100)
18     UNITS "PDUs"
19     MAX-ACCESS read-write
20     STATUS current
21     DESCRIPTION
22         "The maximum number of consecutive LLDPDUs that can be
23         transmitted at any time.
24
25         This object takes its initial value from the
26         lldpV2PortTxCreditMax object on table row creation.
27
28         The value of this object is restored from non-volatile
29         storage after a re-initialization of the management system."
30     REFERENCE
31         "9.2.5.18"
32     DEFVAL { 5 }
33     ::= { lldpV2PortConfigEntryV2 8 }
34
35 lldpV2PortMessageFastTx OBJECT-TYPE
36     SYNTAX Unsigned32(1..3600)
37     UNITS "seconds"
38     MAX-ACCESS read-write
39     STATUS current
40     DESCRIPTION
41         "The interval at which LLDP frames are transmitted on
42         behalf of this LLDP agent during fast transmission period
43         (e.g., when a new neighbor is detected).
44
45         This object takes its initial value from the
46         lldpV2PortMessageFastTx object on table row creation.
47
48         The value of this object is restored from non-volatile
49         storage after a re-initialization of the management system."
50     REFERENCE
51         "9.2.5.4"
52     DEFVAL { 1 }
53     ::= { lldpV2PortConfigEntryV2 9 }
54
55 lldpV2PortTxFastInit OBJECT-TYPE
56     SYNTAX Unsigned32(1..8)
57     MAX-ACCESS read-write
58     STATUS current
59     DESCRIPTION

```

1 "The initial value used to initialize the txFast variable
2 which determines the number of transmissions that are
3 made in fast transmission mode.
4
5 This object takes its initial value from the
6 lldpV2PortTxFastInit object on table row creation.
7
8 The value of this object is restored from non-volatile
9 storage after a re-initialization of the management system."

10 REFERENCE
11 "9.2.5.20"
12 DEFVAL { 4 }
13 ::= { lldpV2PortConfigEntryV2 10 }
14
15

16 lldpV2PortConfigNotificationEnableV2 OBJECT-TYPE
17 SYNTAX TruthValue
18 MAX-ACCESS read-write
19 STATUS current
20 DESCRIPTION
21 "The lldpV2PortConfigNotificationEnableV2 controls, on a per
22 agent basis, whether or not notifications from the agent
23 are enabled. The value true(1) means that notifications are
24 enabled; the value false(2) means that they are not."
25 DEFVAL { false }
26 ::= { lldpV2PortConfigEntryV2 11 }
27

28 lldpV2PortConfigTLVsTxEnableV2 OBJECT-TYPE
29 SYNTAX BITS {
30 portDesc(0),
31 sysName(1),
32 sysDesc(2),
33 sysCap(3)
34 }
35 MAX-ACCESS read-write
36 STATUS current
37 DESCRIPTION
38 "The lldpV2PortConfigTLVsTxEnableV2, defined as a bitmap,
39 includes the basic set of LLDP TLVs whose transmission is
40 allowed on the local LLDP agent by the network management.
41 Each bit in the bitmap corresponds to a TLV type associated
42 with a specific optional TLV.
43
44 It should be noted that the organizationally-specific TLVs
45 are excluded from the lldpV2PortConfigTLVsTxEnable bitmap.
46
47 LLDP Organization Specific Information Extension MIBs should
48 have similar configuration objects to control transmission
49 of their organizationally defined TLVs.
50
51 The bit 'portDesc(0)' indicates that LLDP agent should
52 transmit 'Port Description TLV'.
53
54 The bit 'sysName(1)' indicates that LLDP agent should transmit
55 'System Name TLV'.
56
57 The bit 'sysDesc(2)' indicates that LLDP agent should transmit
58 'System Description TLV'.
59

```

1         The bit 'sysCap(3)' indicates that LLDP agent should transmit
2         'System Capabilities TLV'.
3
4         There is no bit reserved for the management address TLV type
5         since transmission of management address TLVs are controlled
6         by another object, lldpV2ConfigManAddrTable.
7
8         The default value for lldpV2PortConfigTLVsTxEnable object is
9         empty set, which means no enumerated values are set.
10
11        The value of this object is restored from non-volatile
12        storage after a re-initialization of the management system."
13    REFERENCE
14        "9.1.2.1"
15    DEFVAL { { } }
16    ::= { lldpV2PortConfigEntryV2 12 }
17
18
19 --
20 -- lldpV2DestAddressTable: Destination MAC addresses used by LLDP
21 --
22
23 lldpV2DestAddressTable    OBJECT-TYPE
24     SYNTAX      SEQUENCE OF LldpV2DestAddressTableEntry
25     MAX-ACCESS  not-accessible
26     STATUS      current
27     DESCRIPTION
28         "The table that contains the set of MAC addresses used
29         by LLDP for transmission and reception of LLDPDUs."
30     ::= { lldpV2Configuration 9 }
31
32 lldpV2DestAddressTableEntry    OBJECT-TYPE
33     SYNTAX      LldpV2DestAddressTableEntry
34     MAX-ACCESS  not-accessible
35     STATUS      current
36     DESCRIPTION
37         "Destination MAC address information for LLDP.
38
39         This configuration parameter identifies a MAC address
40         corresponding to a LldpV2DestAddressTableIndex value.
41
42         Rows in this table are created as necessary, to support
43         MAC addresses needed by other tables in the MIB that
44         are indexed by MAC address.
45
46         A given row in this table cannot be deleted if the MAC
47         address table index value is in use in any other table
48         in the MIB.
49
50         The contents of this table are persistent across
51         re-initializations or re-boots."
52     INDEX { lldpV2AddressTableIndex }
53     ::= { lldpV2DestAddressTable 1 }
54
55 LldpV2DestAddressTableEntry ::= SEQUENCE {
56     lldpV2AddressTableIndex      LldpV2DestAddressTableIndex,
57     lldpV2DestMacAddress         MacAddress }
58
59 lldpV2AddressTableIndex    OBJECT-TYPE

```

```

1  SYNTAX      LldpV2DestAddressTableIndex
2  MAX-ACCESS  not-accessible
3  STATUS      current
4  DESCRIPTION
5      "The index value used to identify the destination
6      MAC address associated with this entry.
7
8      The value of this object is used as an index to the
9      lldpV2DestAddressTable."
10 ::= { lldpV2DestAddressTableEntry 1 }
11
12 lldpV2DestMacAddress OBJECT-TYPE
13 SYNTAX      MacAddress
14 MAX-ACCESS  read-only
15 STATUS      current
16 DESCRIPTION
17     "The MAC address associated with this entry.
18
19     The octet string identifies an individual or a group
20     MAC address that is in use by LLDP as a destination
21     MAC address.
22
23     The MAC address is encoded in the octet string in
24     canonical format (see IEEE Std 802)."
```

```

25 ::= { lldpV2DestAddressTableEntry 2 }
26
27 --
28 -- lldpV2ManAddrConfigTxPortsTable : selection of management addresses
29 -- to be transmitted on a specified set of port/destination
30 -- address pairs.
31 --
32
33 lldpV2ManAddrConfigTxPortsTable OBJECT-TYPE
34 SYNTAX      SEQUENCE OF LldpV2ManAddrConfigTxPortsEntry
35 MAX-ACCESS  not-accessible
36 STATUS      current
37 DESCRIPTION
38     "The table that controls selection of LLDP management address
39     TLV instances to be transmitted on individual port/
40     destination address pairs."
41 ::= { lldpV2Configuration 10 }
42
43 lldpV2ManAddrConfigTxPortsEntry OBJECT-TYPE
44 SYNTAX      LldpV2ManAddrConfigTxPortsEntry
45 MAX-ACCESS  not-accessible
46 STATUS      current
47 DESCRIPTION
48     "LLDP configuration information that specifies the set
49     of port/destination address pairs on which the local
50     system management address instance is transmitted.
51
52     Each active lldpManAddrConfigTxPortsTableV2Entry is
53     restored from non-volatile storage and re-created
54     after a re-initialization of the management system."
```

```

55 INDEX {
56     lldpV2ManAddrConfigIfIndex,
57     lldpV2ManAddrConfigDestAddressIndex,
58     lldpV2ManAddrConfigLocManAddrSubtype,
59     lldpV2ManAddrConfigLocManAddr }

```

```

1      ::= { lldpV2ManAddrConfigTxPortsTable 1 }
2
3 LldpV2ManAddrConfigTxPortsEntry ::= SEQUENCE {
4     lldpV2ManAddrConfigIfIndex      InterfaceIndex,
5     lldpV2ManAddrConfigDestAddressIndex  LldpV2DestAddressTableIndex,
6     lldpV2ManAddrConfigLocManAddrSubtype  AddressFamilyNumbers,
7     lldpV2ManAddrConfigLocManAddr      LldpV2ManAddress,
8     lldpV2ManAddrConfigTxEnable        TruthValue,
9     lldpV2ManAddrConfigRowStatus        RowStatus
10 }
11
12 lldpV2ManAddrConfigIfIndex OBJECT-TYPE
13     SYNTAX      InterfaceIndex
14     MAX-ACCESS  not-accessible
15     STATUS      current
16     DESCRIPTION
17         "The interface index value used to identify the port
18         associated with this entry. Its value is an index into
19         the interfaces MIB.
20
21         The value of this object is used as an index to the
22         lldpV2PortConfigTable.
23
24         The value in this column of the table MUST match
25         the IfIndex value specified in the BridgePort table."
26     ::= { lldpV2ManAddrConfigTxPortsEntry 1 }
27
28 lldpV2ManAddrConfigDestAddressIndex OBJECT-TYPE
29     SYNTAX      LldpV2DestAddressTableIndex
30     MAX-ACCESS  not-accessible
31     STATUS      current
32     DESCRIPTION
33         "The index value used to identify the destination
34         MAC address associated with this entry. Its value identifies
35         the row in the lldpV2DestAddressTable where the MAC address
36         can be found.
37
38         The value of this object is used as an index to the
39         lldpV2PortConfigTable."
40     ::= { lldpV2ManAddrConfigTxPortsEntry 2 }
41
42
43 lldpV2ManAddrConfigLocManAddrSubtype OBJECT-TYPE
44     SYNTAX      AddressFamilyNumbers
45     MAX-ACCESS  not-accessible
46     STATUS      current
47     DESCRIPTION
48         "The type of management address identifier encoding used in
49         the associated 'lldpLocManagmentAddr' object.
50
51         It should be noted that only a subset of the possible
52         address encodings enumerated in AddressFamilyNumbers
53         are appropriate for use as a LLDP management
54         address, either because some are just not applicable or
55         because the maximum size of a LldpV2ManAddress octet string
56         would prevent the use of some address identifier encodings."
57     REFERENCE
58         "8.5.9.3"
59     ::= { lldpV2ManAddrConfigTxPortsEntry 3 }

```

```

1
2 lldpV2ManAddrConfigLocManAddr OBJECT-TYPE
3     SYNTAX      LldpV2ManAddress
4     MAX-ACCESS  not-accessible
5     STATUS      current
6     DESCRIPTION
7         "The string value used to identify the management address
8         component associated with the local system. The purpose of
9         this address is to contact the management entity."
10    REFERENCE
11        "8.5.9.4"
12    ::= { lldpV2ManAddrConfigTxPortsEntry 4 }
13
14
15
16 lldpV2ManAddrConfigTxEnable OBJECT-TYPE
17     SYNTAX      TruthValue
18     MAX-ACCESS  read-create
19     STATUS      current
20     DESCRIPTION
21         "A Boolean controlling the transmission of system
22         management address instance for the specified port,
23         destination, subtype, and MAN address used to index
24         this table. If set to the default value of false,
25         no transmission occurs. If set to true, the
26         appropriate information is transmitted out of the
27         port specified in the row's index."
28    REFERENCE
29        "9.1.2.1"
30    DEFVAL { false } -- not transmitted
31    ::= { lldpV2ManAddrConfigTxPortsEntry 5 }
32
33 lldpV2ManAddrConfigRowStatus OBJECT-TYPE
34     SYNTAX RowStatus
35     MAX-ACCESS read-create
36     STATUS current
37     DESCRIPTION
38         "Indicates the status of an entry in this table, and is used
39         to create/delete entries.
40         The corresponding instances of the following objects
41         must be set before this object can be made active(1):
42             lldpV2ManAddrConfigDestAddressIndex
43             lldpV2ManAddrConfigLocManAddrSubtype
44             lldpV2ManAddrConfigLocManAddr
45             lldpV2ManAddrConfigTxEnable
46
47         The corresponding instances of the following objects
48         may not be changed while this object is active(1):
49             lldpV2ManAddrConfigDestAddressIndex
50             lldpV2ManAddrConfigLocManAddrSubtype
51             lldpV2ManAddrConfigLocManAddr "
52    ::= { lldpV2ManAddrConfigTxPortsEntry 6 }
53
54 --
55 -- *****
56 --
57 --             L L D P       S T A T S
58 --
59 -- *****

```

```
1 --
2 -- LLDP Stats Group
3
4 lldpV2StatsRemTablesLastChangeTime OBJECT-TYPE
5     SYNTAX      TimeStamp
6     MAX-ACCESS  read-only
7     STATUS      current
8     DESCRIPTION
9         "The value of sysUpTime object (defined in IETF RFC 3418)
10        at the time an entry is created, modified, or deleted in the
11        in tables associated with the lldpV2RemoteSystemsData objects
12        and all LLDP extension objects associated with remote systems.
13
14        An NMS can use this object to reduce polling of the
15        lldpV2RemoteSystemsData objects."
16 ::= { lldpV2Statistics 1 }
17
18 lldpV2StatsRemTablesInserts OBJECT-TYPE
19     SYNTAX      ZeroBasedCounter32
20     UNITS       "table entries"
21     MAX-ACCESS  read-only
22     STATUS      current
23     DESCRIPTION
24         "The number of times the complete set of information
25         advertised by a particular MSAP has been inserted into tables
26         contained in lldpV2RemoteSystemsData and lldpV2Extensions objects.
27
28         The complete set of information received from a particular
29         MSAP should be inserted into related tables. If partial
30         information cannot be inserted for a reason such as lack
31         of resources, all of the complete set of information should
32         be removed.
33
34         This counter should be incremented only once after the
35         complete set of information is successfully recorded
36         in all related tables. Any failures during inserting
37         information set that result in deletion of previously
38         inserted information should not trigger any changes in
39         lldpV2StatsRemTablesInserts since the insert is not completed
40         yet or in lldpStatsRemTablesDeletes since the deletion
41         would only be a partial deletion. If the failure was the
42         result of lack of resources, the lldpStatsRemTablesDrops
43         counter should be incremented once."
44 ::= { lldpV2Statistics 2 }
45
46 lldpV2StatsRemTablesDeletes OBJECT-TYPE
47     SYNTAX      ZeroBasedCounter32
48     UNITS       "table entries"
49     MAX-ACCESS  read-only
50     STATUS      current
51
52     DESCRIPTION
53         "The number of times the complete set of information
54         advertised by a particular MSAP has been deleted from
55         tables contained in lldpV2RemoteSystemsData and lldpV2Extensions
56         objects.
57
58         This counter should be incremented only once when the
59         complete set of information is completely deleted from all
```

```

1         related tables. Partial deletions, such as deletion of
2         rows associated with a particular MSAP from some tables,
3         but not from all tables are not allowed, thus should not
4         change the value of this counter."
5 ::= { lldpV2Statistics 3 }
6
7 lldpV2StatsRemTablesDrops OBJECT-TYPE
8     SYNTAX      ZeroBasedCounter32
9     UNITS       "table entries"
10    MAX-ACCESS  read-only
11
12    STATUS      current
13    DESCRIPTION
14        "The number of times the complete set of information
15        advertised by a particular MSAP could not be entered into
16        tables contained in lldpV2RemoteSystemsData and lldpV2Extensions
17        objects because of insufficient resources."
18 ::= { lldpV2Statistics 4 }
19
20 lldpV2StatsRemTablesAgeouts OBJECT-TYPE
21     SYNTAX      ZeroBasedCounter32
22     UNITS       "table entries"
23     MAX-ACCESS  read-only
24     STATUS      current
25     DESCRIPTION
26         "The number of times the complete set of information
27         advertised by a particular MSAP has been deleted from tables
28         contained in lldpV2RemoteSystemsData and lldpV2Extensions objects
29         because the information timeliness interval has expired.
30
31         This counter should be incremented only once when the complete
32         set of information is completely invalidated (aged out)
33         from all related tables. Partial ageing, similar to deletion
34         case, is not allowed, and thus, should not change the value
35         of this counter."
36 ::= { lldpV2Statistics 5 }
37
38 --
39 -- TX statistics
40 -- Indexed by port (via ifIndex) and
41 -- destination MAC address.
42 --
43
44 lldpV2StatsTxPortTable OBJECT-TYPE
45     SYNTAX      SEQUENCE OF LldpV2StatsTxPortEntry
46     MAX-ACCESS  not-accessible
47     STATUS      current
48     DESCRIPTION
49         "A table containing LLDP transmission statistics for
50         individual port/destination address combinations.
51         Entries are not required to exist in
52         this table while the lldpPortConfigEntry object is equal to
53         'disabled(4)'."
54 ::= { lldpV2Statistics 6 }
55
56 lldpV2StatsTxPortEntry OBJECT-TYPE
57     SYNTAX      LldpV2StatsTxPortEntry
58     MAX-ACCESS  not-accessible
59     STATUS      current

```



```

1      DESCRIPTION
2          "LLDP frame transmission statistics for a particular port
3          and destination MAC address.
4          The port is contained in the same chassis as the
5          LLDP agent.
6
7          All counter values in a particular entry shall be
8          maintained on a continuing basis and shall not be deleted
9          upon expiration of rxInfoTTL timing counters in the LLDP
10         remote systems MIB of the receipt of a shutdown frame from
11         a remote LLDP agent.
12
13         All statistical counters associated with a particular
14         port on the local LLDP agent become frozen whenever the
15         adminStatus is disabled for the same port.
16
17         Rows in this table can only be created for MAC addresses
18         that can validly be used in association with the type of
19         interface concerned, as defined by Table 7-2."
20     INDEX { lldpV2StatsTxIfIndex,
21             lldpV2StatsTxDestMACAddress }
22     ::= { lldpV2StatsTxPortTable 1 }
23
24 lldpV2StatsTxPortEntry ::= SEQUENCE {
25     lldpV2StatsTxIfIndex          InterfaceIndex,
26     lldpV2StatsTxDestMACAddress   LldpV2DestAddressTableIndex,
27     lldpV2StatsTxPortFramesTotal  Counter32,
28     lldpV2StatsTxLLDPDULengthErrors Counter32 }
29
30 lldpV2StatsTxIfIndex OBJECT-TYPE
31     SYNTAX      InterfaceIndex
32     MAX-ACCESS  not-accessible
33     STATUS      current
34     DESCRIPTION
35         "The interface index value used to identify the port
36         associated with this entry. Its value is an index
37         into the interfaces MIB.
38
39         The value of this object is used as an index to the
40         lldpV2StatsTxPortTable."
41     ::= { lldpV2StatsTxPortEntry 1 }
42
43 lldpV2StatsTxDestMACAddress OBJECT-TYPE
44     SYNTAX      LldpV2DestAddressTableIndex
45     MAX-ACCESS  not-accessible
46     STATUS      current
47     DESCRIPTION
48         "The index value used to identify the destination
49         MAC address associated with this entry. Its value identifies
50         the row in the lldpV2DestAddressTable where the MAC address
51         can be found.
52
53         The value of this object is used as an index to the
54         lldpV2StatsTxPortTable."
55     ::= { lldpV2StatsTxPortEntry 2 }
56
57 lldpV2StatsTxPortFramesTotal OBJECT-TYPE
58     SYNTAX      Counter32
59     UNITS        "LLDP frames"

```

```

1  MAX-ACCESS      read-only
2  STATUS          current
3  DESCRIPTION
4      "The number of LLDP frames transmitted by this LLDP agent
5      on the indicated port to the destination MAC address
6      associated with this row of the table."
7  REFERENCE
8      "9.2.7.5"
9  ::= { lldpV2StatsTxPortEntry 3 }
10
11 lldpV2StatsTxLLDPDULengthErrors OBJECT-TYPE
12     SYNTAX      Counter32
13     UNITS        "LLDP frames"
14     MAX-ACCESS  read-only
15     STATUS      current
16     DESCRIPTION
17         "The number of LLDPDU Length Errors recorded for the Port."
18     REFERENCE
19         "9.2.7.8"
20     ::= { lldpV2StatsTxPortEntry 4 }
21
22 --
23 -- lldpV2StatsRxPortTable - RX statistics
24 -- This table is indexed by ifIndex and destination MAC address.
25 --
26
27 lldpV2StatsRxPortTable OBJECT-TYPE
28     SYNTAX      SEQUENCE OF LldpV2StatsRxPortEntry
29     MAX-ACCESS  not-accessible
30     STATUS      current
31     DESCRIPTION
32         "A table containing LLDP reception statistics for individual
33         ports and destination MAC addresses.
34         Entries are not required to exist in this table while
35         the lldpPortConfigEntry object is equal to 'disabled(4)'."
36     ::= { lldpV2Statistics 7 }
37
38 lldpV2StatsRxPortEntry OBJECT-TYPE
39     SYNTAX      LldpV2StatsRxPortEntry
40     MAX-ACCESS  not-accessible
41     STATUS      current
42     DESCRIPTION
43         "LLDP frame reception statistics for a particular port.
44         The port is contained in the same chassis as the
45         LLDP agent.
46
47         All counter values in a particular entry shall be
48         maintained on a continuing basis and shall not be deleted
49         upon expiration of rxInfoTTL timing counters in the LLDP
50         remote systems MIB of the receipt of a shutdown frame from
51         a remote LLDP agent.
52
53         All statistical counters associated with a particular
54         port on the local LLDP agent become frozen whenever the
55         adminStatus is disabled for the same port.
56
57         Rows in this table can only be created for MAC addresses
58         that can validly be used in association with the type of
59         interface concerned, as defined by Table 7-2.
```

```

1
2         The contents of this table is persistent across
3         re-initializations or re-boots."
4     INDEX { lldpV2StatsRxDestIfIndex,
5             lldpV2StatsRxDestMACAddress }
6     ::= { lldpV2StatsRxPortTable 1 }
7
8 lldpV2StatsRxPortEntry ::= SEQUENCE {
9     lldpV2StatsRxDestIfIndex      InterfaceIndex,
10    lldpV2StatsRxDestMACAddress    LldpV2DestAddressTableIndex,
11    lldpV2StatsRxPortFramesDiscardedTotal Counter32,
12    lldpV2StatsRxPortFramesErrors   Counter32,
13    lldpV2StatsRxPortFramesTotal     Counter32,
14    lldpV2StatsRxPortTLVsDiscardedTotal Counter32,
15    lldpV2StatsRxPortTLVsUnrecognizedTotal Counter32,
16    lldpV2StatsRxPortAgeoutsTotal    ZeroBasedCounter32
17 }
18
19 lldpV2StatsRxDestIfIndex OBJECT-TYPE
20     SYNTAX      InterfaceIndex
21     MAX-ACCESS  not-accessible
22     STATUS      current
23     DESCRIPTION
24         "The interface index value used to identify the port
25         associated with this entry. Its value is an index
26         into the interfaces MIB.
27
28         The value of this object is used as an index to the
29         lldpStatsRxPortV2Table."
30     ::= { lldpV2StatsRxPortEntry 1 }
31
32 lldpV2StatsRxDestMACAddress OBJECT-TYPE
33     SYNTAX      LldpV2DestAddressTableIndex
34     MAX-ACCESS  not-accessible
35     STATUS      current
36     DESCRIPTION
37         "The index value used to identify the destination
38         MAC address associated with this entry. Its value identifies
39         the row in the lldpV2DestAddressTable where the MAC address
40         can be found.
41
42         The value of this object is used as an index to the
43         lldpStatsRxPortV2Table."
44     ::= { lldpV2StatsRxPortEntry 2 }
45
46
47 lldpV2StatsRxPortFramesDiscardedTotal OBJECT-TYPE
48     SYNTAX      Counter32
49     UNITS        "LLDP frames"
50     MAX-ACCESS  read-only
51     STATUS      current
52     DESCRIPTION
53         "The number of LLDP frames received by this LLDP agent on
54         the indicated port, and then discarded for any reason.
55         This counter can provide an indication that LLDP header
56         formatting problems may exist with the local LLDP agent in
57         the sending system or that LLDPDU validation problems may
58         exist with the local LLDP agent in the receiving system."
59     REFERENCE

```

```

1         "9.2.7.2"
2     ::= { lldpV2StatsRxPortEntry 3 }
3
4 lldpV2StatsRxPortFramesErrors OBJECT-TYPE
5     SYNTAX      Counter32
6     UNITS       "LLDP frames"
7     MAX-ACCESS  read-only
8     STATUS      current
9     DESCRIPTION
10        "The number of invalid LLDP frames received by this LLDP
11        agent on the indicated port, while this LLDP agent is enabled."
12    REFERENCE
13        "9.2.7.3"
14    ::= { lldpV2StatsRxPortEntry 4 }
15
16 lldpV2StatsRxPortFramesTotal OBJECT-TYPE
17     SYNTAX      Counter32
18     UNITS       "LLDP frames"
19     MAX-ACCESS  read-only
20     STATUS      current
21     DESCRIPTION
22        "The number of valid LLDP frames received by this LLDP agent
23        on the indicated port, while this LLDP agent is enabled."
24    REFERENCE
25        "9.2.7.4"
26    ::= { lldpV2StatsRxPortEntry 5 }
27
28 lldpV2StatsRxPortTLVsDiscardedTotal OBJECT-TYPE
29     SYNTAX      Counter32
30     UNITS       "TLVs"
31     MAX-ACCESS  read-only
32     STATUS      current
33     DESCRIPTION
34        "The number of LLDP TLVs discarded for any reason by this LLDP
35        agent on the indicated port."
36    REFERENCE
37        "9.2.7.6"
38    ::= { lldpV2StatsRxPortEntry 6 }
39
40 lldpV2StatsRxPortTLVsUnrecognizedTotal OBJECT-TYPE
41     SYNTAX      Counter32
42     UNITS       "TLVs"
43     MAX-ACCESS  read-only
44     STATUS      current
45     DESCRIPTION
46        "The number of LLDP TLVs received on the given port that
47        are not recognized by this LLDP agent on the indicated port.
48
49        An unrecognized TLV is referred to as the TLV whose type value
50        is in the range of reserved TLV types (000 1001 - 111 1110)
51        in Table 8-1 of IEEE Std 802.1AB-2015. An unrecognized
52        TLV may be a basic management TLV from a later LLDP version."
53    REFERENCE
54        "9.2.7.7"
55    ::= { lldpV2StatsRxPortEntry 7 }
56
57 lldpV2StatsRxPortAgeoutsTotal OBJECT-TYPE
58     SYNTAX      ZeroBasedCounter32
59     MAX-ACCESS  read-only

```

```

1      STATUS      current
2      DESCRIPTION
3          "The counter that represents the number of age-outs that
4          occurred on a given port. An age-out is the number of
5          times the complete set of information advertised by a
6          particular MSAP has been deleted from tables contained in
7          lldpV2RemoteSystemsData and lldpV2Extensions objects because
8          the information timeliness interval has expired.
9
10         This counter is similar to lldpV2StatsRemTablesAgeouts, except
11         that the counter is on a per port basis. This enables NMS to
12         poll tables associated with the lldpV2RemoteSystemsData objects
13         and all LLDP extension objects associated with remote systems
14         on the indicated port only.
15
16         This counter is set to zero during agent initialization
17         and its value should not be saved in non-volatile storage.
18
19         This counter is incremented only once when the
20         complete set of information is invalidated (aged out) from
21         all related tables on a particular port. Partial ageing
22         is not allowed."
23      REFERENCE
24          "9.2.7.1"
25      ::= { lldpV2StatsRxPortEntry 8 }
26
27
28  -- *****
29  --
30  --          L O C A L      S Y S T E M      D A T A
31  --
32  -- *****
33
34  lldpV2LocChassisIdSubtype  OBJECT-TYPE
35      SYNTAX      LldpV2ChassisIdSubtype
36      MAX-ACCESS  read-only
37      STATUS      current
38      DESCRIPTION
39          "The type of encoding used to identify the chassis
40          associated with the local system."
41      REFERENCE
42          "8.5.2.2"
43      ::= { lldpV2LocalSystemData 1 }
44
45  lldpV2LocChassisId  OBJECT-TYPE
46      SYNTAX      LldpV2ChassisId
47      MAX-ACCESS  read-only
48      STATUS      current
49      DESCRIPTION
50          "The string value used to identify the chassis component
51          associated with the local system."
52      REFERENCE
53          "8.5.2.3"
54      ::= { lldpV2LocalSystemData 2 }
55
56  lldpV2LocSysName  OBJECT-TYPE
57      SYNTAX      SnmpAdminString (SIZE(0..255))
58      MAX-ACCESS  read-only
59      STATUS      current

```

```

1      DESCRIPTION
2          "The string value used to identify the system name of the
3          local system. If the local agent supports IETF RFC 3418,
4          lldpLocSysName object should have the same value as sysName
5          object."
6      REFERENCE
7          "8.5.6.2"
8      ::= { lldpV2LocalSystemData 3 }
9
10 lldpV2LocSysDesc OBJECT-TYPE
11     SYNTAX      SnmpAdminString (SIZE(0..255))
12     MAX-ACCESS  read-only
13     STATUS      current
14     DESCRIPTION
15         "The string value used to identify the system description
16         of the local system. If the local agent supports IETF RFC 3418,
17         lldpLocSysDesc object should have the same value as sysDesc
18         object."
19     REFERENCE
20         "8.5.7.2"
21     ::= { lldpV2LocalSystemData 4 }
22
23 lldpV2LocSysCapSupported OBJECT-TYPE
24     SYNTAX      LldpV2SystemCapabilitiesMap
25     MAX-ACCESS  read-only
26     STATUS      current
27     DESCRIPTION
28         "The bitmap value used to identify which system capabilities
29         are supported on the local system."
30     REFERENCE
31         "8.5.8.1"
32     ::= { lldpV2LocalSystemData 5 }
33
34 lldpV2LocSysCapEnabled OBJECT-TYPE
35     SYNTAX      LldpV2SystemCapabilitiesMap
36     MAX-ACCESS  read-only
37     STATUS      current
38     DESCRIPTION
39         "The bitmap value used to identify which system capabilities
40         are enabled on the local system."
41     REFERENCE
42         "8.5.8.2"
43     ::= { lldpV2LocalSystemData 6 }
44
45
46 --
47 -- lldpV2LocPortTable : Port specific Local system data
48 -- Indexed by ifIndex.
49 --
50
51 lldpV2LocPortTable OBJECT-TYPE
52     SYNTAX      SEQUENCE OF LldpV2LocPortEntry
53     MAX-ACCESS  not-accessible
54     STATUS      current
55     DESCRIPTION
56         "This table contains one row per port of information
57         associated with the local system known to this agent."
58     ::= { lldpV2LocalSystemData 7 }
59

```

```

1 lldpV2LocPortEntry OBJECT-TYPE
2     SYNTAX      LldpV2LocPortEntry
3     MAX-ACCESS  not-accessible
4     STATUS      current
5     DESCRIPTION
6         "Information about a particular port component.
7
8         Entries may be created and deleted in this table by the
9         agent.
10
11        Rows in this table can only be created for MAC addresses
12        that can validly be used in association with the type of
13        interface concerned, as defined by Table 7-2.
14
15        The contents of this table is persistent across
16        re-initializations or re-boots."
17     INDEX      { lldpV2LocPortIfIndex }
18     ::= { lldpV2LocPortTable 1 }
19
20 LldpV2LocPortEntry ::= SEQUENCE {
21     lldpV2LocPortIfIndex      InterfaceIndex,
22     lldpV2LocPortIdSubtype    LldpV2PortIdSubtype,
23     lldpV2LocPortId           LldpV2PortId,
24     lldpV2LocPortDesc         SnmpAdminString
25 }
26
27 lldpV2LocPortIfIndex  OBJECT-TYPE
28     SYNTAX      InterfaceIndex
29     MAX-ACCESS  not-accessible
30     STATUS      current
31     DESCRIPTION
32         "The interface index value used to identify the port
33         associated with this entry. Its value is an index
34         into the interfaces MIB.
35
36         The value of this object is used as an index to the
37         lldpV2LocPortTable."
38     ::= { lldpV2LocPortEntry 1 }
39
40 lldpV2LocPortIdSubtype OBJECT-TYPE
41     SYNTAX      LldpV2PortIdSubtype
42     MAX-ACCESS  read-only
43     STATUS      current
44     DESCRIPTION
45         "The type of port identifier encoding used in the associated
46         'lldpLocPortId' object."
47     REFERENCE
48         "8.5.3.2"
49     ::= { lldpV2LocPortEntry 2 }
50
51 lldpV2LocPortId  OBJECT-TYPE
52     SYNTAX      LldpV2PortId
53     MAX-ACCESS  read-only
54     STATUS      current
55     DESCRIPTION
56         "The string value used to identify the port component
57         associated with a given port in the local system."
58     REFERENCE
59         "8.5.3.3"

```

```

1 ::= { lldpV2LocPortEntry 3 }
2
3 lldpV2LocPortDesc OBJECT-TYPE
4     SYNTAX      SnmpAdminString (SIZE(0..255))
5     MAX-ACCESS  read-only
6     STATUS      current
7     DESCRIPTION
8         "The string value used to identify the IEEE 802 LAN station's port
9         description associated with the local system. If the local
10        agent supports IETF RFC 2863, lldpLocPortDesc object should
11        have the same value of ifDescr object."
12    REFERENCE
13        "8.5.5.2"
14    ::= { lldpV2LocPortEntry 4 }
15
16 --
17 -- lldpV2LocManAddrTable : Management addresses of the local system
18 --
19
20 lldpV2LocManAddrTable OBJECT-TYPE
21     SYNTAX      SEQUENCE OF LldpV2LocManAddrEntry
22     MAX-ACCESS  not-accessible
23     STATUS      current
24     DESCRIPTION
25         "This table contains management address information on the
26         local system known to this agent."
27     ::= { lldpV2LocalSystemData 8 }
28
29 lldpV2LocManAddrEntry OBJECT-TYPE
30     SYNTAX      LldpV2LocManAddrEntry
31     MAX-ACCESS  not-accessible
32     STATUS      current
33     DESCRIPTION
34         "Management address information about a particular chassis
35         component. There may be multiple management addresses
36         configured on the system identified by a particular
37         lldpLocChassisId. Each management address should have
38         distinct 'management address type' (lldpV2LocManAddrSubtype) and
39         'management address' (lldpLocManAddr.)
40
41         Entries may be created and deleted in this table by the
42         agent.
43
44         Since a variable length octetstring is used as an index
45         in a table, the address length is encoded as part of the OID
46         (as per IETF RFC 2578)."
```

INDEX { lldpV2LocManAddrSubtype,
lldpV2LocManAddr }

```

50 ::= { lldpV2LocManAddrTable 1 }
51
52 LldpV2LocManAddrEntry ::= SEQUENCE {
53     lldpV2LocManAddrSubtype  AddressFamilyNumbers,
54     lldpV2LocManAddr         LldpV2ManAddress,
55     lldpV2LocManAddrLen      Unsigned32,
56     lldpV2LocManAddrIfSubtype LldpV2ManAddrIfSubtype,
57     lldpV2LocManAddrIfId     Unsigned32,
58     lldpV2LocManAddrOID      OBJECT IDENTIFIER
59 }
```



```
1
2 lldpV2LocManAddrSubtype OBJECT-TYPE
3     SYNTAX      AddressFamilyNumbers
4     MAX-ACCESS  not-accessible
5     STATUS      current
6     DESCRIPTION
7         "The type of management address identifier encoding used in
8         the associated 'lldpLocManagmentAddr' object.
9
10        It should be noted that only a subset of the possible
11        address encodings enumerated in AddressFamilyNumbers
12        are appropriate for use as a LLDP management
13        address, either because some are just not applicable or
14        because the maximum size of a LldpV2ManAddress octet string
15        would prevent the use of some address identifier encodings."
16     REFERENCE
17         "8.5.9.3"
18     ::= { lldpV2LocManAddrEntry 1 }
19
20 lldpV2LocManAddr OBJECT-TYPE
21     SYNTAX      LldpV2ManAddress
22     MAX-ACCESS  not-accessible
23     STATUS      current
24     DESCRIPTION
25         "The string value used to identify the management address
26         component associated with the local system. The purpose of
27         this address is to contact the management entity."
28     REFERENCE
29         "8.5.9.4"
30     ::= { lldpV2LocManAddrEntry 2 }
31
32 lldpV2LocManAddrLen OBJECT-TYPE
33     SYNTAX      Unsigned32
34     MAX-ACCESS  read-only
35     STATUS      current
36     DESCRIPTION
37         "The total length of the management address subtype and the
38         management address fields in LLDPDUs transmitted by the
39         local LLDP agent.
40
41        The management address length field is needed so that the
42        receiving systems that do not implement SNMP are not
43        required to implement an Internet Assigned Numbers
44        Authority (IANA) family numbers/address length equivalency
45        table in order to decode the management address."
46     REFERENCE
47         "8.5.9.2"
48     ::= { lldpV2LocManAddrEntry 3 }
49
50 lldpV2LocManAddrIfSubtype OBJECT-TYPE
51     SYNTAX      LldpV2ManAddrIfSubtype
52     MAX-ACCESS  read-only
53     STATUS      current
54     DESCRIPTION
55         "The enumeration value that identifies the interface numbering
56         method used for defining the interface number
57         (lldpV2LocManAddrIfId), associated with the local system."
58     REFERENCE
59         "8.5.9.5"
```

```

1 ::= { lldpV2LocManAddrEntry 4 }
2
3 lldpV2LocManAddrIfId OBJECT-TYPE
4     SYNTAX      Unsigned32
5     MAX-ACCESS  read-only
6     STATUS      current
7     DESCRIPTION
8         "The integer value used to identify the interface number
9         regarding the management address component associated with
10        the local system."
11    REFERENCE
12        "8.5.9.6"
13    ::= { lldpV2LocManAddrEntry 5 }
14
15 lldpV2LocManAddrOID OBJECT-TYPE
16     SYNTAX      OBJECT IDENTIFIER
17     MAX-ACCESS  read-only
18     STATUS      current
19     DESCRIPTION
20         "The OID value used to identify the type of hardware component
21         or protocol entity associated with the management address
22         advertised by the local system agent."
23    REFERENCE
24        "8.5.9.8"
25    ::= { lldpV2LocManAddrEntry 6 }
26
27
28 -- *****
29 --
30 --             R E M O T E       S Y S T E M S       D A T A
31 --
32 -- *****
33
34
35 --
36 -- lldpV2RemTable
37 -- Indexed by ifIndex and destination MAC address.
38 --
39
40 lldpV2RemTable OBJECT-TYPE
41     SYNTAX      SEQUENCE OF LldpV2RemEntry
42     MAX-ACCESS  not-accessible
43     STATUS      current
44     DESCRIPTION
45         "This table contains one or more rows per physical network
46         connection known to this agent. The agent may wish to ensure
47         that only one lldpRemEntry is present for each local port
48         and destination MAC address,
49         or it may choose to maintain multiple lldpRemEntries for
50         the same local port and destination MAC address.
51
52         The following procedure may be used to retrieve remote
53         systems information updates from an LLDP agent:
54
55             1. NMS polls all tables associated with remote systems
56                and keeps a local copy of the information retrieved.
57                NMS polls periodically the values of the following
58                objects:
59                a. lldpV2StatsRemTablesInserts

```

```

1          b. lldpV2StatsRemTablesDeletes
2          c. lldpV2StatsRemTablesDrops
3          d. lldpV2StatsRemTablesAgeouts
4          e. lldpV2StatsRxPortAgeoutsTotal for all ports.
5
6      2. LLDP agent updates remote systems MIB objects, and
7         sends out notifications to a list of notification
8         destinations.
9
10     3. NMS receives the notifications and compares the new
11        values of objects listed in step 1.
12
13        Periodically, NMS should poll the object
14        lldpV2StatsRemTablesLastChangeTime to find out if anything
15        has changed since the last poll. If something has
16        changed, NMS polls the objects listed in step 1 to
17        figure out what kind of changes occurred in the tables.
18
19        If value of lldpV2StatsRemTablesInserts has changed,
20        then NMS walks all tables by employing TimeFilter
21        with the last-pollled time value. This request
22        returns new objects or objects whose values have been
23        updated since the last poll.
24
25        If value of lldpV2StatsRemTablesAgeouts has changed,
26        then NMS walks the lldpStatsRxPortAgeoutsTotal and
27        compares the new values with previously recorded ones.
28        For ports whose lldpStatsRxPortAgeoutsTotal value is
29        greater than the recorded value, NMS can
30        retrieve objects associated with those ports from
31        table(s) without employing a TimeFilter (which is
32        performed by specifying 0 for the TimeFilter).
33
34        lldpV2StatsRemTablesDeletes and lldpV2StatsRemTablesDrops
35        objects are provided for informational purposes."
36 ::= { lldpV2RemoteSystemsData 1 }
37
38 lldpV2RemEntry OBJECT-TYPE
39     SYNTAX      LldpV2RemEntry
40     MAX-ACCESS  not-accessible
41     STATUS      current
42     DESCRIPTION
43         "Information about a particular physical network connection.
44         Entries may be created and deleted in this table by the agent,
45         if a physical topology discovery process is active.
46
47         Rows in this table can only be created for MAC addresses
48         that can validly be used in association with the type of
49         interface concerned, as defined by Table 7-2.
50
51         The contents of this table is persistent across
52         re-initializations or re-boots."
53     INDEX      {
54         lldpV2RemTimeMark,
55         lldpV2RemLocalIfIndex,
56         lldpV2RemLocalDestMACAddress,
57         lldpV2RemIndex
58     }
59 ::= { lldpV2RemTable 1 }

```

```

1
2 LldpV2RemEntry ::= SEQUENCE {
3     lldpV2RemTimeMark      TimeFilter,
4     lldpV2RemLocalIfIndex  InterfaceIndex,
5     lldpV2RemLocalDestMACAddress LldpV2DestAddressTableIndex,
6     lldpV2RemIndex         Unsigned32,
7     lldpV2RemChassisIdSubtype LldpV2ChassisIdSubtype,
8     lldpV2RemChassisId     LldpV2ChassisId,
9     lldpV2RemPortIdSubtype LldpV2PortIdSubtype,
10    lldpV2RemPortId        LldpV2PortId,
11    lldpV2RemPortDesc      SnmpAdminString,
12    lldpV2RemSysName       SnmpAdminString,
13    lldpV2RemSysDesc       SnmpAdminString,
14    lldpV2RemSysCapSupported LldpV2SystemCapabilitiesMap,
15    lldpV2RemSysCapEnabled  LldpV2SystemCapabilitiesMap,
16    lldpV2RemRemoteChanges  TruthValue,
17    lldpV2RemTooManyNeighbors TruthValue
18 }
19
20 lldpV2RemTimeMark OBJECT-TYPE
21     SYNTAX      TimeFilter
22     MAX-ACCESS  not-accessible
23     STATUS      current
24     DESCRIPTION
25         "A TimeFilter for this entry. See the TimeFilter textual
26         convention in IETF RFC 4502 and
27         http://www.ietf.org/IESG/Implementations/RFC2021-Implementation.txt
28         to see how TimeFilter works."
29     REFERENCE
30         "IETF RFC 4502 section 6"
31     ::= { lldpV2RemEntry 1 }
32
33
34 lldpV2RemLocalIfIndex OBJECT-TYPE
35     SYNTAX      InterfaceIndex
36     MAX-ACCESS  not-accessible
37     STATUS      current
38     DESCRIPTION
39         "The interface index value used to identify the port
40         associated with this entry. Its value is an index
41         into the interfaces MIB
42
43         The value of this object is used as an index to the
44         lldpV2RemTable."
45     ::= { lldpV2RemEntry 2 }
46
47 lldpV2RemLocalDestMACAddress OBJECT-TYPE
48     SYNTAX      LldpV2DestAddressTableIndex
49     MAX-ACCESS  not-accessible
50     STATUS      current
51     DESCRIPTION
52         "The index value used to identify the destination
53         MAC address associated with this entry. Its value identifies
54         the row in the lldpV2DestAddressTable where the MAC address
55         can be found.
56
57         The value of this object is used as an index to the
58         lldpV2RemTable."
59     ::= { lldpV2RemEntry 3 }

```

```
1
2
3 lldpV2RemIndex    OBJECT-TYPE
4     SYNTAX          Unsigned32(1..2147483647)
5     MAX-ACCESS      not-accessible
6     STATUS          current
7     DESCRIPTION
8         "This object represents an arbitrary local integer value used
9         by this agent to identify a particular connection instance,
10        unique only for the indicated remote system.
11
12        An agent is encouraged to assign monotonically increasing
13        index values to new entries, starting with one, after each
14        reboot. It is considered unlikely that the lldpRemIndex
15        can wrap between reboots."
16    ::= { lldpV2RemEntry 4 }
17
18 lldpV2RemChassisIdSubtype OBJECT-TYPE
19     SYNTAX          LldpV2ChassisIdSubtype
20     MAX-ACCESS      read-only
21     STATUS          current
22     DESCRIPTION
23         "The type of encoding used to identify the chassis associated
24         with the remote system."
25     REFERENCE
26         "8.5.2.2"
27    ::= { lldpV2RemEntry 5 }
28
29 lldpV2RemChassisId OBJECT-TYPE
30     SYNTAX          LldpV2ChassisId
31     MAX-ACCESS      read-only
32     STATUS          current
33     DESCRIPTION
34         "The string value used to identify the chassis component
35         associated with the remote system."
36     REFERENCE
37         "8.5.2.3"
38    ::= { lldpV2RemEntry 6 }
39
40 lldpV2RemPortIdSubtype OBJECT-TYPE
41     SYNTAX          LldpV2PortIdSubtype
42     MAX-ACCESS      read-only
43     STATUS          current
44     DESCRIPTION
45         "The type of port identifier encoding used in the associated
46         'lldpRemPortId' object."
47     REFERENCE
48         "8.5.3.2"
49    ::= { lldpV2RemEntry 7 }
50
51 lldpV2RemPortId OBJECT-TYPE
52     SYNTAX          LldpV2PortId
53     MAX-ACCESS      read-only
54     STATUS          current
55     DESCRIPTION
56         "The string value used to identify the port component
57         associated with the remote system."
58     REFERENCE
59         "8.5.3.3"
```

```
1 ::= { lldpV2RemEntry 8 }
2
3 lldpV2RemPortDesc OBJECT-TYPE
4     SYNTAX      SnmpAdminString (SIZE(0..255))
5     MAX-ACCESS  read-only
6     STATUS      current
7     DESCRIPTION
8         "The string value used to identify the description of
9         the given port associated with the remote system."
10    REFERENCE
11        "8.5.5.2"
12    ::= { lldpV2RemEntry 9 }
13
14 lldpV2RemSysName OBJECT-TYPE
15     SYNTAX      SnmpAdminString (SIZE(0..255))
16     MAX-ACCESS  read-only
17     STATUS      current
18     DESCRIPTION
19         "The string value used to identify the system name of the
20         remote system."
21    REFERENCE
22        "8.5.6.2"
23    ::= { lldpV2RemEntry 10 }
24
25 lldpV2RemSysDesc OBJECT-TYPE
26     SYNTAX      SnmpAdminString (SIZE(0..255))
27     MAX-ACCESS  read-only
28     STATUS      current
29     DESCRIPTION
30         "The string value used to identify the system description
31         of the remote system."
32    REFERENCE
33        "8.5.7.2"
34    ::= { lldpV2RemEntry 11 }
35
36 lldpV2RemSysCapSupported OBJECT-TYPE
37     SYNTAX      LldpV2SystemCapabilitiesMap
38     MAX-ACCESS  read-only
39     STATUS      current
40     DESCRIPTION
41         "The bitmap value used to identify which system capabilities
42         are supported on the remote system."
43    REFERENCE
44        "8.5.8.1"
45    ::= { lldpV2RemEntry 12 }
46
47 lldpV2RemSysCapEnabled OBJECT-TYPE
48     SYNTAX      LldpV2SystemCapabilitiesMap
49     MAX-ACCESS  read-only
50     STATUS      current
51     DESCRIPTION
52         "The bitmap value used to identify which system capabilities
53         are enabled on the remote system."
54    REFERENCE
55        "8.5.8.2"
56    ::= { lldpV2RemEntry 13 }
57
58 lldpV2RemRemoteChanges OBJECT-TYPE
59     SYNTAX      TruthValue
```

```

1     MAX-ACCESS    read-only
2     STATUS        current
3     DESCRIPTION
4         "Indicates that there are changes in the remote systems
5         MIB, as determined by the variable remoteChanges."
6     REFERENCE
7         "9.2.5.10"
8     ::= { lldpV2RemEntry 14 }
9
10    lldpV2RemTooManyNeighbors OBJECT-TYPE
11        SYNTAX      TruthValue
12        MAX-ACCESS  read-only
13        STATUS      current
14        DESCRIPTION
15            "Indicates that there are too many neighbors
16            as determined by the variable tooManyNeighbors."
17        REFERENCE
18            "9.2.5.16"
19        ::= { lldpV2RemEntry 15 }
20
21 --
22 -- lldpV2RemManAddrTable : Management addresses of the remote system
23 -- Version 2 includes additional index values for ifIndex and
24 -- destination MAC address.
25 --
26
27    lldpV2RemManAddrTable OBJECT-TYPE
28        SYNTAX      SEQUENCE OF LldpV2RemManAddrEntry
29        MAX-ACCESS  not-accessible
30        STATUS      current
31        DESCRIPTION
32            "This table contains one or more rows per management address
33            information on the remote system learned on a particular port
34            contained in the local chassis known to this agent."
35        ::= { lldpV2RemoteSystemsData 2 }
36
37    lldpV2RemManAddrEntry OBJECT-TYPE
38        SYNTAX      LldpV2RemManAddrEntry
39        MAX-ACCESS  not-accessible
40        STATUS      current
41        DESCRIPTION
42            "Management address information about a particular chassis
43            component. There may be multiple management addresses
44            configured on the remote system identified by a particular
45            lldpRemIndex whose information is received on
46            an interface of the local system and a given destination
47            MAC address. Each management
48            address should have distinct 'management address
49            type' (lldpRemManAddrSubtype) and 'management address'
50            (lldpRemManAddr).
51
52            Entries may be created and deleted in this table by the
53            agent.
54            Since a variable length octetstring is used as an index
55            in a table, the address length is encoded as part of the OID
56            (as per IETF RFC 2578)."
```

INDEX { lldpV2RemTimeMark,
 lldpV2RemLocalIfIndex,
 lldpV2RemLocalDestMACAddress,

```

1         lldpV2RemIndex,
2         lldpV2RemManAddrSubtype,
3         lldpV2RemManAddr
4     }
5     ::= { lldpV2RemManAddrTable 1 }
6
7
8 LldpV2RemManAddrEntry ::= SEQUENCE {
9     lldpV2RemManAddrSubtype    AddressFamilyNumbers,
10    lldpV2RemManAddr            LldpV2ManAddress,
11    lldpV2RemManAddrIfSubtype   LldpV2ManAddrIfSubtype,
12    lldpV2RemManAddrIfId        Unsigned32,
13    lldpV2RemManAddrOID         OBJECT IDENTIFIER
14 }
15
16 lldpV2RemManAddrSubtype OBJECT-TYPE
17     SYNTAX      AddressFamilyNumbers
18     MAX-ACCESS  not-accessible
19     STATUS      current
20     DESCRIPTION
21         "The type of management address identifier encoding used in
22         the associated 'lldpRemManagmentAddr' object.
23
24         It should be noted that only a subset of the possible
25         address encodings enumerated in AddressFamilyNumbers
26         are appropriate for use as a LLDP management
27         address, either because some are just not apliccable or
28         because the maximum size of a LldpV2ManAddress octet string
29         would prevent the use of some address identifier encodings."
30     REFERENCE
31         "8.5.9.3"
32     ::= { lldpV2RemManAddrEntry 1 }
33
34 lldpV2RemManAddr OBJECT-TYPE
35     SYNTAX      LldpV2ManAddress
36     MAX-ACCESS  not-accessible
37     STATUS      current
38     DESCRIPTION
39         "The string value used to identify the management address
40         component associated with the remote system. The purpose
41         of this address is to contact the management entity."
42     REFERENCE
43         "8.5.9.4"
44     ::= { lldpV2RemManAddrEntry 2 }
45
46 lldpV2RemManAddrIfSubtype OBJECT-TYPE
47     SYNTAX      LldpV2ManAddrIfSubtype
48     MAX-ACCESS  read-only
49     STATUS      current
50     DESCRIPTION
51         "The enumeration value that identifies the interface numbering
52         method used for defining the interface number, associated
53         with the remote system."
54     REFERENCE
55         "8.5.9.5"
56     ::= { lldpV2RemManAddrEntry 3 }
57
58 lldpV2RemManAddrIfId OBJECT-TYPE
59     SYNTAX      Unsigned32

```



```

1     MAX-ACCESS    read-only
2     STATUS        current
3     DESCRIPTION
4         "The integer value used to identify the interface number
5         regarding the management address component associated with
6         the remote system. The value depends upon the value of the
7         lldpV2RemManAddrIfSubtype for the table row."
8     REFERENCE
9         "8.5.9.6"
10    ::= { lldpV2RemManAddrEntry 4 }
11
12 lldpV2RemManAddrOID OBJECT-TYPE
13     SYNTAX        OBJECT IDENTIFIER
14     MAX-ACCESS    read-only
15     STATUS        current
16     DESCRIPTION
17         "The OID value used to identify the type of hardware component
18         or protocol entity associated with the management address
19         advertised by the remote system agent."
20     REFERENCE
21         "8.5.9.8"
22    ::= { lldpV2RemManAddrEntry 5 }
23
24
25 --
26 -- lldpV2RemUnknownTLVTable : Unrecognized TLV information
27 -- This version has additional indexes for
28 -- ifIndex and destination MAC address
29 --
30
31 lldpV2RemUnknownTLVTable OBJECT-TYPE
32     SYNTAX        SEQUENCE OF LldpV2RemUnknownTLVEntry
33     MAX-ACCESS    not-accessible
34     STATUS        current
35     DESCRIPTION
36         "This table contains information about an incoming TLV that
37         is not recognized by the receiving LLDP agent. The TLV may
38         be from a later version of the basic management set.
39
40         This table should only contain TLVs that are found in
41         a single LLDP frame. Entries in this table, associated
42         with an MAC service access point (MSAP, the access point
43         for MAC services provided to the LCC sublayer, defined
44         in IEEE Standards Dictionary Online, which is also
45         identified with a particular lldpRemLocalPortNum,
46         lldpRemIndex pair) are overwritten with most recently
47         received unrecognized TLV from the same MSAP, or they
48         naturally age out when the rxInfoTTL timer
49         (associated with the MSAP) expires."
50     REFERENCE
51         "9.2.8.7.1"
52    ::= { lldpV2RemoteSystemsData 3 }
53
54 lldpV2RemUnknownTLVEntry OBJECT-TYPE
55     SYNTAX        LldpV2RemUnknownTLVEntry
56     MAX-ACCESS    not-accessible
57     STATUS        current
58     DESCRIPTION
59         "Information about an unrecognized TLV received from a

```

```

1         physical network connection. Entries may be created and
2         deleted in this table by the agent, if a physical topology
3         discovery process is active."
4     INDEX {
5         lldpV2RemTimeMark,
6         lldpV2RemLocalIfIndex,
7         lldpV2RemLocalDestMACAddress,
8         lldpV2RemIndex,
9         lldpV2RemUnknownTLVType
10    }
11    ::= { lldpV2RemUnknownTLVTable 1 }
12
13 lldpV2RemUnknownTLVEntry ::= SEQUENCE {
14     lldpV2RemUnknownTLVType      Unsigned32,
15     lldpV2RemUnknownTLVInfo      OCTET STRING
16 }
17
18 lldpV2RemUnknownTLVType OBJECT-TYPE
19     SYNTAX      Unsigned32(9..126)
20     MAX-ACCESS  not-accessible
21     STATUS      current
22     DESCRIPTION
23         "This object represents the value extracted from the type
24         field of the TLV."
25     REFERENCE
26         "9.2.8.7.1"
27     ::= { lldpV2RemUnknownTLVEntry 1 }
28
29 lldpV2RemUnknownTLVInfo OBJECT-TYPE
30     SYNTAX      OCTET STRING (SIZE(0..511))
31     MAX-ACCESS  read-only
32     STATUS      current
33     DESCRIPTION
34         "This object represents the value extracted from the value
35         field of the TLV."
36     REFERENCE
37         "9.2.8.7.1"
38     ::= { lldpV2RemUnknownTLVEntry 2 }
39
40 -----
41 -- Remote Systems Extension Table - Organizationally Defined Information
42 -----
43 --
44 -- lldpV2RemOrgDefInfoTable - indexed by ifIndex and destination
45 -- MAC address.
46 --
47
48 lldpV2RemOrgDefInfoTable OBJECT-TYPE
49     SYNTAX      SEQUENCE OF LldpV2RemOrgDefInfoEntry
50     MAX-ACCESS  not-accessible
51     STATUS      current
52     DESCRIPTION
53         "This table contains one or more rows per physical network
54         connection which advertises the organizationally defined
55         information.
56
57         Note that this table contains one or more rows of
58         organizationally defined information that is not recognized
59         by the local agent.

```

```

1
2         If the local system is capable of recognizing any
3         organizationally defined information, appropriate extension
4         MIBs from the organization should be used for information
5         retrieval."
6 ::= { lldpV2RemoteSystemsData 4 }
7
8 lldpV2RemOrgDefInfoEntry OBJECT-TYPE
9     SYNTAX      LldpV2RemOrgDefInfoEntry
10    MAX-ACCESS   not-accessible
11    STATUS      current
12    DESCRIPTION
13        "Information about the unrecognized organizationally
14        defined information advertised by the remote system.
15        The lldpRemTimeMark, lldpRemLocalPortNum, lldpRemIndex,
16        lldpRemOrgDefInfoOUI, lldpRemOrgDefInfoSubtype, and
17        lldpRemOrgDefInfoIndex are indexes to this table. If there is
18        an lldpRemOrgDefInfoEntry associated with a particular remote
19        system identified by the lldpRemLocalPortNum and lldpRemIndex,
20        then there is an lldpRemEntry associated with the same
21        instance (i.e., using same indexes.) When the lldpRemEntry
22        for the same index is removed from the lldpRemTable, the
23        associated lldpRemOrgDefInfoEntry is removed from
24        the lldpRemOrgDefInfoTable.
25
26        Entries may be created and deleted in this table by the
27        agent."
28    INDEX      { lldpV2RemTimeMark,
29                lldpV2RemLocalIfIndex,
30                lldpV2RemLocalDestMACAddress,
31                lldpV2RemIndex,
32                lldpV2RemOrgDefInfoOUI,
33                lldpV2RemOrgDefInfoSubtype,
34                lldpV2RemOrgDefInfoIndex }
35 ::= { lldpV2RemOrgDefInfoTable 1 }
36
37 LldpV2RemOrgDefInfoEntry ::= SEQUENCE {
38     lldpV2RemOrgDefInfoOUI      OCTET STRING,
39     lldpV2RemOrgDefInfoSubtype   Unsigned32,
40     lldpV2RemOrgDefInfoIndex     Unsigned32,
41     lldpV2RemOrgDefInfo         OCTET STRING
42 }
43
44 lldpV2RemOrgDefInfoOUI OBJECT-TYPE
45     SYNTAX      OCTET STRING (SIZE(3))
46     MAX-ACCESS   not-accessible
47     STATUS      current
48     DESCRIPTION
49         "The Organizationally Unique Identifier (OUI), as defined
50         in IEEE Std 802, is a 24 bit (three octets) globally
51         unique assigned number referenced by various standards,
52         of the information received from the remote system."
53     REFERENCE
54         "8.6.1.3"
55 ::= { lldpV2RemOrgDefInfoEntry 1 }
56
57 lldpV2RemOrgDefInfoSubtype OBJECT-TYPE
58     SYNTAX      Unsigned32(1..255)
59     MAX-ACCESS   not-accessible

```

```

1      STATUS      current
2      DESCRIPTION
3          "The integer value used to identify the subtype of the
4          organizationally defined information received from the
5          remote system.
6
7          The subtype value is required to identify different instances
8          of organizationally defined information that could not be
9          retrieved without a unique identifier that indicates the
10         particular type of information contained in the information
11         string."
12     REFERENCE
13         "8.6.1.4"
14     ::= { lldpV2RemOrgDefInfoEntry 2 }
15
16 lldpV2RemOrgDefInfoIndex OBJECT-TYPE
17     SYNTAX      Unsigned32(1..2147483647)
18     MAX-ACCESS  not-accessible
19     STATUS      current
20     DESCRIPTION
21         "This object represents an arbitrary local integer value
22         used by this agent to identify a particular unrecognized
23         organizationally defined information instance, unique only
24         for the lldpRemOrgDefInfoOUI and lldpRemOrgDefInfoSubtype
25         from the same remote system.
26
27         An agent is encouraged to assign monotonically increasing
28         index values to new entries, starting with one, after each
29         reboot. It is considered unlikely that the
30         lldpRemOrgDefInfoIndex can wrap between reboots."
31     ::= { lldpV2RemOrgDefInfoEntry 3 }
32
33 lldpV2RemOrgDefInfo OBJECT-TYPE
34     SYNTAX      OCTET STRING(SIZE(0..507))
35     MAX-ACCESS  read-only
36     STATUS      current
37     DESCRIPTION
38         "The string value used to identify the organizationally
39         defined information of the remote system. The encoding for
40         this object should be as defined for SnmpAdminString TC."
41     REFERENCE
42         "8.6.1.5"
43     ::= { lldpV2RemOrgDefInfoEntry 4 }
44
45
46 --
47 -- *****
48 --
49 --      L L D P      M I B      N O T I F I C A T I O N S
50 --
51 -- *****
52 --
53
54 lldpV2NotificationPrefix OBJECT IDENTIFIER ::= { lldpV2Notifications 0 }
55
56 lldpV2RemTablesChange NOTIFICATION-TYPE
57     OBJECTS {
58         lldpV2StatsRemTablesInserts,
59         lldpV2StatsRemTablesDeletes,

```

```

1      lldpV2StatsRemTablesDrops,
2      lldpV2StatsRemTablesAgeouts
3  }
4  STATUS      current
5  DESCRIPTION
6      "A lldpV2RemTablesChange notification is sent when the value
7      of lldpV2StatsRemTablesLastChangeTime changes. It can be
8      utilized by an NMS to trigger LLDP remote systems table
9      maintenance polls.
10
11      Note that transmission of lldpV2RemTablesChange
12      notifications are throttled by the agent, as specified by the
13      'lldpV2NotificationInterval' object."
14  ::= { lldpV2NotificationPrefix 1 }
15
16
17 --
18 -- *****
19 --
20 --      L L D P   M I B   C O N F O R M A N C E
21 --
22 -- *****
23 --
24
25 lldpV2Compliances OBJECT IDENTIFIER ::= { lldpV2Conformance 1 }
26 lldpV2Groups      OBJECT IDENTIFIER ::= { lldpV2Conformance 2 }
27
28 -- compliance statements
29
30 lldpV2TxRxCompliance MODULE-COMPLIANCE
31     --V2 to add ifGeneralInformationGroup
32     --and support re-indexed tables
33     STATUS      current
34     DESCRIPTION
35         "A compliance statement for all SNMP entities that
36         implement the LLDP MIB as either a transmitter or
37         a receiver of LLDPDUs.
38
39         This version defines compliance requirements for
40         V2 of the LLDP MIB module."
41     MODULE -- this module
42         MANDATORY-GROUPS { lldpV2ConfigGroup,
43                             ifGeneralInformationGroup
44         }
45
46     ::= { lldpV2Compliances 1 }
47
48 lldpV2TxCompliance MODULE-COMPLIANCE
49     --V2 requirements for transmitters of LLDPDUs
50     --and support re-indexed tables
51     STATUS      current
52     DESCRIPTION
53         "A compliance statement for SNMP entities that implement
54         the LLDP MIB and have the capability of transmitting
55         LLDP frames.
56
57         This version defines compliance requirements for
58         V2 of the LLDP MIB module."
59     MODULE -- this module

```

```
1      MANDATORY-GROUPS { lldpV2ConfigTxGroup,
2                          lldpV2StatsTxGroup,
3                          lldpV2LocSysGroup
4      }
5
6      ::= { lldpV2Compliances 2 }
7
8 lldpV2RxCompliance MODULE-COMPLIANCE
9      --V2 requirements for receivers of LLDPDUs
10     --and support re-indexed tables
11     STATUS current
12     DESCRIPTION
13         "The compliance statement for SNMP entities that implement
14         the LLDP MIB and have the capability of receiving
15         LLDP frames.
16
17         This version defines compliance requirements for
18         V2 of the LLDP MIB module."
19     MODULE -- this module
20         MANDATORY-GROUPS { lldpV2ConfigRxGroup,
21                             lldpV2StatsRxGroup,
22                             lldpV2RemSysGroup,
23                             lldpV2NotificationsGroup
24         }
25
26     ::= { lldpV2Compliances 3 }
27
28 -- MIB groupings
29
30
31 lldpV2ConfigGroup OBJECT-GROUP
32     OBJECTS {
33         lldpV2PortConfigAdminStatusV2
34     }
35     STATUS current
36     DESCRIPTION
37         "The collection of objects that are used to configure the
38         LLDP implementation behavior."
39     ::= { lldpV2Groups 1 }
40
41
42 lldpV2ConfigRxGroup OBJECT-GROUP
43     OBJECTS {
44         lldpV2NotificationInterval,
45         lldpV2PortConfigNotificationEnableV2
46     }
47     STATUS current
48     DESCRIPTION
49         "The collection of objects that are used to configure the
50         LLDP reception implementation behavior."
51     ::= { lldpV2Groups 2 }
52
53
54 lldpV2ConfigTxGroup OBJECT-GROUP
55     OBJECTS {
56         lldpV2MessageTxInterval,
57         lldpV2MessageTxHoldMultiplier,
58         lldpV2ReinitDelay,
59         lldpV2PortConfigTLVsTxEnableV2,
```

```

1      lldpV2ManAddrConfigTxEnable,
2      lldpV2ManAddrConfigRowStatus,
3      lldpV2TxCreditMax,
4      lldpV2MessageFastTx,
5      lldpV2TxFastInit,
6      lldpV2DestMacAddress,
7      lldpV2PortMessageTxInterval,
8      lldpV2PortMessageTxHoldMultiplier,
9      lldpV2PortReinitDelay,
10     lldpV2PortNotificationInterval,
11     lldpV2PortTxCreditMax,
12     lldpV2PortMessageFastTx,
13     lldpV2PortTxFastInit
14 }
15 STATUS current
16 DESCRIPTION
17     "The collection of objects that are used to configure the
18     LLDP transmission implementation behavior."
19 ::= { lldpV2Groups 3 }
20
21
22 lldpV2StatsRxGroup    OBJECT-GROUP
23     OBJECTS {
24         lldpV2StatsRemTablesLastChangeTime,
25         lldpV2StatsRemTablesInserts,
26         lldpV2StatsRemTablesDeletes,
27         lldpV2StatsRemTablesDrops,
28         lldpV2StatsRemTablesAgeouts,
29         lldpV2StatsRxPortFramesDiscardedTotal,
30         lldpV2StatsRxPortFramesErrors,
31         lldpV2StatsRxPortFramesTotal,
32         lldpV2StatsRxPortTLVsDiscardedTotal,
33         lldpV2StatsRxPortTLVsUnrecognizedTotal,
34         lldpV2StatsRxPortAgeoutsTotal
35     }
36 STATUS current
37 DESCRIPTION
38     "The collection of objects that are used to represent LLDP
39     reception statistics."
40 ::= { lldpV2Groups 4 }
41
42
43 lldpV2StatsTxGroup    OBJECT-GROUP
44     OBJECTS {
45         lldpV2StatsTxPortFramesTotal,
46         lldpV2StatsTxLLDPDULengthErrors
47     }
48 STATUS current
49 DESCRIPTION
50     "The collection of objects that are used to represent LLDP
51     transmission statistics."
52 ::= { lldpV2Groups 5 }
53
54
55 lldpV2LocSysGroup    OBJECT-GROUP
56     OBJECTS {
57         lldpV2LocChassisIdSubtype,
58         lldpV2LocChassisId,
59         lldpV2LocPortIdSubtype,

```

```

1      lldpV2LocPortId,
2      lldpV2LocPortDesc,
3      lldpV2LocSysDesc,
4      lldpV2LocSysName,
5      lldpV2LocSysCapSupported,
6      lldpV2LocSysCapEnabled,
7      lldpV2LocManAddrLen,
8      lldpV2LocManAddrIfSubtype,
9      lldpV2LocManAddrIfId,
10     lldpV2LocManAddrOID
11 }
12 STATUS current
13 DESCRIPTION
14     "The collection of objects that are used to represent LLDP
15     Local System Information."
16 ::= { lldpV2Groups 6 }
17
18 lldpV2RemSysGroup OBJECT-GROUP
19 OBJECTS {
20     lldpV2RemChassisIdSubtype,
21     lldpV2RemChassisId,
22     lldpV2RemPortIdSubtype,
23     lldpV2RemPortId,
24     lldpV2RemPortDesc,
25     lldpV2RemSysName,
26     lldpV2RemSysDesc,
27     lldpV2RemSysCapSupported,
28     lldpV2RemSysCapEnabled,
29     lldpV2RemRemoteChanges,
30     lldpV2RemTooManyNeighbors,
31     lldpV2RemManAddrIfSubtype,
32     lldpV2RemManAddrIfId,
33     lldpV2RemManAddrOID,
34     lldpV2RemUnknownTLVInfo,
35     lldpV2RemOrgDefInfo
36 }
37 STATUS current
38 DESCRIPTION
39     "The collection of objects that are used to represent
40     LLDP Remote Systems Information. The objects represent the
41     information associated with the basic TLV set. Please note
42     that even if the agent does not implement some of the optional
43     TLVs, it shall recognize all the optional TLV information
44     that the remote system may advertise."
45 ::= { lldpV2Groups 7 }
46
47 lldpV2NotificationsGroup NOTIFICATION-GROUP
48 NOTIFICATIONS {
49     lldpV2RemTablesChange
50 }
51 STATUS current
52 DESCRIPTION
53     "The collection of notifications used to indicate LLDP MIB
54     data consistency and general status information."
55 ::= { lldpV2Groups 8 }
56
57 END

```


12. LLDP YANG definitions

<<Editor's Note: All of this clause was added by IEEE Std 802.1ABcu-2021.>>

This clause specifies YANG modules that provide control and status monitoring of systems and system components that implement functionality specified in this standard.

This clause:

- a) Introduces the YANG framework that governs the naming and hierarchy of configuration and operational data structures in the data models, and the modeling of network interfaces (12.1).
- b) Describes the information data model and its relationship to the operational processes and managed objects specified in the other clauses of this standard, and provides a UML representation of each data model (12.2).
- c) Describes the structure of the data models, each of which comprises or makes use of one or more YANG modules (12.3).
- d) Includes a relationship description of other modules imported in YANG modules (12.4).
- e) Reviews security considerations applicable to each of the modules, with specific reference to data nodes in the YANG modules that compose the model (12.5).
- f) Includes each of the YANG modules and its data schema (12.6).

12.1 Internet Standard Management Framework

This YANG module uses the YANG 1.1 Data Modeling Language as specified in IETF RFC 7950.

The YANG framework applies hierarchy in the following areas:

- a) The uniform resource name (URN), as specified in IEEE Std 802.
- b) The YANG objects form a hierarchy of configuration and operational data structures that define the YANG model.

<<Editor's Note: The sentence "The structure of the URN is such that ieee is the root (i.e., name-space identifier), followed by the standard, then the working group developing the standard." was deleted from item a) above based on the maintenance request 0339.>>

<<Editor's Note: the reference to 802d has been replaced with a reference to 802 in item a) above since 802d is now included in 802.>>

12.2 Information model for LLDP management

The YANG objects are based on the TLVs detailed in 8.5 and 8.6, and the agent operation variables detailed in 9.1 and 9.2. A UML-like representation of the management model is provided in the following subclauses.

The purpose of a UML-like⁶ diagram is to express the model design on a single piece of paper. The structure of the UML-like representation shows the name of the object followed by a list of properties for the object. The properties indicate its type and accessibility. It should be noted that the UML-like representation is meant to express simplified semantics for the properties. It is not meant to provide the specific datatype as used to encode the object in either MIB or YANG. In the UML-like representation, a box with a white background represents information that comes from sources outside of the IEEE. A box with a gray background represents objects that are defined by this IEEE Standard.

⁶ A description of the UML-like diagrams used in this clause is provided at <https://1.ieee802.org/uml-like-diagrams>

1 The YANG hierarchical structure that incorporates the LLDP YANG modules supported by this standard is
2 represented by Figure Clause 12-1. In the figures in this clause, items that are shaded gray are described in
3 this document, items with no background shading are defined elsewhere. The YANG data model is realized
4 in two YANG modules. One module *ieee802-dot1ab-types* provides data types that are needed by the LLDP
5 configuration and monitoring objects. The *ieee802-dot1ab-lldp* module provides the LLDP configuration
6 and monitoring objects. The ability to augment the port container to support extension TLVs is also shown.
7 The LLDP capabilities are not only applicable to IEEE Std 802.1Q bridges, but also (for example) end
8 stations, and routers.

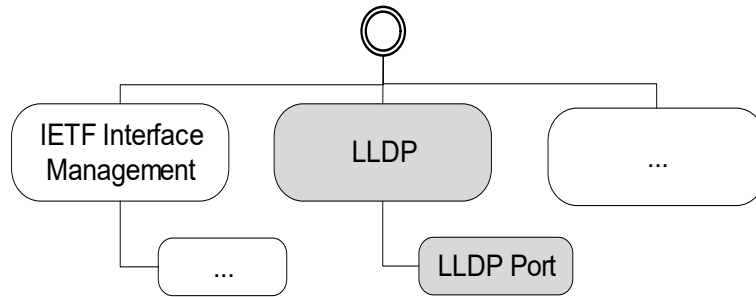


Figure 12-1—YANG root hierarchy with LLDP YANG modules

9 12.2.1 LLDP UML

10 The LLDP configuration and monitoring objects in Figure 12-2 show the objects that are applicable on a
11 per-agent or chassis associated with the LLDP agent.

1

lldp		
uint32	message-fast-tx;	// (9.2.5.5) r-w
uint32	message-tx-hold-multiplier;	// (9.2.5.6) r-w
uint32	message-tx-interval;	// (9.2.5.7) r-w
uint32	reinit-delay;	// (9.2.5.10) r-w
uint32	tx-credit-max;	// (9.2.5.17) r-w
uint32	tx-fast-init;	// (9.2.5.19) r-w
uint32	notification-interval;	// (9.2.5.7) r-w
remote-statistics		
timestamp	last-change-time;	// r
zero-based-counter32	remote-inserts;	// r
zero-based-counter32	remote-deletes;	// r
zero-based-counter32	remote-drops;	// r
zero-based-counter32	remote-ageouts;	// r
local-system-data		
enum	chassis-id-subtype;	// (8.5.2.2) r
string	chassis-id;	// (8.5.2.3) r
string	system-name;	// (8.5.6.2) r
string	system-description;	// (8.5.7.2) r
bits	system-capabilities-supported	// (8.5.8.1) r
bits	system-capabilities-enabled	// (8.5.8.2) r

Figure 12-2—LLDP configuration and monitoring objects

2 12.2.2 LLDP Port UML

3 The LLDP port configuration and monitoring objects in Figure 12-3 show the objects that are applicable to
4 an LLDP port.

1

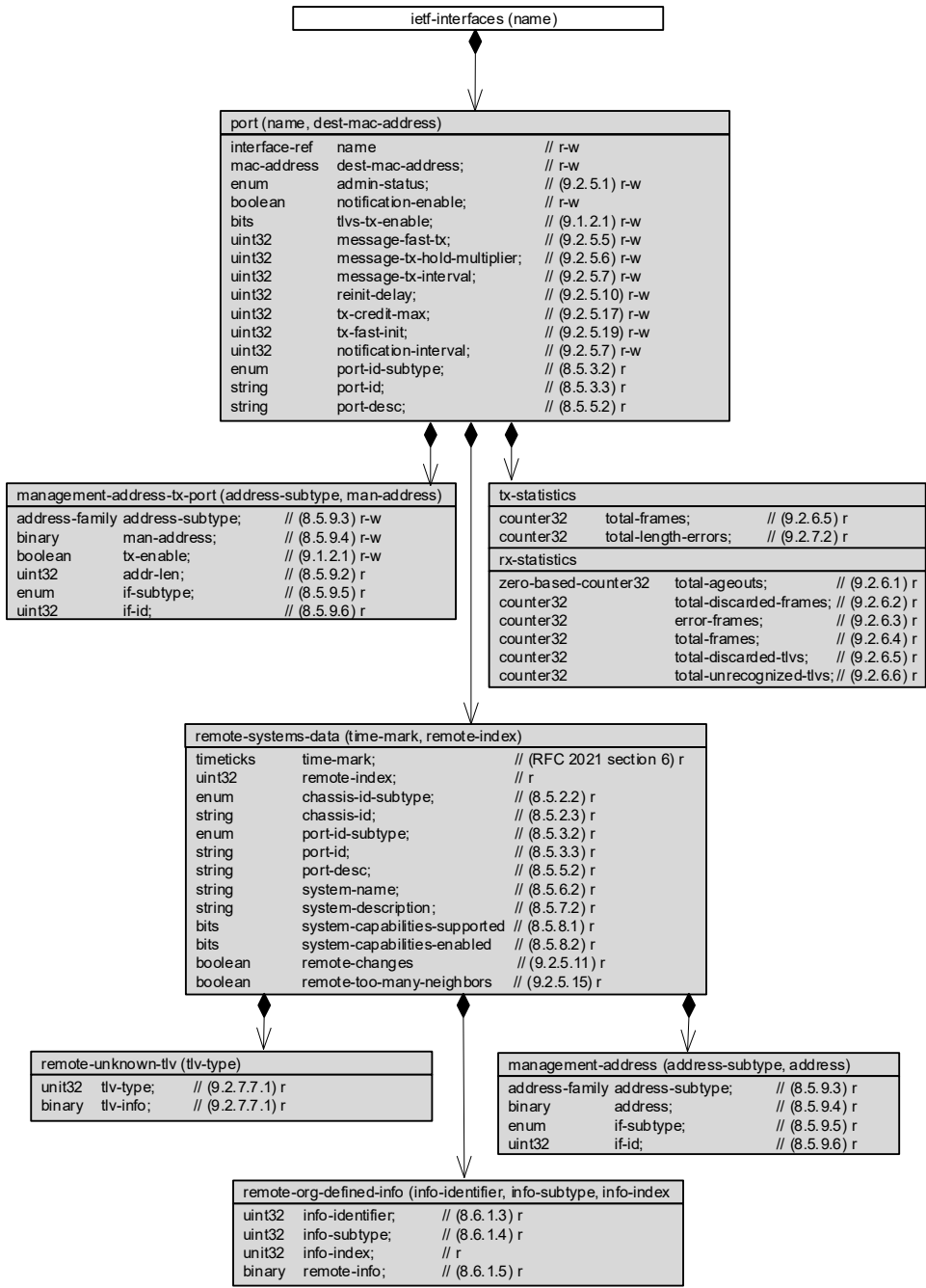


Figure 12-3—LLDP port configuration and monitoring objects

2 12.3 Structure of the LLDP YANG model

3 The IEEE YANG model specified in this standard is divided into two YANG modules. A summary of the
4 modules contained in this clause is represented in Table 12-1.

Table 12-1—Structure of the YANG modules

Module	Subclause	Notes
ieee802-dot1ab-types	Clause 12.6.2.2	Type definitions used for LLDP YANG
ieee802-dot1ab-lldp	Clause 12.6.2.3	LLDP Generic Management

¹ In the YANG module definitions below, if any discrepancy between the DESCRIPTION text and the
² corresponding definition in any other part of this standard occurs, the definitions outside this subclause take
³ precedence.

⁴ **12.3.1 Structure of the *ieee802-dot1ab-lldp* module**

⁵ The *ieee802-dot1ab-lldp* YANG module is divided into several YANG branches (e.g., subtrees). A summary
⁶ of the YANG subtrees associated with this module is presented in Table 12-2.

Table 12-2—Structure of the *ieee802-dot1ab-lldp* module

Branches	References	Notes
lldp		Link Layer Discovery Protocol (LLDP) configuration and operational information
message-fast-tx	9.2.5.5	
message-tx-hold-multiplier	9.2.5.6	
message-tx-interval	9.2.5.7	
reinit-delay	9.2.5.10	
tx-credit-max	9.2.5.17	
tx-fast-init	9.2.5.19	
notification-interval	9.2.5.7	
remote-statistics	10.5.2	LLDP remote operational statistics data
local-system-data	10.5.2	LLDP local system operational data
port	9.2.4	LLDP configuration information for a particular port
management-address-tx-port	8.5.9	Set of ports on which the local system management address instance will be transmitted
tx-statistics	9.2.6	LLDP frame transmission statistics for a particular port
rx-statistics	9.2.6	LLDP frame reception statistics for a particular port
remote-system-data		Information about a particular physical network connection

1 12.4 Relationship to other YANG modules

2 This clause describes how the *ieee802-dot1ab-lldp* YANG module is related to the YANG modules that are
3 imported.

4 12.4.1 IEEE LLDP Types Module

5 The *ieee802-dot1ab-types* module provides reusable types that are used by the *ieee802-dot1ab-lldp* module.

6 12.4.2 IETF Routing Module

7 The *ietf-routing* YANG module (IETF RFC 8349) is a module defined by the IETF for management of a
8 routing subsystem. This document only uses the address-family identity, which is used as the base identity
9 for address families.

10 12.4.3 IETF YANG Types Module

11 The *ietf-yang-types* YANG module (IETF RFC 6991) contains a set of derived YANG types. This document
12 leverages timestamp, timeticks, zero-based-counter32, and counter32.

13 12.4.4 IETF Interfaces YANG Module

14 The *ietf-interfaces* YANG module (IETF RFC 8343) contains a set of YANG definitions for managing
15 network interfaces. This document models a port as an interface.

16 12.4.5 IEEE 802 Types Module

17 The *ieee802-types* module provides reusable types that are used in IEEE 802 standards.

18 The type for mac-addresses defined in *ieee802-types* has a pattern that allows upper and lower case letters.
19 To avoid issues with string comparison, it is suggested to only use upper case for the letters in the
20 hexadecimal numbers. Implementers using code comparing MAC addresses should note that there is still an
21 issue with a difference between the IETF mac-address definition and the IEEE mac-address definition.

22 12.5 Security considerations

23 The YANG modules defined in this clause are designed to be accessed via a network configuration protocol
24 (e.g., NETCONF protocol). In the case of NETCONF, the lowest NETCONF layer is the secure transport
25 layer and the mandatory to implement secure transport is SSH. The NETCONF access control model
26 provides the means to restrict access for particular NETCONF users to a pre-configured subset of all
27 available NETCONF protocol operations and content.

28 It is the responsibility of a system's implementor and administrator to ensure that the protocol entities in the
29 system that support NETCONF, and any other remote configuration protocols that make use of these YANG
30 modules, are properly configured to allow access only to those users who have legitimate rights to read or
31 write data nodes. This standard does not specify how the credentials of those users are to be stored or
32 validated.

33 12.5.1 Security considerations of the *ieee802-dot1ab-lldp* YANG module

34 There are several management objects defined in the *ieee802-dot1ab-lldp* YANG module that are
35 configurable (i.e., read-write) and/or operational (i.e., read-only). Such objects may be considered sensitive
36 or vulnerable in some network environments. A network configuration protocol, such as NETCONF (IETF

1 RFC 6241), can support protocol operations that can edit or delete YANG module configuration data
2 (e.g., edit-config, delete-config, copy-config). If this is done in a non-secure environment without proper
3 protection, then negative effects on the network operation are possible.

4 The following objects in the *ieee802-dot1ab-lldp* YANG module can be manipulated to interfere with the
5 operation of LLDP. This could, for example, be used to return incorrect information about neighbor nodes,
6 or cause a denial-of-service attack.

7 lldp/message-fast-tx
8 lldp/message-tx-hold-multiplier
9 lldp/message-tx-interval
10 lldp/reinit-delay
11 lldp/tx-credit-max
12 lldp/tx-fast-init
13 lldp/notification-interval
14 lldp/port/name
15 lldp/port/dest-mac-address
16 lldp/port/admin-status
17 lldp/port/notification-enable
18 lldp/port/tlvs-tx-enable
19 lldp/port/message-fast-tx
20 lldp/port/message-tx-hold-multiplier
21 lldp/port/message-tx-interval
22 lldp/port/reinit-delay
23 lldp/port/tx-credit-max
24 lldp/port/tx-fast-init
25 lldp/port/notification-interval
26 lldp/port/management-address-tx-port/address-subtype
27 lldp/port/management-address-tx-port/man-address
28 lldp/port/management-address-tx-port/tx-enable

29 Some of the readable data in this YANG module may be considered sensitive or vulnerable in some network
30 environments. It is important to control all types of access (e.g., including NETCONF get and get-config
31 operations) to these objects and possibly to even encrypt the values of these objects when sending them over
32 the network. For example the system name and other information about the remote systems could provide
33 information about the configuration and topology of the network and could be considered a privacy threat.

34 **12.6 Definition of the YANG modules^{7, 8}**

35 **12.6.1 YANG schema definitions**

36 A simplified graphical representation of the data model is used in this document. The meaning of the
37 symbols in these diagrams is as follows:

38 — Brackets “[“ and “]” enclose list keys.

⁷ Copyright release for YANG modules: Users of this standard may freely reproduce the YANG modules contained in this subclause so that they can be used for their intended purpose.

⁸ An ASCII version of the YANG module(s) can be obtained by Web browser from the IEEE 802.1 Website at <https://1.ieee802.org/yang-modules/>.

- 1 — Abbreviations before data node names: “rw” means configuration (read-write), and “ro” means state data (read-only).
- 2
- 3 — Symbols after data node names: “?” means an optional node, “!” means a presence container, and
- 4 “*” denotes a list and leaf-list.
- 5 — Parentheses enclose choice and case nodes, and case nodes are also marked with a colon (“:”).
- 6 — Ellipsis (“...”) stand for contents of subtrees that are not shown.

7 12.6.1.1 YANG schema definition for *ieee802-dot1ab-lldp* YANG module

```

8 module: ieee802-dot1ab-lldp
9   +--rw lldp
10     +--rw message-fast-tx?          uint32
11     +--rw message-tx-hold-multiplier?  uint32
12     +--rw message-tx-interval?        uint32
13     +--rw reinit-delay?              uint32
14     +--rw tx-credit-max?             uint32
15     +--rw tx-fast-init?              uint32
16     +--rw notification-interval?      uint32
17     +--ro remote-statistics
18       | +--ro last-change-time?      yang:timestamp
19       | +--ro remote-inserts?        yang:zero-based-counter32
20       | +--ro remote-deletes?       yang:zero-based-counter32
21       | +--ro remote-drops?         yang:zero-based-counter32
22       | +--ro remote-ageouts?       yang:zero-based-counter32
23     +--ro local-system-data
24       | +--ro chassis-id-subtype?    ieee:chassis-id-subtype-type
25       | +--ro chassis-id?           ieee:chassis-id-type
26       | +--ro system-name?          string
27       | +--ro system-description?    string
28       | +--ro system-capabilities-supported?
29       | | lldp-types:system-capabilities-map
30       | +--ro system-capabilities-enabled?
31       | | lldp-types:system-capabilities-map
32     +--rw port* [name dest-mac-address]
33       +--rw name                    if:interface-ref
34       +--rw dest-mac-address        ieee:mac-address
35       +--rw admin-status?           enumeration
36       +--rw notification-enable?    boolean
37       +--rw tlvs-tx-enable?         bits
38       +--rw message-fast-tx?        uint32
39       +--rw message-tx-hold-multiplier?  uint32
40       +--rw message-tx-interval?      uint32
41       +--rw reinit-delay?           uint32
42       +--rw tx-credit-max?          uint32
43       +--rw tx-fast-init?           uint32
44       +--rw notification-interval?    uint32
45       +--rw management-address-tx-port* [address-subtype man-address]
46         | +--rw address-subtype      identityref
47         | +--rw man-address          lldp-types:man-addr-type
48         | +--rw tx-enable?           boolean
49         | +--ro addr-len?            uint32
50         | +--ro if-subtype?          lldp-types:man-addr-if-subtype
51         | +--ro if-id?              uint32
52       +--ro port-id-subtype?         ieee:port-id-subtype-type
53       +--ro port-id?                ieee:port-id-type
54       +--ro port-desc?              string
55       +--ro tx-statistics
56         | +--ro total-frames?        yang:counter32
57         | +--ro total-length-errors?  yang:counter32
58       +--ro rx-statistics
59         | +--ro total-ageouts?       yang:zero-based-counter32
60         | +--ro total-discarded-frames? yang:counter32

```



```

1      | +--ro error-frames?                yang:counter32
2      | +--ro total-frames?            yang:counter32
3      | +--ro total-discarded-tlvs?    yang:counter32
4      | +--ro total-unrecognized-tlvs? yang:counter32
5      +--ro remote-systems-data* [time-mark remote-index]
6          +--ro time-mark                yang:timeticks
7          +--ro remote-index            uint32
8          +--ro remote-too-many-neighbors? boolean
9          +--ro remote-changes?         boolean
10         +--ro chassis-id-subtype?     ieee:chassis-id-subtype-type
11         +--ro chassis-id?             ieee:chassis-id-type
12         +--ro port-id-subtype?        ieee:port-id-subtype-type
13         +--ro port-id?                ieee:port-id-type
14         +--ro port-desc?              string
15         +--ro system-name?            string
16         +--ro system-description?     string
17         +--ro system-capabilities-supported?
18         | lldp-types:system-capabilities-map
19         +--ro system-capabilities-enabled?
20         | lldp-types:system-capabilities-map
21         +--ro management-address* [address-subtype address]
22         | +--ro address-subtype       identityref
23         | +--ro address               lldp-types:man-addr-type
24         | +--ro if-subtype?           lldp-types:man-addr-if-subtype
25         | +--ro if-id?                uint32
26         +--ro remote-unknown-tlv* [tlv-type]
27         | +--ro tlv-type              uint32
28         | +--ro tlv-info?             binary
29         +--ro remote-org-defined-info*
30         | [info-identifier info-subtype info-index]
31         | +--ro info-identifier       uint32
32         | +--ro info-subtype          uint32
33         | +--ro info-index            uint32
34         | +--ro remote-info?         binary
35
36     notifications:
37         +---n remote-table-change
38             +--ro remote-insert?      -> /lldp/remote-statistics/remote-inserts
39             +--ro remote-delete?      -> /lldp/remote-statistics/remote-deletes
40             +--ro remote-drops?       -> /lldp/remote-statistics/remote-drops
41             +--ro remote-ageouts?     -> /lldp/remote-statistics/remote-ageouts

```

42

43 12.6.2 YANG data model definitions

44 12.6.2.1 Definition for the ieee802-types module

```

45 module ieee802-types {
46     namespace urn:ieee:std:802.1Q:yang:ieee802-types;
47     prefix ieee;
48     organization
49         "IEEE 802.1 Working Group";
50     contact
51         "WG-URL: http://ieee802.org/1/
52         WG-EMail: stds-802-1-1@ieee.org
53
54     Contact: IEEE 802.1 Working Group Chair
55     Postal: C/O IEEE 802.1 Working Group
56             IEEE Standards Association
57             445 Hoes Lane
58             Piscataway, NJ 08854
59             USA

```

```
1
2   E-mail: stds-802-1-chairs@ieee.org";
3 description
4   "This module contains a collection of generally useful derived data types
5   for IEEE YANG models. Copyright(C) IEEE (2022). All rights reserved. This
6   version of this YANG module is part of IEEE Std 802.1ABcu; see the
7   standard itself for full legal notices.";
8 revision 2022-03-16 {
9   description
10    "Published as part of IEEE Std 802.1ABcu.";
11   reference
12    "IEEE Std 802.1AB-2016";
13 }
14 revision 2020-06-04 {
15   description
16    "Published as part of IEEE Std 802.1Qcx-2020. Second version.";
17   reference
18    "IEEE Std 802.1Qcx-2020, Bridges and Bridged Networks - YANG Data Model
19    for Connectivity Fault Management.";
20 }
21 revision 2018-03-07 {
22   description
23    "Published as part of IEEE Std 802.1Q-2018. Initial version.";
24   reference
25    "IEEE Std 802.1Q-2018, Bridges and Bridged Networks.";
26 }
27 typedef mac-address {
28   type string {
29     pattern "[0-9a-fA-F]{2}(-[0-9a-fA-F]{2}){5}";
30   }
31   description
32    "The mac-address type represents a MAC address in the canonical format
33    and hexadecimal format specified by IEEE Std 802. The hexadecimal
34    representation uses uppercase characters.";
35   reference
36    "3.1 of IEEE Std 802-2014
37    8.1 of IEEE Std 802-2014";
38 }
39 typedef chassis-id-subtype-type {
40   type enumeration {
41     enum chassis-component {
42       value 1;
43       description
44        "Represents a chassis identifier based on the value of
45        entPhysicalAlias object (defined in IETF RFC 2737) for a chassis
46        component (i.e., an entPhysicalClass value of chassis(3)).";
47     }
48     enum interface-alias {
49       value 2;
50       description
51        "Represents a chassis identifier based on the value of ifAlias
52        object (defined in IETF RFC 2863) for an interface on the
53        containing chassis.";
54     }
55     enum port-component {
56       value 3;
57       description
58        "Represents a chassis identifier based on the value of
59        entPhysicalAlias object (defined in IETF RFC 2737) for a port or
60        backplane component (i.e., entPhysicalClass value of port(10) or
61        backplane(4)), within the containing chassis.";
62     }
63     enum mac-address {
64       value 4;
```

```
1      description
2          "Represents a chassis identifier based on the value of a unicast
3          source address (encoded in network byte order and IEEE 802.3
4          canonical bit order), of a port on the containing chassis as
5          defined in IEEE Std 802-2014.";
6      }
7      enum network-address {
8          value 5;
9          description
10             "Represents a chassis identifier based on a network address,
11             associated with a particular chassis. The encoded address is
12             actually composed of two fields. The first field is a single octet,
13             representing the IANA AddressFamilyNumbers value for the specific
14             address type, and the second field is the network address value.";
15     }
16     enum interface-name {
17         value 6;
18         description
19             "Represents a chassis identifier based on the value of ifName
20             object (defined in IETF RFC 2863) for an interface on the
21             containing chassis.";
22     }
23     enum local {
24         value 7;
25         description
26             "Represents a chassis identifier based on a locally defined value.";
27     }
28 }
29 description
30     "The source of a chassis identifier.";
31 reference
32     "IEEE Std 802-2014
33     IETF RFC 2737
34     IETF RFC 2863";
35 }
36 typedef chassis-id-type {
37     type string {
38         length "1..255";
39     }
40     description
41         "The format of a chassis identifier string. Objects of this type are
42         always used with an associated chassis-id-subtype object, which
43         identifies the format of the particular chassis-id object instance.
44
45         If the associated chassis-id-subtype object has a value of
46         chassis-component, then the octet string identifies a particular
47         instance of the entPhysicalAlias object (defined in IETF RFC 2737) for
48         a chassis component (i.e., an entPhysicalClass value of chassis(3)).
49
50         If the associated chassis-id-subtype object has a value of
51         interface-alias, then the octet string identifies a particular instance
52         of the ifAlias object (defined in IETF RFC 2863) for an interface on
53         the containing chassis. If the particular ifAlias object does not
54         contain any values, another chassis identifier type should be used.
55
56         If the associated chassis-id-subtype object has a value of
57         port-component, then the octet string identifies a particular instance
58         of the entPhysicalAlias object (defined in IETF RFC 2737) for a port or
59         backplane component within the containing chassis.
60
61         If the associated chassis-id-subtype object has a value of mac-address,
62         then this string identifies a particular unicast source address
63         (encoded in network byte order and IEEE 802.3 canonical bit order), of
64         a port on the containing chassis as defined in IEEE Std 802-2014.
```

If the associated chassis-id-subtype object has a value of network-address, then this string identifies a particular network address, encoded in network byte order, associated with one or more ports on the containing chassis. The first octet contains the IANA Address Family Numbers enumeration value for the specific address type, and octets 2 through N contain the network address value in network byte order.

If the associated chassis-id-subtype object has a value of interface-name, then the octet string identifies a particular instance of the ifName object (defined in IETF RFC 2863) for an interface on the containing chassis. If the particular ifName object does not contain any values, another chassis identifier type should be used.

If the associated chassis-id-subtype object has a value of local, then this string identifies a locally assigned Chassis ID.";

reference

"IEEE Std 802-2014
IETF RFC 2737
IETF RFC 2863";

```

}
typedef port-id-subtype-type {
  type enumeration {
    enum interface-alias {
      value 1;
      description
        "Represents a port identifier based on the ifAlias MIB object,
        defined in IETF RFC 2863.";
    }
    enum port-component {
      value 2;
      description
        "Represents a port identifier based on the value of
        entPhysicalAlias (defined in IETF RFC 2737) for a port component
        (i.e., entPhysicalClass value of port(10)), within the containing
        chassis.";
    }
    enum mac-address {
      value 3;
      description
        "Represents a port identifier based on a unicast source address
        (encoded in network byte order and IEEE 802.3 canonical bit order),
        which has been detected by the agent and associated with a
        particular port (IEEE Std 802-2014).";
    }
    enum network-address {
      value 4;
      description
        "Represents a port identifier based on a network address, detected
        by the agent and associated with a particular port.";
    }
    enum interface-name {
      value 5;
      description
        "Represents a port identifier based on the ifName MIB object,
        defined in IETF RFC 2863.";
    }
    enum agent-circuit-id {
      value 6;
      description
        "Represents a port identifier based on the agent-local identifier
        of the circuit (defined in RFC 3046), detected by the agent and
        associated with a particular port.";
    }
  }
}

```

```
1     }
2     enum local {
3         value 7;
4         description
5             "Represents a port identifier based on a value locally assigned.";
6     }
7 }
8 description
9     "The source of a particular type of port identifier.";
10 reference
11     "IEEE Std 802-2014
12     IETF RFC 2737
13     IETF RFC 2863
14     IETF RFC 3046";
15 }
16 typedef port-id-type {
17     type string {
18         length "1..255";
19     }
20     description
21         "The format of a port identifier string. Objects of this type are
22         always used with an associated port-id-subtype object, which identifies
23         the format of the particular port-id object instance.
24
25         If the associated port-id-subtype object has a value of
26         interface-alias, then the octet string identifies a particular instance
27         of the ifAlias object (defined in IETF RFC 2863). If the particular
28         ifAlias object does not contain any values, another port identifier
29         type should be used.
30
31         If the associated port-id-subtype object has a value of port-component,
32         then the octet string identifies a particular instance of the
33         entPhysicalAlias object (defined in IETF RFC 2737) for a port or
34         backplane component.
35
36         If the associated port-id-subtype object has a value of mac-address,
37         then this string identifies a particular unicast source address
38         (encoded in network byte order and IEEE 802.3 canonical bit order)
39         associated with the port (IEEE Std 802-2014).
40
41         If the associated port-id-subtype object has a value of
42         network-address, then this string identifies a network address
43         associated with the port. The first octet contains the IANA
44         AddressFamilyNumbers enumeration value for the specific address type,
45         and octets 2 through N contain the networkAddress address value in
46         network byte order.
47
48         If the associated port-id-subtype object has a value of interface-name,
49         then the octet string identifies a particular instance of the ifName
50         object (defined in IETF RFC 2863). If the particular ifName object does
51         not contain any values, another port identifier type should be used.
52
53         If the associated port-id-subtype object has a value of
54         agent-circuit-id, then this string identifies a agent-local identifier
55         of the circuit (defined in RFC 3046).
56
57         If the associated port-id-subtype object has a value of local, then
58         this string identifies a locally assigned port ID.";
59     reference
60         "IEEE Std 802-2014
61         IETF RFC 2737
62         IETF RFC 2863
63         IETF RFC 3046";
64 }
```

1 }
2

3 12.6.2.2 Definition for the *ieee802-dot1ab-types* module

```
4 module ieee802-dot1ab-types {  
5   yang-version "1.1";  
6   namespace urn:ieee:std:802.1Q:yang:ieee802-dot1ab-types;  
7   prefix lldp-types;  
8   organization  
9     "IEEE 802.1 Working Group";  
10  contact  
11    "WG-URL: http://ieee802.org/1/  
12    WG-Email: stds-802-1-1@ieee.org  
13    Contact: IEEE 802.1 Working Group Chair  
14    Postal: C/O IEEE 802.1 Working Group  
15    IEEE Standards Association  
16    445 Hoes Lane  
17    Piscataway, NJ 08854  
18    USA  
19  
20    E-mail: stds-802-1-chairs@ieee.org";  
21  description  
22    "Common types used within ieee802-dot1ab-lldp modules. Copyright(C) IEEE  
23    (2022). All rights reserved. This version of this YANG module is part of  
24    IEEE Std 802.1ABcu; see the standard itself for full legal notices.";  
25  revision 2022-03-15 {  
26    description  
27      "Published as part of IEEE Std 802.1ABcu.";  
28    reference  
29      "IEEE Std 802.1AB-2016";  
30  }  
31  typedef man-addr-if-subtype {  
32    type enumeration {  
33      enum unknown {  
34        value 1;  
35        description  
36          "Interface is not known.";  
37      }  
38      enum port-ref {  
39        value 2;  
40        description  
41          "Interface based on the port-ref MIB object.";  
42      }  
43      enum system-port-number {  
44        value 3;  
45        description  
46          "Interface based on the system port number.";  
47      }  
48    }  
49    description  
50      "Management address interface subtype.";  
51    reference  
52      "8.5.9.5 of IEEE Std 802.1AB-2016";  
53  }  
54  typedef man-addr-type {  
55    type string {  
56      pattern "[0-9A-F]{2}([0-9A-F]{2}){0,30}";  
57    }  
58    description  
59      "Management address associated with the LLDP agent.";  
60    reference  
61      "8.5.9.4 of IEEE Std 802.1AB-2016";  
62  }
```

```

1 typedef system-capabilities-map {
2     type bits {
3         bit other {
4             position 0;
5             description
6                 "System has capabilities other than those listed below.";
7         }
8         bit repeater {
9             position 1;
10            description
11                "System has repeater capability.";
12        }
13        bit bridge {
14            position 2;
15            description
16                "System has bridge capability.";
17        }
18        bit wlan-access-point {
19            position 3;
20            description
21                "System has WLAN access point capability.";
22        }
23        bit router {
24            position 4;
25            description
26                "System has router capability.";
27        }
28        bit telephone {
29            position 5;
30            description
31                "System has telephone capability.";
32        }
33        bit docsis-cable-device {
34            position 6;
35            description
36                "System has DOCSIS Cable Device capability.";
37            reference
38                "IETF RFC 4639";
39        }
40        bit station-only {
41            position 7;
42            description
43                "System has only station capability.";
44        }
45        bit cvlan-component {
46            position 8;
47            description
48                "System has C-VLAN component functionality.";
49        }
50        bit svlan-component {
51            position 9;
52            description
53                "System has S-VLAN component functionality.";
54        }
55        bit two-port-mac-relay {
56            position 10;
57            description
58                "System has Two-port MAC Relay (TPMR) functionality.";
59        }
60    }
61    description
62        "This describes system capabilities.";
63    reference
64        "8.5.8.1 of IEEE Std 802.1AB-2016";

```

```

1  }
2  typedef port-list {
3      type binary {
4          length "0..512";
5      }
6      description
7          "Each octet within this value specifies a set of eight ports, with the
8          first octet specifying ports 1 through 8, the second octet specifying
9          ports 9 through 16, etc. Within each octet, the most significant bit
10         represents the lowest numbered port, and the least significant bit
11         represents the highest numbered port. Thus, each port of the system is
12         represented by a single bit within the value of this object. If that
13         bit has a value of '1' then that port is included in the set of ports;
14         the port is not included if its bit has a value of '0'.";
15     reference
16         "IETF RFC 2674 section 5";
17 }
18 }
19

```

20 12.6.2.3 Definition for *ieee802-dot1ab-lldp* YANG module

```

21 module ieee802-dot1ab-lldp {
22     yang-version "1.1";
23
24     /*** NAMESPACE / PREFIX DEFINITION ***/
25     namespace urn:ieee:std:802.1AB:yang:ieee802-dot1ab-lldp;
26     prefix lldp;
27
28     /*** LINKAGE (IMPORTS / INCLUDES) ***/
29     import ieee802-dot1ab-types {
30         prefix lldp-types;
31     }
32     import ietf-routing {
33         prefix rt;
34     }
35     import ietf-yang-types {
36         prefix yang;
37     }
38     import ietf-interfaces {
39         prefix if;
40     }
41     import ieee802-types {
42         prefix ieee;
43     }
44
45     /*** META INFORMATION ***/
46     organization
47         "Institute of Electrical and Electronics Engineers";
48     contact
49         "WG-URL: http://ieee802.org/1/
50         WG-Email: stds-802-1-1@ieee.org
51         Contact: IEEE 802.1 Working Group Chair
52         Postal: C/O IEEE 802.1 Working Group
53         IEEE Standards Association
54             445 Hoes Lane
55             Piscataway, NJ 08854
56             USA
57
58         E-mail: stds-802-1-chairs@ieee.org";
59     description
60         "Management Information Base module for LLDP configuration, statistics,
61         local system data, and remote systems data components. Copyright(C) IEEE
62         (2022). All rights reserved. This version of this YANG module is part of

```



```

1 IEEE Std 802.1ABcu; see the standard itself for full legal notices.";
2 revision 2022-03-15 {
3   description
4     "Published as part of IEEE Std 802.1ABcu.";
5   reference
6     "IEEE Std 802.1AB-2016";
7 }
8
9 /** GROUPING DEFINITIONS */
10 grouping lldp-cfg {
11   description
12     "LLDP basic configuration group.";
13   leaf message-fast-tx {
14     type uint32 {
15       range "1..3600";
16     }
17     units "ticks";
18     default "1";
19     description
20       "Time interval in timer ticks between transmissions during fast
21       transmission periods (i.e., txFast is non-zero).";
22     reference
23       "9.1.1, 9.2.5.5 of IEEE Std 802.1AB-2016";
24   }
25   leaf message-tx-hold-multiplier {
26     type uint32 {
27       range "2..10";
28     }
29     default "4";
30     description
31       "Multiplier of msg-tx-interval.";
32     reference
33       "9.2.5.6 of IEEE Std 802.1AB-2016";
34   }
35   leaf message-tx-interval {
36     type uint32 {
37       range "1..3600";
38     }
39     units "ticks";
40     default "30";
41     description
42       "Time interval in timer ticks between transmissions during normal
43       transmission periods (i.e., txFast is zero).";
44     reference
45       "9.1.1, 9.2.5.7 of IEEE Std 802.1AB-2016";
46   }
47   leaf reinit-delay {
48     type uint32 {
49       range "1..10";
50     }
51     units "second";
52     default "2";
53     description
54       "Amount of delay (in units of seconds) from when admin-status becomes
55       'disabled' until re-initialization is attempted.";
56     reference
57       "9.2.5.10 of IEEE Std 802.1AB-2016";
58   }
59   leaf tx-credit-max {
60     type uint32 {
61       range "1..10";
62     }
63     default "5";
64     description

```

```

1      "The maximum number of consecutive LLDPDUs that can be transmitted at
2      any time.";
3      reference
4      "9.2.5.17 of IEEE Std 802.1AB-2016";
5  }
6  leaf tx-fast-init {
7      type uint32 {
8          range "1..8";
9      }
10     default "4";
11     description
12     "Initial value for the fast transmitting LLDPDU.";
13     reference
14     "9.2.5.19 of IEEE Std 802.1AB-2016";
15 }
16 leaf notification-interval {
17     type uint32 {
18         range "1..3600";
19     }
20     units "second";
21     default "30";
22     description
23     "Controls the transmission of LLDP notifications.";
24     reference
25     "9.2.5.7 of IEEE Std 802.1AB-2016";
26 }
27 }
28
29 /** SCHEMA DEFINITIONS */
30 container lldp {
31     description
32     "Link Layer Discovery Protocol configuration and operational
33     information.";
34     uses lldp-cfg;
35     container remote-statistics {
36         config false;
37         description
38         "LLDP remote operational statistics data.";
39         leaf last-change-time {
40             type yang:timestamp;
41             description
42             "The value of sysUpTime object.";
43             reference
44             "11.5.1 of IEEE Std 802.1AB-2016:
45             lldpV2StatsRemTablesLastChangeTime";
46         }
47         leaf remote-inserts {
48             type yang:zero-based-counter32;
49             units "table entries";
50             description
51             "The number of times the complete set of information advertised by
52             a particular MSAP has been inserted into tables contained in
53             remote-systems-data and lldpExtensions objects.";
54             reference
55             "11.5.1 of IEEE Std 802.1AB-2016: lldpV2StatsRemTablesInserts";
56         }
57         leaf remote-deletes {
58             type yang:zero-based-counter32;
59             units "table entries";
60             description
61             "The number of times the complete set of information advertised by
62             a particular MSAP has been deleted from tables contained in
63             remote-systems-data and lldpExtensions objects.";
64             reference

```

```

1      "11.5.1 of IEEE Std 802.1AB-2016: lldpV2StatsRemTablesDeletes";
2  }
3  leaf remote-drops {
4      type yang:zero-based-counter32;
5      units "table entries";
6      description
7          "The number of times the complete set of information advertised by
8          a particular MSAP could not be entered into tables contained in
9          remote-systems-data and lldpExtensions objects because of
10         insufficient resources.";
11     reference
12         "11.5.1 of IEEE Std 802.1AB-2016: lldpV2StatsRemTablesDrops";
13 }
14 leaf remote-ageouts {
15     type yang:zero-based-counter32;
16     description
17         "The number of times the complete set of information advertised by
18         a particular MSAP has been deleted from tables contained in
19         remote-systems-data and lldpExtensions objects because the
20         information timeliness interval has expired.";
21     reference
22         "11.5.1 of IEEE Std 802.1AB-2016: lldpV2StatsRemTablesAgeouts";
23 }
24 }
25 container local-system-data {
26     config false;
27     description
28         "LLDP local system operational data.";
29     leaf chassis-id-subtype {
30         type ieee:chassis-id-subtype-type;
31         description
32             "The type of encoding used to identify the chassis associated with
33             the local system.";
34         reference
35             "8.5.2.2 of IEEE Std 802.1AB-2016";
36     }
37     leaf chassis-id {
38         type ieee:chassis-id-type;
39         description
40             "Chassis component associated with the local system.";
41         reference
42             "8.5.2.3 of IEEE Std 802.1AB-2016";
43     }
44     leaf system-name {
45         type string {
46             length "0..255";
47         }
48         description
49             "System name of the local system.";
50         reference
51             "8.5.6.2 of IEEE Std 802.1AB-2016";
52     }
53     leaf system-description {
54         type string {
55             length "0..255";
56         }
57         description
58             "System description of the local system.";
59         reference
60             "8.5.7.2 of IEEE Std 802.1AB-2016";
61     }
62     leaf system-capabilities-supported {
63         type lldp-types:system-capabilities-map;
64         description

```

```

1         "System capabilities are supported on the local system.";
2     reference
3         "8.5.8.1 of IEEE Std 802.1AB-2016";
4 }
5 leaf system-capabilities-enabled {
6     type lldp-types:system-capabilities-map;
7     description
8         "System capabilities that are enabled on the local system.";
9     reference
10        "8.5.8.2 of IEEE Std 802.1AB-2016";
11 }
12 }
13
14 /* LLDP port configuration table */
15 list port {
16     key "name dest-mac-address";
17     description
18         "LLDP configuration information for a particular port.";
19     leaf name {
20         type if:interface-ref;
21         description
22             "The port name used to identify the port component (contained in
23              the local chassis with the LLDP agent) associated with this entry.";
24     }
25     leaf dest-mac-address {
26         type ieee:mac-address;
27         description
28             "Destination MAC address. The ieee:mac-address type has a pattern
29              that allows upper and lower case letters. To avoid issues with
30              string compairson, it is suggested to only use upper case for the
31              letters in the hexadecimal numbers. Implementers using code
32              comparing MAC addresses should note that there is still an issue
33              with a difference between the IETF mac-address definition and the
34              IEEE mac-address definition.";
35     }
36     leaf admin-status {
37         type enumeration {
38             enum tx-only {
39                 value 1;
40                 description
41                     "Transmit LLDP frames only.";
42             }
43             enum rx-only {
44                 value 2;
45                 description
46                     "Receive LLDP frames only.";
47             }
48             enum tx-and-rx {
49                 value 3;
50                 description
51                     "Transmit and Receive LLDP frames.";
52             }
53             enum disabled {
54                 value 4;
55                 description
56                     "Do Not Transmit or Receive LLDP frames.";
57             }
58         }
59         default "tx-and-rx";
60         description
61             "Administrative status of the local LLDP agent.";
62         reference
63             "9.2.5.1 of IEEE Std 802.1AB-2016";
64     }

```

```

1  leaf notification-enable {
2      type boolean;
3      default "false";
4      description
5          "Notification status.";
6  }
7  leaf tlvs-tx-enable {
8      type bits {
9          bit port-desc {
10             position 0;
11             description
12                 "Transmit 'Port Description TLV'.";
13         }
14         bit sys-name {
15             position 1;
16             description
17                 "Transmit 'System Name TLV'.";
18         }
19         bit sys-desc {
20             position 2;
21             description
22                 "Transmit 'System Description TLV'.";
23         }
24         bit sys-cap {
25             position 3;
26             description
27                 "Transmit 'System Capabilities TLV'.";
28         }
29     }
30     description
31         "LLDP TLVs whose transmission is allowed on the local LLDP agent by
32         the network management.";
33     reference
34         "9.1.2.1 of IEEE Std 802.1AB-2016";
35 }
36 uses lldp-cfg;
37 list management-address-tx-port {
38     key "address-subtype man-address";
39     description
40         "Set of ports (represented as a PortList) on which the local system
41         management address instance will be transmitted.";
42     leaf address-subtype {
43         type identityref {
44             base rt:address-family;
45         }
46         description
47             "Type of address.";
48         reference
49             "8.5.9.3 of IEEE Std 802.1AB-2016";
50     }
51     leaf man-address {
52         type lldp-types:man-addr-type;
53         description
54             "Management address associated with this TLV.";
55         reference
56             "8.5.9.4 of IEEE Std 802.1AB-2016";
57     }
58     leaf tx-enable {
59         type boolean;
60         default "false";
61         description
62             "Transmission enabled status.";
63         reference
64             "9.1.2.1 of IEEE Std 802.1AB-2016";

```

```

1      }
2      leaf addr-len {
3          type uint32;
4          config false;
5          description
6              "Length of the management address subtype and the management
7              address fields in LLDPDUs transmitted by the local LLDP agent.";
8          reference
9              "8.5.9.2 of IEEE Std 802.1AB-2016";
10     }
11     leaf if-subtype {
12         type lldp-types:man-addr-if-subtype;
13         config false;
14         description
15             "Interface numbering method used for defining the interface
16             number, associated with the local system.";
17         reference
18             "8.5.9.5 of IEEE Std 802.1AB-2016";
19     }
20     leaf if-id {
21         type uint32;
22         config false;
23         description
24             "Interface number for the management address component associated
25             with the local system.";
26         reference
27             "8.5.9.6 of IEEE Std 802.1AB-2016";
28     }
29 }
30 leaf port-id-subtype {
31     type ieee:port-id-subtype-type;
32     config false;
33     description
34         "Port identifier encoding used in the associated 'port-id' object.";
35     reference
36         "8.5.3.2 of IEEE Std 802.1AB-2016";
37 }
38 leaf port-id {
39     type ieee:port-id-type;
40     config false;
41     description
42         "Port component associated with a given port in the local system.";
43     reference
44         "8.5.3.3 of IEEE Std 802.1AB-2016";
45 }
46 leaf port-desc {
47     type string {
48         length "0..255";
49     }
50     config false;
51     description
52         "802 LAN station's port description associated with the local
53         system.";
54     reference
55         "8.5.5.2 of IEEE Std 802.1AB-2016";
56 }
57 container tx-statistics {
58     config false;
59     description
60         "LLDP frame transmission statistics for a particular port.";
61     leaf total-frames {
62         type yang:counter32;
63         description
64             "A count of all LLDP frames transmitted through the port.";

```

```

1      reference
2      "9.2.6.5 of IEEE Std 802.1AB-2016";
3  }
4  leaf total-length-errors {
5      type yang:counter32;
6      description
7          "A count of all LLDP length errors detected when constructing
8          LLDPDU frames for transmission through the port.";
9      reference
10     "9.2.7.2 of IEEE Std 802.1AB-2016";
11 }
12 }
13 container rx-statistics {
14     config false;
15     description
16         "LLDP frame reception statistics for a particular port.";
17     leaf total-ageouts {
18         type yang:zero-based-counter32;
19         description
20             "A count of the times that a neighbor's information is deleted
21             because of rxInfoTTL timer expiration.";
22         reference
23             "9.2.6.1 of IEEE Std 802.1AB-2016";
24     }
25     leaf total-discarded-frames {
26         type yang:counter32;
27         description
28             "A count of all LLDPDUs received and then discarded.";
29         reference
30             "9.2.6.2 of IEEE Std 802.1AB-2016";
31     }
32     leaf error-frames {
33         type yang:counter32;
34         description
35             "A count of all LLDPDUs received at the port with one or more
36             detectable errors.";
37         reference
38             "9.2.6.3 of IEEE Std 802.1AB-2016";
39     }
40     leaf total-frames {
41         type yang:counter32;
42         description
43             "A count of all LLDP frames received at the port.";
44         reference
45             "9.2.6.4 of IEEE Std 802.1AB-2016";
46     }
47     leaf total-discarded-tlvs {
48         type yang:counter32;
49         description
50             "A count of all TLVs received at the port and discarded for any
51             reason.";
52         reference
53             "9.2.6.5 of IEEE Std 802.1AB-2016";
54     }
55     leaf total-unrecognized-tlvs {
56         type yang:counter32;
57         description
58             "A count of all TLVs not recognized by the receiving LLDP local
59             agent.";
60         reference
61             "9.2.6.6 of IEEE Std 802.1AB-2016";
62     }
63 }
64 list remote-systems-data {

```

```

1  key "time-mark remote-index";
2  config false;
3  description
4    "Information about a particular physical network connection.";
5  leaf time-mark {
6    type yang:timeticks;
7    description
8      "A TimeFilter for this entry.";
9    reference
10     "IETF RFC 2021 section 6";
11  }
12  leaf remote-index {
13    type uint32 {
14      range "1..2147483647";
15    }
16    description
17      "Represents an arbitrary local integer value used to identify a
18      remote system.";
19    reference
20      "11.5.1 of IEEE Std 802.1AB-2016: lldpV2RemIndex";
21  }
22  leaf remote-too-many-neighbors {
23    type boolean;
24    description
25      "Indicates that there are too many neighbors as determined by the
26      variable tooManyNeighbors.";
27    reference
28      "9.2.5.15 of IEEE Std 802.1AB-2016";
29  }
30  leaf remote-changes {
31    type boolean;
32    description
33      "Indicates that there are changes in the remote system's data, as
34      determined by the variable remoteChanges.";
35    reference
36      "9.2.5.11 of IEEE Std 802.1AB-2016";
37  }
38  leaf chassis-id-subtype {
39    type ieee:chassis-id-subtype-type;
40    description
41      "Identify the chassis associated with the remote system.";
42    reference
43      "8.5.2.2 of IEEE Std 802.1AB-2016";
44  }
45  leaf chassis-id {
46    type ieee:chassis-id-type;
47    description
48      "Identify the chassis component associated with the remote
49      system.";
50    reference
51      "8.5.2.3 of IEEE Std 802.1AB-2016";
52  }
53  leaf port-id-subtype {
54    type ieee:port-id-subtype-type;
55    description
56      "The type of port identifier encoding used in the associated
57      'port-id' object.";
58    reference
59      "8.5.3.2 of IEEE Std 802.1AB-2016";
60  }
61  leaf port-id {
62    type ieee:port-id-type;
63    description
64      "Port component associated with the remote system.";

```



```

1      reference
2      "8.5.3.3 of IEEE Std 802.1AB-2016";
3  }
4  leaf port-desc {
5      type string {
6          length "0..255";
7      }
8      description
9          "Description of the given port associated with the remote system.";
10     reference
11         "8.5.5.2 of IEEE Std 802.1AB-2016";
12 }
13 leaf system-name {
14     type string {
15         length "0..255";
16     }
17     description
18         "System name of the remote system.";
19     reference
20         "8.5.6.2 of IEEE Std 802.1AB-2016";
21 }
22 leaf system-description {
23     type string {
24         length "0..255";
25     }
26     description
27         "System description of the remote system.";
28     reference
29         "8.5.7.2 of IEEE Std 802.1AB-2016";
30 }
31 leaf system-capabilities-supported {
32     type lldp-types:system-capabilities-map;
33     description
34         "Capabilities that are supported on the remote system.";
35     reference
36         "8.5.8.1 of IEEE Std 802.1AB-2016";
37 }
38 leaf system-capabilities-enabled {
39     type lldp-types:system-capabilities-map;
40     description
41         "System capabilities that are enabled on the remote system.";
42     reference
43         "8.5.8.2 of IEEE Std 802.1AB-2016";
44 }
45 list management-address {
46     key "address-subtype address";
47     description
48         "Management address information about a particular chassis
49         component.";
50     leaf address-subtype {
51         type identityref {
52             base rt:address-family;
53         }
54         description
55             "Management address identifier encoding.";
56         reference
57             "8.5.9.3 of IEEE Std 802.1AB-2016";
58     }
59     leaf address {
60         type lldp-types:man-addr-type;
61         description
62             "Management address component associated with the remote
63             system.";
64         reference

```

```

1         "8.5.9.4 of IEEE Std 802.1AB-2016";
2     }
3     leaf if-subtype {
4         type lldp-types:man-addr-if-subtype;
5         description
6             "Interface numbering method used for defining the interface
7             number, associated with the remote system.";
8         reference
9             "8.5.9.5 of IEEE Std 802.1AB-2016";
10    }
11    leaf if-id {
12        type uint32;
13        description
14            "Interface number regarding the management address component
15            associated with the remote system.";
16        reference
17            "8.5.9.6 of IEEE Std 802.1AB-2016";
18    }
19 }
20 list remote-unknown-tlv {
21     key "tlv-type";
22     description
23         "Information about an unrecognized TLV received from a physical
24         network connection. Entries may be created and deleted in this
25         table by the agent, if a physical topology discovery process is
26         active.";
27     leaf tlv-type {
28         type uint32 {
29             range "9..126";
30         }
31         description
32             "Type of TLV.";
33         reference
34             "9.2.7.7.1 of IEEE Std 802.1AB-2016";
35     }
36     leaf tlv-info {
37         type binary {
38             length "0..511";
39         }
40         description
41             "Value extracted from TLV.";
42         reference
43             "9.2.7.7.1 of IEEE Std 802.1AB-2016";
44     }
45 }
46 list remote-org-defined-info {
47     key "info-identifier info-subtype info-index";
48     description
49         "Information about the unrecognized organizationally defined
50         information advertised by the remote system.";
51     leaf info-identifier {
52         type uint32 {
53             range "0..16777215";
54         }
55         description
56             "The Organizationally Unique Identifier (OUI) or Company ID
57             (CID).";
58         reference
59             "8.6.1.3 of IEEE Std 802.1AB-2016";
60     }
61     leaf info-subtype {
62         type uint32 {
63             range "1..255";
64         }

```

```

1      description
2          "The subtype of the organizationally defined information
3          received from the remote system.";
4      reference
5          "8.6.1.4 of IEEE Std 802.1AB-2016";
6  }
7  leaf info-index {
8      type uint32 {
9          range "1..2147483647";
10     }
11     description
12         "Arbitrary local integer value.";
13 }
14 leaf remote-info {
15     type binary {
16         length "0..507";
17     }
18     description
19         "The organizationally defined information of the remote system.";
20     reference
21         "8.6.1.5 of IEEE Std 802.1AB-2016";
22 }
23 }
24 }
25 }
26 }
27 notification remote-table-change {
28     description
29         "A rem-table-change notification is sent when the value of
30         remote-table-last-change-time changes. It can be utilized by an NMS to
31         trigger LLDP remote systems table maintenance polls.";
32     leaf remote-insert {
33         type leafref {
34             path '/lldp:lldp/lldp:remote-statistics/lldp:remote-inserts';
35         }
36         description
37             "The number of times the complete set of information advertised by a
38             particular MSAP has been inserted into tables contained in
39             remote-systems-data and lldpExtensions objects.";
40         reference
41             "11.5.1 of IEEE Std 802.1AB-2016: lldpV2StatsRemTablesInserts";
42     }
43     leaf remote-delete {
44         type leafref {
45             path '/lldp:lldp/lldp:remote-statistics/lldp:remote-deletes';
46         }
47         description
48             "The number of times the complete set of information advertised by a
49             particular MSAP has been deleted from tables contained in
50             remote-systems-data and lldpExtensions objects.";
51         reference
52             "11.5.1 of IEEE Std 802.1AB-2016: lldpV2StatsRemTablesDeletes";
53     }
54     leaf remote-drops {
55         type leafref {
56             path '/lldp:lldp/lldp:remote-statistics/lldp:remote-drops';
57         }
58         description
59             "The number of times the complete set of information advertised by a
60             particular MSAP could not be entered into tables contained in
61             remote-systems-data and lldpExtensions objects because of
62             insufficient resources.";
63         reference
64             "11.5.1 of IEEE Std 802.1AB-2016: lldpV2StatsRemTablesDrops";

```

```
1     }
2     leaf remote-ageouts {
3         type leafref {
4             path '/lldp:lldp/lldp:remote-statistics/lldp:remote-ageouts';
5         }
6         description
7             "The number of times the complete set of information advertised by a
8             particular MSAP has been deleted from tables contained in
9             remote-systems-data and lldpExtensions objects because the
10            information timeliness interval has expired.";
11         reference
12            "11.5.1 of IEEE Std 802.1AB-2016: lldpV2StatsRemTablesAgeouts";
13     }
14 }
15 }
16
17
```

1 Annex A

2 (normative)

3 PICS proforma⁶

4 A.1 Introduction

5 The supplier of a protocol implementation that is claimed to conform to this standard shall complete the
6 following Protocol Implementation Conformance Statement (PICS) proforma.

7 A completed PICS proforma is the PICS for the implementation in question. The PICS is a statement of
8 which capabilities and options of the protocol have been implemented. The PICS can have a number of uses,
9 including use

- 10 a) By the protocol implementers, as a checklist to reduce the risk of failure to conform to the standard
11 through oversight.
- 12 b) By the supplier and acquirer—or potential acquirer—of the implementation, as a detailed indication
13 of the capabilities of the implementation, stated relative to the common basis for understanding
14 provided by the standard PICS proforma.
- 15 c) By the user—or potential user—of the implementation, as a basis for initially checking the
16 possibility of interworking with another implementation (note that although interworking can never
17 be guaranteed, failure to interwork can often be predicted from incompatible PICS).
- 18 d) By a protocol tester, as the basis for selecting appropriate tests against which to assess the claim for
19 conformance of the implementation.

20 A.2 Abbreviations and special symbols

21 A.2.1 Status symbols

- 22 M Mandatory
- 23 O Optional
- 24 *O.n* Optional, but support of at least one of the group of options labeled by the same numeral *n* is
25 required
- 26 X Prohibited
- 27 pred: Conditional-item symbol, including predicate identification: See B.3.4
- 28 ¬ Logical negation, applied to a conditional item's predicate

29 A.2.2 General abbreviations

- 30 N/A Not applicable
- 31 PICS Protocol Implementation Conformance Statement

⁶ *Copyright release for PICS proformas:* Users of this standard may freely reproduce the PICS proforma in this annex so that it can be used for its intended purpose and may further publish the completed PICS.

1 A.3 Instructions for completing the PICS proforma

2 A.3.1 General structure of the PICS proforma

3 The first part of the PICS proforma, implementation identification and protocol summary, is to be completed
4 as indicated with the information necessary to identify fully both the supplier and the implementation.

5 The main part of the PICS proforma is a fixed-format questionnaire, divided into several subclauses, each
6 containing a number of individual items. Answers to the questionnaire items are to be provided in the
7 right-most column, either by simply marking an answer to indicate a restricted choice (usually Yes or No), or
8 by entering a value or a set or range of values. (Note that there are some items in which two or more choices
9 from a set of possible answers can apply; all relevant choices are to be marked.)

10 Each item is identified by an item reference in the first column. The second column contains the question to
11 be answered; the third column records the status of the item—whether support is mandatory, optional, or
12 conditional; see also B.3.4. The fourth column contains the reference or references to the material that
13 specifies the item in the main body of this standard, and the fifth column provides the space for the answers.

14 A supplier may also provide (or be required to provide) further information, categorized as either Additional
15 Information or Exception Information. When present, each kind of further information is to be provided in a
16 further subclause of items labeled A_i or X_i , respectively, for cross-referencing purposes, where i is any
17 unambiguous identification for the item (e.g., simply a numeral). There are no other restrictions on its format
18 and presentation.

19 A completed PICS proforma, including any Additional Information and Exception Information, is the
20 Protocol Implementation Conformation Statement for the implementation in question.

21 NOTE—Where an implementation is capable of being configured in more than one way, a single PICS may be able to
22 describe all such configurations. However, the supplier has the choice of providing more than one PICS, each covering
23 some subset of the implementation's configuration capabilities, in case that makes for easier and clearer presentation of
24 the information.

25 A.3.2 Additional information

26 Items of Additional Information allow a supplier to provide further information intended to assist the
27 interpretation of the PICS. It is not intended or expected that a large quantity will be supplied, and a PICS
28 can be considered complete without any such information. Examples might be an outline of the ways in
29 which a (single) implementation can be set up to operate in a variety of environments and configurations, or
30 information about aspects of the implementation that are outside the scope of this standard but that have a
31 bearing on the answers to some items.

32 References to items of Additional Information may be entered next to any answer in the questionnaire and
33 may be included in items of Exception Information.

34 A.3.3 Exception information

35 It may occasionally happen that a supplier will wish to answer an item with mandatory status (after any
36 conditions have been applied) in a way that conflicts with the indicated requirement. No preprinted answer
37 will be found in the Support column for this: instead, the supplier shall write the missing answer into the
38 Support column, together with an X_i reference to an item of Exception Information, and shall provide the
39 appropriate rationale in the Exception item.

- 1 An implementation for which an Exception item is required in this way does not conform to this standard.
2 NOTE—A possible reason for the situation described above is that a defect in this standard has been reported, a
3 correction for which is expected to change the requirement not met by the implementation.

4 **A.3.4 Conditional status**

5 **A.3.4.1 Conditional items**

6 The PICS proforma contains a number of conditional items. These are items for which both the applicability
7 of the item itself, and its status if it does apply—mandatory or optional—are dependent upon whether or not
8 certain other items are supported.

9 Where a group of items is subject to the same condition for applicability, a separate preliminary question
10 about the condition appears at the head of the group, with an instruction to skip to a later point in the
11 questionnaire if the “Not Applicable” answer is selected. Otherwise, individual conditional items are
12 indicated by a conditional symbol in the Status column.

13 A conditional symbol is of the form “pred: S,” where pred is a predicate as described in A.3.4.2, and S is a
14 status symbol, M or O.

15 If the value of the predicate is TRUE (see A.3.4.2), the conditional item is applicable, and its status is
16 indicated by the status symbol following the predicate: the answer column is to be marked in the usual way.
17 If the value of the predicate is FALSE, the “Not Applicable” (N/A) ⁷ answer is to be marked.

18 **A.3.4.2 Predicates**

19 A predicate is one of the following:

- 20 a) An item-reference for an item in the PICS proforma: The value of the predicate is TRUE if the item
21 is marked as supported, and is FALSE otherwise.
22 b) A predicate-name, for a predicate defined as a Boolean expression constructed by combining
23 item-references using the Boolean operator OR: The value of the predicate is TRUE if one or more
24 of the items is marked as supported.
25 c) A predicate-name, for a predicate defined as a Boolean expression constructed by combining
26 item-references using the Boolean operator AND: The value of the predicate is TRUE if all of the
27 items are marked as supported.
28 d) The logical negation symbol “¬” prefixed to an item-reference or predicate-name: The value of the
29 predicate is TRUE if the value of the predicate formed by omitting the “¬” symbol is FALSE, and
30 vice versa.

31 Each item whose reference is used in a predicate or predicate definition, or in a preliminary question for
32 grouped conditional items, is indicated by an asterisk ⁸ in the Item column.

33

34

35

36 **PICS proforma for IEEE Std 802.1AB-2016**

⁷ (N/A) is currently not used in this PICS.

⁸ Asterisks are currently not used in this PICS.

A.3.5 Implementation identification

Supplier	
Contact point for queries about the PICS	
Implementation Name(s) and Version(s)	
Other information necessary for full identification—e.g., name(s) and version(s) of machines and/or operating system names	
NOTE 1—Only the first three items are required for all implementations; other information may be completed as appropriate in meeting the requirement for full identification. NOTE 2—The terms Name and Version should be interpreted appropriately to correspond with a supplier’s terminology (e.g., Type, Series, Model).	

A.3.6 Protocol summary, IEEE Std 802.1AB-2016

Identification of protocol specification	IEEE Std 802.1AB-2016, IEEE Standard for Local and metropolitan area networks—Station and Media Access Control Connectivity Discovery		
Identification of amendments and corrigenda to the PICS proforma that have been completed as part of the PICS	Amd.	:	Corr. :
	Amd.	:	Corr. :
Have any Exception items been required? (See B.3.3: The answer Yes means that the implementation does not conform to IEEE Std 802.1AB-2016)	No [] Yes []		

Date of Statement	
-------------------	--

A.4 Major capabilities and options

[illegible]

A.4 Major capabilities and options

Item	Feature	Status	References	Support
	Which of the following operational modes are implemented?			
optxrx	Transmit and receive	O.1	6.1	Yes [] No []
optx	Transmit only	O.1	6.1	Yes [] No []
oprxx	Receive only	O.1	6.1	Yes [] No []
txmode	Is the transmit mode in conformance with all operational specifications indicated for the Tx mode in Table 9-1?	optxrx OR optx: M	Clause 9	Yes [] N/A []
rxmode	Is the receive module in conformance with all operational specifications indicated for the Rx mode in Table 9-1?	optxrx OR oprx: M	Clause 9	Yes [] N/A []
	Which type of data store/retrieval is implemented?			
mib	SNMP MIB is supported	O.2	11.5, 5.3	Yes [] No []
nomib	SNMP MIB is not supported	O.2	10.1, 5.3	Yes [] No []
snmpmib	Is the MIB module in conformance with the MIB sections indicated in Table 11-1 for the operating mode being implemented?	mib:M	11.5, 5.3	Yes [] N/A []
snmpsupport	Which of the transport mappings defined by IETF RFC 3417 or IETF RFC 4789 is used to support SNMP?			
	IETF RFC 3417	mib:O.3	5.3, 5.4	Yes [] No [] N/A []
	IETF RFC 4789	mib:O.3	5.3, 5.4	Yes [] No [] N/A []
equivstor	If the SNMP is not supported, is functionally equivalent storage and retrieval capability specified in Clause 8, Clause 9, Clause10 provided for the operating mode being implemented?	nomib:M	10.1	Yes [] N/A []

A.4 Major capabilities and options

Item	Feature	Status	References	Support
xlldp	Is the Extended Link Layer Discovery Protocol supported?	O		Yes [] No []
xlldpdu	Are the Extension and Extension Request LLDPDU encapsulation in conformance with the TLV order specified?	xlldp:M	8.2	Yes [] N/A []
xaddr	Are LLDP addressing and LLDP EtherType encoding for Extension and Extension Request LLDPDUs in conformance with the defined requirements?			
	<i>Extension Request LLDPDUs:</i>			
	DA = Return MAC address from Manifest TLV	xlldp:M	7.1.2	Yes [] N/A []
	SA = station MAC address	xlldp:M	7.2	Yes [] N/A []
	LLDP EtherType encoding	xlldp:M	7.4	Yes [] N/A []
	<i>Extension LLDPU:</i>			
	DA = Return MAC address from Extension Request TLV	xlldp:M	7.1.2	Yes [] N/A []
	SA = station MAC address	xlldp:M	7.2	Yes [] N/A []
	LLDP EtherType encoding	xlldp:M	7.4	Yes [] N/A []
yang	Does the implementation support management operations using YANG modules?	O	12.6	Yes[] No[]
yang modules	Is the ieee802-dot1ab-lldp module supported?	yang:M	12.6.2.3	Yes[] No []
	Is the ieee802-dot1ab-types module supported?	yang:M	12.6.2.2	Yes[] No[]

Annex B

(normative)

PTOPO MIB update

The PTOPO MIB should be updated according to the following rules:

- a) If any objects in the LLDP remote systems MIB age out, the equivalent objects in the PTOPO MIB should be deleted.
- b) If the TTL value in the Time To Live TLV is zero, then the port is being shutdown and the PTOPO MIB objects associated with the MSAP identifier should be deleted.
- c) If the TTL field is non-zero, then the appropriate ptopoRemEntry is found or created, based on the data elements included in the LLDP frame. If the indicated entry is dynamic (i.e., ptopoConnIsStatic is FALSE), then the current sysUpTime value is stored in the ptopoConnLastVerifyTime field for the entry.
- d) If a ptopoRemEntry was added then the ptopoConnTabInserts counter is incremented.
- e) If any ptopoRemEntry was added or deleted, or if information other than the ptopoRemLastVerifyTime changed for any entry due to the processing of this LLDP frame, the ptopoLastChangeTime object is set with the current sysUpTime, and a ptopoConfigChange trap event is generated. (See the PTOPO MIB for information on ptopoConfigChange trap generation.)

1 **Annex C**

2 (informative)

3 **Example LLDP transmission frame formats**

4 The LLDP MAC frame format is based on the particular transmission protocol. The following example
5 formats illustrate the indicated LLDP EtherType encoding method.

6 **C.1 Direct-encoded LLDP frame format**

7 The IEEE 802.3 LLDP frame format is illustrated in Figure C.1.

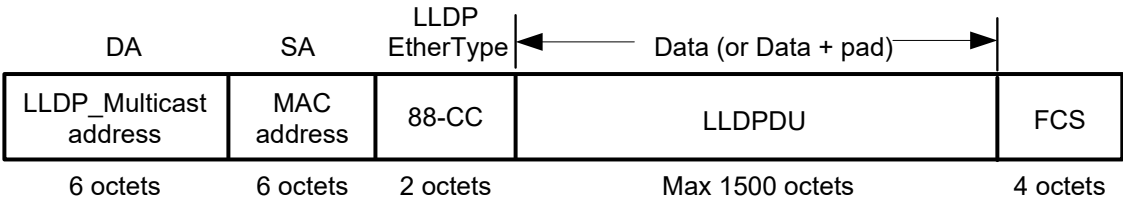


Figure C.1—IEEE 802.3 LLDP frame format

8 NOTE—The illustration shows the simplest form of an LLDP frame on an IEEE 802.3 medium; i.e., where the frame
9 has had no IEEE 802.1Q tag header, or IEEE 802.1AE™ security tag, or any other form of encapsulation applied to it.

10 **C.2 SNAP-encoded LLDP frame format**

11 The IEEE 802.11™ frame format is illustrated in Figure C.2, for the case where the value of To DS and
12 From DS in the Frame Control field are both zero.

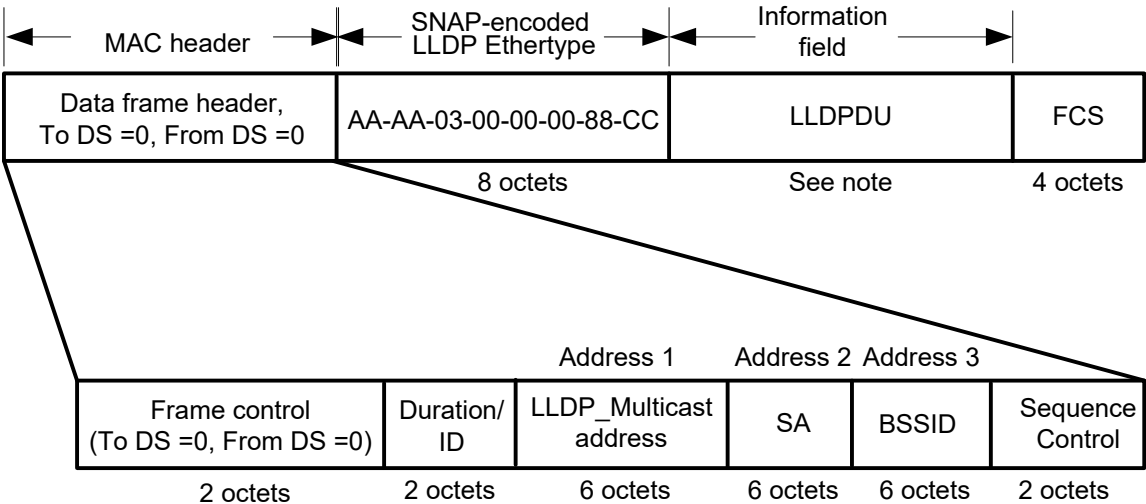


Figure C.2—IEEE 802.11 LLDP frame format

13 NOTE—The illustration shows the simplest form of an LLDP frame on an IEEE 802.11 medium; i.e., where the frame
14 has had no IEEE 802.1Q tag header, or IEEE 802.1AE security tag, or any other form of encapsulation applied to it.

¹ Annex D

² (informative)

³ Bibliography

⁴ [B1] IEEE Std 802.1aj-2009, Standard for Local and Metropolitan Area Networks—Virtual Bridged Local
⁵ Area Networks—Amendment: Two-port Media Access Control (MAC) Relay. ⁶

⁶ [B2] IEEE Std 802.11, Standard for Information technology—Telecommunications and information
⁷ exchange between systems—Local and metropolitan area networks—Specific requirements—Part 11:
⁸ Wireless LAN Medium Access Control (MAC) and Physical Layer (PHY) specifications.

⁹ [B3] IETF RFC 2578 (STD 58), Structure of Management Information for Version 2 of the Simple Network
¹⁰ Management Protocol (SNMPv2), McCloghrie, K., Perkins, D., Schoenwaelder, J., Case, J., Rose, M.,
¹¹ Waldbusser, S., April 1999. ⁷

¹² [B4] IETF RFC 2579 (STD 58), Textual Conventions for Version 2 of the Simple Network Management
¹³ Protocol (SNMPv2), McCloghrie, K., Perkins, D., Schoenwaelder, J., Case, J., Rose, M., Waldbusser, S.,
¹⁴ April 1999.

¹⁵ [B5] IETF RFC 2580 (STD 58), Conformance Statements for SMIPv2, McCloghrie, K., Perkins, D.,
¹⁶ Schoenwaelder, J., Case, J., Rose, M., Waldbusser, S., April 1999.

¹⁷ [B6] IETF RFC 1812, Requirements for IP Version 4 Routers, June 1995.

¹⁸ [B7] IETF RFC 2108, Definitions of Managed Objects for IEEE 802.3 Repeater Devices using SMIPv2,
¹⁹ February 1997.

²⁰ [B8] IETF RFC 2670, Radio Frequency (RF) Interface Management Information Base for MCNS/DOCSIS
²¹ compliant RF interfaces, August 1999.

²² [B9] IETF RFC 2922, Physical Topology MIB, Bierman, A., and Jones, K., November 1998.

²³ [B10] IETF RFC 4293, Management Information Base for the Internet Protocol (IP), April 2006.

²⁴ [B11] IETF RFC 4363, Definitions of Managed Objects for Bridges with Traffic Classes, Multicast
²⁵ Filtering, and Virtual LAN Extensions, January 2006.

²⁶ [B12] IETF RFC 4546, Radio Frequency (RF) Interface Management Information Base for Data over Cable
²⁷ Service Interface Specifications (DOCSIS) 2.0 Compliant RF Interfaces, June 2006.

²⁸ [B13] IETF RFC 7803, Changing the Registration Policy for the NETCONF Capability URNs Registry,
²⁹ February 2016.

³⁰ [B13] IETF RFC 8040, RESTCONF Protocol, January 2017.

⁶ IEEE publications are available from The Institute of Electrical and Electronic Engineers (<http://standards.ieee.org>).

⁷ IETF RFCs are available from the Internet Engineering Task Force (<https://www.rfc-archive.org/>).

- ¹ [B14] ISO/IEC 8802-2:1998, Standard for Information technology—Telecommunications and information
² exchange between systems—Local and metropolitan area networks—Specific requirements—Part 2:
³ Logical link control.⁸

⁸ ISO/IEC publications are available from the International Organization for Standardization (<https://www.iso.org/>), the International Electrotechnical Commission (<https://www.iec.ch/>), and the American National Standards Institute (<https://www.ansi.org/>).